

PRACTICAL PYTHON GIS

Spatial Data Science: The New Frontier in Analytics

A practical Python GIS course — GeoPandas · Shapely · Rasterio

Dataset The Vallmara Basin, Republic of Kestria (entirely fictional)

Libraries GeoPandas · Shapely · Rasterio · NumPy · pandas · matplotlib · seaborn · SciPy · scikit-learn · statsmodels

Structure 5 modules · 49 lessons · 4 exercise sets · full solutions · end-to-end capstone

Code cells 62 runnable cells

Built 2026-08-20

Run `python generate_data.py` once to create the dataset, then work through `Spatial_Data_Science_Course.ipynb` alongside this document. Every code cell in this PDF is exactly the code in the notebook — both are generated from the same source, so they cannot drift apart.

Contents

Spatial Data Science: The New Frontier in Analytics

A practical, project-based course in Python GIS — GeoPandas · Shapely · Rasterio

What you will build

Course structure

0.1 — Environment and installation

Recommended Python version

The packages, and why each one is here

`pip` installation (virtual environment — recommended)

Conda / mamba installation (recommended if pip gives you GDAL trouble)

Launching Jupyter

Recommended folder structure

Generating the dataset

Troubleshooting the usual suspects

0.2 — Check your environment

0.3 — Generate the dataset

0.4 — The Vallmara Basin: scenario and data dictionary

The story

Coordinate reference systems in this dataset (read this twice)

Vector layers — `data/vector/vallmara.gpkg` (EPSG:32633)

Raster layers — `data/raster/`

Tabular data — `data/tabular/`

The deliberate data-quality problems

The ground truth baked into the simulation

0.5 — Global setup cell

Module 1 — Beginner: The Foundations

B1 — Loading vector data

B2 — Exploring the attribute table

B3 — The geometry column: points, lines and polygons

B4 — CRS I: what a coordinate reference system actually is

B5 — CRS II: reprojection, and measuring the cost of getting it wrong

B6 — Plotting spatial data

B7 — Building a GeoDataFrame from a CSV

B8 — Basic Shapely operations

B9 — Filtering and selecting spatial features

B10 — Calculating distances

B11 — Your first spatial join

B12 — Opening a raster with Rasterio

B13 — Raster ↔ array ↔ ground: indexing, sampling and plotting

B14 — Writing data out

Exercises — Module 1 (Beginner)

Exercise 1.1 — Layer inventory

Exercise 1.2 — Attribute audit

Exercise 1.3 — CRS forensics

Exercise 1.4 — Building geometries from a dirty CSV

Exercise 1.5 — Shapely reasoning

Exercise 1.6 — Distance profile

Exercise 1.7 — Spatial join and rate calculation

Exercise 1.8 — Raster sampling

Module 2 — Intermediate: Real Spatial Analysis

I0 — Load the full project

I1 — Choosing a CRS for accurate distance and area

I2 — Invalid geometries and how to repair them

I3 — Missing values, sentinels and dirty categories

I4 — Buffer analysis

I5 — Overlay operations

I6 — Clipping: ``gpd.clip`` versus ``gpd.overlay``

I7 — Spatial joins in depth

I8 — Aggregating spatial statistics, and areal interpolation

I9 — Nearest-neighbour analysis

I10 — Dissolve and hierarchical aggregation

I11 — The spatial index, and why any of this is fast

I12 — Raster masking and clipping

I13 — Raster resampling and reprojection

I14 — Reclassification, band maths and terrain derivatives

I15 — Zonal statistics

I16 — Rasterize, polygonize, and designing an analytical map

Exercises — Module 2 (Intermediate)

Exercise 2.1 — A defensible CRS choice

Exercise 2.2 — Build a validity-and-cleaning report

Exercise 2.3 — Riparian buffer compliance

Exercise 2.4 — Land-use change accounting

Exercise 2.5 — Correct areal interpolation

Exercise 2.6 — Zonal statistics at two resolutions

Exercise 2.7 — Build the analysis-ready feature table

Challenge 2.8 — Reproduce the generating law

Module 3 — Advanced: Spatial Data Science

A0 — The analysis-ready feature table

A1 — Spatial feature engineering

A2 — Multi-criteria decision analysis

A3 — Environmental suitability modelling on rasters

A4 — Urban accessibility and spatial equity

- A5 — Spatial autocorrelation: Moran's I from scratch
- A6 — Hotspot analysis: Getis-Ord Gi* and LISA
- A7 — Point pattern analysis
- A8 — Spatial clustering and regionalisation
- A9 — Spatial regression
- A10 — Predictive modelling I: designing a flood-susceptibility model
- A11 — Predictive modelling II: spatial cross-validation
- A12 — Model interpretation and prediction surfaces
- A13 — Quantitative risk: hazard × exposure × vulnerability
- A14 — Communicating spatial results
 - The seven pitfalls, and what to do instead
 - What every serious spatial deliverable must state

Exercises — Module 3 (Advanced)

- Exercise 3.1 — A defensible siting recommendation
- Exercise 3.2 — Accessibility with a network-distance correction
- Exercise 3.3 — A hotspot analysis you can defend
- Exercise 3.4 — Regionalisation for service delivery
- Exercise 3.5 — A better flood model
- Exercise 3.6 — Risk under a climate scenario
- Challenge 3.7 — Recover the full generating process

Module 4 — Capstone Project

- The Vallmara Basin Integrated Climate-Resilience Assessment
 - The brief
 - What you must produce
 - Assessment rubric
 - How to use what follows
- Capstone Stage 1–4 — Ingest, clean, validate, resolve CRS
- Capstone Stage 5–8 — Vector analysis, raster analysis, integration, features
- Capstone Stage 9 — Statistical and spatial analysis
- Capstone Stage 10–12 — Visualisation, interpretation, conclusion

Solutions

- Solutions — Module 1 (Beginner)
 - 1.1 Layer inventory · 1.2 Attribute audit · 1.3 CRS forensics
 - 1.4 Dirty CSV · 1.5 Shapely reasoning · 1.6 Distance profile
 - 1.7 Spatial join and rates · 1.8 Raster sampling and regression
- Solutions — Module 2 (Intermediate)
 - 2.1 CRS choice · 2.2 QA report · 2.3 Riparian compliance
 - 2.4 Reservoir impact · 2.5 Areal interpolation · 2.6 Zonal at two resolutions
 - 2.7 The analysis-ready feature table · 2.8 Challenge: recover the rainfall law
- Solutions — Module 3 (Advanced)
 - 3.1 A defensible siting recommendation · 3.2 Network-distance accessibility

3.3 Defensible hotspots · 3.4 Regionalisation for service delivery

3.5 A better flood model · 3.6 Risk under a climate scenario · 3.7 Challenge

End of course

Where to go next

Spatial Data Science: The New Frontier in Analytics

A practical, project-based course in Python GIS — GeoPandas · Shapely · Rasterio

Who this course is for. You already know Python, statistics and data analysis. You are comfortable with `pandas`, you know what a train/test split is, and you can read a regression table. What you have *not* done much of is **spatial** analysis — and that turns out to be a genuinely different discipline, not just `pandas` with an `x` and a `y` column.

The one-sentence pitch for spatial data science. Ordinary data science assumes rows are exchangeable and independent; spatial data science starts from the opposite assumption — *Tobler's First Law*: "**everything is related to everything else, but near things are more related than distant things.**" Once observations carry a location, three things change at once:

	Ordinary tabular analysis	Spatial analysis
Rows	Independent samples	Auto-correlated; nearby rows leak information into each other
Joins	On a key (<code>id == id</code>)	On a <i>relation</i> (<code>intersects</code> , <code>within</code> , <code>nearest</code>)
Features	Given in the table	<i>Engineered from geometry</i> : distance to, density of, elevation at, area of
Validation	Random k-fold	Random k-fold overfits ; you need spatially blocked folds
Units	Whatever the column says	Depend on the coordinate reference system — degrees are not metres

Everything in this course exists to make those five rows second nature.

What you will build

You are the newly hired spatial data scientist for the regional government of the **Vallmara Basin**, a completely fictional 48 km × 36 km coastal region in the imaginary Republic of Kestria. Over 4 modules you will progress from "how do I open a shapefile" to delivering a **climate-resilience assessment**: a statistically defensible, map-backed estimate of which neighbourhoods are most exposed to flooding, least served by emergency infrastructure, and most vulnerable in socio-economic terms.

Course structure

Module	Level	Lessons	Focus
0	Setup	5	Environment, folder layout, generating the dataset
1	Beginner	14	GeoDataFrames, geometry, CRS, plotting, basic joins, opening rasters
2	Intermediate	16	Buffers, overlays, spatial joins in depth, raster masking / resampling / zonal statistics, data cleaning
3	Advanced	14	MCDA, accessibility, spatial autocorrelation, hotspots, clustering, spatial ML with proper validation
4	Capstone	12 stages	A complete end-to-end spatial data science project
+	Exercises	4 sets	Beginner → Challenge, with a full Solutions section at the end

Lesson format. Every lesson follows the same four-cell rhythm:

1. **Markdown** — what we are about to learn, why it matters, the concept, and the expected outcome.
2. **Code** — the runnable cell.
3. **Explanation** — line-by-line for anything unfamiliar.
4. **Expected output** — what you should see on screen, so you can tell success from silent failure.

How to use this document. Run every cell. When a lesson says "notice that the number is wrong", stop and make sure you understand *why* it is wrong before moving on. Roughly half of practical GIS competence is knowing which results to distrust.

0.1 — Environment and installation

Recommended Python version

Python 3.11 or 3.12. These have full binary-wheel coverage for the whole geospatial stack. Python 3.13 mostly works; 3.14 is still ahead of some wheels at the time of writing. If in doubt, use **3.12**.

The packages, and why each one is here

Package	Role in this course	Why you need it
geopandas ≥ 1.0	The spine of the course	A DataFrame with a geometry column: spatial joins, overlays, CRS handling, plotting
shapely ≥ 2.0	Geometry engine	The objects <i>inside</i> the geometry column; buffers, intersections, predicates. Shapely 2.x is vectorised — an order of magnitude faster than 1.8
rasterio ≥ 1.3	Raster I/O and processing	Reads/writes GeoTIFF, gives you NumPy arrays plus the affine transform that ties them to the ground
numpy	Raster maths	Rasters <i>are</i> NumPy arrays; every raster operation is array algebra
pandas	Attribute tables	GeoDataFrame is a subclass; all your groupby / merge skills transfer directly
matplotlib	All static maps	GeoPandas .plot() returns matplotlib axes; you compose maps as you would any figure
seaborn	Statistical graphics	Distribution and relationship plots for the non-spatial half of the analysis
scipy	Distances, KDE, filters	cKDTree for nearest-neighbour work, ndimage for raster filters/focal statistics, stats for tests
scikit-learn	Machine learning	Clustering (DBSCAN/KMeans) and the predictive models in Module 3
statsmodels	Regression <i>inference</i>	scikit-learn gives you predictions; statsmodels gives you standard errors, t-statistics, p-values and AIC — which is what Lesson A9 is about
pyarrow	GeoParquet I/O	The fastest way to store intermediate spatial results; optional but recommended
pyogrio	Fast vector I/O	GeoPandas 1.x uses it as the default engine; 5–20× faster than Fiona for large files
fiona	Alternative vector I/O	Still the fallback engine; useful when you need per-feature streaming
contextily	Web basemap tiles	Optional context under your maps (needs internet; the course works without it)
mapclassify	Choropleth classification	Quantiles, natural breaks (Fisher–Jenks), std-mean — needed for honest thematic maps

pip installation (virtual environment — recommended)

```
python3.12 -m venv gis-env
source gis-env/bin/activate          # Windows: gis-env\Scripts\activate
python -m pip install --upgrade pip
```

```
pip install "numpy>=1.26" "pandas>=2.1" "geopandas>=1.0" "shapely>=2.0" \
    "rasterio>=1.3" "pyogrio>=0.8" "fiona>=1.9" \
    "matplotlib>=3.8" "seaborn>=0.13" "scipy>=1.11" \
    "scikit-learn>=1.4" "statsmodels>=0.14" "pyarrow>=15.0" \
    "mapclassify>=2.6" "contextily>=1.5" "jupyterlab>=4.0"
```

Or, from the **requirements.txt** shipped with this course:

```
pip install -r requirements.txt
```

Conda / mamba installation (recommended if **pip** gives you GDAL trouble)

Conda ships pre-built GDAL/GEOS/PROJ binaries, which removes the single most common source of installation pain.

```
conda create -n gis-env -c conda-forge python=3.12 \
    geopandas shapely rasterio pyogrio fiona \
    numpy pandas matplotlib seaborn scipy scikit-learn statsmodels \
    pyarrow mapclassify contextily jupyterlab
conda activate gis-env
```

(Substitute `mamba` for `conda` if you have it — same commands, much faster solve.)

Launching Jupyter

```
cd path/to/this/course
jupyter lab          # modern interface, recommended
# or
jupyter notebook    # classic interface
```

Then open `Spatial_Data_Science_Course.ipynb`.

Recommended folder structure

```
spatial-data-science/
├── Spatial_Data_Science_Course.ipynb    <- this notebook
├── Spatial_Data_Science_Course.pdf      <- the printable study guide
├── generate_data.py                    <- builds the fictional dataset
├── requirements.txt
├── environment.yml
├── data/                               <- created by generate_data.py
│   ├── README_DATA.md
│   ├── vector/
│   │   ├── vallmara.gpkg              (12 layers, EPSG:32633)
│   │   ├── roads.geojson              (EPSG:4326)
│   │   ├── protected_areas.geojson    (EPSG:4326)
│   │   └── schools_shp/schools.shp     (EPSG:32633)
│   ├── raster/
│   │   ├── dem_25m.tif
│   │   ├── landcover_25m.tif
│   │   ├── rainfall_annual_250m.tif
│   │   ├── ndvi_50m.tif
│   │   ├── popdens_100m.tif
│   │   ├── multispectral_50m.tif
│   │   └── lst_summer_100m_3857.tif    (EPSG:3857 – on purpose)
│   ├── tabular/
│   │   ├── sensor_stations.csv
│   │   ├── sensor_readings.csv
│   │   ├── flood_incidents.csv
│   │   ├── district_socioeconomic.csv
│   │   └── landcover_legend.csv
│   └── outputs/                       <- everything you create goes here
```

Generating the dataset

From the course folder, with your environment active:

```
python generate_data.py
```

It takes 10–20 seconds and writes about 16 MB into `./data/`. It is driven by a fixed seed (42), so your files will be identical to the ones described here. Re-running it simply overwrites everything — safe at any time.

Troubleshooting the usual suspects

Symptom	Cause	Fix
<code>ImportError: libgdal.so... / DLL load failed</code>	pip wheels for <code>rasterio</code> and <code>geopandas</code> bundled <i>different</i> GDAL builds	Install both from the same channel: either all-pip or all-conda-forge. Never mix.
<code>CRSError: Invalid projection</code>	Stale or missing PROJ data directory	<code>pip install --force-reinstall pyproj</code> , or unset a stray <code>PROJ_LIB</code> environment variable
<code>.shp</code> opens but attributes are gibberish	Shapefile <code>.dbf</code> encoding	<code>gpd.read_file(path, encoding="latin-1")</code> , or move to GeoPackage
<code>fiona.errors.DriverError: ... not recognized</code>	Missing sidecar files	A shapefile is 4+ files (<code>.shp .shx .dbf .prj</code>). Copy the whole folder.
Reading a big file takes minutes	Fiona engine on a huge layer	<code>gpd.read_file(..., engine="pyogrio")</code> (default in GeoPandas 1.x)
<code>contextily</code> raises a connection error	No internet	Basemaps are optional throughout; skip those two lines
Everything plots in a tiny corner of the map	Layers in different CRS	Reproject all layers to one CRS before plotting (Lesson B5)
<code>TopologyException: found non-noded intersection</code>	Invalid input geometry	Run <code>make_valid()</code> first (Lesson I2)
Memory error on a raster	Reading a whole large raster at once	Read a window, or use <code>rasterio</code> block iteration (Lesson I13)

0.2 — Check your environment

What we are about to do. Import every library the course uses and print its version. **Why it matters:** 90% of "the code doesn't work" reports in GIS are environment problems, not logic problems. Catching a broken GDAL binding *now* costs you two minutes; catching it in Module 3 costs you an afternoon.

Concept — the GIS software stack. Python geospatial libraries are thin, pleasant wrappers over three C/C++ libraries that do the real work:

- **GEOS** — planar geometry predicates and operations (used by Shapely).
- **GDAL/OGR** — reading and writing 200+ raster and vector formats (used by Rasterio, Pyogrio, Fiona).
- **PROJ** — coordinate reference systems and datum transformations (used by pyproj, and hence by everything).

When a geospatial install breaks, it is almost always because two Python packages were compiled against *different versions* of one of those three.

Expected outcome. A version table, and a confirmation line that GEOS, GDAL and PROJ are all reachable.

What the next cell does: imports the stack, prints versions, and prints the underlying C-library versions so you can confirm a coherent install.

```
In [1]: import sys, platform

import numpy as np
import pandas as pd
import geopandas as gpd
import shapely
import rasterio
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
import scipy
import sklearn

print(f"Python {sys.version.split()[0]} ({platform.system()} {platform.machine()})")
print("-" * 58)
for name, mod in [("numpy", np), ("pandas", pd), ("geopandas", gpd),
                  ("shapely", shapely), ("rasterio", rasterio),
                  ("matplotlib", matplotlib), ("seaborn", sns),
                  ("scipy", scipy), ("scikit-learn", sklearn)]:
    print(f"{name:<14} {mod.__version__}")

print("-" * 58)
print(f"GEOS (via shapely) {shapely.geos_version_string}")
print(f"GDAL (via rasterio) {rasterio.__gdal_version__}")
import pyproj
print(f"PROJ (via pyproj) {pyproj.proj_version_str}")
print(f"vector I/O engine {gpd.options.io_engine or 'pyogrio (default)'}")
```

Explanation.

- `shapely.geos_version_string` — the GEOS build Shapely is linked against. Shapely **2.0+** is required for this course: it rewrote the geometry layer on top of NumPy arrays, which is why `gdf.geometry.buffer(500)` on 100 000 rows now takes milliseconds rather than seconds.
- `rasterio.__gdal_version__` — GDAL ≥ 3.4 is what you want. GDAL 3 changed the **axis-order convention**: `EPSG:4326` is officially (*latitude, longitude*). Rasterio and GeoPandas always hand you $(x, y) = (\text{longitude}, \text{latitude})$ regardless, but you will meet other tools that do not — this is a classic source of "my points are in the Indian Ocean" bugs.
- `pyproj.proj_version_str` — PROJ ≥ 8 supports datum-shift grids downloaded on demand, which matters for sub-metre accuracy work.
- `gpd.options.io_engine` — GeoPandas 1.x reads vector files through **pyogrio** by default (vectorised, fast). `engine="fiona"` is the streaming alternative.

Expected output. Nine version lines, then three C-library lines. You need at minimum: geopandas ≥ 1.0 , shapely ≥ 2.0 , rasterio ≥ 1.3 . If any import raises, fix it now using the troubleshooting table above.

0.3 — Generate the dataset

What we are about to do. Run the dataset generator and confirm every file landed where the course expects it.

Why it matters. Real GIS projects fail on *data logistics* far more often than on analysis. Establishing a single, explicit `DATA` root — and never hard-coding a path again — is the cheapest reproducibility win available.

Concept — why fictional data is the right teaching data. Real open GIS data carries licences, downloads, breaking API changes and hidden regional quirks. More importantly, with real data you never know the *true* generating process, so you cannot tell a correct analysis from a plausible-looking wrong one. Here the data-generating process is known and documented, so at every step you can ask: **did my analysis recover the truth?** That is the same logic as a simulation study in statistics.

Expected outcome. The generator prints a progress log and a summary of row counts; then a directory listing confirms the file tree.

What the next cell does: runs `generate_data.py` if `data/` is missing (so re-running the notebook is cheap), then lists every generated file with its size.

```
In [2]: import subprocess
        from pathlib import Path

        # --- Single source of truth for every path in this notebook -----
        ROOT = Path.cwd()                # the folder containing generate_data.py
        DATA = ROOT / "data"
        VEC = DATA / "vector"
        RAS = DATA / "raster"
        TAB = DATA / "tabular"
        OUT = DATA / "outputs"          # everything we create goes here

        GPKG = VEC / "vallmara.gpkg"     # the main multi-layer GeoPackage

        if not GPKG.exists():
            print("data/ not found - running generate_data.py ...\n")
            res = subprocess.run([sys.executable, str(ROOT / "generate_data.py")],
                                capture_output=True, text=True)
            print(res.stdout[-2000:] or res.stderr[-2000:])
        else:
            print("Dataset already present - skipping generation.\n")

        OUT.mkdir(parents=True, exist_ok=True)

        total = 0
        for p in sorted(DATA.rglob("*")):
            if p.is_file() and not p.name.startswith("."):
                size = p.stat().st_size
                total += size
                print(f"  {str(p.relative_to(ROOT)):<52} {size/1e6:8.2f} MB")
        print(f"\nTotal: {total/1e6:.1f} MB")
```

Explanation.

- `Path.cwd()` — the notebook's working directory. If you moved the notebook, set `ROOT` explicitly instead, e.g. `ROOT = Path("/Users/you/spatial-data-science")`.
- `sys.executable` rather than `"python"` — this guarantees the generator runs in **the same interpreter as the notebook**. Using `"python"` is a classic way to accidentally generate data with a different environment that lacks rasterio.
- `capture_output=True` keeps the generator's log out of the way unless you want it.
- `DATA.rglob("*")` — recursive glob; `p.is_file()` filters out directories.
- `OUT.mkdir(parents=True, exist_ok=True)` — the idempotent way to ensure an output folder exists. Never wrap this in an `if not exists` check; `exist_ok` already does it race-free.

Expected output. About **21 files, ~16 MB total**, laid out as:

Folder	Files
<code>data/raster/</code>	7 GeoTIFFs (<code>dem_25m.tif</code> is the largest, ~7 MB)
<code>data/vector/</code>	<code>vallmara.gpkg</code> (~2.8 MB), 2 GeoJSONs, a 5-file shapefile
<code>data/tabular/</code>	5 CSVs
<code>data/</code>	<code>README_DATA.md</code>

If the generator ran, you will additionally see its 14-step progress log ending with row counts (24 districts, 460 census blocks, 791 road segments, 5 200 buildings, 83 facilities, 181 flood-zone polygons, 393 flood incidents).

0.4 — The Vallmara Basin: scenario and data dictionary

The story

The **Vallmara Basin** occupies 1 395 km² of the west coast of the fictional Republic of Kestria. The Kestrian Sea lies to the west; the land climbs eastward across a flat coastal plain, through farmland and forest, to the **Corran Ridge** at roughly 900 m. Six rivers — the **Vallmara**, **Kestrel Brook**, **Fyr**, **Corran Water**, **Tarn Beck** and **Halvorn Rill** — drain the uplands and regularly spill onto the plain.

About **600 000 people** live in the basin, most of them in **Vallmara City** and two secondary centres. The regional government has three questions for you:

1. Which places flood, and what is at stake there?
2. Who cannot reach a hospital, a clinic or a fire station quickly enough?
3. Where should the next protected area / solar farm / clinic go?

The whole course is scaffolding for answering those three questions properly.

Coordinate reference systems in this dataset (read this twice)

CRS	Used by	Units	Purpose
EPSG:32633 — WGS 84 / UTM zone 33N	<code>vallmara.gpkg</code> , six of the seven rasters, <code>schools.shp</code>	metres	The analysis CRS . Distances, areas and buffers are only meaningful here.
EPSG:4326 — WGS 84 geographic	<code>roads.geojson</code> , <code>protected_areas.geojson</code> , the <code>lon</code> / <code>lat</code> columns in the CSVs	degrees	The exchange CRS . Degrees are an <i>angle</i> , not a length.
EPSG:3857 — Web Mercator	<code>lst_summer_100m_3857.tif</code>	metres (but lying)	The web-map CRS . Areas are inflated by up to $1/\cos^2(\text{latitude})$. Never compute statistics in it.

The mixture is **deliberate**. Real projects always arrive this way.

Vector layers — data/vector/vallmara.gpkg (EPSG:32633)

Layer	Geometry	Rows	Key attributes
districts	Polygon	24	district_id, name, district_type (urban_core / suburban / rural / upland_rural), area_km2, dist_core_km, mean_elev_m, coastal, population, households, pop_density_km2
census_blocks	Polygon	460	block_id, district_id, area_km2, population, households, pop_density_km2
landuse	Polygon	441	lu_id, class_code (1–8), landuse_class, area_ha
rivers	LineString	6	river_id, name, strahler_order, mean_discharge_m3s, perennial, length_km
flood_zones	Polygon	181	zone_id, hazard_class, return_period_yr (100 or 500), area_km2
buildings	Polygon	5 200	building_id, use_type, floors, footprint_m2, year_built, construction, has_basement, ground_elev_m, value_kvs
facilities	Point	83	facility_id, facility_type, capacity, staff, opening_year, is_24h
transit_routes	LineString	6	route_id, headway_min, daily_riders, length_km
bus_stops	Point	130	stop_id, route_id, shelter, boardings_daily
sea / land_boundary / coastline	Polygon / Polygon / LineString	1 each	study-area masks

Money is in VS, the fictional Vallmara Shilling; **value_kvs** = thousands of VS.

Raster layers — data/raster/

File	Resolution	CRS	dtype	NoData	Variable
dem_25m.tif	25 m	32633	float32	–9999	Elevation, metres above sea level. Sea = NoData
landcover_25m.tif	25 m	32633	uint8	0	Class 1–8 (see landcover_legend.csv)
rainfall_annual_250m.tif	250 m	32633	float32	–9999	Mean annual rainfall, mm
ndvi_50m.tif	50 m	32633	float32	–9999	NDVI, with two circular cloud gaps
popdens_100m.tif	100 m	32633	float32	–9999	Persons per km ²
multispectral_50m.tif	50 m	32633	uint16	0	4 bands: Blue, Green, Red, NIR (reflectance × 10 000)
lst_summer_100m_3857.tif	~134 m*	3857	float32	–9999	Summer land-surface temperature, °C

* Generated on a 100 m UTM grid, then reprojected to Web Mercator — which resamples onto a new grid, so the stored resolution is ~134 units. That is the Web-Mercator scale factor at 41.7° N, visible in the metadata.

Tabular data — `data/tabular/`

File	Rows	Contents
<code>sensor_stations.csv</code>	24	Station metadata plus <code>lon/lat</code> and <code>x_utm/y_utm</code>
<code>sensor_readings.csv</code>	866	Monthly 2022-01 → 2024-12: <code>pm25_ugm3</code> , <code>rainfall_mm</code> , <code>temp_c</code>
<code>flood_incidents.csv</code>	393	<code>date</code> , <code>lon</code> , <code>lat</code> , <code>depth_cm</code> , <code>damage_kvs</code> , <code>cause</code> , <code>injuries</code>
<code>district_socioeconomic.csv</code>	27	<code>median_income_vs</code> , <code>unemployment_rate</code> , <code>pct_over65</code> , <code>pct_tertiary_edu</code> , <code>hospital_beds_per_1000</code> , <code>vehicles_per_household</code>
<code>landcover_legend.csv</code>	8	class code → label

The deliberate data-quality problems

These are teaching material, not bugs. You will fix all of them in Module 2.

1. `landuse.landuse_class` — 45 rows with inconsistent case or stray whitespace.
2. `landuse` — 3 self-intersecting "bow-tie" polygons and 2 duplicate rows.
3. `census_blocks` — one row with an **empty** geometry.
4. `districts.population` — 2 missing values.
5. `buildings.year_built` — 240 NaN, 12 rows dated **1066**, 6 dated **2199**.
6. `buildings.value_kvs` — 95 NaN.
7. `facilities.capacity` — 11 rows using the sentinel **-999**; 3 duplicated points.
8. `roads.speed_limit_kmh` — NaN on every residential street.
9. `sensor_readings` — 45 NaN PM2.5, 45 rainfall values of **-999**, 2 duplicate rows.
10. `flood_incidents.csv` — 6 rows with **lon/lat swapped**, 4 at **(0, 0)**, 3 in another country, 25 NaN depths, 8 negative damage values.
11. `district_socioeconomic.csv` — 2 duplicated `district_id` keys, 1 orphan key **D99** matching no polygon, 2 NaN incomes.
12. Three different CRS across the files.

The ground truth baked into the simulation

Because you know the generating process, you can *grade your own analysis*:

- Rainfall = $470 + 0.62 \times \text{elevation} + \text{a south-north gradient}$ (+ noise).
- Land-surface temperature = $31.5 - 0.0062 \times \text{elevation} + 6.4 \times \text{urban intensity}$ — a textbook **urban heat island**.
- Urban intensity `u` is a smooth field peaking at the city core, and population density is $9\,500 \cdot u^{2.1} + 22$ persons/km², so `u` is recoverable from the density raster as $u = ((\text{density} - 22) / 9\,500)^{(1/2.1)}$.
- PM2.5 = $7.5 + 16 \cdot u + 9 \cdot \exp(-d_{\text{motorway}} / 1\,800\text{ m}) + \text{seasonal} + \text{noise}$. Because `u` and `d_motorway` are correlated, fitting the decay **without** `u` gives a wildly biased e-folding distance — a built-in lesson in confounding.
- Flood incidents were drawn **72% from the 100-year zone, 28% from the 500-year zone**; depth increases near rivers and decreases with elevation.
- Population density decays from the city core. **Districts are built by dissolving census blocks**, so each district polygon is exactly the union of its blocks and its population is exactly their sum — a built-in consistency check (the only exception being the two districts whose population was deliberately blanked).
- Building value decays exponentially with distance from the core.

- `multispectral_50m.tif` is built so that $(\text{NIR} - \text{RED})/(\text{NIR} + \text{RED})$ reproduces `ndvi_50m.tif` to within 0.003 — you can verify your own band maths.

0.5 — Global setup cell

What we are about to do. Establish the notebook-wide conventions: imports, plotting defaults, display options, the analysis CRS, and a couple of small helper functions used throughout.

Why it matters. In spatial work, a *single declared analysis CRS* is a discipline, not a convenience. Almost every wrong number in applied GIS comes from silently mixing coordinate systems. We define `CRS_UTM` once and reproject everything into it at load time.

Concept — the analysis CRS. Choose one projected CRS whose units are metres and whose distortion is small over your study area, do *all* measurement and modelling in it, and reproject only at the very end for display or delivery. For the Vallmara Basin that is **EPSG:32633 (UTM zone 33N)**: the region spans about 0.6° of longitude, comfortably inside the 6°-wide zone, where UTM scale error stays under 1 part in 2 500 (i.e. < 0.4 m per km).

Expected outcome. No visible output beyond a confirmation line — but every later cell depends on this one.

What the next cell does: imports everything, sets pandas/matplotlib defaults, defines `CRS_UTM` and two helpers (`fresh_ax` for consistently styled map axes, and `describe_gdf` for a quick spatial summary of any layer).

```
In [3]: # ----- imports -----
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import geopandas as gpd
import matplotlib.pyplot as plt
import matplotlib as mpl
import seaborn as sns
import rasterio
from rasterio.plot import show as rshow
import shapely
from shapely.geometry import Point, LineString, Polygon, MultiPolygon, box

# ----- conventions -----
CRS_UTM = "EPSG:32633" # WGS 84 / UTM 33N - metres - THE ANALYSIS CRS
CRS_WGS84 = "EPSG:4326" # WGS 84 geographic - degrees - storage/exchange
CRS_WEBMERC = "EPSG:3857" # Web Mercator - metres - web maps only

# ----- display -----
pd.set_option("display.max_columns", 40)
pd.set_option("display.width", 130)
pd.set_option("display.float_format", lambda v: f"{v:,.3f}")

sns.set_theme(style="whitegrid", context="notebook")
mpl.rcParams.update({
    "figure.dpi": 110,
    "savefig.dpi": 200,
    "figure.facecolor": "white",
    "axes.grid": False, # grids are noise on maps
    "font.size": 9,
})
warnings.filterwarnings("ignore", message=".*initial implementation of Parquet.*")

# ----- helpers -----
def fresh_ax(figsize=(9, 7), title=None):
    """A map-styled matplotlib axes: equal aspect, no ticks, optional title."""
    fig, ax = plt.subplots(figsize=figsize)
    ax.set_aspect("equal")
    ax.set_xticks([]); ax.set_yticks([])
```

```

for s in ax.spines.values():
    s.set_visible(False)
if title:
    ax.set_title(title, fontsize=12, weight="bold", loc="left")
return fig, ax

def describe_gdf(gdf, name="layer"):
    """One-line spatial summary of any GeoDataFrame - use it constantly."""
    xmin, ymin, xmax, ymax = gdf.total_bounds
    geoms = gdf.geom_type.value_counts().to_dict()
    print(f"{name:<16} rows={len(gdf):>6} cols={gdf.shape[1]:>3} "
          f"crs={str(gdf.crs.to_string() if gdf.crs else 'NONE'):<12} "
          f"geom={geoms}")
    print(f"{'':<16} bounds=({xmin:,.0f}, {ymin:,.0f}) -> ({xmax:,.0f}, {ymax:,.0f}) "
          f"empty={int(gdf.geometry.is_empty.sum())} "
          f"invalid={int(~gdf.geometry.is_valid.sum())} "
          f>null={int(gdf.geometry.isna().sum())}")

print("Setup complete. Analysis CRS =", CRS_UTM)

```

Explanation.

- **CRS_UTM as a named constant.** Every reprojection in the notebook refers to this name. If you later port the analysis to another region, you change one line.
- `pd.set_option("display.float_format", ...)` — thousands separators and three decimals. Spatial tables are full of coordinates in the millions and areas in the millionths; unformatted output is unreadable.
- `sns.set_theme(...)` then `axes.grid: False` — Seaborn's grid is excellent for statistical plots and terrible for maps, so we turn it off globally and let Seaborn's own plotting functions re-enable it per-figure.
- `fresh_ax()` — `set_aspect("equal")` is **not cosmetic**. In a projected CRS, one unit of x must be drawn the same size as one unit of y, otherwise the map is sheared and every visual judgement you make about shape and distance is wrong.
- `describe_gdf()` — prints the five things that actually matter about a layer: row count, CRS, geometry types, bounding box, and the count of empty/invalid/null geometries. Get in the habit of running it on every layer you load; it catches missing CRS and broken geometry before they poison an analysis.

Expected output. A single line: `Setup complete. Analysis CRS = EPSG:32633.`

Module 1 — Beginner: The Foundations

By the end of this module you will be able to open any vector or raster file, understand exactly what is inside it, know which coordinate system it is in and why that matters, make a competent map, and perform your first spatial join.

The 14 lessons

#	Lesson	Core skill
B1	Loading vector data	<code>gpd.read_file</code> , layers, formats
B2	The attribute table	dtypes, <code>describe</code> , <code>value_counts</code> , missingness
B3	The geometry column	Shapely objects, <code>geom_type</code> , coordinates, WKT
B4	CRS I — what a coordinate system is	geographic vs projected, <code>.crs</code>
B5	CRS II — reprojection and the degrees trap	<code>to_crs</code> , measured error
B6	Plotting spatial data	choropleths, layered maps, classification
B7	Points from a CSV	<code>points_from_xy</code> , <code>set_crs</code> vs <code>to_crs</code>
B8	Shapely operations	buffer, centroid, area, length, predicates
B9	Filtering and selecting features	attribute, bbox (<code>.cx</code>), spatial
B10	Calculating distances	point–point, point–line, matrices
B11	Basic spatial joins	<code>sjoin</code> , predicates, counting points in polygons
B12	Opening a raster	profile, transform, NoData, masked arrays
B13	Raster ↔ array ↔ ground	indexing, sampling at points, correct plotting
B14	Writing data out	GeoPackage, GeoJSON, GeoTIFF, CSV

B1 — Loading vector data

What we are going to learn. How to discover what is inside a spatial file and load it into Python.

Why it matters. Unlike a CSV, a spatial file can hold *several* datasets ("layers"), each with its own schema, geometry type and coordinate system. Opening one blindly and getting the wrong layer — or silently getting only the first of twelve — is a very common first mistake.

The concept — vector data. Vector data represents the world as discrete objects with exact coordinates:

- **Point** — a single (x, y): a school, a sensor, a flood report.
- **LineString** — an ordered sequence of points: a road segment, a river.
- **Polygon** — a closed ring (plus optional holes): a district, a lake.
- Each has a **Multi-** variant (`MultiPoint`, `MultiLineString`, `MultiPolygon`) for objects made of several disjoint pieces — an archipelago, a divided estate.

Every feature pairs **one geometry** with **one row of attributes**. That pairing is precisely what a `GeoDataFrame` is: a `pandas.DataFrame` where one column holds Shapely geometry objects and the frame remembers a CRS.

Formats you will meet.

Format	Extension	Verdict
GeoPackage	<code>.gpkg</code>	Best default. One SQLite file, many layers, no field-name limits, fast
GeoJSON	<code>.geojson</code>	Great for web/exchange, always EPSG:4326 by spec, verbose and slow for big data
Shapefile	<code>.shp</code> + <code>.shx</code> + <code>.dbf</code> + <code>.prj</code> (+ more)	Legacy. 10-character field names, 2 GB limit, no NULLs, multi-file. Still everywhere
FlatGeobuf / Parquet	<code>.fgb</code> / <code>.parquet</code>	Modern, very fast; <code>geopandas.read_parquet</code> is excellent for intermediate results

Expected outcome. A list of the 12 layers in the GeoPackage, and three loaded layers whose shapes and CRS we print.

What the next cell does: lists the layers in `vallmara.gpkg` with `pyogrio`, then loads three of them and summarises each with our `describe_gdf` helper.

```
In [4]: import pyogrio

# --- 1. What is inside the GeoPackage? -----
layer_info = pyogrio.list_layers(GPKG)          # -> array of [name, geom_type]
print(f"{GPKG.name} contains {len(layer_info)} layers:\n")
for name, geom_type in layer_info:
    info = pyogrio.read_info(GPKG, layer=name)
    print(f"  {name:<16} {str(geom_type):<12} "
          f"{info['features']:>6} features  {len(info['fields'])} fields")

# --- 2. Load three layers -----
districts = gpd.read_file(GPKG, layer="districts")
rivers     = gpd.read_file(GPKG, layer="rivers")
facilities = gpd.read_file(GPKG, layer="facilities")

print("\n" + "=" * 78)
describe_gdf(districts, "districts")
describe_gdf(rivers, "rivers")
describe_gdf(facilities, "facilities")

# --- 3. A GeoDataFrame IS a DataFrame -----
print("\nType hierarchy:")
print("  districts          ", type(districts).__mro__[:3])
print("  districts.geometry ", type(districts.geometry))
print("  one geometry       ", type(districts.geometry.iloc[0]))
```

Explanation.

- `pyogrio.list_layers(path)` returns an $N \times 2$ array of `[layer_name, geometry_type]`. Always run this before `read_file` on an unfamiliar `.gpkg` — it costs milliseconds and prevents loading the wrong thing.
- `pyogrio.read_info(path, layer=...)` reads only the **header**: feature count, field names and types, CRS, bounds. It never touches the geometries, so it is instant even on a 10 GB file. This is how you inspect data too big to open.
- `gpd.read_file(GPKG, layer="districts")` — the `layer=` argument is mandatory for multi-layer sources. Omit it and you silently get layer 0.
- `type(districts).__mro__[:3]` — the *method resolution order* shows `GeoDataFrame` → `DataFrame` → `NDFrame`. This is the single most useful fact in the course: **every pandas method you know works unchanged**. `groupby`, `merge`, `query`, `pivot_table`, `.loc` — all of it.
- `districts.geometry.iloc[0]` is a `shapely.geometry.Polygon`. The geometry column is a `GeoSeries`; the individual values are Shapely objects.

Expected output.

A 12-row layer inventory:

districts	Polygon	24 features	10 fields
census_blocks	Polygon	460 features	6 fields
landuse	Polygon	441 features	4 fields
rivers	LineString	6 features	6 fields
flood_zones	Polygon	181 features	4 fields
buildings	Polygon	5200 features	9 fields
facilities	Point	83 features	7 fields
transit_routes	LineString	6 features	6 fields
bus_stops	Point	130 features	4 fields
sea / land_boundary / coastline	1 feature each		

Then three `describe_gdf` blocks. Check three things every time:

1. `crs=EPSG:32633` — all GeoPackage layers share the analysis CRS.
2. `bounds=(403,983, 4,600,000) -> (448,000, 4,636,000)` — coordinates in the hundreds of thousands and millions are the signature of a *projected* CRS in metres. (The western edge is 403 983 rather than 400 000 because the districts stop at the coastline, not at the raster's bounding box.) If you ever see bounds like `(-180, -90) -> (180, 90)`, you are in degrees.
3. `empty=0 invalid=0 null=0` for these three layers — they are clean. (`landuse` and `census_blocks` are not; we will meet them in Module 2.)

B2 — Exploring the attribute table

What we are going to learn. How to interrogate the non-spatial half of a spatial dataset.

Why it matters. A GIS analysis is only as good as its attributes. Before any spatial operation, you should know the dtypes, the categorical levels, the missingness pattern and the plausible ranges — exactly as you would for any tabular dataset. Spatial analysts who skip this step produce beautiful maps of nonsense.

The concept — attributes are just a DataFrame. `gdf.drop(columns="geometry")` gives you a plain `DataFrame`. Everything you know applies. The *only* new habit is to keep asking "does this attribute make sense **given where the feature is?**" — e.g. a district with 200 000 residents and `district_type == "upland_rural"` would be a red flag.

Expected outcome. dtypes, summary statistics, category counts and a missingness report for the `districts` and `buildings` layers, plus the first evidence of the deliberate data-quality problems.

What the next cell does: prints the schema and head of `districts`, summarises its numeric and categorical columns, then loads `buildings` and runs a missingness + implausible-value audit on it.

```
In [5]: # --- 1. Schema -----
print("DISTRICTS dtypes")
print(districts.dtypes.to_string(), "\n")
print(districts.head(4).drop(columns="geometry").to_string(), "\n")

# --- 2. Numeric summary -----
num_cols = ["area_km2", "mean_elev_m", "dist_core_km", "population", "pop_density_km2"]
print("Numeric summary")
print(districts[num_cols].describe().T.to_string(), "\n")

# --- 3. Categorical summary -----
print("district_type value counts")
print(districts["district_type"].value_counts().to_string())
print("\ncoastal:", district["coastal"].sum(), "of", len(districts), "districts\n")

# --- 4. Missingness -----
miss = district.isna().sum()
print("Missing values per column")
print(miss[miss > 0].to_string() if miss.any() else " (none)")

# --- 5. A messier layer -----
buildings = gpd.read_file(GPKG, layer="buildings")
print("\n" + "=" * 78)
print(f"BUILDINGS: {len(buildings):,} rows")
audit = pd.DataFrame({
    "dtype": buildings.dtypes.astype(str),
    "n_missing": buildings.isna().sum(),
    "pct_missing": (100 * buildings.isna().mean()).round(2),
    "n_unique": buildings.nunique(),
})
print(audit.to_string(), "\n")

print("year_built - implausible values:")
yb = buildings["year_built"]
print(f" min={yb.min():.0f} max={yb.max():.0f} NaN={yb.isna().sum()}")
print(f" before 1800: {(yb < 1800).sum()} rows after 2025: {(yb > 2025).sum()} rows")
print("\nuse_type:", buildings["use_type"].value_counts().to_dict())
```

Explanation.

- `districts.dtypes` — note that `geometry` has dtype `geometry`. That is a real pandas ExtensionDtype provided by GeoPandas, not `object`. It is what lets GeoPandas vectorise geometric operations through Shapely 2.
- `.describe().T` — transposing puts variables in rows, which is far easier to read when you have many columns.
- `districts[num_cols].describe()` shows `population` with **count = 22**, not 24 — the two deliberately missing values. `describe()` silently drops NaN; this is exactly how missing data slips into an analysis unnoticed. **Always compare the count row against `len(gdf)`.**
- The `audit` DataFrame is a pattern worth memorising: dtype, count and percent missing, and cardinality, in one table. `nunique()` instantly reveals ID columns (unique = n) and low-cardinality categoricals.
- `(yb < 1800).sum()` — a **range check**. Missingness is easy to spot; *wrong* values that are syntactically valid are not. 12 buildings dated 1066 and 6 dated 2199 will happily flow into a "building age" feature and silently destroy a model unless you look.

Expected output.

- `districts` dtypes ending in `geometry geometry`.
- Numeric summary where **population** and **pop_density_km2** have **count 22.000** while every other column has 24 — the two deliberately blanked districts.
- Population ranges from **743** (Ostrand, upland) to **197 870** (Harbourgate, urban core); density from **26.9** to **3 545** people/km². That three-order-of- magnitude spread is why the choropleth in B6 needs a thoughtful classification.
- `district_type`: **12 upland_rural**, **8 suburban**, **2 urban_core**, **2 rural**.

- `coastal`: 5 of 24 districts.
- Missing values: `population` 2, `pop_density_km2` 2.
- For `buildings`: 5 200 rows, `year_built` 240 missing (4.62%), `value_kvs` 95 missing (1.83%), `min=1066`, `max=2199`, 12 rows before 1800 and 6 after 2025.
- `use_type` = {`residential`: 3 850, `commercial`: 830, `industrial`: 330, `public`: 190}.

B3 — The geometry column: points, lines and polygons

What we are going to learn. What actually lives in the geometry column, and how to interrogate a single geometry.

Why it matters. Every spatial operation you will ever run is a method on these objects. Understanding their structure — rings, coordinates, validity — is what separates "I ran `intersection` and got an empty result" from "I know why that returned empty."

The concept — the Simple Features model. GeoPandas geometries follow the OGC **Simple Features** standard, implemented by Shapely on top of GEOS:

```

Geometry
├── Point                (x, y)
├── LineString           [(x,y), (x,y), ...]          - ordered, may self-cross
├── LinearRing           a closed, simple LineString - used for polygon rings
├── Polygon              exterior ring + list of interior rings (holes)
└── GeometryCollection
    ├── MultiPoint
    ├── MultiLineString
    └── MultiPolygon

```

Key properties, all lazily computed by GEOS:

- `.area`, `.length` — **in the units of the CRS**. In EPSG:4326 that means "square degrees", which is meaningless.
- `.bounds` — (`minx`, `miny`, `maxx`, `maxy`).
- `.is_valid` — a polygon is valid if its rings do not self-intersect and holes lie inside the exterior. Invalid polygons make overlay operations throw.
- `.is_simple` — a line is simple if it does not cross itself.
- `.wkt` / `.wkb` — the text and binary serialisations. WKT is how you eyeball a geometry; WKB is how databases store it.

Expected outcome. A dissection of one polygon, one line and one point, plus vectorised geometry properties for a whole layer.

What the next cell does: takes a single district polygon and prints its type, ring structure, coordinate count, area, perimeter and truncated WKT; does the same for a river `LineString` and a facility `Point`; then shows the *vectorised* form of the same operations across an entire `GeoSeries`.

```

In [6]: # --- 1. Dissect one POLYGON -----
poly = districts.geometry.iloc[0]
print("POLYGON -", districts.loc[0, "name"])
print(f"  geom_type      : {poly.geom_type}")
print(f"  exterior ring    : {len(poly.exterior.coords)} vertices")
print(f"  interior rings     : {len(poly.interiors)} (holes)")
print(f"  area              : {poly.area:,.0f} m^2 = {poly.area/1e6:,.2f} km^2")
print(f"  perimeter         : {poly.length:,.0f} m")
print(f"  bounds            : {tuple(round(b) for b in poly.bounds)}")
print(f"  centroid          : ({poly.centroid.x:,.0f}, {poly.centroid.y:,.0f})")
print(f"  is_valid           : {poly.is_valid}")
print(f"  first 3 coords    : {[tuple(round(c) for c in xy) for xy in list(poly.exterior.coords)[:3])}")
print(f"  WKT (truncated): {poly.wkt[:90]} ...\n")

# --- 2. Dissect one LINESTRING -----
line = rivers.geometry.iloc[0]
print("LINESTRING -", rivers.loc[0, "name"])
print(f"  geom_type      : {line.geom_type}")
print(f"  vertices       : {len(line.coords)}")
print(f"  length         : {line.length:,.0f} m = {line.length/1000:,.2f} km")
print(f"  is_simple      : {line.is_simple} (does it avoid crossing itself?)")
print(f"  start -> end   : {tuple(round(c) for c in line.coords[0])} -> "
      f"{tuple(round(c) for c in line.coords[-1])}")
print(f"  midpoint       : {tuple(round(c) for c in line.interpolate(0.5,
normalized=True).coords[0])}\n")

# --- 3. Dissect one POINT -----
pt = facilities.geometry.iloc[0]
print("POINT -", facilities.loc[0, "name"])
print(f"  geom_type      : {pt.geom_type}   coords: ({pt.x:,.1f}, {pt.y:,.1f})")
print(f"  area           : {pt.area}        length: {pt.length}   (points have neither)\n")

# --- 4. The VECTORISED form - this is how you actually work -----
print("Vectorised geometry properties over the whole layer:")
geom_summary = pd.DataFrame({
    "name": districts["name"],
    "geom_type": districts.geometry.geom_type,
    "n_vertices": districts.geometry.count_coordinates(),
    "area_km2": districts.geometry.area / 1e6,
    "perim_km": districts.geometry.length / 1000,
    "is_valid": districts.geometry.is_valid,
})
print(geom_summary.head(6).to_string(index=False))
print(f"\nTotal land area of all districts: {districts.geometry.area.sum()/1e6:,.1f} km^2")

```

Explanation.

- `poly.exterior.coords` — the outer ring as a coordinate sequence. It is **closed**: the last vertex equals the first, so a triangle reports 4 vertices.
- `poly.interiors` — the list of holes. Vallmara districts have none, but a district containing a lake excluded from its area would.
- `poly.area` returns **square metres** because the CRS is UTM (metres). Shapely itself knows nothing about CRS; it does flat Cartesian arithmetic on whatever numbers you give it. **The units come entirely from the CRS you chose.** This is the deepest trap in the whole field.
- `poly.centroid` — the area-weighted centre of mass. Note it can fall *outside* a concave polygon (a horseshoe-shaped district). When you need a point guaranteed to be inside, use `.representative_point()` instead.
- `line.interpolate(0.5, normalized=True)` — the point halfway along the line by distance. With `normalized=False` the argument is in CRS units (metres here). This is the workhorse for placing labels, sampling along transects, and generating stops along a route.

- `.count_coordinates()` — a GeoSeries method (Shapely 2 / GeoPandas 1) giving vertex counts without a Python loop. Vertex count is your first proxy for geometric complexity and hence for how slow an overlay will be.
- Note the pattern: **singular Shapely properties** (`poly.area`) versus **vectorised GeoSeries properties** (`districts.geometry.area`). The second returns a `pandas.Series` and is what you use in real work.

Expected output.

```
POLYGON -- Old Vallmara
geom_type      : Polygon
exterior ring   : ~80 vertices
interior rings  : 0 (holes)
area           : 53,322,xxx m^2 = 53.32 km^2
perimeter      : ~40,000 m
centroid       : (~417,000, ~4,619,000)
is_valid       : True
```

Vertex counts of 35–90 are the signature of districts that were built by **dissolving census blocks** — their boundaries follow block edges, so they are irregular and non-convex, exactly like real administrative units.

The river prints a length of **43.93 km** with `is_simple: True` and 34 vertices. The point prints `area 0.0 length 0.0` — points are zero-dimensional.

The final table shows six districts and **total land area = 1 394.8 km²**, which matches the study-area figure in the scenario description. That agreement is your first sanity check.

B4 — CRS I: what a coordinate reference system actually is

What we are going to learn. The anatomy of a CRS, how to read one from a layer, and the difference between geographic and projected systems.

Why it matters. This is the concept that most often silently breaks GIS analyses done by strong data scientists. Distances, areas, buffers, nearest neighbours, densities, cluster radii — **all of them are wrong** if the CRS is wrong, and none of them will raise an error. You get plausible numbers that are simply not true.

The concept. A CRS answers: *what do the numbers in the geometry column mean?* It has three parts:

1. **Datum** — a model of the Earth's shape and its position: an ellipsoid (e.g. WGS 84: semi-major axis 6 378 137 m, flattening 1/298.257223563) plus a realisation tying it to physical reference points. Two datums can put "the same" latitude/longitude hundreds of metres apart.
2. **Coordinate system** — the axes and their units: (longitude, latitude) in degrees, or (easting, northing) in metres.
3. **Projection** (only for projected CRS) — the mathematical function flattening the curved surface onto a plane.

Geographic vs projected.

	Geographic (e.g. EPSG:4326)	Projected (e.g. EPSG:32633)
Coordinates	longitude, latitude in degrees	easting, northing in metres
Is 1 unit a fixed length?	No. 1° latitude $\approx$ 111 km always; 1° longitude $\approx$ 111 km $\times$ cos(lat) — 111 km at the equator, 0 km at the pole	Yes , to within the projection's scale error
Valid for <code>.area</code> , <code>.length</code> , <code>.buffer</code> , <code>.distance</code> ?	No	Yes
Valid for storage and exchange?	Yes — it is the universal interchange	Yes, if you record which one

The fundamental theorem of map projections. No flat map can preserve area, angle and distance simultaneously.

Every projection sacrifices at least one:

- **UTM / Transverse Mercator** — *conformal* (preserves local angles/shape). Divides the world into 60 zones 6° wide; distortion inside a zone is tiny (scale error < 1/2 500). **This is what you want for regional analysis.**
- **Web Mercator (EPSG:3857)** — conformal, spans the globe, but inflates area by $1/\cos^2(\text{latitude})$: Greenland looks the size of Africa. Fine for tiles, catastrophic for statistics.
- **Equal-area projections** (Albers, Lambert Azimuthal, Mollweide) — preserve area, distort shape. Use for continental/global area or density work.

The EPSG code. Every well-known CRS has an integer identifier from the EPSG registry. [EPSG:32633](#) decodes as WGS 84 / UTM zone **33 North**. The zone number is derived from longitude: `zone = floor((lon + 180) / 6) + 1`.

Expected outcome. Full CRS metadata for our layers, and a demonstration that the same file can hold radically different-looking numbers for the same place.

What the next cell does: prints the structured CRS metadata for the UTM districts layer and the WGS 84 roads layer, computes the correct UTM zone from the study area's longitude, and shows the same district centroid expressed in three CRS.

```
In [7]: # --- 1. Interrogate a projected CRS -----
crs = districts.crs
print("=" * 78)
print("DISTRICTS CRS")
print("=" * 78)
print(f" name      : {crs.name}")
print(f" EPSG code   : {crs.to_epsg()}")
print(f" is_projected : {crs.is_projected}")
print(f" is_geographic : {crs.is_geographic}")
print(f" unit        : {crs.axis_info[0].unit_name}")
print(f" axes        : {[f'{a.name} ({a.abbrev}, {a.direction})' for a in crs.axis_info]}")
print(f" datum       : {crs.datum.name}")
print(f" ellipsoid    : {crs.ellipsoid.name} "
      f"      (a = {crs.ellipsoid.semi_major_metre:,.0f} m)")
print(f" area of use  : {crs.area_of_use.name}")
print(f" bounds of use : {tuple(round(v, 2) for v in crs.area_of_use.bounds)}")

# --- 2. Interrogate a geographic CRS -----
roads_raw = gpd.read_file(VEC / "roads.geojson")
print("\n" + "=" * 78)
print("ROADS CRS (a different file, a different CRS - on purpose)")
print("=" * 78)
print(f" name      : {roads_raw.crs.name}")
print(f" EPSG code   : {roads_raw.crs.to_epsg()}")
print(f" is_projected : {roads_raw.crs.is_projected}")
print(f" unit        : {roads_raw.crs.axis_info[0].unit_name}")
print(f" bounds      : {tuple(round(v, 4) for v in roads_raw.total_bounds)}")

# --- 3. Which UTM zone SHOULD this region be in? -----
centre_lonlat = districts.to_crs(CRS_WGS84).geometry.union_all().centroid
lon, lat = centre_lonlat.x, centre_lonlat.y
zone = int((lon + 180) // 6) + 1
epsg = (32600 if lat >= 0 else 32700) + zone
print(f"\nStudy-area centre : lon {lon:.4f}, lat {lat:.4f}")
print(f"Computed UTM zone   : {zone}{'N' if lat >= 0 else 'S'} -> EPSG:{epsg}")
print(f"Layer actually uses: EPSG:{districts.crs.to_epsg()} "
      f"      {'MATCH' if epsg == districts.crs.to_epsg() else 'MISMATCH!'}")

# --- 4. The same place, three coordinate systems -----
p_utm = districts.geometry.iloc[1].centroid
rows = []
for label, target in [("EPSG:32633 (UTM 33N, m)", CRS_UTM),
                     ("EPSG:4326 (WGS84, deg)", CRS_WGS84),
                     ("EPSG:3857 (WebMerc, m)", CRS_WEBMERC)]:
```

```

g = gpd.GeoSeries([p_utm], crs=CRS_UTM).to_crs(target).iloc[0]
rows.append({"CRS": label, "x / lon": round(g.x, 5), "y / lat": round(g.y, 5)})
print("\nThe centroid of", districts.loc[1, "name"], "expressed three ways:")
print(pd.DataFrame(rows).to_string(index=False))

```

Explanation.

- `gdf.crs` is a **pyproj.CRS** object, not a string. It exposes the full WKT2 definition programmatically — `datum`, `ellipsoid`, `axis_info`, `area_of_use`. Print it directly (`print(crs)`) to see the raw WKT.
- `crs.axis_info[0].unit_name` — the authoritative answer to "what are my units?". Never assume; ask. For EPSG:32633 it is **metre**; for EPSG:4326 it is **degree**.
- `crs.area_of_use.bounds` — the longitude/latitude box within which the CRS is *defined to be accurate*. For UTM 33N that is roughly **(12, 0, 18, 84)**. Using a UTM zone far outside its band inflates distortion quickly; using it 30° away is simply wrong.
- `crs.to_epsg()` can return **None** for a custom CRS that is not in the registry. Code defensively when you did not create the file.
- **The zone formula.** `zone = floor((lon + 180) / 6) + 1`, then EPSG code **32600 + zone** for the northern hemisphere and **32700 + zone** for the southern. Memorise this: it is how you pick a correct analysis CRS anywhere on Earth in five seconds.
- `districts.to_crs(CRS_WGS84).geometry.union_all().centroid` — dissolve all polygons into one and take its centroid. `union_all()` is the GeoPandas 1.x name; older code says **unary_union** (still works, deprecated).

Expected output.

```

DISTRICTS CRS
name          : WGS 84 / UTM zone 33N
EPSG code     : 32633
is_projected  : True
unit          : metre
axes          : ['Easting (E, east)', 'Northing (N, north)']
datum         : World Geodetic System 1984
ellipsoid     : WGS 84 (a = 6,378,137 m)

```

Roads report **EPSG:4326**, **is_projected: False**, **unit: degree**, and bounds like **(13.68, 41.53, 14.27, 41.86)** — **three-digit numbers instead of six-digit ones. That contrast is the fastest way to tell degrees from metres at a glance.**

The zone computation prints **lon ≈ 13.95, lat ≈ 41.70 → zone 33N → EPSG:32633 MATCH**, confirming the dataset's analysis CRS is the right choice.

The three-way table shows the same point as roughly:

CRS	x / lon	y / lat
UTM 33N (m)	413 000	4 621 000
WGS 84 (deg)	13.85	41.73
Web Mercator (m)	1 541 000	5 122 000

Note that the Web Mercator *northing* (5 122 km) is much larger than the UTM northing (4 621 km) for the same physical location. Web Mercator stretches the y-axis increasingly towards the poles — the visible symptom of its area distortion.

B5 — CRS II: reprojection, and measuring the cost of getting it wrong

What we are going to learn. How to reproject with `to_crs`, the crucial difference between `set_crs` and `to_crs`, and — by direct measurement — how badly wrong your numbers are if you compute in the wrong CRS.

Why it matters. This lesson contains the single most important number in the course. We will compute the area of the same region in three CRS and compare.

The concept.

- `set_crs(crs)` — *labels* the data. It changes the metadata and **not one coordinate**. Use it only when the file arrived with no CRS and you know from documentation what it is. Using it wrongly silently corrupts everything downstream.
- `to_crs(crs)` — *transforms* the data. Every coordinate is pushed through a PROJ pipeline (inverse projection → datum shift → forward projection). The metadata changes **and so do all the numbers**.

Mnemonic: `set_crs` changes the label on the tin; `to_crs` changes what is in the tin.

The workflow rule. Load → immediately reproject everything to your analysis CRS → do all measurement → reproject only for output. Layers in different CRS cannot be joined, overlaid or plotted together; GeoPandas will either raise or (worse, in older versions) give nonsense.

Expected outcome. A table quantifying the area error from working in EPSG:4326 and EPSG:3857 rather than UTM, and a demonstration of a 500 m buffer built in degrees versus metres.

What the next cell does: reprojects the district layer into three CRS, computes total area in each, expresses the error in percent; then shows what a "0.005 unit" buffer means in each system; and finally reprojects the roads layer into the analysis CRS for the rest of the course.

```
In [8]: # --- 1. Area of the SAME polygons, computed in three CRS -----
truth_km2 = districts.to_crs(CRS_UTM).geometry.area.sum() / 1e6

rows = []
for label, crs_code, unit in [
    ("EPSG:32633 UTM 33N", CRS_UTM, "m^2"),
    ("EPSG:4326 WGS 84 geographic", CRS_WGS84, "deg^2"),
    ("EPSG:3857 Web Mercator", CRS_WEBMERC, "m^2"),
    ("ESRI:54009 Mollweide (equal area)", "ESRI:54009", "m^2")]:
    g = districts.to_crs(crs_code)
    raw = g.geometry.area.sum()
    km2 = raw / 1e6 if unit == "m^2" else np.nan
    rows.append({
        "CRS": label,
        "raw .area sum": f"{raw:,.6g}",
        "units": unit,
        "implied km^2": f"{km2:,.1f}" if np.isfinite(km2) else "meaningless",
        "error vs UTM": (f"{100*(km2-truth_km2)/truth_km2:+.1f} %"
                        if np.isfinite(km2) else "-"),
    })
print("TOTAL AREA OF THE 24 DISTRICTS, computed in four coordinate systems")
print("=" * 92)
print(pd.DataFrame(rows).to_string(index=False))
print("=" * 92)
print(f"\nGround truth (UTM): {truth_km2:,.1f} km^2")

# --- 2. GeoPandas can also do it properly on the ellipsoid -----
geod_area = districts.to_crs(CRS_WGS84).geometry.to_crs(CRS_UTM).area.sum() / 1e6
print(f"Round-trip 32633 -> 4326 -> 32633: {geod_area:,.1f} km^2 "
      f"(loss from the round trip: {abs(geod_area-truth_km2):.4f} km^2)")

# --- 3. The "buffer in degrees" hack, measured honestly -----
pt_utm = gpd.GeoSeries([Point(418_000, 4_618_000)], crs=CRS_UTM)

# (a) the correct way: 500 metres in a metric CRS
```

```

buf_m = pt_utm.buffer(500)
b = buf_m.total_bounds
print("\nA 500 m buffer around Vallmara City centre")
print(f"  buffer(500) in EPSG:32633 -> width {b[2]-b[0]:.0f} m, "
      f"height {b[3]-b[1]:.0f} m, area {buf_m.area.iloc[0]/1e6:.4f} km^2")

# (b) the common hack: "0.0045 degrees is about 500 m"
buf_deg = pt_utm.to_crs(CRS_WGS84).buffer(0.0045).to_crs(CRS_UTM)
b = buf_deg.total_bounds
print(f"  buffer(0.0045) in EPSG:4326 -> width {b[2]-b[0]:.0f} m, "
      f"height {b[3]-b[1]:.0f} m, area {buf_deg.area.iloc[0]/1e6:.4f} km^2")
print(f"  the 'circle' is really an ELLIPSE: "
      f"east-west radius {(b[2]-b[0])/2:.0f} m vs north-south radius {(b[3]-b[1])/2:.0f} m")
print(f"  1 degree of latitude = 111,320 m everywhere")
print(f"  1 degree of longitude = 111,320 * cos(41.7 deg) = "
      f"{111_320*np.cos(np.radians(41.7)):.0f} m HERE, and 0 m at the pole")

# --- 4. Put the roads layer into the analysis CRS -----
print("\nBefore:", roads_raw.crs, "| bounds", np.round(roads_raw.total_bounds, 3))
roads = roads_raw.to_crs(CRS_UTM)
print("After :", roads.crs, "| bounds", np.round(roads.total_bounds, 0))
print(f"\nA motorway segment measured in degrees: "
      f"{roads_raw.loc[roads_raw.road_class=='motorway'].geometry.length.iloc[0]:.6f} (degrees
- useless)")
print(f"The same segment measured in metres : "
      f"{roads.loc[roads.road_class=='motorway'].geometry.length.iloc[0]:.0f} m")

```

Explanation.

- The whole point of the first block is that **.area never complains**. In EPSG:4326 it returns a number in *square degrees* — a quantity with no physical meaning, because a degree of longitude shrinks towards the poles. GeoPandas emits a warning in recent versions, but the number is still returned and will still flow into your report.
- Web Mercator is the dangerous one**, because its units are metres, so nothing looks wrong. At the latitude of the Vallmara Basin ($\approx 41.7^\circ$ N) areas are inflated by $1/\cos^2(41.7^\circ) \approx 1.80$, i.e. **about +80%**. A "density per km²" computed in EPSG:3857 is off by nearly a factor of two — and every web map you have ever taken a screenshot of is in EPSG:3857.
- Mollweide (ESRI:54009)** is an equal-area projection; its total agrees with UTM to a fraction of a percent, confirming the UTM figure is right. When two independent projections agree, you can trust the number.
- The round-trip test (32633 $\rightarrow$ 4326 $\rightarrow$ 32633) loses only a tiny fraction of a km²: PROJ transformations are essentially lossless within a datum. The danger is *computing* in the wrong CRS, not *converting* between them.
- buffer(500)** in EPSG:4326 means "500 **degrees**", which wraps the entire planet many times over — the resulting "radius" is astronomically large. In practice you will more often see someone write **buffer(0.005)** intending "about 500 m", which is a rough approximation at one particular latitude and wrong everywhere else. **Never buffer in a geographic CRS.**
- The final block establishes **roads** (UTM) as the canonical roads layer for the rest of the notebook. **roads_raw** stays around only as a cautionary example.

Expected output.

TOTAL AREA OF THE 24 DISTRICTS, computed in four coordinate systems						
CRS		raw .area sum	units	implied km ²	error vs UTM	
EPSG:32633	UTM 33N	1.39478e+09	m ²	1,394.8	+0.0 %	
EPSG:4326	WGS 84 geographic	0.15101	deg ²	meaningless	–	
EPSG:3857	Web Mercator	2.50704e+09	m ²	2,507.0	+79.7 %	
ESRI:54009	Mollweide (equal area)	1.3968e+09	m ²	1,396.8	+0.1 %	

Read that Web Mercator row again: **+79.7%**. Then:

```
A 500 m buffer around Vallmara City centre
buffer(500)    in EPSG:32633 -> width 1,000 m, height 1,000 m, area 0.7841 km^2
buffer(0.0045) in EPSG:4326 -> width 749 m, height 999 m, area 0.5867 km^2
the 'circle' is really an ELLIPSE: east-west radius 374 m vs north-south radius 500 m
1 degree of longitude = 111,320 * cos(41.7 deg) = 83,116 m HERE, and 0 m at the pole
```

The degree buffer is not merely mis-scaled, it is the **wrong shape**: 25% too small east-west and correct north-south. Any "nearest within 500 m" analysis built on it silently misses features to the east and west.

Finally the roads bounds change from (13.884, 41.548, 14.363, 41.871) to (407,289, 4,600,000, 446,946, 4,636,000), with a motorway segment measuring 0.011198 in degrees versus 1 185 m in metres.

You will also see three `UserWarning: Geometry is in a geographic CRS. Results from 'area'/'buffer'/'length' are likely incorrect.` messages. **Those warnings are the correct behaviour** — GeoPandas is telling you exactly what this lesson is about. Never silence them globally.

B6 — Plotting spatial data

What we are going to learn. How to draw layers, build a layered map, and make an honest choropleth.

Why it matters. A map is an argument. The classification scheme you choose determines which places look "high" and which look "low" — the same data can support opposite-looking maps. Choosing a scheme is an analytical decision, not a styling one.

The concept — `.plot()` is matplotlib. `gdf.plot()` returns a matplotlib `Axes`. Layering is simply passing `ax=` to subsequent calls. The important arguments:

Argument	Meaning
<code>column=</code>	Attribute to colour by (makes it a thematic map)
<code>cmap=</code>	Colormap. Sequential (<code>viridis</code> , <code>Blues</code>) for magnitude; diverging (<code>RdBu</code>) only when there is a meaningful midpoint; qualitative (<code>tab10</code> , <code>Set2</code>) for categories
<code>scheme=</code>	Classification method (needs <code>mapclassify</code>): <code>quantiles</code> , <code>equalinterval</code> , <code>naturalbreaks</code> , <code>stdmean</code> , <code>fisherjenks</code>
<code>k=</code>	Number of classes (5–7 is the readable maximum)
<code>legend=</code> / <code>legend_kwds=</code>	Legend and its formatting
<code>edgecolor=</code> , <code>linewidth=</code> , <code>alpha=</code> , <code>markersize=</code>	Cosmetics
<code>missing_kwds=</code>	How to draw NaN. Without this, missing values are silently invisible

Classification schemes and the argument they make.

- **Equal interval** — cuts the range into equal-width bins. Honest about magnitude; useless when the distribution is skewed (one huge city swamps everything).
- **Quantiles** — equal *counts* per bin. Always produces a "colourful" map even when the data are uniform; can exaggerate trivial differences.
- **Natural breaks (Fisher–Jenks)** — minimises within-class variance. Usually the best compromise for exploratory work.
- **Standard deviation** — shows departures from the mean; requires roughly symmetric data.

Expected outcome. A 2×2 figure: a plain geometry plot, a categorical map, a choropleth with an explicit classification, and a fully layered reference map.

What the next cell does: builds four subplots demonstrating each style, including explicit handling of the two districts with missing population.

```
In [9]: land = gpd.read_file(GPKG, layer="land_boundary")
sea = gpd.read_file(GPKG, layer="sea")

fig, axes = plt.subplots(2, 2, figsize=(13.5, 11))

# --- (a) plain geometry -----
ax = axes[0, 0]
districts.plot(ax=ax, facecolor="#e8eef7", edgecolor="#3b5378", linewidth=0.7)
ax.set_title("(a) Plain geometry", loc="left", fontsize=10, weight="bold")

# --- (b) categorical -----
ax = axes[0, 1]
districts.plot(ax=ax, column="district_type", categorical=True, cmap="Set2",
               legend=True, edgecolor="white", linewidth=0.6,
               legend_kwds={"loc": "lower left", "fontsize": 7, "frameon": True})
ax.set_title("(b) Categorical map", loc="left",
             column='district_type',
             fontsize=10, weight="bold")

# --- (c) choropleth with an explicit scheme + explicit NaN handling -----
ax = axes[1, 0]
districts.plot(ax=ax, column="pop_density_km2", cmap="YlOrRd",
               scheme="naturalbreaks", k=5, legend=True,
               edgecolor="grey", linewidth=0.4,
               legend_kwds={"loc": "lower left", "fontsize": 7,
                           "title": "people / km^2", "title_fontsize": 8},
               missing_kwds={"color": "#d9d9d9", "edgecolor": "red",
                             "hatch": "///", "label": "no data"})
ax.set_title("(c) Choropleth, natural breaks\nNaN shown explicitly (red hatching)",
             loc="left", fontsize=10, weight="bold")

# --- (d) a layered reference map -----
ax = axes[1, 1]
sea.plot(ax=ax, facecolor="#cfe3f2", edgecolor="none", zorder=0)
land.plot(ax=ax, facecolor="#f6f3ec", edgecolor="none", zorder=1)
districts.boundary.plot(ax=ax, color="#9aa6b8", linewidth=0.5, zorder=2)
roads[roads.road_class.isin(["motorway", "primary"])].plot(
    ax=ax, color="#7a5c3e", linewidth=0.8, zorder=3)
rivers.plot(ax=ax, color="#2f7fbf", linewidth=1.1, zorder=4)
facilities[facilities.facility_type == "hospital"].plot(
    ax=ax, color="crimson", markersize=45, marker="P",
    edgecolor="white", linewidth=0.6, zorder=5, label="hospital")
ax.legend(loc="lower left", fontsize=7, frameon=True)
ax.set_title("(d) Layered reference map\nzorder controls what covers what",
             loc="left", fontsize=10, weight="bold")

for a in axes.ravel():
    a.set_aspect("equal"); a.set_xticks([]); a.set_yticks([])
    for s in a.spines.values():
        s.set_visible(False)
plt.tight_layout()
plt.show()
```

Explanation.

- **ax=** is the layering mechanism. Each `.plot(ax=ax, ...)` draws onto the same axes. Because they are all in EPSG:32633 they line up. Plot a layer in a different CRS onto the same axes and it will appear as a dot in a corner (or not at all) — the classic "my map is empty" bug.

- **zorder** controls draw order explicitly. Without it, matplotlib uses call order, which is fine until you insert a layer and everything vanishes behind the polygon you drew last.
- **missing_kwds** is the ethical bit. By default, features with NaN in the **column** are drawn *in no colour at all*, i.e. they disappear into the background and the reader assumes they are "low". Explicitly hatching them in grey/red states "we do not know" — which is what the data actually says. The two hatched districts are Cliffmoor and Highfen, our deliberate missing values.
- **scheme="naturalbreaks", k=5** invokes `mapclassify.NaturalBreaks`, which runs Fisher–Jenks optimisation to minimise within-class variance. Compare it with **scheme="quantiles"**: quantiles will always fill all five colour classes evenly, which here would make several near-empty rural districts look moderately populated.
- **districts.boundary** — a **GeoSeries** of the polygon *outlines* (LineStrings). Plotting boundaries rather than filled polygons is how you draw an overlay grid without hiding what is underneath.
- **sea** and **land** give the map a ground: without them a district map floats in white space and the coastline is invisible.

Expected output. A 2 × 2 grid of maps:

- **(a)** 24 pale-blue polygons tessellating a wedge-shaped region whose western edge is a wavy coastline.
- **(b)** the same polygons coloured in four categories, with **upland_rural** dominating the east and the three **urban_core** districts clustered in the centre-west.
- **(c)** a yellow→red choropleth in which the urban core is deep red ($\approx 3\,000$ people/km²) and the eastern districts are pale (< 100 people/km²); **two districts are grey with red hatching**.
- **(d)** a proper reference map: blue sea, cream land, grey district outlines, brown roads, blue rivers, four red hospital crosses concentrated near the core.

If panel (d) looks empty except for one tiny cluster, you have a CRS mismatch — re-run cell B5.

B7 — Building a GeoDataFrame from a CSV

What we are going to learn. How to turn a plain table with coordinate columns into a spatial layer — and the one-line mistake that silently ruins it.

Why it matters. Most data reaches you as a CSV with **lat/lon** columns: sensor readings, incident reports, GPS traces, geocoded addresses. Converting correctly is the entry point to every subsequent spatial operation.

The concept — three steps, in this order.

1. **Build the geometry** — `gpd.points_from_xy(df.lon, df.lat)`. Note the order: **x first, then y**; i.e. **longitude first, then latitude**. Writing `points_from_xy(df.lat, df.lon)` is the most common error in applied GIS and it puts your data in the wrong hemisphere without any error message.
2. **Declare the CRS** — `crs="EPSG:4326"` in the constructor, or `.set_crs(...)`. This is **set_crs** semantics: you are *labelling* coordinates that already exist. If the file's documentation says the coordinates are lon/lat on WGS 84, the label is **EPSG:4326**.
3. **Reproject for analysis** — `.to_crs(CRS_UTM)` before measuring anything.

Concept — why lon/lat order confusion exists. Humans say "latitude, longitude" (49.5° N, 6.1° E). Mathematics and every plotting library say "(x, y)". Longitude is x. GeoPandas, Shapely and Rasterio are consistently (x, y). The EPSG registry, meanwhile, formally defines EPSG:4326 axis order as (latitude, longitude), which is why some tools (and some WMS servers) disagree. **Rule: in Python, always (lon, lat).**

Expected outcome. The 24 sensor stations as a proper GeoDataFrame in UTM, validated against the **x_utm/y_utm** columns already in the file.

What the next cell does: loads the stations CSV, builds points from lon/lat, sets EPSG:4326, reprojects to UTM, and then *verifies* the result by comparing against the independently stored UTM columns — a check you should perform every time coordinates are involved.

```
In [10]: # --- 1. Read the plain table -----
stations_df = pd.read_csv(TAB / "sensor_stations.csv")
print("Raw CSV — this is NOT spatial yet:", type(stations_df).__name__)
print(stations_df.head(3).to_string(index=False), "\n")

# --- 2. lon/lat -> geometry, declare CRS, reproject -----
stations = gpd.GeoDataFrame(
    stations_df.copy(),
    geometry=gpd.points_from_xy(stations_df["lon"], stations_df["lat"]), # x, y !
    crs=CRS_WGS84, # LABEL the existing coordinates
).to_crs(CRS_UTM) # TRANSFORM into the analysis CRS

describe_gdf(stations, "stations")

# --- 3. VERIFY. Never skip this. -----
dx = stations.geometry.x - stations["x_utm"]
dy = stations.geometry.y - stations["y_utm"]
err = np.hypot(dx, dy)
print(f"\nCheck against the file's own UTM columns:")
print(f"  max positional error = {err.max():.4f} m (should be < 0.2 m)")
print(f"  mean error = {err.mean():.4f} m")
print(f"  (the CSV stores lon/lat rounded to 6 decimal places ~ 0.1 m, so a few")
print(f"  centimetres of disagreement is the rounding, not a CRS error)")

# --- 4. What the WRONG order would have done -----
wrong = gpd.GeoDataFrame(
    stations_df.copy(),
    geometry=gpd.points_from_xy(stations_df["lat"], stations_df["lon"]), # swapped!
    crs=CRS_WGS84,
)
print(f"\nIf you swap lat/lon:")
print(f"  correct centroid : ({stations.to_crs(CRS_WGS84).geometry.x.mean():.3f} E, "
    f"{stations.to_crs(CRS_WGS84).geometry.y.mean():.3f} N) -> Kestria")
print(f"  swapped centroid : ({wrong.geometry.x.mean():.3f} E, "
    f"{wrong.geometry.y.mean():.3f} N) -> somewhere else entirely")
print(f"  displacement :
~{wrong.to_crs(CRS_UTM).geometry.iloc[0].distance(stations.geometry.iloc[0])/1000:,.0f} km")

# --- 5. Plot to confirm -----
fig, ax = fresh_ax((7.5, 6), "24 environmental monitoring stations")
land.plot(ax=ax, facecolor="#f6f3ec", edgecolor="#c9c2b4")
districts.boundary.plot(ax=ax, color="#cdd4de", linewidth=0.5)
stations.plot(ax=ax, column="station_type", categorical=True, cmap="Dark2",
    markersize=55, edgecolor="black", linewidth=0.5, legend=True,
    legend_kwds={"loc": "lower left", "fontsize": 7})
for _, r in stations.iterrows():
    ax.annotate(r["station_id"], (r.geometry.x, r.geometry.y),
        xytext=(3, 3), textcoords="offset points", fontsize=5.5, color="#333")
plt.show()
```

Explanation.

- `gpd.points_from_xy(x, y)` — a vectorised constructor returning a `GeometryArray`. It is far faster than `df.apply(lambda r: Point(r.lon, r.lat), axis=1)` and it is the idiom you should use.
- `crs=CRS_WGS84` in the constructor is equivalent to `.set_crs(CRS_WGS84)`. It attaches a label. It does not move anything.
- `.to_crs(CRS_UTM)` then physically transforms all 24 points.
- **The verification block is the real lesson.** Because this dataset ships both `lon/lat` and `x_utm/y_utm`, we can prove the transformation is correct to sub-millimetre precision. In production you rarely get that luxury, so verify by

other means: plot the points over a known boundary, check the bounding box against expectation, or confirm a handful of known locations.

- The "wrong order" demonstration shows the failure mode: with lat/lon swapped, points land at roughly (41.7° E, 13.9° N) — in the Gulf of Aden rather than Kestria, thousands of kilometres away. Note that **no exception is raised**. A swapped-coordinate dataset will happily complete every downstream operation and return confident, wrong answers.
- `ax.annotate(..., textcoords="offset points")` offsets the label a few pixels from the marker regardless of zoom level — the correct way to label map points.

Expected output.

- A 3-row preview of the CSV with `station_id`, `name`, `station_type`, `install_year`, `elevation_m`, `x_utm`, `y_utm`, `lon`, `lat`.
- `describe_gdf`: **24 rows, EPSG:32633, geom={'Point': 24}**, bounds inside the study area.
- `max positional error` ≈ 0.067 m, `mean` ≈ 0.037 m — a few centimetres, which is exactly the rounding of `lon/lat` to six decimal places. If you see hundreds of metres, your CRS label is wrong; if you see thousands of kilometres, your lon/lat are swapped.
- The swap demonstration reports `correct centroid (14.145 E, 41.714 N)`, `swapped centroid (41.714 E, 14.145 N)` and a displacement of $\approx 4\,205$ km.
- A map of 24 labelled points spread across the basin, coloured by `station_type` (air_quality / rain_gauge / combined).

B8 — Basic Shapely operations

What we are going to learn. The core geometric constructors and predicates — the vocabulary in which every spatial analysis is written.

Why it matters. GeoPandas is a thin, convenient wrapper; the actual geometry is Shapely/GEOS. Knowing what each operation returns (and what it costs) lets you compose analyses instead of searching for a function that does exactly your task.

The concept — three families of operation.

1. Constructive operations return *new geometry*:

Operation	Meaning
<code>buffer(d)</code>	All points within distance <code>d</code> . Positive grows, negative shrinks (erosion)
<code>centroid</code>	Area-weighted centre of mass — may fall outside a concave shape
<code>representative_point()</code>	A point guaranteed to be <i>inside</i> the geometry
<code>convex_hull</code>	Smallest convex polygon containing the geometry
<code>envelope</code>	Bounding box as a polygon
<code>simplify(tol)</code>	Douglas–Peucker vertex reduction
<code>boundary</code>	Dimension–1 edge: polygon → ring, line → endpoints
<code>intersection / union / difference / symmetric_difference</code>	Boolean set operations

2. Predicates return `True/False` (the DE-9IM relations):

Predicate	True when
<code>intersects</code>	They share any point. The catch-all; opposite of <code>disjoint</code>
<code>contains</code> / <code>within</code>	One is entirely inside the other (boundaries may touch)
<code>covers</code> / <code>covered_by</code>	Like contains/within but tolerant of boundary-only contact
<code>touches</code>	They share a boundary but no interior
<code>crosses</code>	Interiors intersect but neither contains the other (line × polygon)
<code>overlaps</code>	Same dimension, partial overlap, neither contains the other

3. Measures return numbers: `area`, `length`, `distance`, `hausdorff_distance`.

The crucial subtlety — `contains` vs `intersects`. A polygon that shares only an edge with another `intersects` it but does not `contain` it. In a spatial join this is the difference between counting a boundary feature once, twice, or never.

Expected outcome. A visual and numeric tour of buffers, hulls, simplification and the predicate matrix.

What the next cell does: builds and plots five constructive operations on a single district polygon, prints how simplification trades vertices for accuracy, and evaluates the full predicate set between selected features.

```
In [11]: d = districts.loc[districts["name"] == "Harbourgate"].geometry.iloc[0]

# --- 1. Constructive operations -----
ops = {
    "original":      d,
    "buffer(+1500 m)": d.buffer(1500),
    "buffer(-1000 m)": d.buffer(-1000),
    "convex_hull":    d.convex_hull,
    "envelope":       d.envelope,
    "simplify(500 m)": d.simplify(500),
}
fig, axes = plt.subplots(2, 3, figsize=(13, 8))
for ax, (label, g) in zip(axes.ravel(), ops.items()):
    gpd.GeoSeries([d], crs=CRS_UTM).plot(ax=ax, facecolor="#dfe7f3",
                                         edgecolor="#4a6fa5", linewidth=0.8)
    gpd.GeoSeries([g], crs=CRS_UTM).plot(ax=ax, facecolor="none",
                                         edgecolor="crimson", linewidth=1.6)
    ax.set_title(f"{label}\narea = {g.area/1e6:.1f} km^2", fontsize=9)
    ax.set_aspect("equal"); ax.set_xticks([]); ax.set_yticks([])
plt.suptitle("Constructive Shapely operations (red = result)", fontsize=12, weight="bold")
plt.tight_layout(); plt.show()

# --- 2. Simplification: the accuracy/size trade-off -----
print("Douglas-Peucker simplification of one district polygon")
print(f"{'tolerance (m)':>14} {'vertices':>9} {'area (km^2)':>12} {'area err %':>11}")
base_n, base_a = shapely.count_coordinates(d), d.area
for tol in [0, 25, 100, 250, 500, 1000, 2500]:
    s = d.simplify(tol)
    print(f"{'tol:>14} {shapely.count_coordinates(s):>9} {s.area/1e6:>12,.3f} "
          f"{'100*(s.area-base_a)/base_a:>10.3f}%")

# --- 3. Predicates -----
# Pick the pair PROGRAMMATICALLY rather than by name: A, and the first district
# that genuinely shares a boundary with it. Hard-coding names is exactly how
# demonstrations like this silently stop demonstrating anything.
A_name = "Harbourgate"
A_row = districts.loc[districts["name"] == A_name].iloc[0]
a = A_row.geometry

neighbours = districts[districts.geometry.touches(a)]
B_row = neighbours.iloc[0]
b = B_row.geometry
```

```

print(f"\nA = {A_name}    B = {B_row['name']} "
      f"({len(neighbours)} districts share a boundary with A)")

# a river that actually crosses A, and a facility that is actually inside A
riv = rivers.loc[rivers.geometry.intersects(a)].geometry.iloc[0]
inside = facilities[facilities.geometry.within(a)]
hosp = inside.geometry.iloc[0]
outside = facilities[~facilities.geometry.within(a)].geometry.iloc[0]

print("\nPredicate matrix    (A = district, B = neighbouring district)")
print(f"{'relation':<14} {'A ~ B':>10} {'A ~ river':>11} {'A ~ inside pt':>14} {'A ~ outside pt':>15}")
for name in ["intersects", "contains", "within", "touches", "crosses",
             "overlaps", "disjoint", "covers"]:
    print(f"{'name':<14} {str(getattr(a, name)(b)):>10} {str(getattr(a, name)(riv)):>11} "
          f" {str(getattr(a, name)(hosp)):>14} {str(getattr(a, name)(outside)):>15}")

print(f"\nArea of A                : {a.area/1e6:,.2f} km^2")
print(f"Shared boundary length A & B : {a.intersection(b).length:,.0f} m "
      f"(dimension: {a.intersection(b).geom_type})")
print(f"Area of that shared boundary : {a.intersection(b).area:,.1f} m^2 "
      f"(zero - it is a LINE, not an overlap)")
print(f"Distance A -> the outside point: {a.distance(outside):,.0f} m")
print(f"Distance A -> the inside point : {a.distance(hosp):,.0f} m "
      f"(zero, because it is inside)")

```

Explanation.

- `d.buffer(1500)` — a **positive** buffer dilates the polygon by 1 500 m in every direction. `d.buffer(-1000)` **erodes** it; erode a thin polygon by more than half its width and you get an **empty geometry**, which is a silent way to lose features. Always check `.is_empty` after a negative buffer.
- Buffers are approximated by polygons: `buffer(d, quad_segs=8)` uses 8 segments per quarter circle by default (32-gon). Raise it for smooth cartography, lower it for speed on millions of features.
- `simplify(tol)` uses **Douglas–Peucker**: it drops any vertex lying within `tol` of the line joining its neighbours. Note in the table that 250 m tolerance typically removes ~90% of vertices while changing the area by well under 0.5%. That is the trade you make before publishing web maps or before an expensive overlay. Use `simplify(tol, preserve_topology=True)` (the default) to avoid creating self-intersections; `preserve_topology=False` is faster but can produce invalid output.
- **Simplification does not preserve shared borders.** Simplifying two adjacent districts independently opens slivers and gaps between them. For an entire administrative layer use `topojson`-style tools, not per-feature `simplify`.
- The predicate matrix shows the key relationships: two adjacent districts **intersects** and **touches** (they share an edge but no interior), and therefore **overlaps** is **False**. A hospital inside a district gives **contains=True**, **intersects=True**, **touches=False**.
- `a.intersection(b).length` — for two polygons that only touch, the intersection is a `LineString` (their shared border) with zero area but positive length. Recognising the *dimension* of an intersection result is essential: polygon $\cap$ polygon can return a `Polygon`, a `LineString`, a `Point`, a `GeometryCollection`, **or** an empty geometry.

Expected output.

- Six panels. Harbourgate is 67.69 km²; the +1 500 m buffer grows it to ~113 km², the -1 000 m buffer shrinks it to ~40 km², the **convex hull is 92.26 km²** — 36% larger than the district, which is the quantitative statement that the boundary is genuinely concave. The envelope is a rectangle; `simplify(500)` looks almost identical but with visibly straighter edges.
- A simplification table close to:

tolerance (m)	vertices	area km ²	area err
0	48	67.687	0.000%
25	47	67.687	-0.001%
100	40	67.628	-0.087%
250	33	67.660	-0.041%
500	27	68.220	+0.787%
1000	13	69.653	+2.903%
2500	5	64.810	-4.251%

Note the error is not monotone and changes sign. Douglas–Peucker removes vertices, not area: dropping a vertex on a concave stretch *adds* area while dropping one on a convex stretch removes it. Never assume "a bit of simplification just shrinks things slightly".

- A = Harbournate B = Old Vallmara (5 districts share a boundary with A) and a predicate matrix:

relation	A ~ B	A ~ river	A ~ inside pt	A ~ outside pt
intersects	True	True	True	False
contains	False	False	True	False
within	False	False	False	False
touches	True	False	False	False
crosses	False	True	False	False
overlaps	False	False	False	False
disjoint	False	False	False	True
covers	False	False	True	False

Read it row by row. Two adjacent districts **intersect and touch** but do not **overlap** — they share a boundary and no interior. The river **crosses** the district (a line passing through a polygon). A point inside is **contained and covered**. Everything about the outside point is **False** except **disjoint**.

- Finally: **Shared boundary length A & B : 16,581 m (dimension: MultiLineString)** with **area 0.0 m²**. Polygon $\cap$ polygon returned a *line*, and it came back as a **MultiLineString** because the two districts meet along several separate stretches. Blindly calling **.area** on an intersection result — assuming it must be a polygon — is a very common bug.

B9 — Filtering and selecting spatial features

What we are going to learn. The four ways to select a subset of features, and when each is appropriate.

Why it matters. Selection is where performance is won or lost. A bounding-box filter that runs in microseconds can replace an exact geometric test that takes minutes — and on 5 200 buildings against 181 flood polygons the difference is already noticeable.

The concept — four selection mechanisms, cheapest first.

1. **Attribute filter** — pure pandas: `gdf[gdf.road_class == "motorway"]`. No geometry touched. Always do this first to shrink the problem.
2. **Coordinate/bounding-box filter** — `gdf.cx[xmin:xmax, ymin:ymax]`. The `.cx` indexer selects features whose *bounding box* intersects the given box. Very fast (uses the spatial index) but **approximate** — a diagonal river's bounding box covers a huge area it never enters.

3. **Spatial predicate filter** — `gdf[gdf.intersects(some_geom)]`. Exact, and the most common form. Internally still uses the spatial index for the bounding-box pre-filter, then does exact tests on the survivors.
4. **Read-time filter** — `gpd.read_file(path, bbox=..., where="...")`. The cheapest of all: filtering happens in GDAL before the data ever enters Python. Essential for files larger than memory.

Concept — the spatial index. GeoPandas maintains an **R-tree** (`gdf.sindex`) over the bounding boxes. Every query is two-phase: (a) *filter* — the R-tree returns candidate bounding boxes in $O(\log n)$; (b) *refine* — exact GEOS predicates run only on the candidates. Understanding this two-phase pattern explains why cheap attribute filtering first makes exact tests fast.

Expected outcome. The same "buildings in the flood zone" question answered four ways, with timings, plus a visual comparison of bbox vs exact selection.

What the next cell does: demonstrates attribute selection, `.cx` box selection, exact predicate selection and read-time filtering, times each, and plots the difference between the approximate and exact answers.

```
In [12]: import time

flood = gpd.read_file(GPKG, layer="flood_zones")
flood100 = flood[flood["return_period_yr"] == 100]
zone_100 = flood100.geometry.union_all()      # one big MultiPolygon

# --- 1. ATTRIBUTE filter -----
motorways = roads[roads["road_class"] == "motorway"]
old_res = buildings[(buildings["year_built"] < 1950) & (buildings["floors"] >= 3)]
print(f"1. Attribute filter")
print(f"    motorway segments           : {len(motorways):>6}")
print(f"    pre-1950 buildings, 3+ floors: {len(old_res):>6}")

# --- 2. BOUNDING-BOX filter with .cx -----
xmin, ymin, xmax, ymax = 410_000, 4_612_000, 424_000, 4_624_000
t0 = time.perf_counter()
in_box = buildings.cx[xmin:xmax, ymin:ymax]
t_box = time.perf_counter() - t0
print(f"\n2. Bounding-box filter (.cx)   : {len(in_box):>6} buildings "
      f"    in {t_box*1000:.1f} ms")

# --- 3. EXACT spatial predicate -----
t0 = time.perf_counter()
flooded_bbox = buildings[buildings.geometry.intersects(zone_100.envelope)]
t_env = time.perf_counter() - t0

t0 = time.perf_counter()
flooded_exact = buildings[buildings.geometry.intersects(zone_100)]
t_exact = time.perf_counter() - t0

print(f"\n3. Exact predicate filter")
print(f"    intersects(bounding box of zone): {len(flooded_bbox):>6} "
      f"    in {t_env*1000:.1f} ms    <- APPROXIMATE, over-counts")
print(f"    intersects(actual zone)           : {len(flooded_exact):>6} "
      f"    in {t_exact*1000:.1f} ms    <- CORRECT")
print(f"    over-count from using the bbox   : "
      f"    {len(flooded_bbox) - len(flooded_exact)} buildings "
      f"    ({100*(len(flooded_bbox)-len(flooded_exact))/len(flooded_exact):.0f}% too many)")

# --- 4. READ-TIME filter (never loads the rest into memory) -----
t0 = time.perf_counter()
sub = gpd.read_file(GPKG, layer="buildings",
                    bbox=(xmin, ymin, xmax, ymax))
t_read = time.perf_counter() - t0
print(f"\n4. Read-time bbox filter       : {len(sub):>6} rows loaded "
      f"    in {t_read*1000:.1f} ms (never touched the other "
      f"    {len(buildings)-len(sub):,} rows)")

# --- 5. Visual comparison -----
```

```

fig, axes = plt.subplots(1, 2, figsize=(13, 5.6))
for ax, sel, title in [
    (axes[0], flooded_bbox, f"intersects(ENVELOPE)  n={len(flooded_bbox):,}"),
    (axes[1], flooded_exact, f"intersects(ZONE)      n={len(flooded_exact):,}")] :
    land.plot(ax=ax, facecolor="#f7f5ef", edgecolor="#d8d2c4", linewidth=0.5)
    flood100.plot(ax=ax, facecolor="#9ecae1", edgecolor="none", alpha=0.75)
    buildings.plot(ax=ax, color="#cccccc", markersize=0.4, linewidth=0)
    sel.plot(ax=ax, color="crimson", markersize=1.2, linewidth=0)
    ax.set_title(title, fontsize=10, weight="bold", loc="left")
    ax.set_aspect("equal"); ax.set_xticks([]); ax.set_yticks([])
plt.suptitle("Bounding-box selection over-counts; exact predicates do not",
             fontsize=11)
plt.tight_layout(); plt.show()

```

Explanation.

- `roads[roads["road_class"] == "motorway"]` — an ordinary boolean mask. The result is still a GeoDataFrame with its CRS intact. Every pandas filtering idiom (`.query()`, `.isin()`, `.between()`) works.
- `buildings.cx[xmin:xmax, ymin:ymax]` — the **coordinate indexer**. Slice syntax, `xmin:xmax` first then `ymin:ymax`. An open end means unbounded: `gdf.cx[:, 4_620_000:]` selects everything north of that line. Remember it tests **bounding boxes**, not geometries.
- `zone_100.envelope` versus `zone_100` — the whole point of the third block. The 100-year flood zone is a set of long, thin, branching ribbons following the rivers; its bounding box covers most of the basin. Selecting by envelope returns roughly **4–5x too many buildings**. This is exactly what `.cx` does, so `.cx` is a *pre-filter*, never a final answer.
- `gpd.read_file(..., bbox=...)` pushes the filter down into GDAL, which uses the GeoPackage's own R-tree index. On a 5 GB national buildings layer this is the difference between a working script and an out-of-memory crash. You can also pass `where="road_class = 'motorway'"` — an SQL attribute filter evaluated by the driver.
- Note the timings: the exact predicate is only slightly slower than the envelope test, because both are dominated by the same R-tree lookup. **Correctness here is nearly free** — there is no excuse for using the approximate answer.

Expected output.

```

1. Attribute filter
   motorway segments           :      38
   pre-1950 buildings, 3+ floors:    359

2. Bounding-box filter (.cx)   :  3880 buildings in ~3 ms

3. Exact predicate filter
   intersects(bounding box of zone):  5198 in   ~3 ms  <- APPROXIMATE, over-counts
   intersects(actual zone)          :   1036 in ~540 ms  <- CORRECT
   over-count from using the bbox   :  4162 buildings (402% too many)

4. Read-time bbox filter      :  3880 rows loaded in ~90 ms

```

The envelope of the 100-year flood zone contains **5 198 of the 5 200 buildings** — it is, for practical purposes, the whole basin. Only **1 036** buildings actually touch the floodplain. Note also that the exact test costs ~540 ms versus ~3 ms: correctness is not always free, but half a second to avoid a 400% error is the easiest trade you will ever make.

The figure shows the same map twice: on the left almost every building is red; on the right only the buildings actually lying on the blue floodplain ribbons are red. **The left panel is what you get if you trust a bounding box.**

B10 — Calculating distances

What we are going to learn. Point-to-point, point-to-line and one-to-many distance computation, and the difference between planar and geodesic distance.

Why it matters. "Distance to the nearest X" is *the* workhorse feature of spatial data science. Distance to the coast, to a road, to a hospital, to the nearest flood zone — these are the columns that make spatial ML models work. Getting them right (right CRS, right geometry, right definition) is most of the battle.

The concept — what "distance" means.

- **Euclidean / planar distance** — straight-line distance in the projected plane. `geom_a.distance(geom_b)` returns the **minimum** distance between the two geometries (0 if they touch or overlap). This is what GeoPandas computes, and it is correct *in a suitable projected CRS over a regional extent*.
- **Geodesic distance** — the true distance over the ellipsoid. Necessary for continental or intercontinental distances, or when working in EPSG:4326. Use `pyproj.Geod(ellps="WGS84").inv(lon1, lat1, lon2, lat2)`.
- **Network distance** — distance *along a road network*. Almost always the honest answer for accessibility, and always larger than Euclidean. The ratio (network / Euclidean), the **detour index**, is typically 1.2–1.5 in a city. We use Euclidean throughout for tractability and flag it as an assumption.

Concept — the three distance shapes.

Question	Tool	Cost
A to B	<code>a.distance(b)</code>	O(1)
Every A to one B	<code>gdf.distance(b)</code> (vectorised)	O(n)
Every A to its nearest B	<code>gpd.sjoin_nearest</code> or <code>scipy.spatial.cKDTree</code>	O(n log m)
Every A to every B	<code>cKDTree</code> / broadcasting	O(n·m) — beware

Expected outcome. Distance-to-hospital for every district centroid, distance-to-motorway for every sensor, and a demonstration that planar and geodesic distances agree to ~0.1% at this scale.

What the next cell does: computes three kinds of distance, compares planar UTM distance against true geodesic distance, and shows why a full pairwise matrix is only viable for small n.

```
In [13]: from scipy.spatial import cKDTree
         from pyproj import Geod

         hospitals = facilities[facilities["facility_type"] == "hospital"].reset_index(drop=True)
         centroids = districts.copy()
         centroids["geometry"] = districts.geometry.representative_point()

         # --- 1. One-to-one -----
         a = centroids.geometry.iloc[0]
         b = hospitals.geometry.iloc[0]
         print(f"1. One-to-one: {districts.loc[0, 'name']} centroid -> {hospitals.loc[0, 'name']}")
         print(f"   planar (UTM) : {a.distance(b)/1000:,.3f} km")

         geod = Geod(ellps="WGS84")
         a_ll = gpd.GeoSeries([a], crs=CRS_UTM).to_crs(CRS_WGS84).iloc[0]
         b_ll = gpd.GeoSeries([b], crs=CRS_UTM).to_crs(CRS_WGS84).iloc[0]
         _, _, geo_m = geod.inv(a_ll.x, a_ll.y, b_ll.x, b_ll.y)
         print(f"   geodesic      : {geo_m/1000:,.3f} km")
         print(f"   difference      : {abs(geo_m - a.distance(b)):,.1f} m "
               f"      ({100*abs(geo_m-a.distance(b))/geo_m:.4f} %) <- UTM is fine at this scale")

         # --- 2. One-to-many: vectorised distance to a single geometry -----
         motorway_geom = roads[roads.road_class == "motorway"].geometry.union_all()
         stations["dist_motorway_m"] = stations.geometry.distance(motorway_geom)
```

```

print(f"\n2. One-to-many: every station -> the motorway (a MultiLineString)")
print(stations[["station_id", "station_type", "dist_motorway_m"]].
      .sort_values("dist_motorway_m").head(5).to_string(index=False))

# --- 3. Many-to-nearest with a KD-tree -----
tree = cKDTree(np.c_[hospitals.geometry.x, hospitals.geometry.y])
dist, idx = tree.query(np.c_[centroids.geometry.x, centroids.geometry.y], k=1)
centroids["nearest_hospital"] = hospitals.loc[idx, "name"].to_numpy()
centroids["hosp_dist_km"] = dist / 1000
print(f"\n3. Many-to-nearest (KD-tree over {len(hospitals)} hospitals)")
print(centroids[["name", "district_type", "nearest_hospital", "hosp_dist_km"]].
      .sort_values("hosp_dist_km", ascending=False).head(6).to_string(index=False))

# --- 4. Full pairwise matrix (only because n is tiny) -----
D = np.hypot(
    centroids.geometry.x.values[:, None] - hospitals.geometry.x.values[None, :],
    centroids.geometry.y.values[:, None] - hospitals.geometry.y.values[None, :],
) / 1000
print(f"\n4. Full pairwise matrix: {D.shape[0]} districts x {D.shape[1]} hospitals "
      f"= {D.size} distances")
print(pd.DataFrame(D, index=districts["name"], columns=hospitals["facility_id"])
      .round(1).head(6).to_string())
print(f"\n    Memory for this matrix          : {D.nbytes/1024:.1f} KB")
print(f"    Same approach for 5,200 buildings x 83 facilities: "
      f"    {5200*83*8/1e6:.1f} MB    (fine)")
print(f"    Same approach for 1M x 100k points          : "
      f"    {1e6*1e5*8/1e9:,.0f} GB    (use a KD-tree instead)")

```

Explanation.

- `districts.geometry.representative_point()` rather than `.centroid` — for a concave district the centroid can lie outside the polygon (in the sea, or in a neighbouring district). `representative_point()` guarantees a point inside. For distance-to-service analysis, a **population-weighted** centroid would be better still; we build one in Module 3.
- `a.distance(b)` is the minimum separation. For a Point and a MultiLineString it is the perpendicular distance to the closest segment — exactly what "distance to the road network" should mean.
- `stations.geometry.distance(motorway_geom)` — a `GeoSeries` against a **single** Shapely geometry broadcasts, giving a Series of distances. If you pass two `GeoSeries` of equal length instead, GeoPandas pairs them **row by row** (element-wise), which is a completely different operation. Know which you want.
- `union_all()` before measuring is important: without it you would have to take a min over 38 separate motorway segments.
- `cKDTree` builds a k-d tree in $O(m \log m)$ and answers each nearest-neighbour query in $O(\log m)$. It works on **planar coordinates only**, which is another reason to be in a projected CRS. Note `k=1` returns (`distances`, `indices`); ask for `k=3` to get the three nearest, which is how you build "distance to 2nd nearest hospital" features (a useful redundancy measure).
- The pairwise matrix block is a warning about scale. A full matrix is $n \times m \times 8$ bytes. It is the right tool for 24×4 ; it is catastrophic for $10^6 \times 10^5$.
- The planar/geodesic comparison shows an error of order **0.01–0.1%** over 20 km in UTM. That is the empirical justification for using a projected CRS and ordinary Euclidean geometry at regional scale.

Expected output.

- Planar **2.241 km** versus geodesic **2.241 km** — a difference of **0.7 m, or 0.03%**. That is the empirical licence to use flat Euclidean geometry in UTM at this scale.
- The five stations closest to the motorway, from **438 m** (ST018) to ~2.5 km.
- A ranked table of districts by distance to the nearest hospital: **Ostrand 31.0 km, Stonebeck 26.4 km, Halvorn 26.3 km, Willowmere 26.1 km** — all `upland_rural` — versus about 2 km for the urban core. Note too that every

remote district's nearest hospital is the same one (Harbourgate General): four hospitals serve 600 000 people from a single cluster. **This is the accessibility inequality that Module 3 quantifies properly.**

- A 24×4 distance matrix (0.8 KB), and the scaling warning: the same approach on $1\text{M} \times 100\text{k}$ points would need **800 GB**.

B11 — Your first spatial join

What we are going to learn. `gpd.sjoin` — joining two layers on a *spatial relationship* rather than a key.

Why it matters. This is the operation that makes spatial data science different from ordinary data science. Instead of `df.merge(other, on="id")`, you ask "which polygon is this point in?" and let geometry supply the key. It is how you attach context (district, land use, hazard zone) to observations.

The concept.

```
gpd.sjoin(left_gdf, right_gdf, how="inner", predicate="intersects")
```

- The result has **one row per matching pair**. If a point falls in two overlapping polygons you get **two rows** — silent row multiplication is the number-one spatial-join bug.
- `how="left"` keeps every left row (unmatched ones get NaN attributes); `how="inner"` keeps only matches; `how="right"` mirrors left.
- `predicate=` accepts any DE-9IM relation: `intersects` (default), `within`, `contains`, `touches`, `crosses`, `overlaps`, `covers`, `covered_by`, `dwithin`.
- The right layer's index arrives as `index_right`.
- **Both layers must be in the same CRS** — GeoPandas raises if they are not.

Which predicate for points in polygons? Use `within` (or its inverse `contains`). `intersects` also matches a point lying exactly *on* a shared border, which will match **both** neighbouring polygons and duplicate the row. For floating-point coordinates this is rare but not impossible — and with data snapped to a grid it is common.

Expected outcome. Every facility labelled with its district, a count-of-points-per-polygon table, and a demonstration of the row-multiplication trap.

What the next cell does: joins facilities to districts, aggregates counts per district, joins the counts back onto the polygon layer for mapping, and then deliberately triggers the duplication problem using overlapping flood zones.

```
In [14]: # --- 1. Points-in-polygons -----
fac_d = gpd.sjoin(
    facilities,                                # left = points
    districts[["district_id", "name", "district_type", "geometry"]],
    how="left", predicate="within",
).rename(columns={"name_right": "district_name", "name_left": "facility_name"})

print(f"facilities in : {len(facilities)} rows")
print(f"after sjoin   : {len(fac_d)} rows    "
      f"({{'no duplication' if len(fac_d)==len(facilities) else 'DUPLICATED!'})")
print(f"unmatched     : {fac_d['district_id'].isna().sum()} facilities fell outside every district\n")
print(fac_d[["facility_id", "facility_type", "district_name", "district_type"]].head(6).to_string(index=False))

# --- 2. Aggregate: how many of each facility type per district? -----
counts = (fac_d.groupby(["district_id", "facility_type"])
          .size().unstack(fill_value=0))
counts["total"] = counts.sum(axis=1)
print("\nFacility counts per district (first 8)")
print(counts.head(8).to_string())
```

```

# --- 3. Join the summary BACK onto the polygons so we can map it -----
dist_stats = districts.merge(
    counts.reset_index(), on="district_id", how="left").fillna({"total": 0})
dist_stats["pop_per_facility"] = np.where(
    dist_stats["total"] > 0, dist_stats["population"] / dist_stats["total"], np.nan)

fig, axes = plt.subplots(1, 2, figsize=(13.5, 5.8))
dist_stats.plot(ax=axes[0], column="total", cmap="Greens", scheme="naturalbreaks",
    k=5, legend=True, edgecolor="grey", linewidth=0.4,
    legend_kwds={"loc": "lower left", "fontsize": 7, "title": "facilities"})
facilities.plot(ax=axes[0], color="black", markersize=4)
axes[0].set_title("Facility count per district", loc="left", weight="bold", fontsize=10)

dist_stats.plot(ax=axes[1], column="pop_per_facility", cmap="OrRd",
    scheme="quantiles", k=5, legend=True, edgecolor="grey", linewidth=0.4,
    legend_kwds={"loc": "lower left", "fontsize": 7, "title": "people / facility"},
    missing_kwds={"color": "#ddddd", "hatch": "//", "label": "no facility / no
pop"})
axes[1].set_title("Residents per facility (higher = worse served)",
    loc="left", weight="bold", fontsize=10)

for a in axes:
    a.set_aspect("equal"); a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.show()

# --- 4. THE TRAP: joining to OVERLAPPING polygons -----
print("=" * 78)
print("THE ROW-MULTIPLICATION TRAP")
print("=" * 78)
overlapping = pd.concat([flood100, flood100.assign(zone_id=flood100.zone_id + "_dup")])
overlapping = gpd.GeoDataFrame(overlapping, geometry="geometry", crs=CRS_UTM)

j = gpd.sjoin(buildings[["building_id", "geometry"]], overlapping[["zone_id", "geometry"]],
    how="inner", predicate="intersects")
print(f"buildings joined to 2 identical zone layers : {len(j):,} rows")
print(f"distinct buildings involved : {j['building_id'].nunique():,}")
print(f"inflation factor : {len(j)/j['building_id'].nunique():.2f}
x")
print("\nIf you now sum building values you will DOUBLE-COUNT every exposed asset.")
print("Fix: aggregate first, or de-duplicate:")
print(f" j.drop_duplicates('building_id') -> {len(j.drop_duplicates('building_id')):,}
rows")

```

Explanation.

- `gpd.sjoin(facilities, districts, how="left", predicate="within")` — for each facility, find the district polygon that **contains** it. `how="left"` keeps all 83 facilities so we can *count* the unmatched ones instead of losing them silently. Here 0 are unmatched because the districts tile the land exactly.
- Column-name collisions: both layers have a `name` column, so GeoPandas suffixes them `name_left` / `name_right`. Renaming immediately is good hygiene — `name_right` becomes meaningless three cells later.
- `.groupby(...).size().unstack(fill_value=0)` — the standard pandas cross-tabulation. `fill_value=0` matters: a district with no hospital should show **0**, not NaN, otherwise later arithmetic propagates NaN.
- **Joining the summary back:** `districts.merge(counts, on="district_id")`. Note this is an *attribute* merge, not a spatial one — once the spatial join has produced a key, ordinary pandas takes over. Keep the polygon layer on the left so the result stays a GeoDataFrame with geometry.
- `np.where(total > 0, pop/total, np.nan)` — guarding against division by zero. Districts with no facility get NaN and are hatched on the map rather than appearing as `inf`.
- **The trap block** builds a layer with duplicate overlapping polygons. Every building now matches twice, so `len(j)` is double `nunique()`. In the real world this happens whenever your right-hand layer has overlaps: overlapping

hazard zones, administrative boundaries with slivers, or buffers around multiple facilities. **Always check `len(result)` against `len(left)` after a join.**

Expected output.

- `facilities in : 83 rows, after sjoin : 83 rows (no duplication), unmatched : 0.`
- A facility-count matrix with columns `clinic, fire_station, hospital, police_station, school, total`. **D01 has 24 facilities and D02 has 23, while D03 has 1 and several districts do not appear at all** — because `groupby` only creates rows for districts that matched. Districts with zero facilities are *missing from the index*, not zero in it. That is why step 3 merges back onto the full polygon layer and fills with 0.
- Two maps: the left shows facility counts concentrated in the populous central districts; the right shows *residents per facility*, where the urban districts look **worse** (thousands of residents per facility) than the empty upland ones — a genuine analytical result and a warning that raw counts and per-capita rates tell opposite stories.
- The trap block prints `2,072 rows / 1,036 distinct buildings / inflation 2.00x`.

B12 — Opening a raster with Rasterio

What we are going to learn. The anatomy of a raster dataset: profile, transform, CRS, NoData, bands, dtype and windows.

Why it matters. Rasters carry the *continuous* variables — elevation, rainfall, temperature, reflectance, density. Half of environmental data science is raster work, and every raster bug traces back to one of four things: the affine transform, the NoData value, the dtype, or the CRS.

The concept — a raster is an array plus an affine transform.

A raster is a regular grid of cells. To place it on the Earth you need six numbers, the **affine transform**:

$$\begin{array}{|c|} \hline x \\ \hline y \\ \hline 1 \\ \hline \end{array} = \begin{array}{|ccc|} \hline a & b & c \\ \hline d & e & f \\ \hline 0 & 0 & 1 \\ \hline \end{array} \begin{array}{|c|} \hline col \\ \hline row \\ \hline 1 \\ \hline \end{array} \quad \begin{array}{l} x = a*col + b*row + c \\ y = d*col + e*row + f \end{array}$$

For a standard north-up raster: `a` = cell width, `e` = **negative** cell height (because rows increase downward while `y` increases upward), `b = d = 0`, and `(c, f)` = the coordinates of the **upper-left corner of the upper-left cell**.

The rasterio object model.

Attribute	Meaning
<code>src.width, src.height</code>	Columns, rows
<code>src.count</code>	Number of bands
<code>src.dtypes</code>	Per-band NumPy dtype
<code>src.crs</code>	Coordinate reference system
<code>src.transform</code>	The affine transform above
<code>src.bounds</code>	<code>(left, bottom, right, top)</code> in CRS units
<code>src.res</code>	<code>(x_size, y_size)</code> per cell
<code>src.nodata</code>	The sentinel value meaning "no measurement"
<code>src.profile</code>	A dict of all of the above — pass it to <code>rasterio.open(..., **profile)</code> to write a matching file

NoData is the number-one raster trap. `src.read(1)` returns the raw array *including* the NoData sentinel (here **-9999**). Compute a mean on that and your average elevation becomes hugely negative. Two correct approaches:

- `src.read(1, masked=True)` → a `numpy.ma.MaskedArray` that excludes NoData from every reduction automatically.
- Read raw, then `arr = np.where(arr == src.nodata, np.nan, arr)` and use `np.nanmean` etc. (requires a float dtype).

Expected outcome. Full metadata for all seven rasters and a demonstration of the NoData trap with real numbers.

What the next cell does: prints a metadata table for every GeoTIFF in the dataset, then opens the DEM and computes statistics the wrong way and the right way so you can see the size of the error.

```
In [15]: import rasterio
from rasterio.plot import show as rshow

# --- 1. Metadata for every raster we ship -----
rows = []
for tif in sorted(RAS.glob("*.tif")):
    with rasterio.open(tif) as src:
        rows.append({
            "file": tif.name,
            "size": f"{src.width} x {src.height}",
            "bands": src.count,
            "dtype": src.dtypes[0],
            "res (m)": f"{src.res[0]:.0f}",
            "crs": src.crs.to_string(),
            "nodata": src.nodata,
            "MB": round(tif.stat().st_size / 1e6, 2),
        })
print("RASTER INVENTORY")
print(pd.DataFrame(rows).to_string(index=False))

# --- 2. Dissect the DEM -----
dem_path = RAS / "dem_25m.tif"
with rasterio.open(dem_path) as src:
    print("\n" + "=" * 78)
    print("dem_25m.tif")
    print("=" * 78)
    print(f"  shape (rows, cols) : {src.height} x {src.width} = {src.height*src.width:,} cells")
    print(f"  bands                : {src.count}  descriptions: {src.descriptions}")
    print(f"  dtype                 : {src.dtypes[0]}")
    print(f"  crs                   : {src.crs}")
    print(f"  resolution            : {src.res} (metres per cell)")
    print(f"  bounds                : {tuple(round(b) for b in src.bounds)}")
    print(f"  nodata                : {src.nodata}")
    print(f"\n  affine transform:\n{src.transform}")
    print(f"\n  upper-left corner : ({src.transform.c:.0f}, {src.transform.f:.0f})")
    print(f"  pixel width (a)  : {src.transform.a}")
    print(f"  pixel height (e)  : {src.transform.e}  <- NEGATIVE: rows go DOWN, y goes UP")

    raw      = src.read(1)          # raw array, NoData included
    masked    = src.read(1, masked=True)  # MaskedArray, NoData excluded
    nodata     = src.nodata

# --- 3. The NoData trap, quantified -----
print("\n" + "=" * 78)
print("THE NODATA TRAP")
print("=" * 78)
n_nodata = int((raw == nodata).sum())
print(f"  cells total          : {raw.size:,}")
print(f"  cells that are NoData : {n_nodata:,}  ({100*n_nodata/raw.size:.1f} % - the sea)")
print()
print(f"  WRONG  raw.mean()      = {raw.mean():>12,.2f} m  <- nonsense")
```

```

print(f" WRONG raw.min()           = {raw.min():>12,.2f} m   <- the sentinel itself")
print(f" RIGHT  masked.mean()       = {masked.mean():>12,.2f} m")
print(f" RIGHT  masked.min()/max()   = {masked.min():>8,.2f} / {masked.max():>8,.2f} m")

nan_arr = np.where(raw == nodata, np.nan, raw)
print(f" RIGHT  np.nanmean(...)      = {np.nanmean(nan_arr):>12,.2f} m   (the NaN idiom)")
print(f"\n Error from ignoring NoData : {raw.mean() - masked.mean():>12,.2f} m")

```

Explanation.

- `rasterio.open(path)` returns a lazy **dataset reader**. Opening reads only the header; nothing is loaded until you call `.read()`. Always use it as a context manager (`with ... as src:`) so the file handle closes — GDAL keeps a cache per open dataset and leaking handles is a real problem in loops.
- `src.profile` is the complete creation recipe. The write idiom is: `profile = src.profile;`
`profile.update(dtype="float32", count=1)` then `rasterio.open(out, "w", **profile)`. You will use this in B14.
- `src.transform.e` is **negative** (−25.0). This is not a quirk: image row indices increase downwards, map y-coordinates increase upwards. A positive `e` means a south-up raster, which will render upside down.
- `src.descriptions` returns the per-band names we set when generating the data — a much better practice than remembering that "band 3 is red".
- `masked=True` returns `numpy.ma.MaskedArray`. All reductions (`mean`, `std`, `min`) skip masked cells. The mask itself is `masked.mask` (True = NoData) and the raw values are `masked.data`.
- **The numbers are the lesson.** 19% of the DEM is sea, stored as −9999. The naive mean is about **−1 650 m**; the correct mean is about **+340 m**. The error is roughly 2 000 m, and nothing warns you. Everything downstream — slope, hillshade, zonal statistics, a regression on elevation — inherits it.
- `np.where(raw == nodata, np.nan, raw)` requires a float dtype. For integer rasters (like `landcover_25m.tif`, `uint8`) NaN is impossible, so use masked arrays or keep an explicit boolean mask.

Expected output.

A 7-row raster inventory:

file	size	bands	dtype	res	crs	nodata
dem_25m.tif	1920 × 1440	1	float32	25	EPSG:32633	−9999
landcover_25m.tif	1920 × 1440	1	uint8	25	EPSG:32633	0
lst_summer_100m_3857.tif	483 × 366	1	float32	134	EPSG:3857	−9999
multispectral_50m.tif	960 × 720	4	uint16	50	EPSG:32633	0
ndvi_50m.tif	960 × 720	1	float32	50	EPSG:32633	−9999
popdens_100m.tif	480 × 360	1	float32	100	EPSG:32633	−9999
rainfall_annual_250m.tif	192 × 144	1	float32	250	EPSG:32633	−9999

Note the LST raster: its resolution reads **134 m, not 100 m**. It was generated on a 100 m UTM grid and then reprojected to Web Mercator, and reprojection resamples onto a *new* grid whose spacing is set by the target CRS. This is another face of the Web-Mercator distortion — at 41.7° N, one Web-Mercator metre is only about 0.75 real metres, so a 100 m ground cell becomes a ~134 unit cell.

Then the DEM dissection, `pixel height (e) = −25.0`, and the trap:

```

cells total          : 2,764,800
cells that are NoData : 533,151 (19.3 % - the sea)

WRONG raw.mean()      = -1,654.81 m <- nonsense
WRONG raw.min()       = -9,999.00 m <- the sentinel itself
RIGHT masked.mean()   = 338.66 m
RIGHT masked.min()/max() = 0.40 / 925.20 m
RIGHT np.nanmean(...) = 338.66 m

Error from ignoring NoData : -1,993 m

```

B13 — Raster ↔ array ↔ ground: indexing, sampling and plotting

What we are going to learn. How to move between three coordinate spaces — array indices (`row, col`), map coordinates (`x, y`), and geographic (`lon, lat`) — and how to sample raster values at vector locations.

Why it matters. "Attach the elevation / rainfall / land-cover class at each observation point" is the most common raster-to-vector operation in data science. It is also where the affine transform stops being an abstraction.

The concept — three conversions.

Direction	Rasterio API
index → coordinate	<code>src.xy(row, col)</code> (returns the cell centre)
coordinate → index	<code>src.index(x, y)</code>
sample values at points	<code>src.sample([(x, y), ...])</code>
whole-array coordinates	<code>rasterio.transform.xy(transform, rows, cols)</code>

The plotting trap. `plt.imshow(array)` labels the axes with row/column indices, so your raster will not line up with vector layers. Two fixes:

- `rasterio.plot.show(src, ax=ax)` — reads the transform and does it correctly.
- `plt.imshow(arr, extent=rasterio.plot.plotting_extent(src))` — pass the extent (`left, right, bottom, top`) explicitly.

Sampling caveat — the raster value is a cell average. Sampling at a point returns the value of the cell that contains it. For a 250 m rainfall grid, that is the average over 6.25 hectares, not a point measurement. When you need a smoother estimate, resample bilinearly (Lesson I13) or take a focal mean.

Expected outcome. A verified round-trip between index and coordinate space, elevation and rainfall attached to all 24 sensor stations, and a correctly georeferenced plot with vector layers on top.

What the next cell does: demonstrates index↔coordinate conversion, samples four rasters at the station points in one pass, validates the sampled elevation against the elevation column already in the CSV, and produces a properly aligned DEM map.

```

In [16]: from rasterio.plot import plotting_extent

with rasterio.open(RAS / "dem_25m.tif") as src:
    dem = src.read(1, masked=True)
    dem_transform, dem_crs, dem_extent = src.transform, src.crs, plotting_extent(src)

    # --- 1. index <=> coordinate round trip -----
    row, col = 700, 900
    x, y = src.xy(row, col)                    # cell CENTRE in map coordinates

```

```

r2, c2 = src.index(x, y) # and back again
print("INDEX <=> COORDINATE")
print(f"  array index (row={row}, col={col})")
print(f"    -> map coords  ({x:,.1f}, {y:,.1f})  [EPSG:32633, metres]")
print(f"    -> back to idx (row={r2}, col={c2})    "
      f"'round trip OK' if (r2, c2) == (row, col) else 'MISMATCH'")
print(f"    -> value      {dem[row, col]:,.1f} m")

# manual arithmetic, to prove there is no magic
a, e, c0, f0 = src.transform.a, src.transform.e, src.transform.c, src.transform.f
print(f"\n  by hand: x = c + (col+0.5)*a = {c0:,.0f} + {col+0.5}*{a} = {c0+
(col+0.5)*a:,.1f}")
print(f"              y = f + (row+0.5)*e = {f0:,.0f} + {row+0.5}*{e} = {f0+
(row+0.5)*e:,.1f}")

# --- 2. Sample several rasters at the station points -----
coords = [(p.x, p.y) for p in stations.geometry]

samples = {}
for name, fn, band in [("elev_m", "dem_25m.tif", 1),
                       ("rain_mm", "rainfall_annual_250m.tif", 1),
                       ("ndvi", "ndvi_50m.tif", 1),
                       ("landcover", "landcover_25m.tif", 1),
                       ("popdens", "popdens_100m.tif", 1)]:
    with rasterio.open(RAS / fn) as src:
        vals = np.array([v[band - 1] for v in src.sample(coords)], dtype="float64")
        vals[vals == src.nodata] = np.nan # honour NoData!
        samples[name] = vals

sampled = stations[["station_id", "station_type", "elevation_m"]].copy()
for k, v in samples.items():
    sampled[k] = v
legend = pd.read_csv(TAB / "landcover_legend.csv").set_index("class_code")["landuse_class"]
sampled["landcover_label"] = sampled["landcover"].map(legend)

print("\nRASTER VALUES SAMPLED AT THE 24 STATIONS (first 8)")
print(sampled.head(8).to_string(index=False))

# --- 3. Validate against the elevation already stored in the CSV -----
err = (sampled["elev_m"] - sampled["elevation_m"]).abs()
print(f"\nValidation: |sampled elevation - CSV elevation| "
      f"max = {err.max():.2f} m, mean = {err.mean():.2f} m")
print("  (small differences are expected: the CSV stored the value at generation time)")

# --- 4. A CORRECTLY georeferenced plot -----
fig, axes = plt.subplots(1, 2, figsize=(14, 5.8))

axes[0].imshow(dem, cmap="terrain")
axes[0].set_title("WRONG: plt.imshow(array)\naxes are row/col indices",
                  loc="left", fontsize=10, weight="bold", color="crimson")
axes[0].set_xlabel("column"); axes[0].set_ylabel("row")

im = axes[1].imshow(dem, cmap="terrain", extent=dem_extent)
districts.boundary.plot(ax=axes[1], color="black", linewidth=0.35)
rivers.plot(ax=axes[1], color="#1f6fb4", linewidth=1.0)
stations.plot(ax=axes[1], color="red", markersize=22, edgecolor="white", linewidth=0.5)
axes[1].set_title("RIGHT: extent=plotting_extent(src)\nvectors align perfectly",
                  loc="left", fontsize=10, weight="bold", color="darkgreen")
axes[1].set_aspect("equal"); axes[1].set_xticks([]); axes[1].set_yticks([])
plt.colorbar(im, ax=axes[1], shrink=0.8, label="elevation (m)")
plt.tight_layout(); plt.show()

```

Explanation.

- `src.xy(row, col)` returns the **centre** of the cell. If you need a corner, use `src.xy(row, col, offset="ul")`. Forgetting the half-cell offset introduces a systematic half-pixel shift — 12.5 m on this DEM — which matters when you are comparing two grids of different resolution.
- `src.index(x, y)` floors to the containing cell. Coordinates outside the raster return out-of-range indices **without raising**, so validate before indexing.
- The "by hand" block reproduces `src.xy` from the six affine numbers. Do this once and the transform stops being mysterious.
- `src.sample(coords)` is a **generator** yielding one array per point (length = band count). It reads only the blocks it needs, so it is efficient even on huge rasters. Because it returns tuples, we take `v[band-1]`.
- `vals[vals == src.nodata] = np.nan` — `sample()` does *not* apply the mask. Points over the sea return -9999 and will quietly become "the lowest elevation in your dataset". This one line is the difference between a working feature and a poisoned one.
- Mapping `landcover` codes to labels via the legend CSV turns an opaque integer into an interpretable category — do this immediately, not at report time.
- The two panels show the plotting trap. The left axes run 0–1920 and 0–1440 (pixels); the right axes are in metres and the districts, rivers and stations overlay exactly. If your vectors ever appear as a tiny cluster in one corner of a raster, you forgot `extent=`.

Expected output.

```
INDEX <-> COORDINATE
array index (row=700, col=900)
  -> map coords (422,512.5, 4,618,487.5) [EPSG:32633, metres]
  -> back to idx (row=700, col=900) round trip OK
  -> value      119.7 m

by hand: x = c + (col+0.5)*a = 400,000 + 900.5*25.0 = 422,512.5
         y = f + (row+0.5)*e = 4,636,000 + 700.5*(-25.0) = 4,618,487.5
```

A sample table beginning with `ST001 ... elev_m 169.2, rain_mm 634.7, ndvi 0.581, landcover 3.0 -> Cropland, popdens 1793`. Coastal stations show `elev_m` of a few metres with `Built-up/Wetland` cover; upland stations show 400–750 m with `Forest/Shrubland`. Rainfall ranges roughly **480–1 050 mm**, tracking elevation — the orographic effect built into the data.

Validation error should be **< 0.1 m**. Then two panels: pixel-indexed (wrong) and metre-indexed with perfectly aligned vectors (right).

B14 — Writing data out

What we are going to learn. How to save vector layers, rasters and tables in the formats you will actually be asked for.

Why it matters. An analysis nobody can open is not an analysis. Format choice also has real consequences: a shapefile will silently truncate your carefully named columns to 10 characters and convert your booleans to strings.

The concept — matching format to purpose.

Purpose	Format	Call
Your own intermediate results	GeoPackage or GeoParquet	<code>gdf.to_file(p, layer=..., driver="GPKG") / gdf.to_parquet(p)</code>
Sharing with a GIS user	GeoPackage	same
Web / API	GeoJSON (EPSG:4326!)	<code>gdf.to_crs(4326).to_file(p, driver="GeoJSON")</code>
A colleague who insists	Shapefile	<code>gdf.to_file(p, driver="ESRI Shapefile")</code> — and check the field names
A derived raster	GeoTIFF	<code>rasterio.open(p, "w", **profile)</code>
Non-spatial summary	CSV	<code>df.to_csv(p, index=False)</code>

Writing a raster — the profile pattern. Copy the source profile, update what changed, write. This guarantees the output is georeferenced identically:

```
with rasterio.open(src_path) as src:
    profile = src.profile
    profile.update(dtype="float32", count=1, nodata=-9999,
                  compress="deflate", tiled=True)
    with rasterio.open(out_path, "w", **profile) as dst:
        dst.write(new_array.astype("float32"), 1)
```

Compression is free money. `compress="deflate"` with `predictor=2` (integers) or `predictor=3` (floats) routinely halves GeoTIFF size at negligible CPU cost. `tiled=True` makes windowed reads fast.

Expected outcome. Four output files written to `data/outputs/`, plus a demonstration of what a shapefile does to your column names.

What the next cell does: writes a derived GeoPackage layer, a GeoJSON in EPSG:4326, a CSV summary and a derived GeoTIFF (elevation above the district mean); then round-trips a layer through a shapefile to show the field-name truncation.

```
In [17]: # --- 1. GeoPackage (best default) -----
out_gpkg = OUT / "module1_results.gpkg"
dist_stats_out = dist_stats[["district_id", "name", "district_type", "area_km2",
                             "population", "total", "pop_per_facility", "geometry"]]
dist_stats_out = dist_stats_out.rename(columns={"total": "n_facilities"})
dist_stats_out.to_file(out_gpkg, layer="district_facility_stats", driver="GPKG")
print(f"GeoPackage : {out_gpkg.name} ({out_gpkg.stat().st_size/1024:.0f} KB)")

# --- 2. GeoJSON for the web - ALWAYS reproject to 4326 -----
out_json = OUT / "stations_wgs84.geojson"
stations.to_crs(CRS_WGS84).to_file(out_json, driver="GeoJSON")
print(f"GeoJSON : {out_json.name} ({out_json.stat().st_size/1024:.0f} KB)")

# --- 3. Plain CSV summary -----
out_csv = OUT / "district_summary.csv"
dist_stats_out.drop(columns="geometry").to_csv(out_csv, index=False)
print(f"CSV : {out_csv.name} ({out_csv.stat().st_size/1024:.0f} KB)")

# --- 4. A derived GeoTIFF: elevation relative to the regional mean -----
out_tif = OUT / "dem_anomaly_25m.tif"
with rasterio.open(RAS / "dem_25m.tif") as src:
    arr = src.read(1, masked=True)
    anomaly = (arr - arr.mean()).astype("float32")
    profile = src.profile.copy()
    profile.update(dtype="float32", count=1, nodata=-9999.0,
                  compress="deflate", predictor=3, tiled=True)
    with rasterio.open(out_tif, "w", **profile) as dst:
```

```

        dst.write(anomaly.filled(-9999.0), 1)
        dst.set_band_description(1, "Elevation anomaly vs regional mean (m)")
    print(f"GeoTIFF      : {out_tif.name} ({out_tif.stat().st_size/1e6:.2f} MB)")

    # verify it reads back correctly
    with rasterio.open(out_tif) as chk:
        back = chk.read(1, masked=True)
        print(f"          read back: shape {back.shape}, "
              f"          f'mean {back.mean():.3f} (should be ~0), crs {chk.crs}")

    # --- 5. What a SHAPEFILE does to your schema -----
    print("\n" + "=" * 78)
    print("SHAPEFILE FIELD-NAME TRUNCATION")
    print("=" * 78)
    demo = dist_stats_out.rename(columns={
        "district_type": "district_classification_type",
        "pop_per_facility": "population_per_facility_ratio",
        "n_facilities": "number_of_public_facilities",
    })
    shp_path = OUT / "shapefile_demo" / "districts.shp"
    shp_path.parent.mkdir(exist_ok=True)
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        demo.to_file(shp_path, driver="ESRI Shapefile")

    back_shp = gpd.read_file(shp_path)
    print(pd.DataFrame({"written": demo.columns[:-1],
                       "read back from .shp":
                           list(back_shp.columns[:-1])}).to_string(index=False))
    print(f"\nFiles created for ONE shapefile: "
          f"{sorted(p.suffix for p in shp_path.parent.glob('districts.*'))}")

    # --- 6. And the modern alternative -----
    try:
        out_pq = OUT / "districts.parquet"
        dist_stats_out.to_parquet(out_pq)
        print(f"\nGeoParquet : {out_pq.name} ({out_pq.stat().st_size/1024:.0f} KB) "
              f"    - long column names preserved, ~10x faster to read than GeoJSON")
    except ImportError:
        print("\nGeoParquet : skipped (pip install pyarrow to enable it)")

```

Explanation.

- `to_file(path, layer=..., driver="GPKG")` — writing to an existing GeoPackage **adds** a layer; writing the same layer name again replaces it. This makes a GeoPackage an excellent project database: one file, many results, versioned by layer name.
- `stations.to_crs(CRS_WGS84).to_file(..., driver="GeoJSON")` — the GeoJSON specification (RFC 7946) mandates WGS 84 lon/lat. GeoPandas will happily write projected coordinates into a GeoJSON, producing a file that every web map will misplace. **Always reproject first.**
- The raster block shows the profile pattern. Note `anomaly.filled(-9999.0)`: a MaskedArray must be converted back to a plain array *with the sentinel re-inserted* before writing, otherwise the mask is lost and NoData cells are written as whatever garbage was underneath.
- `predictor=3` is the floating-point predictor; use `predictor=2` for integers and omit it for already-compressed data. Combined with `compress="deflate"` it typically halves the file.
- `set_band_description` — self-documenting rasters. Anyone opening your GeoTIFF in QGIS sees "Elevation anomaly vs regional mean (m)" instead of "Band 1".
- **The shapefile demo is the punchline.** `district_classification_type` becomes `district_c`, `population_per_facility_ratio` becomes `populatio`, and `number_of_public_facilities` becomes `number_of`. Truncation can even produce *collisions*, in which case GDAL appends digits. A shapefile also cannot store a true NULL in a numeric field, cannot exceed 2 GB, and needs 4–6 sidecar files that must travel together.

- GeoParquet is the modern intermediate format: columnar, compressed, preserves dtypes and long names, and reads roughly an order of magnitude faster than GeoJSON.

Expected output.

```
GeoPackage : module1_results.gpkg (120 KB)
GeoJSON    : stations_wgs84.geojson (9 KB)
CSV        : district_summary.csv (1 KB)
GeoTIFF    : dem_anomaly_25m.tif (5.20 MB)
            read back: shape (1440, 1920), mean 0.000 (should be ~0), crs EPSG:32633
GeoParquet : districts.parquet (33 KB)
```

Then the shapefile schema comparison:

written	read back from .shp
district_id	district_i
name	name
district_classification_type	district_c
area_km2	area_km2
population	population
number_of_public_facilities	number_of_
population_per_facility_ratio	populati_1

Look at the last row. `population_per_facility_ratio` truncates to `populati...`, which **collides** with an earlier field, so GDAL renamed it `populati_1`. Your column is now called something you never chose, and a script that reads the file by name breaks. This is not a hypothetical: it is what happens every time somebody emails you a shapefile.

Finally, the five sidecar files `['.cpg', '.dbf', '.prj', '.shp', '.shx']` — lose the `.prj` and the CRS is gone; lose the `.dbf` and the attributes are gone.

Exercises — Module 1 (Beginner)

Attempt these before looking at the **Solutions** section at the end of the notebook. Each one is answerable with material from B1–B14 alone.

Exercise 1.1 — Layer inventory

Objective. Produce a summary table of every vector layer available in the project (all 12 GeoPackage layers plus the two GeoJSON files), with columns: `source`, `layer`, `n_features`, `geometry_type`, `crs`, `n_columns`, and the area (km²) or length (km) as appropriate for the geometry type. All measurements must be in EPSG:32633.

Exercise 1.2 — Attribute audit

Objective. Write a reusable function `audit(gdf, name)` that reports, for any layer: row count, column count, per-column dtype, count and percentage of missing values, number of unique values, and — for numeric columns — min,

median, max. Run it on `census_blocks` and identify **every** column with a data-quality problem. State what you think each problem is.

Exercise 1.3 — CRS forensics

Objective. For the `protected_areas.geojson` layer: 1. Report its CRS and total area computed *in its native CRS*. 2. Compute the total area in EPSG:32633, EPSG:3857 and ESRI:54009. 3. Report the percentage error of each versus the UTM value. 4. Explain, in two sentences, which value you would publish and why.

Exercise 1.4 — Building geometries from a dirty CSV

Objective. Load `flood_incidents.csv` and build a GeoDataFrame in EPSG:32633. The file contains coordinate errors on purpose. Without hard-coding row numbers, identify and quarantine: * rows at (0, 0), * rows whose lon/lat are swapped, * rows outside the study region.

Report how many rows you kept, how many you quarantined, and which rule caught each one. (*Hint: the study area's WGS 84 bounds are approximately lon 13.6–14.3, lat 41.5–41.9.*)

Exercise 1.5 — Shapely reasoning

Objective. For the district named `Marnvik`: 1. What is its area, perimeter and "compactness" (Polsby–Popper score, $4\pi A / P^2$ — 1.0 is a perfect circle)? 2. Which districts share a boundary with it? (Use `touches`.) 3. Does its centroid lie inside the polygon? Compare `.centroid` with `.representative_point()`. 4. By how many square kilometres does a 1 km inward buffer shrink it, and what percentage of the original is that?

Exercise 1.6 — Distance profile

Objective. For every one of the 24 sensor stations compute the distance to: the coastline, the nearest river, the nearest primary-or-better road, and the nearest hospital. Produce a tidy DataFrame and answer: which station is the most *remote* by the sum of all four distances, and which is the most connected?

Exercise 1.7 — Spatial join and rate calculation

Objective. Join `bus_stops` to `census_blocks` and compute, per district: number of bus stops, total population, and **bus stops per 10 000 residents**. Map the result with an explicit classification scheme and explicit NaN handling. Which district type is worst served, and is the raw count or the rate the more honest statistic to publish?

Exercise 1.8 — Raster sampling

Objective. Sample `dem_25m.tif`, `rainfall_annual_250m.tif` and `ndvi_50m.tif` at the centroid of every census block. Then: 1. Report how many blocks returned NoData for each raster, and why. 2. Fit and report the simple linear

regression `rainfall ~ elevation`. 3. Compare the fitted slope with the true generating coefficient (0.62 mm/m, stated in the scenario). How close did you get, and why is it not exact?

Module 2 — Intermediate: Real Spatial Analysis

Module 1 taught you to *handle* spatial data. Module 2 teaches you to *analyse* it — and, just as importantly, to clean it first. Every technique here appears again in the capstone.

The 16 lessons

#	Lesson	Core skill
I1	Choosing a CRS for measurement	Quantified distortion; when UTM is wrong
I2	Invalid geometries and how to repair them	<code>is_valid</code> , <code>explain_validity</code> , <code>make_valid</code>
I3	Missing values, sentinels and dirty categories	The full cleaning pipeline
I4	Buffer analysis	Fixed, variable and dissolved buffers
I5	Overlay operations	intersection, union, difference, symmetric difference
I6	Clipping	<code>gpd.clip</code> vs <code>gpd.overlay</code> , and when each is right
I7	Spatial joins in depth	predicates, cardinality, <code>sjoin_nearest</code>
I8	Aggregating spatial statistics	area-weighted aggregation, apportionment
I9	Nearest-neighbour analysis	<code>sjoin_nearest</code> , k-NN, distance bands
I10	Dissolve and hierarchical aggregation	<code>dissolve</code> , <code>aggfunc</code> , topology
I11	The spatial index and performance	R-trees, <code>sindex.query</code> , why joins are fast
I12	Raster masking and clipping	<code>rasterio.mask</code> , windows, cropping
I13	Raster resampling and reprojection	<code>Resampling</code> , <code>WarpedVRT</code> , alignment
I14	Reclassification and band maths	NDVI, slope, aspect, hillshade
I15	Zonal statistics	Raster values summarised by polygon
I16	Rasterize / polygonize and analytical map design	Vector ↔ raster round trip

I0 — Load the full project

What we are about to do. Load every layer we will need in Modules 2–4 into a consistent, named set of variables, all in the analysis CRS.

Why it matters. From here on, lessons build on one another. A single loading cell that guarantees every layer is in EPSG:32633 removes an entire class of bug and makes the rest of the notebook re-runnable from this point.

Concept — the "load and normalise" boundary. Professional spatial pipelines have a hard boundary between *ingestion* (read, reproject, rename, type-cast) and *analysis*. Everything upstream of the boundary is allowed to be messy; nothing downstream is. We are drawing that boundary here.

What the next cell does: loads all 12 GeoPackage layers, the two GeoJSON files and the four CSVs, reprojects everything to `CRS_UTM`, and prints a summary so you can confirm CRS consistency at a glance.

```

In [18]: # ---- vector layers -----
districts = gpd.read_file(GPKG, layer="districts")
blocks = gpd.read_file(GPKG, layer="census_blocks")
landuse = gpd.read_file(GPKG, layer="landuse")
rivers = gpd.read_file(GPKG, layer="rivers")
flood = gpd.read_file(GPKG, layer="flood_zones")
buildings = gpd.read_file(GPKG, layer="buildings")
facilities = gpd.read_file(GPKG, layer="facilities")
routes = gpd.read_file(GPKG, layer="transit_routes")
stops = gpd.read_file(GPKG, layer="bus_stops")
sea = gpd.read_file(GPKG, layer="sea")
land = gpd.read_file(GPKG, layer="land_boundary")
coastline = gpd.read_file(GPKG, layer="coastline")

roads = gpd.read_file(VEC / "roads.geojson").to_crs(CRS_UTM)
protected = gpd.read_file(VEC / "protected_areas.geojson").to_crs(CRS_UTM)

# ---- tabular -----
socio = pd.read_csv(TAB / "district_socioeconomic.csv")
readings = pd.read_csv(TAB / "sensor_readings.csv", parse_dates=["date"])
incidents_raw = pd.read_csv(TAB / "flood_incidents.csv", parse_dates=["date"])
lc_legend = pd.read_csv(TAB / "landcover_legend.csv")

stations_df = pd.read_csv(TAB / "sensor_stations.csv")
stations = gpd.GeoDataFrame(
    stations_df,
    geometry=gpd.points_from_xy(stations_df.lon, stations_df.lat),
    crs=CRS_WGS84).to_crs(CRS_UTM)

# ---- convenience single geometries -----
LAND_GEOM = land.geometry.iloc[0]
SEA_GEOM = sea.geometry.iloc[0]
COAST_GEOM = coastline.geometry.iloc[0]

VECTORS = {
    "districts": districts, "blocks": blocks, "landuse": landuse,
    "rivers": rivers, "flood": flood, "buildings": buildings,
    "facilities": facilities, "routes": routes, "stops": stops,
    "roads": roads, "protected": protected, "stations": stations,
}

print(f"{'layer':<12}{'rows':>7}{'cols':>6} {'crs':<12}{'geometry types':<28}"
      f"{'empty':>6}{'invalid':>8}")
print("-" * 84)
for name, g in VECTORS.items():
    print(f"{'name':<12}{'len(g)':>7}{'g.shape[1]':>6} {g.crs.to_string():<12}"
          f"{'str(dict(g.geom_type.value_counts()))':<28}"
          f"{'int(g.geometry.is_empty.sum()):>6}{'int((~g.geometry.is_valid).sum()):>8}")

print("-" * 84)
print(f"All in one CRS: {len({g.crs.to_string() for g in VECTORS.values()}) == 1}")
print(f"\nTabular: socio {socio.shape}, readings {readings.shape}, "
      f"incidents {incidents_raw.shape}, legend {lc_legend.shape}")

```

Explanation.

- Everything is read once and reprojected at the door. `roads` and `protected` arrive in EPSG:4326 and are converted immediately; nothing downstream ever needs to think about it again.
- `VECTORS` is a dictionary of the layers so we can iterate over them for audits. This is a small but high-leverage habit: any check you write once (CRS consistency, validity, empty geometry) now runs on every layer for free.
- The summary line `All in one CRS: True` is computed from a set comprehension over the CRS strings. Assert this in production code — a mixed-CRS project is a wrong-answer generator.

- `parse_dates=["date"]` on the two time-stamped CSVs — otherwise `date` is a string and every temporal operation silently does lexicographic comparison.
- `incidents_raw` is deliberately named `_raw`: it still contains the swapped coordinates and null-island rows. We clean it in I3.

Expected output. A 12-row table. Note in particular:

- every `crs` reads `EPSG:32633`, and `All in one CRS: True`;
- `landuse` shows **3 invalid** geometries;
- `blocks` shows **1 empty** geometry;
- `protected` shows a mixture `{'Polygon': 4, 'MultiPolygon': 1}` — the layer deliberately mixes geometry types;
- all other layers are clean.

Those three anomalies are the subject of the next two lessons.

I1 — Choosing a CRS for accurate distance and area

What we are going to learn. How to select a measurement CRS defensibly, and how to quantify the error of the alternatives.

Why it matters. In B5 we saw that Web Mercator inflates area by 80%. But "use UTM" is not a universal answer either — UTM is wrong for a study area spanning several zones, and wrong at high latitudes. You need a decision rule.

The concept — a decision procedure.

Situation	Use	Why
Region inside one UTM zone (< ~500 km east–west)	UTM zone (EPSG:326xx / 327xx)	Conformal, scale error < 1/2 500
Country or region spanning zones	A national grid (British National Grid, Lambert-93, ...) or a custom Lambert Conformal Conic	Designed for that footprint
Area statistics over a large region	An equal-area projection (Albers Equal Area with local standard parallels, Lambert Azimuthal Equal Area)	Area is preserved exactly
Distance from one origin	Azimuthal Equidistant centred on that origin	Distances from the centre are exact
Global work	Equal Earth / Mollweide (area), or compute geodesically	No projection is good everywhere
Web display only	Web Mercator	Tiles

Concept — scale factor. A conformal projection preserves angles but scales distance by a factor `k` that varies with position. For UTM, `k = 0.9996` on the central meridian and rises to ≈ 1.00097 at the zone edge. Areas scale as `k2`. So worst-case UTM area error inside a zone is about $\pm 0.2\%$ — negligible for most work, and three hundred times better than Web Mercator.

The practical rule. Compute the same quantity in two independent projections. If they agree, you are fine. If they disagree, you have chosen badly. This is the projection equivalent of a unit test.

Expected outcome. A table of the same three measurements (area, length, distance) across five CRS, with errors relative to a geodesic ground truth.

What the next cell does: measures district area, river length and a long point-to-point distance in five different CRS, computes geodesic ground truth with `pyproj.Geod`, and reports the error of each.

```

In [19]: from pyproj import Geod
geod = Geod(ellps="WGS84")

CANDIDATES = {
    "EPSG:32633 UTM 33N (correct zone)": "EPSG:32633",
    "EPSG:32632 UTM 32N (wrong zone)": "EPSG:32632",
    "EPSG:3857 Web Mercator": "EPSG:3857",
    "ESRI:54009 Mollweide (equal area)": "ESRI:54009",
    "ESRI:54034 Cylindrical Equal Area": "ESRI:54034",
}

# ----- ground truth on the ellipsoid -----
poly_ll = districts.to_crs(CRS_WGS84).geometry.union_all()
true_area_km2 = abs(geod.geometry_area_perimeter(poly_ll)[0]) / 1e6

riv_ll = rivers.to_crs(CRS_WGS84).geometry.iloc[0]
true_len_km = geod.geometry_length(riv_ll) / 1000

p1 = districts.to_crs(CRS_WGS84).geometry.iloc[0].representative_point()
p2 = districts.to_crs(CRS_WGS84).geometry.iloc[-1].representative_point()
true_dist_km = geod.inv(p1.x, p1.y, p2.x, p2.y)[2] / 1000

# ----- the same three measurements in each candidate CRS -----
rows = []
for label, crs_code in CANDIDATES.items():
    d = districts.to_crs(crs_code)
    r = rivers.to_crs(crs_code)
    a = d.geometry.union_all().area / 1e6
    L = r.geometry.iloc[0].length / 1000
    q1 = d.geometry.iloc[0].representative_point()
    q2 = d.geometry.iloc[-1].representative_point()
    D = q1.distance(q2) / 1000
    rows.append({
        "CRS": label,
        "area km2": round(a, 1),    "area err %": round(100*(a-true_area_km2)/true_area_km2, 2),
        "river km": round(L, 2),    "len err %": round(100*(L-true_len_km)/true_len_km, 2),
        "dist km": round(D, 2),     "dist err %": round(100*(D-true_dist_km)/true_dist_km, 2),
    })

print("GEODESIC GROUND TRUTH (computed on the WGS84 ellipsoid)")
print(f" total district area : {true_area_km2:,.1f} km^2")
print(f" Vallmara River len : {true_len_km:,.2f} km")
print(f" cross-basin distance: {true_dist_km:,.2f} km\n")
print(pd.DataFrame(rows).to_string(index=False))

# ----- what the UTM scale factor actually is here -----
from pyproj import CRS as PCRS, Transformer
lons = np.array([13.9, 14.14, 14.4])
tr = Transformer.from_crs(CRS_WGS84, CRS_UTM, always_xy=True)
print("\nUTM 33N point scale factor across the study area "
      "(central meridian = 15 deg E):")
for lon in lons:
    x, y = tr.transform(lon, 41.72)
    # numerical estimate of scale: length of a 1 km geodesic vs its projected length
    lon2, lat2, _ = geod.fwd(lon, 41.72, 90, 1000)
    x2, y2 = tr.transform(lon2, lat2)
    k = np.hypot(x2 - x, y2 - y) / 1000
    print(f" lon {lon:5.2f} deg -> k = {k:.6f} "
          f"({(k-1)*1e6:+.0f} ppm, area factor k^2 = {k**2:.6f})")

```

Explanation.

- `geod.geometry_area_perimeter(geom)` and `geod.geometry_length(geom)` compute **geodesic** area and length directly on the ellipsoid from lon/lat geometry. This is the ground truth: no projection, no distortion. It is slower

than planar arithmetic, which is why we do not use it for every operation — but it is exactly the right tool for *validating* a projection choice.

- `geod.inv(lon1, lat1, lon2, lat2)` returns `(forward_azimuth, back_azimuth, distance_m)`. The third element is the geodesic distance.
- **EPSG:32632 (UTM zone 32N) is included on purpose.** It is the *adjacent* zone — an easy mistake if you compute the zone from the wrong corner of your data. Watch its error: still small, but several times worse than the correct zone, and it grows as you move east.
- **Mollweide and Cylindrical Equal Area** both preserve area, so their area errors are near zero — but look at their *length* and *distance* errors, which are large. Equal-area projections badly distort shape and distance. There is no single "accurate" CRS; there is only "accurate for the quantity you are measuring".
- The scale-factor block measures `k` empirically: project a 1 km geodesic and see how long it comes out. At the western edge of the basin `k` is around 0.9997 and at the eastern edge around 0.9998, i.e. errors of a few hundred **parts per million**. That is 20–30 cm per kilometre — irrelevant for policy analysis, potentially relevant for cadastral surveying.

Expected outcome.

GEODESIC GROUND TRUTH (computed on the WGS84 ellipsoid)
 total district area : 1,395.7 km²
 Vallmara River len : 43.95 km
 cross-basin distance: 33.12 km

CRS	area km ²	area err	river km	len err	dist km	dist err
UTM 33N (correct)	1 394.8	−0.07%	43.93	−0.03%	33.09	−0.10%
UTM 32N (wrong zone)	1 400.9	+0.37%	44.03	+0.19%	32.74	−1.16%
Web Mercator	2 507.0	+79.6%	58.81	+33.8%	44.39	+34.0%
Mollweide (equal area)	1 396.8	+0.08%	44.60	+1.49%	31.71	−4.26%
Cylindrical Equal Area	1 395.7	−0.00%	51.40	+16.97%	40.27	+21.59%

and empirical scale factors:

```
lon 13.90 deg -> k = 0.999702  (−298 ppm, area factor k^2 = 0.999404)
lon 14.14 deg -> k = 0.999662  (−338 ppm)
lon 14.40 deg -> k = 0.999630  (−370 ppm)
```

Read the table row by row:

- **UTM 33N** is the only CRS with all three errors under 0.1%.
- **UTM 32N**, the *adjacent* zone, is 5–10× worse. Picking the zone from the wrong corner of your bounding box is a real and easy mistake.
- **Web Mercator** inflates area by 79.6% and *both* length and distance by 34% — exactly $1/\cos(41.7^\circ) = 1.34$, with area going as the square.
- **Cylindrical Equal Area** nails area to −0.00% and is catastrophically wrong about length (+17%) and distance (+22%).

That last row is the real lesson: **there is no "accurate CRS", only a CRS that is accurate for the quantity you are measuring**. An equal-area projection is perfect for a density map and useless for a service-area analysis.

The scale factors show `k ≈ 0.9997`, i.e. UTM is quietly shrinking every distance by about 300 parts per million — 30 cm per kilometre. Irrelevant for policy analysis, relevant for surveying. Knowing the number is what lets you say which.

12 — Invalid geometries and how to repair them

What we are going to learn. What makes a geometry invalid, how to diagnose it, and the three repair strategies.

Why it matters. An invalid polygon is a landmine. It may sit quietly in your data for weeks and then blow up an overlay with `TopologyException: found non-noded intersection`, or — worse — return a *silently wrong* area. Validity checking is the first thing you do to any polygon layer you did not create yourself.

The concept — the OGC validity rules for a polygon.

1. Rings must be **closed** (first vertex = last vertex).
2. Rings must be **simple** — they must not self-intersect.
3. Interior rings (holes) must lie **inside** the exterior ring.
4. Rings may touch at a **finite number of points**, never along a line.
5. The interior must be **connected** — a hole may not split the polygon in two.

The classic violation is the **bow-tie**: a four-vertex "polygon" whose edges cross, so it is really two triangles joined at a point. Its `.area` is the *difference* of the two lobes, not the sum — which is how invalid geometry produces plausible-but-wrong numbers.

The three repairs.

Method	What it does	When to use
<code>make_valid(geom)</code>	GEOS <code>MakeValid</code> : rigorous, preserves all input area , may return a <code>GeometryCollection</code> or <code>MultiPolygon</code>	The correct default
<code>geom.buffer(0)</code>	Buffers by zero, which re-nodes the rings. Fast, but silently discards parts of a bow-tie	Legacy trick; fine for tiny slivers, dangerous otherwise
<code>set_precision(geom, grid)</code>	Snaps coordinates to a grid, removing near-degenerate edges	When invalidity comes from floating-point noise

Critical: `make_valid` can change the geometry *type*. A repaired bow-tie becomes a `MultiPolygon`; a repaired polygon with a line-touching hole may become a `GeometryCollection` containing a polygon and a line. Always check the type afterwards and `explode()` or filter as needed.

Expected outcome. The three planted invalid polygons diagnosed by name, repaired three different ways, and the area differences quantified.

What the next cell does: finds the invalid geometries in `landuse`, prints GEOS's explanation of each, repairs them with all three strategies, compares the resulting areas and geometry types, and plots one bow-tie before and after.

```
In [20]: from shapely.validation import explain_validity, make_valid
         from shapely import set_precision

         # --- 1. Diagnose -----
         bad_mask = ~landuse.geometry.is_valid
         bad = landuse[bad_mask]
         print(f"Invalid geometries in `landuse`: {len(bad)} of {len(landuse)}\n")
         for i, row in bad.iterrows():
             print(f" {row['lu_id']} {row['landuse_class']:<12} -> {explain_validity(row.geometry)}")

         # --- 2. Compare the three repair strategies -----
         def try_area(fn, g):
             """Apply a repair and report its area, or why it failed."""
             try:
                 r = fn(g)
                 return f"{r.area:,.0f}", r.geom_type
             except Exception as exc:
                 return f"FAILED ({type(exc).__name__})", "-"
```

```

print("\n" + "=" * 104)
print("REPAIR COMPARISON (all areas in m^2)")
print("=" * 104)
print(f"{'lu_id':<8}{'broken .area':>14}{'make_valid':>13}{'buffer(0)':>13}"
      f"{'precision->valid':>24}{'valid->precision':>18} {'type':<14}")
print("-" * 104)
for i, row in bad.iterrows():
    g = row.geometry
    mv_a, mv_t = try_area(make_valid, g)
    b0_a, _ = try_area(lambda q: q.buffer(0), g)
    sp1, _ = try_area(lambda q: set_precision(q, 0.001), g) # wrong order
    sp2, _ = try_area(lambda q: set_precision(make_valid(q), 0.001), g) # right order
    print(f"{'row['lu_id']':<8}{g.area:>14,.0f}{mv_a:>13}{b0_a:>13}"
          f"{'sp1:>24}{sp2:>18} {mv_t:<14}")
print("-" * 104)
print("ORDER MATTERS: set_precision() on an *invalid* geometry can throw;")
print("          repair first, then snap. make_valid() -> set_precision().")

# --- 3. Why the numbers differ -----
g = bad.geometry.iloc[0]
mv = make_valid(g)
print("\nThe bow-tie, explained:")
print(f" the broken geometry reports area : {g.area:,.0f} m^2")
print(f" make_valid gives a {mv.geom_type} of {len(mv.geoms)} parts:")
for k, part in enumerate(mv.geoms):
    print(f"      part {k+1}: {part.area:,.0f} m^2")
print(f" total true area          : {mv.area:,.0f} m^2")
try:
    print(f" buffer(0) area          : {g.buffer(0).area:,.0f} m^2")
except Exception as exc:
    print(f" buffer(0)          : raised {type(exc).__name__}")
    print(f"      {str(exc)[:96]}")
    print(f"      -> on this geometry the legacy trick does not merely lose area,")
    print(f"      it fails outright. make_valid() is the only safe choice.")

# --- 4. Repair the whole layer the right way -----
landuse_fixed = landuse.copy()
landuse_fixed["geometry"] = landuse_fixed.geometry.make_valid()
# make_valid can emit GeometryCollections; keep only the polygonal parts
landuse_fixed = landuse_fixed.explode(index_parts=False, ignore_index=True)
landuse_fixed = landuse_fixed[landuse_fixed.geometry.geom_type.isin(
    ["Polygon", "MultiPolygon"])]
landuse_fixed = landuse_fixed[~landuse_fixed.geometry.is_empty]

print(f"\nLayer repair: {len(landuse)} rows -> {len(landuse_fixed)} rows "
      f"(explode split multi-part results)")
print(f" invalid remaining : {int((~landuse_fixed.geometry.is_valid).sum())}")
print(f" total area before : {landuse.geometry.area.sum()/1e6:,.3f} km^2")
print(f" total area after  : {landuse_fixed.geometry.area.sum()/1e6:,.3f} km^2")

# --- 5. Picture it -----
fig, axes = plt.subplots(1, 2, figsize=(11, 4.6))
gpd.GeoSeries([g], crs=CRS_UTM).plot(ax=axes[0], facecolor="#ffd4d4",
                                     edgecolor="crimson", linewidth=2)
axes[0].set_title(f"BROKEN bow-tie\n.area reports {g.area:,.0f} m^2",
                  fontsize=10, weight="bold", color="crimson")
gpd.GeoSeries(list(mv.geoms), crs=CRS_UTM).plot(
    ax=axes[1], facecolor="#d6ecd6", edgecolor="green", linewidth=2)
axes[1].set_title(f"make_valid() -> {mv.geom_type}, {len(mv.geoms)} parts\n"
                  f"true area {mv.area:,.0f} m^2",
                  fontsize=10, weight="bold", color="green")
for a in axes:
    a.set_aspect("equal"); a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.show()

```

Explanation.

- `explain_validity(geom)` returns a human-readable GEOS diagnostic **and the coordinate where the problem is**, e.g. `Self-intersection[413950 4609450]`. That coordinate is gold when you have to go back to the data provider.
- `landuse.geometry.make_valid()` is the vectorised GeoSeries form (GeoPandas 1.x). For a single geometry use `shapely.validation.make_valid(g)`.
- **The area comparison is the point of the lesson.** The bow-tie's broken `.area` is the *signed* sum of its two lobes — if they are similar sizes the reported area can be near zero, or even correct-looking by accident. `make_valid` returns a `MultiPolygon` with both lobes and the true total. `buffer(0)` returns only one lobe, so it under-reports by roughly half **with no warning at all**.
- `set_precision(g, 0.001)` snaps coordinates to a 1 mm grid. This does not fix a genuine bow-tie, but it is the right tool when invalidity is caused by coordinates that differ in the 12th decimal place — which is what you get after a chain of reprojections.
- `explode(index_parts=False, ignore_index=True)` splits multi-part geometries into one row each. After `make_valid` this is usually what you want, because a `MultiPolygon` carrying a repaired bow-tie is really two separate features.
- The final filter on `geom_type` removes any stray `LineString` or `Point` fragments that `make_valid` may produce when a polygon had zero-width spikes. **Skipping this filter is a common cause of "why does my polygon layer have lines in it?"**

Expected output.

```
Invalid geometries in `landuse`: 3 of 441
```

```
LU9001  Shrubland    -> Self-intersection[...]
LU9002  Shrubland    -> Self-intersection[...]
LU9003  Shrubland    -> Self-intersection[...]
```

A repair-comparison table which, for each bow-tie, reads:

	value
broken <code>.area</code>	0 m ²
<code>make_valid</code>	405 000 m ² , <code>MultiPolygon</code> of 2 parts
<code>buffer(0)</code>	202 500 m ² — exactly half
<code>set_precision</code> then <code>make_valid</code>	FAILED (<code>GEOSException</code>)
<code>make_valid</code> then <code>set_precision</code>	405 000 m ²

Stare at the first row. **The broken polygon reports an area of exactly zero**, even though it covers 40.5 hectares of ground. These bow-ties are symmetric, so the two lobes have equal and opposite signed area and they cancel exactly. Nothing raises, nothing warns; a `groupby("landuse_class").area.sum()` would simply under-report shrubland by 1.2 km² and you would never know.

`buffer(0)` returns **exactly half** the true area — it keeps one lobe and silently discards the other. And note the fourth row: calling `set_precision` *before* repairing raises a `GEOSException`. Snapping assumes a valid input. **Repair first, snap second.**

Layer repair takes 441 rows to **444** (the explode splits each repaired bow-tie into 2 parts), leaves **0 invalid**, and *increases* total area from **1 408.500 km² to 1 409.715 km²** — the 1.215 km² the broken geometries were hiding. Then a two-panel figure: a red bow-tie, and the green two-triangle repair.

Note also that the land-use layer totals ~1 408 km² while the districts total 1 394.8 km². Land use is derived from a 50 m raster and its polygons overlap the coastline slightly, so it over-covers. Whenever two layers that *should* describe the same ground disagree on total area, you have found either a topology problem or a resolution artefact. Never average the two — find out which one is authoritative.

13 — Missing values, sentinels and dirty categories

What we are going to learn. A complete, auditable cleaning pipeline for spatial data: missingness, sentinel values, impossible values, inconsistent categories, duplicate keys and bad coordinates.

Why it matters. You know how to clean tabular data. Spatial data adds two new failure modes: **coordinates can be wrong in ways that still parse**, and **joins can silently multiply or drop rows**. Both produce confident, wrong maps.

The concept — five classes of dirt, and the right response to each.

Class	Example here	Response
Explicit missing	<code>districts.population</code> NaN x2	Keep as NaN. Never fill with 0
Sentinel disguised as data	<code>capacity = -999</code> , <code>rainfall = -999</code>	Convert to NaN <i>before</i> any statistic
Impossible value	<code>year_built = 1066</code> or <code>2199</code>	Range-check against domain knowledge, then NaN
Inconsistent category	"Forest", "FOREST", " forest "	Normalise: strip, casefold, then map to a controlled vocabulary
Bad geometry / coordinates	(0,0), swapped lon/lat, out of region	Quarantine into a separate frame, never delete silently

The cardinal rule of cleaning: quarantine, do not delete. Every row you drop should land in a named "rejects" frame with a `reason` column. That frame is what you show your client when they ask "why does your total differ from ours?"

Concept — why sentinels are worse than NaN. `-999` is a perfectly valid float. `mean()`, `std()`, `groupby` and scikit-learn will all consume it happily. A NaN at least propagates visibly. Always convert sentinels at the ingestion boundary.

Expected outcome. Four cleaned datasets plus a rejects frame, with a printed audit trail of exactly what was changed and why.

What the next cell does: cleans the flood-incident CSV (bad coordinates), the sensor readings (sentinels, duplicates), the socio-economic table (duplicate and orphan keys) and the land-use categories (case/whitespace) — printing a before/ after count for every rule.

```
In [21]: audit_log = []

def log(step, before, after, note=""):
    audit_log.append({"step": step, "rows_before": before, "rows_after": after,
                     "delta": after - before, "note": note})

# ===== INCIDENTS =====
inc = incidents_raw.copy()
n0 = len(inc)
inc["reject_reason"] = pd.NA

# rule 1: null island
inc.loc[(inc.lon == 0) & (inc.lat == 0), "reject_reason"] = "null_island_(0,0)"

# rule 2: lon/lat swapped -> the pair is invalid as (lon, lat) but valid reversed
STUDY = dict(lon=(13.6, 14.5), lat=(41.4, 42.0))
in_box = lambda lo, la: (lo.between(*STUDY["lon"])) & (la.between(*STUDY["lat"]))
```

```

swapped = (~in_box(inc.lon, inc.lat)) & in_box(inc.lat, inc.lon)
inc.loc[swapped & inc.reject_reason.isna(), "reject_reason"] = "lon_lat_swapped"

# rule 3: simply outside the study region
outside = (~in_box(inc.lon, inc.lat)) & inc.reject_reason.isna()
inc.loc[outside, "reject_reason"] = "outside_study_region"

rejects = inc[inc.reject_reason.notna()].copy()
inc_ok = inc[inc.reject_reason.isna()].drop(columns="reject_reason").copy()

# rule 4: repair what is repairable - the swapped rows are recoverable
repaired = rejects[rejects.reject_reason == "lon_lat_swapped"].copy()
repaired[["lon", "lat"]] = repaired[["lat", "lon"]].to_numpy()
repaired = repaired.drop(columns="reject_reason")
inc_ok = pd.concat([inc_ok, repaired], ignore_index=True)

# rule 5: impossible attribute values
inc_ok.loc[inc_ok.damage_kvs < 0, "damage_kvs"] = np.nan

incidents = gpd.GeoDataFrame(
    inc_ok, geometry=gpd.points_from_xy(inc_ok.lon, inc_ok.lat),
    crs=CRS_WGS84).to_crs(CRS_UTM)

print("FLOOD INCIDENTS")
print(f" raw rows : {n0}")
print(f"rejects.reject_reason.value_counts().to_string().replace("\n", "\n ")")
print(f" recovered by un-swapping : {len(repaired)}")
print(f" clean rows : {len(incidents)}")
print(f" negative damages -> NaN : {int((inc.damage_kvs < 0).sum())}")
print(f" depth_cm still missing : {int(incidents.depth_cm.isna().sum())}")
log("incidents", n0, len(incidents), "quarantined bad coordinates")

# ===== SENSOR DATA =====
rd = readings.copy()
n0 = len(rd)
rd = rd.drop_duplicates(subset=["station_id", "date"], keep="first")
SENTINELS = [-999, -9999]
for col in ["pm25_ugm3", "rainfall_mm", "temp_c"]:
    hit = rd[col].isin(SENTINELS).sum()
    rd.loc[rd[col].isin(SENTINELS), col] = np.nan
    if hit:
        print(f"\nSENSOR READINGS: {col}: {hit} sentinel values -> NaN")
print(f" duplicate (station_id, date) rows removed : {n0 - len(rd)}")
print(f" remaining NaN: " +
      ", ".join(f"{c}={int(rd[c].isna().sum())}" for c in
                ["pm25_ugm3", "rainfall_mm", "temp_c"]))
print(f" MEAN RAINFALL before cleaning: {readings.rainfall_mm.mean():>9,.2f} mm <-
polluted")
print(f" after cleaning: {rd.rainfall_mm.mean():>9,.2f} mm <- correct")
readings_clean = rd
log("readings", n0, len(rd), "sentinels -> NaN, duplicates dropped")

# ===== SOCIO-ECONOMIC =====
so = socio.copy()
n0 = len(so)
dupe_ids = so.district_id[so.district_id.duplicated()].unique()
so = so.drop_duplicates(subset="district_id", keep="first")
valid_ids = set(districts.district_id)
orphans = sorted(set(so.district_id) - valid_ids)
so_ok = so[so.district_id.isin(valid_ids)].copy()
missing_ids = sorted(valid_ids - set(so_ok.district_id))
print(f"\nSOCIO-ECONOMIC TABLE")
print(f" raw rows : {n0}")
print(f" duplicated district_id keys : {list(dupe_ids)}")
print(f" orphan keys (no polygon) : {orphans}")
print(f" districts with no socio row : {missing_ids if missing_ids else 'none'}")
print(f" clean rows : {len(so_ok)} (should equal 24)")
socio_clean = so_ok

```

```

log("socio", n0, len(so_ok), "dedup + orphan removal")

# ===== LAND USE =====
lu = landuse_fixed.copy()
raw_levels = lu.landuse_class.nunique()
lu["landuse_class"] = (lu.landuse_class.str.strip().str.lower()
                       .str.replace(r"\s+", " ", regex=True))
CONTROLLED = {c.lower(): c for c in lc_legend.landuse_class}
lu["landuse_class"] = lu.landuse_class.map(CONTROLLED)
print(f"\nLAND USE CATEGORIES")
print(f" distinct labels before normalisation : {raw_levels}")
print(f" distinct labels after                  : {lu.landuse_class.nunique()}")
print(f" unmapped (NaN) after controlled map   : {int(lu.landuse_class.isna().sum())}")
# De-duplicate on (id, GEOMETRY), not on the id alone: after the make_valid /
# explode step the two halves of a repaired bow-tie legitimately share an lu_id.
n_before = len(lu)
lu = lu.assign(_wkb=lu.geometry.to_wkb())
lu = lu.drop_duplicates(subset=["lu_id", "_wkb"], keep="first").drop(columns="_wkb")
print(f" exact duplicate rows removed          : {n_before - len(lu)}")
lu["lu_id"] = [f"LU{i+1:04d}" for i in range(len(lu))] # re-issue unique ids
landuse_clean = lu
log("landuse", n_before, len(lu), "category normalisation + dedup")

# ===== FACILITIES =====
fa = facilities.copy()
n0 = len(fa)
# 2a. duplicated features: same id AND same location
fa = fa.assign(_wkb=fa.geometry.to_wkb())
fa = fa.drop_duplicates(subset=["facility_id", "_wkb"], keep="first").drop(columns="_wkb")
n_dup = n0 - len(fa)
# 2b. sentinel capacity
n_sent = int((fa.capacity == -999).sum())
fa.loc[fa.capacity == -999, "capacity"] = np.nan
print(f"\nFACILITIES")
print(f" raw rows                  : {n0}")
print(f" exact duplicate rows removed : {n_dup}")
print(f" capacity == -999 -> NaN      : {n_sent}")
print(f" MEAN CAPACITY before cleaning: {facilities.capacity.mean():>9,.1f} <- polluted")
print(f"                          after cleaning: {fa.capacity.mean():>9,.1f} <- correct")
print(f" clean rows                  : {len(fa)}")
facilities_clean = fa
log("facilities", n0, len(fa), "dedup + capacity sentinel")

# ===== BUILDINGS =====
bl = buildings.copy()
n0 = len(bl)
bad_year = (bl.year_built < 1800) | (bl.year_built > 2025)
print(f"\nBUILDINGS")
print(f" year_built outside 1800-2025 -> NaN : {int(bad_year.sum())}")
bl.loc[bad_year, "year_built"] = np.nan
print(f" MEAN year_built before : {buildings.year_built.mean():>9,.1f} <- dragged by 1066/2199")
print(f"                          after  : {bl.year_built.mean():>9,.1f}")
bl["building_age"] = 2025 - bl["year_built"]
buildings_clean = bl
log("buildings", n0, len(bl), "impossible years -> NaN")

print("\n" + "=" * 78)
print("AUDIT TRAIL")
print(pd.DataFrame(audit_log).to_string(index=False))

```

Explanation.

- **reject_reason as a column, not a filter.** Every rule stamps a reason rather than dropping the row. The rejects frame survives, so the cleaning is fully auditable and reversible.

- **The swap-detection rule is the interesting one.** We do not look for "impossible latitudes > 90". These rows have lat = 13.9, which is a perfectly legal latitude — it is just in Africa. The test is *relational*: the pair fails as (`lon`, `lat`) but succeeds when reversed. That is strong evidence of a swap, and it lets us **recover** the rows rather than discard them.
- Order matters: null-island is checked first, because (`0,0`) also fails the in-box test and would otherwise be misdiagnosed as "outside region".
- `drop_duplicates(subset=["station_id", "date"])` — de-duplicate on the **logical key**, not on all columns. Two readings for the same station and month are a data error even if some other column differs.
- **The rainfall before/after comparison is the money line.** With 45 sentinel values of -999 in 866 rows, the mean rainfall reads about **9 mm/month**; cleaned, it is about **62 mm/month**. A factor of seven, from 5% contamination.
- The socio-economic block checks the join in **both directions**: orphan keys (rows with no polygon) *and* missing keys (polygons with no row). Only checking one direction is how you end up with a choropleth that quietly omits a district.
- **The controlled vocabulary map** (`CONTROLLED`) is stricter than `.str.title()`. Anything that does not map becomes NaN and is *counted*, so a new unexpected label ("`Forestry`", say) is caught rather than silently title-cased into a new category.

Expected output.

```
FLOOD INCIDENTS
raw rows                : 393
lon_lat_swapped         6
null_island_(0,0)       4
outside_study_region    3
recovered by un-swapping : 6
clean rows              : 386
negative damages -> NaN : 8
depth_cm still missing  : 25

SENSOR READINGS: rainfall_mm: 45 sentinel values -> NaN
duplicate (station_id, date) rows removed : 2
remaining NaN: pm25_ugm3=45, rainfall_mm=45, temp_c=0
MEAN RAINFALL before cleaning:      9.52 mm <- polluted
                  after cleaning:    66.05 mm <- correct

SOCIO-ECONOMIC TABLE
duplicated district_id keys : ['D05', 'D12']
orphan keys (no polygon)    : ['D99']
districts with no socio row : none
clean rows                  : 24 (should equal 24)

LAND USE CATEGORIES
distinct labels before normalisation : 25
distinct labels after                 : 8
exact duplicate rows removed         : 2

FACILITIES
exact duplicate rows removed : 3
capacity == -999 -> NaN       : 11
MEAN CAPACITY before cleaning: 259.2 <- polluted
                  after cleaning: 448.8 <- correct

BUILDINGS
year_built outside 1800-2025 -> NaN : 18
MEAN year_built before : 1,978.4   after : 1,980.4
```

Four numbers deserve a second look.

1. **Rainfall: 9.52 → 66.05 mm/month.** Forty-five sentinel values in 866 rows — 5% contamination — moved the mean by a factor of **seven**.
2. **Capacity: 259.2 → 448.8.** Eleven **-999** values dragged the mean down by 42%. Both of these would have sailed through any pipeline that only checked for NaN.
3. **386 clean incidents from 393 raw.** 4 null-island and 3 foreign rows quarantined, but **6 swapped rows recovered**. A blanket "drop anything outside the bounding box" would have discarded all 13 and lost 6 genuine observations.
4. **Buildings: mean year 1978.4 → 1980.4.** Only a 2-year shift — because 18 bad rows out of 5 200 is 0.35%. Contamination hurts in proportion to its share. This is why you check *every* field rather than assuming the impact is small.

Finally the audit trail — six cleaning steps with row counts before and after. **That table is what you attach to the report.**

I4 — Buffer analysis

What we are going to learn. Fixed, variable and dissolved buffers, and the three ways buffering goes wrong.

Why it matters. The buffer is the fundamental proximity primitive: "*which buildings are within 250 m of a river?*", "*what land is within 1 km of a school?*", "*how much of the protected area lies within the noise corridor of the motorway?*". Almost every regulatory GIS question is a buffer question.

The concept. `geom.buffer(d)` returns every point within distance `d`. Key parameters:

Parameter	Effect
<code>distance</code>	Positive dilates; negative erodes (polygons only)
<code>resolution</code> (a.k.a. <code>quad_segs</code>)	Segments per quarter-circle. Default 8 → a 32-gon. Higher = smoother = slower
<code>cap_style</code>	<code>round</code> (1, default), <code>flat</code> (2), <code>square</code> (3) — how line ends are treated
<code>join_style</code>	<code>round</code> (1), <code>mitre</code> (2), <code>bevel</code> (3) — how corners are treated
<code>single_sided</code>	Buffer one side of a line only — useful for road verges, riparian strips

Three ways buffers go wrong.

1. **Wrong CRS.** Buffering in degrees. Covered in B5; still the most common.
2. **Overlapping buffers double-count.** 40 schools each buffered 1 km produce 40 overlapping discs. Their total `.area` is **not** the area served — you must `union_all()` (dissolve) first. This error inflates "population served" numbers routinely.
3. **Buffering then intersecting is not the same as intersecting then buffering.** Order matters; think about which one your question asks.

Concept — variable-distance buffers. Real regulations are rarely uniform: a riparian protection zone might be 50 m for a first-order stream and 200 m for a fourth-order river. GeoPandas buffers element-wise when you pass an **array** of distances, which makes this a one-liner.

Expected outcome. A riparian protection zone with river-order-dependent widths, a dissolved school catchment showing the double-counting error quantified, and a single-sided motorway verge.

What the next cell does: builds three buffer types, quantifies the overlapping-buffer error, and maps all three.

```

In [22]: # --- 1. VARIABLE-distance buffer: riparian zone scaled by Strahler order -----
WIDTH_BY_ORDER = {2: 50.0, 3: 120.0, 4: 200.0}
rivers_b = rivers.copy()
rivers_b["protect_m"] = rivers_b.strahler_order.map(WIDTH_BY_ORDER)
riparian = rivers_b.copy()
riparian["geometry"] = rivers_b.geometry.buffer(rivers_b["protect_m"].to_numpy())
riparian["zone_area_ha"] = riparian.geometry.area / 1e4

print("VARIABLE-WIDTH RIPARIAN PROTECTION ZONE")
print(riparian[["river_id", "name", "strahler_order", "protect_m",
               "length_km", "zone_area_ha"]].to_string(index=False))
print(f"\n sum of individual zone areas : {riparian.zone_area_ha.sum():>10,.1f} ha")
riparian_union = riparian.geometry.union_all()
print(f" DISSOLVED (union) area      : {riparian_union.area/1e4:>10,.1f} ha")
print(f" double-counted overlap      : {riparian.zone_area_ha.sum() - riparian_union.area/
1e4:>10,.1f} ha")

# --- 2. FIXED buffer + the double-counting trap -----
schools = facilities_clean[facilities_clean.facility_type == "school"].copy()
RADIUS = 1500.0
school_buf = schools.copy()
school_buf["geometry"] = schools.geometry.buffer(RADIUS)

naive_area = school_buf.geometry.area.sum() / 1e6
dissolved = school_buf.geometry.union_all()
true_area = dissolved.area / 1e6

print("\n" + "=" * 78)
print(f"SCHOOL CATCHMENTS: {len(schools)} schools, {RADIUS:.0f} m radius")
print("=" * 78)
print(f" naive sum of buffer areas : {naive_area:>8,.1f} km^2   <- WRONG")
print(f" dissolved (union) area    : {true_area:>8,.1f} km^2   <- correct")
print(f" inflation                 : {100*(naive_area-true_area)/true_area:>8,.1f} %")
print(f" a single 1.5 km disc is    : {np.pi*RADIUS**2/1e6:>8,.2f} km^2")

# population served, computed both ways
blocks_c = blocks[~blocks.geometry.is_empty].copy()
blocks_c["geometry_pt"] = blocks_c.geometry.representative_point()
pts = gpd.GeoDataFrame(blocks_c.drop(columns="geometry"),
                       geometry="geometry_pt", crs=CRS_UTM)
naive_pop = gpd.sjoin(pts, school_buf[["facility_id", "geometry"]],
                     predicate="within").population.sum()
true_pop = pts[pts.geometry.within(dissolved)].population.sum()
print(f"\n population 'served', naive sjoin : {naive_pop:>10,.0f}   <- counts people once per
school")
print(f" population served, dissolved      : {true_pop:>10,.0f}")
print(f" over-count                          : {naive_pop/max(true_pop,1):>10,.2f} x")

# --- 3. SINGLE-SIDED buffer: a motorway verge -----
mw = roads[roads.road_class == "motorway"].geometry.union_all()
verge_r = mw.buffer(120, single_sided=True)
verge_l = mw.buffer(-120, single_sided=True)
print(f"\nSINGLE-SIDED motorway verge (120 m):")
print(f" right side area : {verge_r.area/1e4:>10,.1f} ha")
print(f" left side area  : {verge_l.area/1e4:>10,.1f} ha")
print(f" two-sided 120 m : {mw.buffer(120).area/1e4:>10,.1f} ha")

# --- 4. Map -----
fig, axes = plt.subplots(1, 3, figsize=(16.5, 5.6))
for ax in axes:
    land.plot(ax=ax, facecolor="#f7f5ef", edgecolor="#d8d2c4", linewidth=0.5)
    ax.set_aspect("equal"); ax.set_xticks([]); ax.set_yticks([])

riparian.plot(ax=axes[0], column="strahler_order", cmap="Blues", alpha=0.75,
             legend=True, legend_kwds={"shrink": 0.55, "label": "Strahler order"})
rivers.plot(ax=axes[0], color="navy", linewidth=0.7)
axes[0].set_title("(a) Variable-width riparian zone\n50 / 120 / 200 m by river order",

```

```

loc="left", fontsize=10, weight="bold")

school_buf.plot(ax=axes[1], facecolor="#f4a582", edgecolor="#d6604d",
                alpha=0.35, linewidth=0.4)
gpd.GeoSeries([dissolved], crs=CRS_UTM).boundary.plot(ax=axes[1], color="black",
                                                    linewidth=1.0)
schools.plot(ax=axes[1], color="black", markersize=6)
axes[1].set_title("(b) 42 overlapping 1.5 km catchments\nblack = dissolved outline",
                  loc="left", fontsize=10, weight="bold")

gpd.GeoSeries([verge_r], crs=CRS_UTM).plot(ax=axes[2], facecolor="#7fbf7b", alpha=0.8)
gpd.GeoSeries([verge_l], crs=CRS_UTM).plot(ax=axes[2], facecolor="#af8dc3", alpha=0.8)
roads[roads.road_class == "motorway"].plot(ax=axes[2], color="black", linewidth=0.8)
axes[2].set_title("(c) Single-sided buffers\ngreen = right, purple = left",
                  loc="left", fontsize=10, weight="bold")
plt.tight_layout(); plt.show()

```

Explanation.

- `rivers_b.geometry.buffer(rivers_b["protect_m"].to_numpy())` — passing a NumPy array of the same length buffers **element-wise**. This is Shapely 2 vectorisation; in Shapely 1.8 you needed an `apply`. Note `.to_numpy()`: passing the Series directly can align on index rather than position, which is a subtle bug source.
- **The riparian overlap** is small (the rivers rarely run within 400 m of each other), but non-zero where they converge near the coast. Always check.
- **The school-catchment block is the important one.** 42 discs of 7.07 km² each sum to ~297 km²; the dissolved union is far smaller because the schools cluster in town. The naive sum over-states served area by a large margin.
- The **population** version of the same error is worse, because it is the number that ends up in the report. `gpd.sjoin(pts, school_buf)` returns one row per (block, school) pair, so a block inside five catchments is counted five times. The dissolved version counts each person once. In a real service-coverage study this is the difference between "we serve 1.4 million people" and "we serve 450 000 people" in a region of 600 000 — an obviously impossible number that nevertheless gets published.
- `buffer(d, single_sided=True)` offsets to one side only; the **sign** of `d` chooses the side (positive = left of the direction of travel in Shapely 2 / GEOS convention — verify empirically, as we do here, rather than trusting a remembered rule).
- Note that `verge_r.area + verge_l.area` is close to, but not equal to, the two-sided buffer: the two-sided version includes round caps at the ends and merges self-overlaps at tight bends.

Expected outcome.

```

VARIABLE-WIDTH RIPARIAN PROTECTION ZONE
RV01 Vallmara River  order 4  200 m  43.93 km  ~1,760 ha
RV02 Kestrel Brook   order 3  120 m  ...
...
sum of individual zone areas :    4,813.9 ha
DISSOLVED (union) area      :    4,217.5 ha
double-counted overlap      :        596.4 ha

SCHOOL CATCHMENTS: 42 schools, 1500 m radius
naive sum of buffer areas :    296.6 km^2  <- WRONG
dissolved (union) area    :    ~220 km^2  <- correct
inflation                  :    ~35 %
a single 1.5 km disc is   :    7.07 km^2

population 'served', naive sjoin :  ~700,000  <- counts people once per school
population served, dissolved   :  ~440,000
over-count                     :    ~1.6 x

SINGLE-SIDED motorway verge (120 m):
right side area : 540.1 ha
left side area  : 539.5 ha
two-sided 120 m : 1,084.1 ha

```

The riparian overlap is **596 ha, 12% of the naive total** — the rivers converge near the coast and their protection zones merge there.

The population figure is the one that matters: the naive spatial join claims **~700 000 people served** in a region whose entire population is 644 000. A number larger than the population is at least obviously wrong. The dangerous version is the area figure, where 296.6 km² versus ~220 km² looks plausible either way.

Then a three-panel figure. In panel (b) the overlapping discs are visibly stacked over the urban core — that visual pile-up is the double-counting.

15 — Overlay operations

What we are going to learn. The four set operations on polygon layers, and what each one does to the attributes.

Why it matters. Overlay is how you answer "*how much of X is inside Y?*" — the question behind land-use change, hazard exposure, and every impact assessment ever written.

The concept — `gpd.overlay(df1, df2, how=...)`.

how	Result geometry	Attributes	Typical question
"intersection"	Only the overlapping parts	From both layers	"What land use lies in the flood zone?"
"union"	Every piece from both, split where they overlap	Both, NaN where absent	"Give me every distinct combination"
"difference"	Parts of df1 not in df2	df1 only	"Which land is <i>outside</i> the protected area?"
"symmetric_difference"	Parts in exactly one layer	Both, NaN where absent	"Where do the two datasets disagree?"
"identity"	All of df1, split by df2	Both, NaN outside df2	"Tag df1 with df2 where it applies"

The key mental model. Overlay **splits geometry**. If a land-use polygon straddles a flood-zone boundary, **intersection** returns only the part inside, as a *new, smaller polygon*. The attributes are copied unchanged — which means **every absolute quantity must be recomputed after an overlay**. If your land-use polygon said `area_ha = 120` and the intersection kept a third of it, the attribute still says 120. Recompute, or you will over-report by 3×.

Concept — the sliver problem. Overlaying two layers digitised independently produces thousands of tiny "sliver" polygons along boundaries that *should* coincide. Filter them by area (or by a thinness ratio $4\pi A/P^2$) before analysing.

Expected outcome. A land-use × flood-zone intersection with correctly recomputed areas, plus a demonstration of all four operations on the same pair.

What the next cell does: intersects land use with the 100-year flood zone, shows what happens if you forget to recompute area, runs all four overlay modes on a simplified pair, and reports the sliver distribution.

```
In [23]: lu = landuse_clean[["lu_id", "landuse_class", "area_ha", "geometry"]].copy()
fz100 = flood[flood.return_period_yr == 100][["zone_id", "hazard_class", "geometry"]].copy()

# --- 1. INTERSECTION: which land uses are in the 100-year flood zone? -----
hit = gpd.overlay(lu, fz100, how="intersection", keep_geom_type=True)
hit["area_ha_true"] = hit.geometry.area / 1e4

print(f"land-use polygons      : {len(lu):>6}")
print(f"flood-zone polygons     : {len(fz100):>6}")
print(f"intersection pieces      : {len(hit):>6}    <- geometry was SPLIT\n")

wrong = hit.groupby("landuse_class", observed=True)["area_ha"].sum()
right = hit.groupby("landuse_class", observed=True)["area_ha_true"].sum()
cmp = pd.DataFrame({"inherited area_ha (WRONG)": wrong,
                    "recomputed area (right)": right})
cmp["over-report factor"] = (cmp.iloc[:, 0] / cmp.iloc[:, 1]).round(1)
print("LAND USE INSIDE THE 100-YEAR FLOOD ZONE")
print(cmp.sort_values("recomputed area (right)", ascending=False).round(1).to_string())
print(f"\nTOTAL inherited : {wrong.sum():>12,.0f} ha    <- nonsense")
print(f"TOTAL recomputed: {right.sum():>12,.0f} ha    "
      f"({right.sum()/100:,.1f} km^2 of the {LAND_GEOM.area/1e6:,.0f} km^2 basin)")

# --- 2. All four overlay modes on ONE pair -----
a = gpd.GeoDataFrame({"lab": ["A"]}, geometry=[districts.geometry.iloc[1]], crs=CRS_UTM)
b = gpd.GeoDataFrame({"lab": ["B"]},
                      geometry=[districts.geometry.iloc[1].centroid.buffer(4200)],
                      crs=CRS_UTM)

fig, axes = plt.subplots(1, 5, figsize=(17.5, 4))
modes = ["intersection", "union", "difference", "symmetric_difference", "identity"]
for ax, how in zip(axes, modes):
    a.boundary.plot(ax=ax, color="#3b5378", linewidth=1.2)
    b.boundary.plot(ax=ax, color="#c1666b", linewidth=1.2)
    res = gpd.overlay(a, b, how=how, keep_geom_type=False)
    res.plot(ax=ax, facecolor="#7fb3a3", edgecolor="black", alpha=0.75, linewidth=0.5)
    ax.set_title(f"{how}\n{n}{len(res)} feature(s), {res.geometry.area.sum()/1e6:,.1f} km^2",
                 fontsize=9, weight="bold")
    ax.set_aspect("equal"); ax.set_xticks([]); ax.set_yticks([])
plt.suptitle("gpd.overlay: blue = layer A (a district), red = layer B (a disc)",
             fontsize=11)
plt.tight_layout(); plt.show()

# --- 3. Slivers -----
def sliver_report(gdf, label):
    areas = gdf.geometry.area
    print(f"\nSLIVER DIAGNOSIS - {label} ({len(gdf)} pieces)")
    print(f" {'threshold':>12} {'n rows':>8} {'% rows':>8} {'% of area':>11}")
    for t in [1, 10, 100, 1_000, 10_000]:
        n = int((areas < t).sum())
        print(f" {t:>10,} m^2 {n:>8} {100*n/len(gdf):>7.1f}% "
              f"{100*areas[areas < t].sum()/areas.sum():>10.4f}%")
```

```
# (a) our real result: both layers came off the SAME 25 m raster grid
sliver_report(hit, "land use x flood zone (co-registered layers)")

# (b) what happens with two INDEPENDENTLY digitised layers. We simulate that by
#     overlaying the districts with a copy shifted by 12 m – about the accuracy
#     of two different survey campaigns.
from shapely.affinity import translate
shifted = districts[["district_id", "geometry"]].copy()
shifted["geometry"] = shifted.geometry.apply(lambda g: translate(g, 12, -12))
shifted = shifted.rename(columns={"district_id": "district_id_b"})
slivers = gpd.overlay(districts[["district_id", "geometry"]], shifted,
                     how="intersection", keep_geom_type=True)
mismatch = slivers[slivers.district_id != slivers.district_id_b]
sliver_report(mismatch, "districts x districts shifted 12 m (mis-registration)")
print(f"\n -> {len(mismatch)} spurious polygons created by a 12 m shift, "
      f"carrying {mismatch.geometry.area.sum()/1e4:,.1f} ha in total")
print(f" -> filtering to area >= 1,000 m^2 leaves "
      f"{int((mismatch.geometry.area >= 1000).sum())} of them")
```

Explanation.

- `gpd.overlay(lu, fz100, how="intersection")` computes the pairwise intersection of every land-use polygon with every flood-zone polygon that it touches. The result has **one row per intersecting pair** and carries columns from both inputs.
- `keep_geom_type=True` discards non-polygonal results. Without it, two polygons that merely *touch* contribute a zero-area LineString to your polygon layer, and every subsequent `.area` call returns 0 for those rows. **Set it explicitly; the default has changed across versions.**
- The **area_{ha} vs area_{ha_true} comparison is the lesson.** `areaha` was computed on the *whole* land-use polygon before the split, and `overlay` copied it verbatim onto each fragment. Summing it over-reports by whatever factor the polygons were cut by — here typically **3–10×**, and unboundedly more if one big polygon is cut into many pieces. Any absolute attribute (area, population, count, value) is invalid after an overlay until you recompute or apportion it. We do the apportionment properly in I8.
- `observed=True` in `groupby` — required when grouping by a categorical to avoid materialising empty categories. Harmless here, essential on real data.
- The **five-panel figure** is worth studying. `union` produces *more* features than either input, because every overlapping region becomes its own polygon. `identity` keeps all of A but splits it where B crosses — it is `intersection` plus `difference` of A, and is what you want when tagging one layer with another without losing coverage.
- **Sliver diagnosis:** count pieces below a series of area thresholds and check what fraction of *area* they represent. The characteristic sliver signature is "many rows, negligible area" — e.g. 5% of rows carrying 0.001% of area. That is your licence to drop them. If small pieces carry meaningful area, they are not slivers; they are real small features, and dropping them is a bug.

Expected output.

```
land-use polygons      :    442
flood-zone polygons    :     11
intersection pieces     :    167  <- geometry was SPLIT
```

LAND USE INSIDE THE 100-YEAR FLOOD ZONE

	inherited area_ha (WRONG)	recomputed (right)	factor
Forest	198,214.8	6,963.0	28.5
Grassland	35,773.5	3,523.0	10.2
Bare rock / sparse	8,715.1	1,957.7	4.5
Water	1,893.2	1,885.0	1.0
Built-up	10,106.6	1,792.7	5.6
Cropland	32,619.6	1,782.1	18.3
Shrubland	783.5	328.6	2.4
Wetland	833.0	277.2	3.0

```
TOTAL inherited :    288,939 ha  <- nonsense
TOTAL recomputed:    18,509 ha  (185.1 km^2 of the 1,395 km^2 basin)
```

The inherited total, 288 939 ha = **2 889 km²**, is more than twice the area of the entire basin. Any quantity that exceeds the size of your study area is a free sanity check — take it.

Note the per-class factors: **Forest over-reports 28.5x** because a few very large forest polygons are clipped to thin ribbons along the rivers, while **Water over-reports 1.0x** because water polygons lie almost entirely inside the flood zone and are barely cut at all. The distortion is not a constant you can divide out; it depends on how each polygon was clipped.

Then the five-panel overlay figure, and two sliver reports:

- **Co-registered layers** (land use × flood zone, both derived from the same 25 m grid): **zero** pieces under 100 m². Slivers are not an inevitable by-product of overlay.
- **Mis-registered layers** (districts × districts shifted 12 m): **78 spurious polygons carrying 406 ha**. That is what a 12 m disagreement between two survey campaigns costs you — and note that filtering at 1 000 m² removes only 4 of them, because most slivers along a 12 m offset are *long*, not small. **Filter slivers on thinness ($4\pi A/P^2$), not on area alone.**

I6 — Clipping: `gpd.clip` versus `gpd.overlay`

What we are going to learn. The difference between clipping and intersecting, and when each is the right tool.

Why it matters. They look interchangeable and are not. Choosing wrongly either loses the attributes you needed or explodes your row count.

The concept.

	<code>gpd.clip(gdf, mask)</code>	<code>gpd.overlay(gdf, other, how="intersection")</code>
Mask	A geometry, GeoSeries or GeoDataFrame — treated as one shape	A full GeoDataFrame
Rows out	≤ rows in . One row per <i>input</i> feature, trimmed	One row per intersecting pair — can be far more
Attributes	Only from <code>gdf</code>	From both layers
Use it when	"Cut this layer to my study area"	"Cross-tabulate these two layers"

Mental model. `clip` is a **cookie cutter**: the mask is a stencil, not a dataset. `overlay` is a **join that splits geometry**: both sides contribute attributes.

Two practical warnings.

1. `clip` dissolves the mask internally. If your mask GeoDataFrame has 24 districts, clipping does **not** tag each output feature with its district — it just trims everything to the outline of all 24 combined. If you want the tag, you need `overlay` or `sjoin`.
2. `clip` can return **mixed geometry types** — clipping a polygon layer with a mask that only grazes some polygons yields LineStrings and Points at the tangencies. Filter with `keep_geom_type=True` (GeoPandas ≥ 0.14) or manually.

Expected outcome. Roads clipped to a protected area versus overlaid with it, with the row counts and attributes compared side by side.

What the next cell does: clips the road network to the protected areas and overlays it with the same layer, compares row counts, attributes and total length, then shows the mixed-geometry-type trap.

```
In [24]: pa = protected[["pa_id", "name", "designation", "geometry"]].copy()

# --- 1. CLIP: cookie-cutter -----
roads_clipped = gpd.clip(roads, pa, keep_geom_type=True)

# --- 2. OVERLAY: attribute-carrying intersection -----
roads_overlaid = gpd.overlay(roads, pa, how="intersection", keep_geom_type=True)

print("ROADS INSIDE PROTECTED AREAS")
print("=" * 78)
print(f"  input road segments          : {len(roads):>6}")
print(f"  gpd.clip      -> rows          : {len(roads_clipped):>6}")
print(f"  gpd.overlay   -> rows          : {len(roads_overlaid):>6}")
print(f"  clip total length              : {roads_clipped.length.sum()/1000:>9.2f} km")
print(f"  overlay total length          : {roads_overlaid.length.sum()/1000:>9.2f} km")
print(f"\n  clip columns   : {list(roads_clipped.columns)[:6]} ...")
print(f"  overlay columns: {[c for c in roads_overlaid.columns if c in
                        ['road_id', 'road_class', 'pa_id', 'name_2', 'designation']] } "
      f"<- carries the RESERVE identity too")

# --- 3. Only overlay can answer 'which road in which reserve?' -----
by_pa = (roads_overlaid.assign(km=roads_overlaid.length/1000)
         .groupby(["pa_id", "road_class"], observed=True)["km"].sum()
         .unstack(fill_value=0).round(2))
print("\nRoad kilometres by reserve and class (only OVERLAY can produce this):")
print(by_pa.to_string())

# --- 4. Clip a POLYGON layer to a study sub-area -----
study = box(408_000, 4_608_000, 428_000, 4_628_000)
study_gs = gpd.GeoSeries([study], crs=CRS_UTM)

lu_clip_all = gpd.clip(landuse_clean, study_gs) # no filter
lu_clip_poly = gpd.clip(landuse_clean, study_gs, keep_geom_type=True)
print("\n" + "=" * 78)
print("THE MIXED-GEOMETRY-TYPE TRAP")
print("=" * 78)
print(f"  clip without keep_geom_type : {len(lu_clip_all):>5} rows, "
      f"types {dict(lu_clip_all.geom_type.value_counts())}")
print(f"  clip with      keep_geom_type : {len(lu_clip_poly):>5} rows, "
      f"types {dict(lu_clip_poly.geom_type.value_counts())}")
print(f"  area sum, unfiltered        : {lu_clip_all.geometry.area.sum()/1e6:>9.3f} km^2")
print(f"  area sum, filtered          : {lu_clip_poly.geometry.area.sum()/1e6:>9.3f} km^2")
print("\n  The AREA totals agree – so why does it matter? Because a mixed-type")
print("  layer breaks the moment anything assumes polygons:")
with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    for label, layer in [("unfiltered", lu_clip_all), ("filtered", lu_clip_poly)]:
        n_null_ring = int(layer.geometry.exterior.isna().sum())
```

```

try:
    layer.to_file(OUT / f"_clip_{label}.shp", driver="ESRI Shapefile")
    wmsg = "wrote .shp OK"
except Exception as exc:
    wmsg = f"write to .shp FAILED -> {type(exc).__name__}"
print(f"    {label:<11} rows with no exterior ring: {n_null_ring:>3}    {wmsg}")
print("\n keep_geom_type=True also EXPLODES GeometryCollections into their")
print(" polygon parts, which is why the filtered layer has MORE rows, not fewer.")

# --- 5. Map -----
fig, axes = plt.subplots(1, 2, figsize=(13.5, 5.8))
land.plot(ax=axes[0], facecolor="#f7f5ef", edgecolor="#d8d2c4", linewidth=0.5)
roads.plot(ax=axes[0], color="#cccccc", linewidth=0.4)
pa.plot(ax=axes[0], facecolor="#c7e9c0", edgecolor="#238b45", alpha=0.7, linewidth=0.9)
roads_clipped.plot(ax=axes[0], color="crimson", linewidth=1.1)
axes[0].set_title(f"gpd.clip -> {len(roads_clipped)} rows\nroads trimmed to the reserves",
                  loc="left", fontsize=10, weight="bold")

land.plot(ax=axes[1], facecolor="#f7f5ef", edgecolor="#d8d2c4", linewidth=0.5)
landuse_clean.plot(ax=axes[1], color="#e8e4d9", edgecolor="none")
lu_clip_poly.plot(ax=axes[1], column="landuse_class", cmap="tab10", legend=True,
                  legend_kwds={"loc": "lower left", "fontsize": 6.5},
                  edgecolor="white", linewidth=0.2)
study_gs.boundary.plot(ax=axes[1], color="black", linewidth=1.4, linestyle="--")
axes[1].set_title("gpd.clip of land use to a 20 x 20 km study box",
                  loc="left", fontsize=10, weight="bold")

for a in axes:
    a.set_aspect("equal"); a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.show()

```

Explanation.

- `gpd.clip(roads, pa)` treats the five reserve polygons as **one stencil**. The output has at most one row per input road segment, keeps only the road attributes, and answers "how much road is inside protected land?".
- `gpd.overlay(roads, pa, how="intersection")` produces one row per (road segment, reserve) pair, and carries `pa_id` / `designation` through. Only this version can answer "how much road is inside **which** reserve?" — which is the question a report actually asks.
- Note the total lengths are **identical**. The two operations cut the same geometry; they differ only in bookkeeping. (They would differ if reserves overlapped each other, in which case overlay would double-count.)
- Column-name collision: both layers have `name`, so overlay produces `name_1` and `name_2`. Rename immediately.
- `keep_geom_type=True` is doing real work in step 4. Without it, land-use polygons that merely touch the study box contribute zero-area LineStrings, and those rows carry attributes that will be counted by any subsequent `groupby` while contributing 0 to the area. You get categories that "exist" with zero extent — a confusing artefact that is hard to trace back.

Expected outcome.

```

ROADS INSIDE PROTECTED AREAS
input road segments      :      791
gpd.clip    -> rows      :        22
gpd.overlay  -> rows      :        22
clip total length       :      19.25 km
overlay total length    :      19.25 km
overlay columns: [..., 'pa_id', 'name_2', 'designation'] <- reserve identity too

```

Identical row counts and identical total length — the two operations cut the same geometry. Then the table only `overlay` can produce:

road_class	motorway	primary	secondary	track
PA02	0.00	4.29	4.43	5.99
PA04	1.99	0.00	0.00	0.00
PA90	0.00	0.00	0.00	2.54

Fyrdal Wetland Park (PA02) has 14.7 km of road through it, including 4.3 km of primary road. That is an ecological finding, and `clip` cannot state it.

Then the geometry-type trap:

```
clip without keep_geom_type : 168 rows, {'Polygon': 151, 'GeometryCollection': 16, 'LineString': 1}
clip with    keep_geom_type : 173 rows, {'Polygon': 173}
area sum, unfiltered          : 372.595 km^2
area sum, filtered           : 372.595 km^2
  unfiltered rows with no exterior ring: 17   write to .shp FAILED -> FeatureError
  filtered   rows with no exterior ring:  0   wrote .shp OK
```

Three things to notice. The **area totals agree**, so a summary statistic will not warn you. Seventeen rows have **no exterior ring**, so any code doing `geom.exterior` gets `None` and crashes downstream. And **writing the layer to a shapefile fails outright**, because a shapefile must hold a single geometry type. Note too that the filtered layer has *more* rows (173 vs 168): `keep_geom_type` explodes each `GeometryCollection` into its polygon parts rather than dropping it.

Finally two maps: red road segments inside green reserves, and a land-use map clipped to a dashed 20 × 20 km box.

17 — Spatial joins in depth

What we are going to learn. Predicate choice, join cardinality, and how to guarantee a join did what you think it did.

Why it matters. In B11 we saw a spatial join silently double a row count. Here we build the discipline that prevents it: **state the expected cardinality before you join, then assert it afterwards.**

The concept — cardinality is a design decision.

Intent	Cardinality	How to guarantee it
Tag each point with its polygon	1:1	<code>predicate="within"</code> on a <i>non-overlapping</i> polygon layer, then <code>assert len(out) == len(left)</code>
Count points per polygon	N:1, then aggregate	<code>sjoin</code> then <code>groupby(...).size()</code>
Every polygon a point falls in	1:N, intentionally	<code>predicate="intersects"</code> , keep duplicates
Attach nearest feature	1:1	<code>sjoin_nearest(..., max_distance=...)</code>

Predicate cheat-sheet for the common cases.

- **Points in polygons** → `within`. Use `intersects` only if you deliberately want boundary points matched to both neighbours.
- **Lines in polygons** → there is no single right answer. `intersects` matches a road that merely clips a corner; `within` matches only roads entirely inside. For "how much road is in this district", neither is right — you need `overlay` and length recomputation (I6).
- **Polygons to polygons** → almost always `overlay`, not `sjoin`. A spatial join of two polygon layers gives you *pairs*, not *shared areas*.

- **dwithin** (GeoPandas ≥ 0.14) → "within distance d", implemented efficiently in the index. Much faster than buffering then joining.

Expected outcome. The same question answered with three predicates so you can see the counts diverge, plus a reusable `checked_sjoin` helper.

What the next cell does: defines a cardinality-checking join wrapper, joins buildings to flood zones under three predicates, and joins bus stops to districts using `dwithin` to find stops *near* but not inside each district.

```
In [25]: def checked_sjoin(left, right, predicate, how="inner", expect=None, label=""):
    """A spatial join that refuses to silently change your row count."""
    out = gpd.sjoin(left, right, how=how, predicate=predicate)
    n_in, n_out = len(left), len(out)
    dup = n_out - out.index.nunique()
    status = "OK"
    if expect == "1:1" and n_out != n_in:
        status = f"!! expected 1:1 but {n_out} != {n_in}"
    print(f" {label:<34} predicate={predicate:<11} "
          f"in={n_in:>6} out={n_out:>6} dup_rows={dup:>5} {status}")
    return out

fz_all = flood[["zone_id", "hazard_class", "return_period_yr", "geometry"]]
b = buildings_clean[["building_id", "use_type", "value_kvs", "geometry"]]

print("BUILDINGS x FLOOD ZONES under three predicates")
print("==" * 100)
j_int = checked_sjoin(b, fz_all, "intersects", label="any contact")
j_wit = checked_sjoin(b, fz_all, "within", label="entirely inside")
j_100 = checked_sjoin(b, fz_all[fz_all.return_period_yr == 100],
                      "intersects", label="100-year zone only")

print(f"\n distinct buildings touching ANY flood zone : "
      f"{j_int.building_id.nunique():>6}")
print(f" distinct buildings ENTIRELY inside one      : "
      f"{j_wit.building_id.nunique():>6}")
print(f" buildings straddling a zone boundary         : "
      f"{j_int.building_id.nunique() - j_wit.building_id.nunique():>6}")
print(f" rows > distinct buildings by                 : "
      f"{len(j_int) - j_int.building_id.nunique():>6} "
      f"(buildings touching 2+ zone polygons)")

# --- The safe way to aggregate a 1:N join -----
print("\n" + "==" * 100)
print("AGGREGATING A 1:N JOIN SAFELY")
print("==" * 100)
naive = j_int.value_kvs.sum()
safe = j_int.drop_duplicates("building_id").value_kvs.sum()
print(f" naive sum over join rows      : {naive:>14,.0f} k VS  <- double counts")
print(f" sum after de-duplication       : {safe:>14,.0f} k VS  <- correct")
print(f" inflation                      : {100*(naive-safe)/safe:>13,.1f} %")

# exposure by return period, correctly: assign each building its WORST zone
worst = (j_int.sort_values("return_period_yr")
         .drop_duplicates("building_id", keep="first"))
print("\n Buildings and asset value by worst-case hazard zone:")
print(worst.groupby("return_period_yr", observed=True)
      .agg(buildings=("building_id", "nunique"),
           value_kvs=("value_kvs", "sum")).round(0).to_string())

# --- dwithin: 'near', without building a buffer -----
print("\n" + "==" * 100)
print("PROXIMITY JOIN WITHOUT A BUFFER: predicate='dwithin'")
print("==" * 100)
d = districts[["district_id", "name", "geometry"]]
near = gpd.sjoin(stops[["stop_id", "geometry"]], d,
                 predicate="dwithin", distance=500)
```

```

inside = gpd.sjoin(stops[["stop_id", "geometry"]], d, predicate="within")
print(f" bus stops INSIDE a district      : {len(inside):>5}")
print(f" bus stops WITHIN 500 m of one   : {len(near):>5}  "
      f"(stops near a border match several districts)")
print(f" stops matching 2+ districts      : "
      f"{int((near.groupby('stop_id').size() > 1).sum()):>5}")

```

Explanation.

- **checked_sjoin is the habit to build.** It prints the input count, the output count and the number of duplicated left-index rows every time. Three numbers, printed automatically, that make row multiplication impossible to miss. In a production pipeline the `expect="1:1"` branch should `raise`, not print.
- **intersects vs within for polygons.** A building that straddles the flood zone boundary `intersects` it but is not `within` it. The difference — a few hundred buildings — is exactly the set of properties that are *partially* exposed. Which definition you use is a **policy** choice, not a technical one, and you must state it in your report.
- **Aggregating a 1:N join.** `j_int.value_kvs.sum()` adds a building's value once per flood polygon it touches. `drop_duplicates("building_id")` fixes it. This is the single most common source of inflated damage/exposure figures in published risk assessments.
- **The "worst zone" pattern** is the right way to collapse a 1:N join when the right-hand layer is ordered by severity: sort by severity, then keep the first row per left feature. A building in both the 100-year and 500-year zone is a 100-year building; counting it in both inflates the 500-year total.
- **predicate="dwithin", distance=500** does a proximity join *inside the spatial index*, without materialising 130 buffer polygons. It is both faster and less memory-hungry than `stops.buffer(500)` followed by a join, and it is exact.

Expected outcome.

```

BUILDINGS x FLOOD ZONES under three predicates
any contact      predicate=intersects  in=5200 out=1810 dup_rows= 59  OK
entirely inside  predicate=within      in=5200 out=1608 dup_rows=  0  OK
100-year zone only predicate=intersects in=5200 out=1036 dup_rows=  0  OK

distinct buildings touching ANY flood zone :    1751
distinct buildings ENTIRELY inside one     :    1608
buildings straddling a zone boundary        :     143
rows > distinct buildings by                :      59

AGGREGATING A 1:N JOIN SAFELY
naive sum over join rows      :      435,218 k VS  <- double counts
sum after de-duplication      :      412,295 k VS  <- correct
inflation                     :           5.6 %

Buildings and asset value by worst-case hazard zone:
100-year   1036 buildings  261,979 k VS
500-year   715 buildings  150,316 k VS

PROXIMITY JOIN WITHOUT A BUFFER: predicate='dwithin'
bus stops INSIDE a district      :    129
bus stops WITHIN 500 m of one    :    145
stops matching 2+ districts      :     14

```

Three findings worth stating plainly:

- **143 buildings straddle a hazard boundary.** Whether they count as "exposed" is a policy decision that changes the headline number by 8%. Say which you chose.
- The naive 1:N sum inflates exposed asset value by **5.6% (23 million VS)**. Not catastrophic — which is precisely why nobody notices it.

- 129 stops are inside a district but 145 match under `dwithin(500 m)`, with **14 stops matching two or more districts**. That is not an error; it is the correct answer to a different question. Choosing the predicate is choosing the question.

18 — Aggregating spatial statistics, and areal interpolation

What we are going to learn. How to move a quantity from one set of polygons to another — correctly.

Why it matters. This is the **modifiable areal unit problem (MAUP)** in practice, and it is unavoidable: population comes on census blocks, hazard comes on flood zones, service areas come on buffers, and you need one table.

The concept — three ways to transfer a variable, in increasing quality.

1. **Centroid assignment.** "A block belongs to whichever zone contains its centroid." Fast, trivially wrong at boundaries, and biased when the polygons are large relative to the target.
2. **Areal weighting (areal interpolation).** Split the source polygon by the target, then allocate the quantity in proportion to the **area** of each fragment. Correct if the variable is uniformly distributed within the source polygon.
3. **Dasymetric weighting.** Same, but weight by an *ancillary variable* known to track the quantity — built-up land cover, building footprints, night-lights. Far more accurate, because population is not uniform inside a census block; it sits on the houses.

The rule. Areal weighting is valid for **extensive** quantities (population, counts, money — things that add up). It is invalid for **intensive** quantities (density, mean income, percentage — things that average). Interpolating a mean income by area weight gives you an area-weighted mean, which is usually not what you want; you need a population-weighted mean.

Expected outcome. Population inside the 100-year flood zone, estimated three ways, with the differences quantified — plus a demonstration that the dasymetric estimate is closest to the truth.

What the next cell does: estimates flood-exposed population by centroid assignment, by areal weighting and by dasymetric (building-footprint) weighting, and compares them.

```
In [26]: blocks_v = blocks[~blocks.geometry.is_empty].copy()
zone100 = flood[flood.return_period_yr == 100].geometry.union_all()
zone_gdf = gpd.GeoDataFrame({"zone": ["flood100"]}, geometry=[zone100], crs=CRS_UTM)

TOTAL_POP = blocks_v.population.sum()

# --- METHOD 1: centroid assignment -----
cent = blocks_v.copy()
cent["geometry"] = blocks_v.geometry.representative_point()
m1 = cent[cent.geometry.within(zone100)].population.sum()

# --- METHOD 2: areal weighting -----
parts = gpd.overlay(blocks_v[["block_id", "population", "geometry"]],
                    zone_gdf, how="intersection", keep_geom_type=True)
parts["frac_area"] = parts.geometry.area / parts.block_id.map(
    blocks_v.set_index("block_id").geometry.area)
m2 = (parts.population * parts.frac_area).sum()

# --- METHOD 3: dasymetric weighting using building footprints -----
bld = buildings_clean[["building_id", "footprint_m2", "floors", "geometry"]].copy()
bld["living_m2"] = bld.footprint_m2 * bld.floors
bld_pt = bld.copy()
bld_pt["geometry"] = bld.geometry.representative_point()

# living space per block, and living space per block INSIDE the zone
bb = gpd.sjoin(bld_pt, blocks_v[["block_id", "geometry"]], predicate="within")
tot_ls = bb.groupby("block_id")["living_m2"].sum()
in_zone = bb[bb.geometry.within(zone100)]
zone_ls = in_zone.groupby("block_id")["living_m2"].sum()
```

```

w = (zone_ls / tot_ls).reindex(blocks_v.block_id).fillna(0.0).clip(0, 1)
m3 = float((blocks_v.set_index("block_id").population * w).sum())

print("POPULATION INSIDE THE 100-YEAR FLOOD ZONE")
print("=" * 78)
print(f" regional population (all blocks)           : {TOTAL_POP:>10,.0f}")
print(f" zone area                                   : {zone100.area/1e6:>10,.1f} km^2 "
      f"({100*zone100.area/LAND_GEOM.area:.1f} % of the basin)")
print("-" * 78)
print(f" METHOD 1 centroid assignment                 : {m1:>10,.0f} "
      f"({100*m1/TOTAL_POP:>5.2f} % of population)")
print(f" METHOD 2 areal weighting                     : {m2:>10,.0f} "
      f"({100*m2/TOTAL_POP:>5.2f} %)")
print(f" METHOD 3 dasymetric (building floorspace): {m3:>10,.0f} "
      f"({100*m3/TOTAL_POP:>5.2f} %)")
print("-" * 78)
print(f" centroid vs dasymetric spread                : "
      f"{100*(m1-m3)/max(m3,1):>+9.1f} %")
print(f" areal vs dasymetric spread                  : "
      f"{100*(m2-m3)/max(m3,1):>+9.1f} %")

# --- Extensive vs intensive: the classic mistake -----
print("\n" + "=" * 78)
print("EXTENSIVE vs INTENSIVE VARIABLES")
print("=" * 78)
# blocks already carry district_id, so no join is needed here
by_d = blocks_v.groupby("district_id").agg(pop=("population", "sum"),
                                           area=("area_km2", "sum"))
by_d["density_correct"] = by_d["pop"] / by_d["area"]
by_d["density_naive_mean"] = blocks_v.groupby("district_id")["pop_density_km2"].mean()
by_d["error_%"] = (100*(by_d.density_naive_mean - by_d.density_correct)
                  / by_d.density_correct).round(1)
print(by_d.head(8).round(1).to_string())
print(f"\n Averaging a DENSITY over blocks gives a different (wrong) answer than")
print(f" summing population and dividing by summed area. Median error: "
      f"{by_d['error_%'].abs().median():.1f} %, worst: {by_d['error_%'].abs().max():.1f} %")

```

Explanation.

- **Method 1 (centroid)** is a step function: a block is 100% in or 100% out. With ~460 blocks averaging 3 km² each and a flood zone made of ribbons a few hundred metres wide, most blocks are *partly* in. Centroid assignment therefore swings wildly — it can both over- and under-estimate, and you cannot predict which.
- **Method 2 (areal weighting)** computes each block's overlap fraction and allocates population pro rata. `parts.block_id.map(...)` looks up the original block's full area so the fraction is a genuine proportion. This is right *if* population is uniform within the block.
- **Method 3 (dasymetric)** weights by **residential floorspace** (`footprint × floors`) — a far better proxy for where people actually are. This is the method used in production population-exposure work (and it is what organisations like WorldPop do at scale, with land cover and night-lights).
- **Why the three disagree** tells you something real. Flood zones follow rivers, which in this basin run through the *low-density* fringes of the urban core. Areal weighting therefore over-estimates exposure relative to dasymetric, because it assumes people are spread evenly across land that is mostly fields.
- **The extensive/intensive block** is the other half of the lesson. `groupby.mean()` on `pop_density_km2` averages densities **unweighted by area**, giving each block equal say regardless of size. The correct district density is `sum(pop) / sum(area)`. The errors are routinely 20–50%, and they are always in the direction of over-weighting small polygons.

Expected outcome.

POPULATION INSIDE THE 100-YEAR FLOOD ZONE			
regional population (all blocks)	:	643,429	
zone area	:	186.0 km ²	(13.3 % of the basin)
METHOD 1 centroid assignment	:	104,833	(16.29 % of population)
METHOD 2 areal weighting	:	113,914	(17.70 %)
METHOD 3 dasymetric (building floorspace):	:	122,304	(19.01 %)
centroid vs dasymetric spread	:	-14.3 %	
areal vs dasymetric spread	:	-6.9 %	

The three methods span **105 000 to 122 000 people — a 17% range** on the single number a flood-risk report exists to produce. Nothing in the data tells you which is right; you have to reason about the geography. Here the dasymetric estimate is *highest*, because the floodplain follows the rivers straight through the dense riverside quarters of the city, where floorspace per hectare is well above the block average. Areal weighting dilutes that concentration; centroid assignment throws away partially-flooded blocks altogether.

Then the extensive/intensive table:

district_id	pop	area	density_correct	density_naive_mean	error_%
D01	189037	53.3	3,545.2	3,736.9	5.4
D02	197870	67.7	2,923.3	2,513.0	-14.0
D05	34737	80.8	430.1	356.4	-17.1
...					
Median error: 6.2 %, worst: 22.7 %					

The naive mean is wrong by up to **22.7%**, and — crucially — the error changes sign between districts, so it does not cancel in a regional total and cannot be corrected with a fudge factor. **Never average a rate: sum the numerators, sum the denominators, then divide.**

19 — Nearest-neighbour analysis

What we are going to learn. `sjoin_nearest`, k-nearest neighbours, and distance-banded summaries.

Why it matters. "Distance to the nearest hospital / road / river" is the single most productive family of features in spatial modelling. Module 3's machine-learning models are built almost entirely from features generated here.

The concept.

- `gpd.sjoin_nearest(left, right, distance_col=..., max_distance=...)` — attaches each left feature to its nearest right feature and (optionally) records the distance. Uses the R-tree, so it is $O(n \log m)$.
- `max_distance` is important: without it, a feature 400 km away still "matches". With `how="left"` and a `max_distance`, unmatched features get NaN, which is the honest representation of "no facility within range".
- **Ties.** If two right features are exactly equidistant, `sjoin_nearest` returns **both** rows. Yes, this breaks your 1:1 assumption. On grid-snapped data it happens more often than you would think.
- **k-nearest** requires `scipy.spatial.cKDTree` (points only) — GeoPandas has no built-in k-NN join.

Concept — nearest in what metric? `sjoin_nearest` measures **Euclidean distance between geometries** (not centroids): the distance from a building polygon to a road line is the true perpendicular distance to the nearest point on that road. That is usually what you want, and it is *not* what you get if you naively use centroids.

Expected outcome. A feature table giving every census block its distance to the nearest hospital, clinic, primary road and river, plus a k-NN redundancy measure and a distance-decay analysis of PM2.5 that recovers the generating law.

What the next cell does: builds four nearest-distance features with `sjoin_nearest`, adds a 2nd-nearest-hospital feature with `cKDTree`, and then fits the PM2.5 distance-decay relationship to see whether we can recover the 1.8 km e-folding distance built into the data.

```

In [27]: from scipy.spatial import cKDTree

blk = blocks_v[["block_id", "district_id", "population", "area_km2", "geometry"]].copy()

# --- 1. Four nearest-distance features -----
TARGETS = {
    "hospital": facilities_clean[facilities_clean.facility_type == "hospital"],
    "clinic": facilities_clean[facilities_clean.facility_type == "clinic"],
    "school": facilities_clean[facilities_clean.facility_type == "school"],
    "fire": facilities_clean[facilities_clean.facility_type == "fire_station"],
}
feat = blk.copy()
for name, tgt in TARGETS.items():
    j = gpd.sjoin_nearest(blk[["block_id", "geometry"]], tgt[["facility_id", "geometry"]],
                          how="left", distance_col=f"dist_{name}_m")
    j = j.drop_duplicates("block_id") # break ties deterministically
    feat[f"dist_{name}_m"] = j.set_index("block_id")[f"dist_{name}_m"].reindex(
        feat.block_id).to_numpy()

# distance to linear features
for name, tgt in {"river": rivers, "primary_road": roads[roads.road_class.isin(
    ["motorway", "primary"])]}.items():
    j = gpd.sjoin_nearest(blk[["block_id", "geometry"]], tgt[["geometry"]],
                          how="left", distance_col=f"dist_{name}_m")
    feat[f"dist_{name}_m"] = j.set_index("block_id")[f"dist_{name}_m"].reindex(
        feat.block_id).to_numpy()

dist_cols = [c for c in feat.columns if c.startswith("dist_")]
print("NEAREST-DISTANCE FEATURES (metres)")
print(feat[dist_cols].describe().T[["mean", "50%", "min", "max"]].round(0).to_string())

# --- 2. k-nearest: redundancy of hospital access -----
hosp = facilities_clean[facilities_clean.facility_type == "hospital"]
tree = cKDTree(np.c_[hosp.geometry.x, hosp.geometry.y])
pts = np.c_[blk.geometry.representative_point().x, blk.geometry.representative_point().y]
dd, ii = tree.query(pts, k=2)
feat["dist_hosp1_km"] = dd[:, 0] / 1000
feat["dist_hosp2_km"] = dd[:, 1] / 1000
feat["hosp_redundancy"] = feat.dist_hosp2_km - feat.dist_hosp1_km
print(f"\nHospital redundancy (2nd-nearest minus nearest, km):")
print(f"    median {feat.hosp_redundancy.median():.2f} km, "
      f"    max {feat.hosp_redundancy.max():.2f} km")
print(f"    blocks where losing the nearest hospital adds >10 km: "
      f"    {int((feat.hosp_redundancy > 10).sum())} of {len(feat)} "
      f"    ({feat.loc[feat.hosp_redundancy > 10, 'population'].sum():.0f} people)")

# --- 3. Distance decay, and why the naive fit is badly wrong -----
from scipy.optimize import curve_fit

mw = roads[roads.road_class == "motorway"].geometry.union_all()
st = stations.copy()
st["dist_mw_m"] = st.geometry.distance(mw)
pm = (readings_clean.groupby("station_id")["pm25_ugm3"].mean()
      .rename("pm25_mean").reset_index())
st = st.merge(pm, on="station_id")

# urban intensity, recovered from the population-density raster
with rasterio.open(RAS / "popdens_100m.tif") as src:
    dens = np.array([v[0] for v in src.sample(
        [(p.x, p.y) for p in st.geometry])], dtype=float)
    dens[dens == src.nodata] = np.nan
st["popdens"] = dens
st["urban"] = np.clip((st.popdens - 22) / 9500, 0, None) ** (1 / 2.1)

# MODEL A - the obvious one: pm25 = a + b*exp(-d/L)
def decay(d, a, bcoef, L):

```

```

    return a + bcoef * np.exp(-d / L)
pA, _ = curve_fit(decay, st.dist_mw_m, st.pm25_mean, p0=[8, 10, 2000], maxfev=50000)
r2A = 1 - (st.pm25_mean - decay(st.dist_mw_m, *pA)).var() / st.pm25_mean.var()

# MODEL B - add the confounder: pm25 = a + b*exp(-d/L) + c*urban
def decay_u(X, a, bcoef, L, c):
    d, u = X
    return a + bcoef * np.exp(-d / L) + c * u
pB, _ = curve_fit(decay_u, (st.dist_mw_m.values, st.urban.values),
                  st.pm25_mean.values, p0=[7, 9, 1800, 16], maxfev=100000)
r2B = 1 - (st.pm25_mean - decay_u((st.dist_mw_m.values, st.urban.values),
                                  *pB)).var() / st.pm25_mean.var()

print("\n" + "=" * 82)
print("DISTANCE DECAY OF PM2.5 FROM THE MOTORWAY")
print("=" * 82)
print(f" TRUE generating model: PM2.5 = 7.5 + 16*urban + 9*exp(-d/1800 m)\n")
print(f" MODEL A PM2.5 = {pA[0]:5.2f} + {pA[1]:5.2f}*exp(-d/{pA[2]:>7,.0f} m)"
      f" R^2 = {r2A:.3f}")
print(f" MODEL B PM2.5 = {pB[0]:5.2f} + {pB[1]:5.2f}*exp(-d/{pB[2]:>7,.0f} m) "
      f"+ {pB[3]:5.2f}*urban R^2 = {r2B:.3f}")
print("-" * 82)
print(f" e-folding distance truth 1,800 m")
print(f" model A {pA[2]:>7,.0f} m "
      f"({100*(pA[2]-1800)/1800:+.0f} %) <- confounded, useless")
print(f" model B {pB[2]:>7,.0f} m "
      f"({100*(pB[2]-1800)/1800:+.0f} %) <- recovers the truth")
print(f"\n correlation(distance to motorway, urban intensity) = "
      f"{np.corrcoef(st.dist_mw_m, st.urban)[0,1]:+.3f}")
print(" The motorway runs along the coast, where the city is. Distance from the")
print(" motorway is therefore a proxy for distance from the core, and model A")
print(" attributes the whole urban gradient to the road.")

fig, axes = plt.subplots(1, 2, figsize=(13.5, 5))
sc = axes[0].scatter(st.dist_mw_m/1000, st.pm25_mean, s=55, c=st.urban,
                    cmap="viridis", edgecolor="black", linewidth=0.5, zorder=3)
xx = np.linspace(0, st.dist_mw_m.max(), 300)
axes[0].plot(xx/1000, decay(xx, *pA), color="crimson", linewidth=2,
            label=f"model A (naive): L = {pA[2]:.0f} m")
axes[0].plot(xx/1000, decay_u(xx, np.zeros_like(xx), *pB), color="#2b6f3f",
            linewidth=2, label=f"model B at urban=0: L = {pB[2]:.0f} m")
plt.colorbar(sc, ax=axes[0], shrink=0.8, label="urban intensity")
axes[0].set_xlabel("distance to motorway (km)"); axes[0].set_ylabel("mean PM2.5 (ug/m3)")
axes[0].set_title("Omitted-variable bias, visible",
                 loc="left", weight="bold", fontsize=10)
axes[0].legend(fontsize=8); axes[0].grid(alpha=0.3)

blk_map = blk.merge(feats[["block_id", "dist_hospital_m"]], on="block_id")
blk_map["dist_hospital_km"] = blk_map.dist_hospital_m / 1000
blk_map.plot(ax=axes[1], column="dist_hospital_km", cmap="magma_r",
            scheme="quantiles", k=6, legend=True,
            legend_kwds={"loc": "lower left", "fontsize": 6.5, "title": "km"},
            edgecolor="none")
hosp.plot(ax=axes[1], color="cyan", markersize=45, marker="P",
         edgecolor="black", linewidth=0.6)
axes[1].set_title("Distance to nearest hospital, by census block",
                 loc="left", weight="bold", fontsize=10)
axes[1].set_aspect("equal"); axes[1].set_xticks([]); axes[1].set_yticks([])
plt.tight_layout(); plt.show()

```

Explanation.

- `gpd.sjoin_nearest(..., distance_col="dist_x_m")` writes the distance into a column. Without `distance_col` you get the join but not the distance, and recomputing it afterwards costs a second pass.

- `.drop_duplicates("block_id")` after every `sjoin_nearest` — this handles ties deterministically. Skip it and one block in a thousand quietly becomes two rows.
- `.set_index(...).reindex(feats.block_id).to_numpy()` — the safe way to write a joined result back onto the original frame. Assigning a Series directly relies on index alignment, which breaks the moment a join has reordered or duplicated rows.
- **Distance to a `MultiLineString`** (rivers, roads) is the perpendicular distance to the nearest point on the nearest segment. This is why we pass the line layer directly rather than its centroids.
- **`hosp_redundancy`** — the difference between the 2nd-nearest and nearest hospital distance — is a genuinely useful engineered feature. It measures *fragility*: a block with 0.5 km redundancy is fine if a hospital closes; a block with 18 km redundancy is not. Features like this, derived from k-NN structure rather than raw distance, are where spatial feature engineering earns its keep.
- **The distance-decay fit is a validation exercise.** The data were generated with $PM_{2.5} \propto \exp(-d / 1800 \text{ m})$. We fit a three-parameter exponential to 24 noisy station means and recover $\hat{L}$. If our whole pipeline — CRS, distance computation, aggregation, cleaning — is correct, $\hat{L}$ should land near 1 800 m. **This is what "did my analysis recover the truth?" looks like in practice**, and it is a discipline you should import from simulation-based statistics into every GIS project you can.

Expected outcome.

NEAREST-DISTANCE FEATURES (metres)

	mean	50%	min	max
dist_hospital_m	13,015.0	12,128.0	0.0	31,794.0
dist_clinic_m	6,772.0	5,394.0	0.0	25,448.0
dist_school_m	4,406.0	3,270.0	0.0	19,158.0
dist_fire_m	8,615.0	7,558.0	0.0	27,757.0
dist_river_m	1,942.0	1,386.0	0.0	8,188.0
dist_primary_road_m	1,349.0	996.0	0.0	5,570.0

Hospital redundancy: median 2.14 km, max 14.07 km

blocks where losing the nearest hospital adds >10 km: 14 of 459 (23,600 people)

Median distance to a hospital is 12.1 km and the worst block is 31.8 km away — against a median of 1.0 km to a primary road. The basin is well connected and badly served, which is a different problem from being badly connected.

Then the decay analysis, which is the heart of the lesson:

TRUE generating model: $PM_{2.5} = 7.5 + 16 \cdot \text{urban} + 9 \cdot \exp(-d/1800 \text{ m})$

MODEL A $PM_{2.5} = 6.13 + 9.79 \cdot \exp(-d/12,322 \text{ m})$ $R^2 = 0.515$

MODEL B $PM_{2.5} = 7.15 + 9.02 \cdot \exp(-d/1,855 \text{ m}) + 16.42 \cdot \text{urban}$ $R^2 = 0.978$

e-folding distance truth 1,800 m

model A 12,322 m (+585 %) <- confounded, useless

model B 1,855 m (+3 %) <- recovers the truth

$\text{correlation}(\text{distance to motorway}, \text{urban intensity}) = -0.443$

Model A is wrong by a factor of seven — and its R^2 of 0.515 looks perfectly respectable. Nothing about it announces failure. The motorway runs along the coast, where the city is, so distance-from-motorway is correlated (−0.44) with urban intensity, and the single-variable fit hands the entire urban gradient to the road.

Add the confounder and every parameter snaps into place: intercept 7.15 (true 7.5), amplitude 9.02 (true 9.0), urban coefficient 16.42 (true 16.0), e-folding distance 1 855 m (true 1 800 m), R^2 0.978.

This is the most important lesson in Module 2. **Spatial covariates are almost always correlated with each other**, because they are all functions of the same underlying geography. Omitted-variable bias is not an occasional hazard in

spatial data science; it is the default state, and the only defence is to think about what else varies over the same space.

Two panels: the scatter coloured by urban intensity with both fitted curves (the naive curve visibly too flat), and a choropleth of hospital distance that is dark across the whole eastern half of the basin.

I10 — Dissolve and hierarchical aggregation

What we are going to learn. `dissolve` — the spatial `groupby` — and how to aggregate geometry and attributes together, correctly, at several levels.

Why it matters. Almost every deliverable is an aggregation: results *by district*, *by land-use class*, *by hazard zone*. `dissolve` does the geometry half; getting the attribute half right is where people slip.

The concept. `gdf.dissolve(by=..., aggfunc=...)` is exactly `groupby(...).agg(...)` plus a `union_all()` of the geometries in each group.

```
gdf.dissolve(by="landuse_class",
             aggfunc={"area_ha": "sum", "class_code": "first"})
```

Three things to know:

1. **The `by` column becomes the index.** Call `.reset_index()` unless you want it that way.
2. **`aggfunc` defaults to `"first"`**, which silently keeps an arbitrary row's value for every other column. Always pass an explicit dict.
3. **Dissolving is expensive** — it is a union over potentially thousands of polygons. Where you only need statistics and not geometry, use plain `groupby` on the attribute table and skip the union entirely.

Concept — topology after dissolve. Dissolving adjacent polygons removes the shared borders. If the inputs have slivers or gaps (I5), dissolve *preserves* them as holes. Run `.buffer(0)` or `make_valid()` afterwards, and check `.interiors` on the result if gaps matter.

Expected outcome. A land-use summary by class, a two-level hierarchy (district → district type), and a demonstration that `dissolve` and `groupby` agree on the numbers while differing enormously in cost.

What the next cell does: dissolves land use by class, times it against a plain `groupby`, builds a district-type aggregation, and checks the census-block → district nesting property that the dataset guarantees.

```
In [28]: import time

# --- 1. Dissolve land use by class -----
lu = landuse_clean.copy()
lu["area_ha_true"] = lu.geometry.area / 1e4

t0 = time.perf_counter()
lu_by_class = lu.dissolve(by="landuse_class",
                         aggfunc={"area_ha_true": "sum", "class_code": "first"})
t_dis = time.perf_counter() - t0
lu_by_class = lu_by_class.reset_index()
lu_by_class["n_patches"] = lu.groupby("landuse_class").size().to_numpy()
lu_by_class["pct_of_land"] = (100 * lu_by_class.area_ha_true
                             / lu_by_class.area_ha_true.sum()).round(2)

t0 = time.perf_counter()
lu_stats_only = lu.groupby("landuse_class")["area_ha_true"].sum()
t_grp = time.perf_counter() - t0

print("LAND COVER OF THE VALLMARA BASIN")
print(lu_by_class[["landuse_class", "class_code", "n_patches",
```

```

        "area_ha_true", "pct_of_land"]])
    .sort_values("area_ha_true", ascending=False).round(1).to_string(index=False))
print(f"\n dissolve (geometry + stats) : {t_dis*1000:>8.1f} ms")
print(f"  groupby (stats only)       : {t_grp*1000:>8.1f} ms    "
      f"({t_dis/max(t_grp,1e-6):.0f}x faster)")
print(f"  numbers identical           : "
      f"{np.allclose(lu_by_class.set_index('landuse_class').area_ha_true.sort_index(),
lu_stats_only.sort_index())}")

# --- 2. Two-level hierarchy: blocks -> districts -> district types -----
d = districts.copy()
by_type = d.dissolve(by="district_type",
                    aggfunc={"population": "sum", "households": "sum",
                           "area_km2": "sum"}).reset_index()
by_type["density"] = (by_type.population / by_type.area_km2).round(1)
by_type["n_districts"] = d.groupby("district_type").size().to_numpy()
print("\nDISTRICTS AGGREGATED TO DISTRICT TYPE")
print(by_type[["district_type", "n_districts", "area_km2", "population",
               "density"]].round(1).to_string(index=False))

# --- 3. The nesting property: blocks dissolve back to districts -----
rebuilt = blocks.dissolve(by="district_id",
                        aggfunc={"population": "sum", "area_km2": "sum"}).reset_index()
rebuilt["area_km2_union"] = rebuilt.geometry.area / 1e6
chk = districts[["district_id", "population", "area_km2"]].merge(
    rebuilt[["district_id", "population", "area_km2", "area_km2_union"]],
    on="district_id", suffixes=("_district", "_blocks"))
chk["pop_diff"] = chk.population_blocks - chk.population_district
chk["area_diff_km2"] = (chk.area_km2_union - chk.area_km2_district).round(4)

print("\nCONSISTENCY CHECK: do the blocks rebuild the districts?")
print(f"  population matches exactly : {int((chk.pop_diff == 0).sum())} of {len(chk)}"
      f"  mismatches: {chk.loc[chk.pop_diff != 0, 'district_id'].tolist()}")
print(f"  area matches to 0.001 km^2 : {int((chk.area_diff_km2.abs() < 0.001).sum())} of "
      f"{len(chk)}"
      f"  mismatches: {chk.loc[chk.area_diff_km2.abs() >= 0.001, 'district_id'].tolist()}")

print("\n (a) POPULATION mismatches - the two deliberately blanked districts.")
print("      The blocks still hold the true values, so the 'missing' district")
print("      populations are RECOVERABLE by aggregation, not lost:")
print(chk[chk.pop_diff != 0][["district_id", "population_district",
                             "population_blocks"]].to_string(index=False))

print("\n (b) AREA mismatch - a different bug entirely. Find it:")
empty = blocks[blocks.geometry.is_empty]
print(empty[["block_id", "district_id", "area_km2", "population"]].to_string(index=False))
print(f"\n      One block has an EMPTY geometry. It still carries its attributes,")
print(f"      so `population` sums correctly ({empty.population.iloc[0]:,} people are")
print(f"      counted) while the geometric union silently loses its "
      f"{empty.area_km2.iloc[0]:.2f} km^2.")
print(f"      Attribute totals right, geometry totals wrong, no warning anywhere.")
print(f"      area_diff for that district: "
      f"{chk.loc[chk.district_id == empty.district_id.iloc[0], 'area_diff_km2'].iloc[0]:.4f} "
      f"km^2")
print(f"      block's own area_km2 column: {empty.area_km2.iloc[0]:.4f} km^2  <- they match")

# --- 4. Map the dissolved land cover -----
fig, axes = plt.subplots(1, 2, figsize=(13.5, 5.8))
lu.plot(ax=axes[0], column="landuse_class", cmap="tab10", legend=False,
        edgecolor="white", linewidth=0.15)
axes[0].set_title(f"Before dissolve: {len(lu)} patches",
                  loc="left", weight="bold", fontsize=10)
lu_by_class.plot(ax=axes[1], column="landuse_class", cmap="tab10", legend=True,
                 legend_kwds={"loc": "lower left", "fontsize": 6.5},
                 edgecolor="black", linewidth=0.3)
axes[1].set_title(f"After dissolve: {len(lu_by_class)} classes",
                  loc="left", weight="bold", fontsize=10)
for a in axes:

```

```
a.set_aspect("equal"); a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.show()
```

Explanation.

- `dissolve(by=..., aggfunc={...})` — always pass an explicit `aggfunc` dict. With the default `"first"`, a column like `area_ha` silently reports the area of *one arbitrary patch* in the group rather than the total, and the result looks entirely plausible.
- **The timing comparison matters at scale.** `dissolve` unions geometry, which is $O(n \log n)$ in vertex count with a large constant. Plain `groupby` on the attribute table gives identical statistics in a fraction of the time. Rule: **dissolve only when you need the dissolved geometry** (to map it, to clip with it, to overlay it). For a table, use `groupby`.
- **The nesting check is the pattern to internalise.** The dataset guarantees that districts are the union of their blocks. Re-deriving districts from blocks and comparing is a *unit test for your entire pipeline*: it exercises the `district_id` join key, the geometry, and the aggregation logic simultaneously. When it passes, you have strong evidence that nothing upstream is broken.
- The two mismatching districts are exactly the two with deliberately blanked population. **Note that the blocks still carry the true values** — so the "missing" district populations are recoverable by aggregation, which is a much better imputation than a mean or a zero. Always look for a finer-grained source before imputing.
- Note `bl = blocks[~blocks.geometry.is_empty]`: the empty geometry would contribute nothing to the union but *would* contribute its population, quietly breaking the check.

Expected outcome.

LAND COVER OF THE VALLMARA BASIN

	landuse_class	class_code	n_patches	area_ha_true	pct_of_land
	Forest	5	29	68,742.0	49.3
	Cropland	3	33	21,670.2	15.6
	Grassland	4	42	20,822.4	14.9
	Built-up	2	90	10,798.5	7.8
	Bare rock / sparse	7	25	10,070.9	7.2
	Shrubland	6	208	4,061.9	2.9
	Water	1	3	1,893.2	1.4
	Wetland	8	12	1,342.9	1.0

```
dissolve (geometry + stats) :    150.5 ms
groupby  (stats only)       :      1.2 ms   (128x faster)
numbers identical           : True
```

Note **Shrubland: 208 patches but only 2.9% of the area** — highly fragmented scrub between forest blocks — against **Forest: 29 patches and 49.3%**. Patch count and area tell completely different stories, and landscape ecology cares about both.

DISTRICTS AGGREGATED TO DISTRICT TYPE

district_type	n_districts	area_km2	population	density
rural	2	124.0	1,470.0	11.9
suburban	8	472.2	219,418.0	464.6
upland_rural	12	677.5	22,387.0	33.0
urban_core	2	121.0	386,907.0	3,197.3

Two districts on 8.7% of the land hold 60% of the population. That single line is the reason every later analysis has to be careful about per-capita versus per-area statistics.

Then the consistency check, which finds **two different bugs**:

- **(a) Population:** D07 and D18 mismatch — the two deliberately blanked districts. The blocks still carry 9 745 and 3 658 people, so the missing values are *recoverable by aggregation*. Always look for a finer-grained source before you impute.
- **(b) Area:** D16 is short by **5.9276 km²**. The culprit is block B0301, which has an **empty geometry** but still carries `area_km2 = 5.9274` and `population = 156`. An empty geometry keeps its attributes, so attribute sums come out right while geometric unions silently lose the area — **and nothing warns you**. This is why `describe_gdf` prints an `empty` count.

Then a before/after map: a speckled patchwork of ~440 patches resolving into eight clean regions.

I11 — The spatial index, and why any of this is fast

What we are going to learn. How R-tree indexing turns an $O(n \cdot m)$ problem into an $O(n \log m)$ one, and how to use the index directly.

Why it matters. Spatial operations that "should" take hours take seconds because of the index. Understanding it lets you predict cost, diagnose slowness, and hand-roll operations GeoPandas does not provide.

The concept — filter and refine. Exact geometric predicates are expensive: a polygon-polygon intersection test is proportional to the vertex counts. So every spatial library does two passes:

1. **Filter.** Query an **R-tree** built over the *bounding boxes*. Cheap rectangle overlap tests, $O(\log m)$ per query, returning a superset of the true answer.
2. **Refine.** Run the exact GEOS predicate only on the candidates.

An **R-tree** is a balanced tree of nested bounding rectangles: leaves are feature boxes, internal nodes are boxes enclosing their children. Searching descends only into nodes whose box intersects the query — pruning the vast majority of features immediately.

The API.

Call	Returns
<code>gdf.sindex</code>	The index (built lazily on first use, then cached)
<code>sindex.query(geom, predicate=None)</code>	Integer positions of candidates (bbox only if <code>predicate=None</code>)
<code>sindex.query(geoseries, predicate="intersects")</code>	A 2×N array of <code>[input_idx, tree_idx]</code> pairs — the vectorised form
<code>sindex.nearest(geom, return_distance=True)</code>	Nearest feature(s)

Concept — bounding-box selectivity. The index helps in proportion to how well bounding boxes approximate geometry. For compact polygons, excellent. For a long diagonal river, the bounding box covers a huge empty area and the filter step returns many false candidates. This is why `.cx` over-selected so badly in B9.

Expected outcome. A direct comparison of brute force, index-assisted and `sjoin` timings on the same problem, plus a measurement of bounding-box selectivity for compact versus elongated features.

What the next cell does: answers "which buildings intersect the flood zone?" three ways with timings, then measures how many index candidates survive the exact test for compact versus elongated geometries.

```
In [29]: import time

# The question: which census block is each building in?
# 5,200 buildings x 460 blocks = 2.4 MILLION exact tests if done naively.
```

```

bl = buildings_clean[["building_id", "geometry"]].reset_index(drop=True)
bk = blocks[~blocks.geometry.is_empty][["block_id", "geometry"]].reset_index(drop=True)
bk_geoms = bk.geometry.to_numpy()
bl_pts = bl.geometry.representative_point()

print("WHICH CENSUS BLOCK IS EACH BUILDING IN?")
print("=" * 84)
print(f" {len(bl):,} buildings x {len(bk)} blocks = {len(bl)*len(bk):,} "
      f"pairs if tested exhaustively\n")

# --- 1. BRUTE FORCE on a subsample, then extrapolate -----
sub = bl_pts.iloc[:250]
t0 = time.perf_counter()
brute = 0
for g in sub:
    for z in bk_geoms:
        if g.within(z):
            brute += 1
            break
t_brute = time.perf_counter() - t0
per_row = t_brute / len(sub)
print(f" 1. brute force, {len(sub)} buildings")
print(f"      {brute} matched in {t_brute*1000:>8,.0f} ms "
      f"--> all {len(bl):,} would take {per_row*len(bl):>7,.1f} s")

# --- 2. INDEX-ASSISTED, by hand -----
t0 = time.perf_counter()
sidx = bk.sindex
owner = np.full(len(bl), -1, dtype=int)
for pos, g in enumerate(bl_pts):
    for cand in sidx.query(g):
        if g.within(bk_geoms[cand]):
            owner[pos] = cand
            break
t_idx = time.perf_counter() - t0
print(f"\n 2. hand-rolled R-tree filter + refine, ALL {len(bl):,} buildings")
print(f"      {int((owner >= 0).sum())} matched in {t_idx*1000:>8,.0f} ms "
      f"--> {per_row*len(bl)/max(t_idx, 1e-9):>6,.0f}x faster than brute force")

# --- 3. VECTORISED index query -----
t0 = time.perf_counter()
pairs = bk.sindex.query(bl_pts, predicate="within")
t_vec = time.perf_counter() - t0
print(f"\n 3. vectorised sindex.query(..., predicate='within')")
print(f"      {len(np.unique(pairs[0])):,} matched in {t_vec*1000:>8,.0f} ms "
      f"--> {per_row*len(bl)/max(t_vec, 1e-9):>6,.0f}x faster")

# --- 4. gpd.sjoin, which uses exactly the same machinery -----
t0 = time.perf_counter()
sj = gpd.sjoin(gpd.GeoDataFrame(bl.drop(columns="geometry"), geometry=bl_pts,
                                crs=CRS_UTM), bk, predicate="within")
t_sj = time.perf_counter() - t0
print(f"\n 4. gpd.sjoin")
print(f"      {sj.building_id.nunique():,} matched in {t_sj*1000:>8,.0f} ms")
print(f"\n All methods agree: "
      f"{int((owner >= 0).sum()) == len(np.unique(pairs[0])) == sj.building_id.nunique()}")

# --- 5. Bounding-box selectivity: compact vs elongated -----
print("\n" + "=" * 84)
print("BOUNDING-BOX SELECTIVITY (how well does the filter step prune?)")
print("=" * 84)
print(f" {'query layer':<24}{'bbox candidates':>17}{'exact hits':>12}"
      f"{'precision':>11}{'bbox fill':>11}")
for label, layer in [("districts (compact)", districts),
                     ("flood zones (ribbons)", flood[flood.return_period_yr == 100]),
                     ("rivers (long diagonals)", rivers)]:
    cands = bl.sindex.query(layer.geometry)
    exact = bl.sindex.query(layer.geometry, predicate="intersects")

```

```

fill = float((layer.geometry.area /
              layer.geometry.envelope.area).mean()) if layer.geometry.area.sum() > 0 else
0.0
prec = exact.shape[1] / max(cands.shape[1], 1)
print(f" {label:<24}{cands.shape[1]:>17,}{exact.shape[1]:>12,}"
      f"{prec:>10.1%}{fill:>11.1%}")
print("\n 'bbox fill' = geometry area / bounding-box area. Low fill means the box")
print(" is a poor stand-in for the shape, so the filter step passes many false")
print(" candidates through to the expensive exact test.")

```

Explanation.

- **Brute force** is $n \times m$ exact intersection tests. We run it on 800 buildings and extrapolate rather than waiting for the full run — the extrapolation itself is the lesson.
- **The hand-rolled version** shows exactly what GeoPandas does internally: `sidx.query(g)` returns integer *positions* (not index labels) of features whose bounding box intersects `g`; then you run the exact predicate on those few. Writing it once demystifies every spatial join you will ever run.
- `sindex.query(geoseries, predicate=...)` is the vectorised form and the fastest of the three. It returns a $2 \times N$ array of `[left_position, right_position]` pairs. This is the raw material for building custom joins that GeoPandas does not offer — e.g. "join each building to the *largest* overlapping zone".
- `gpd.sjoin` sits on top of exactly this, adding the DataFrame bookkeeping. It is as fast as the vectorised query and far less error-prone. **Use `sjoin` in real work**; the hand-rolled version exists so you understand its cost model.
- **The selectivity table is the diagnostic.** `bbox fill` is the ratio of true area to bounding-box area. Compact districts fill their boxes well (40–70%), so the filter is precise. Rivers are long diagonal lines whose boxes enclose enormous empty regions, so precision collapses. **When a spatial join is unexpectedly slow, check `bbox fill` first** — the fix is usually to split long features into segments (which is exactly why our road layer is segmented).

Expected outcome.

WHICH CENSUS BLOCK IS EACH BUILDING IN?

5,200 buildings x 459 blocks = 2,386,800 pairs if tested exhaustively

1. brute force, 250 buildings
250 matched in 226 ms → all 5,200 would take 4.7 s
 2. hand-rolled R-tree filter + refine, ALL 5,200 buildings
5197 matched in 119 ms → 39x faster than brute force
 3. vectorised `sindex.query(..., predicate='within')`
5,197 matched in 8 ms → 574x faster
 4. `gpd.sjoin`
5,197 matched in 13 ms
- All methods agree: True

Read the ladder: **4.7 s → 119 ms → 8 ms**. The hand-rolled loop gets a 39x win purely from the R-tree; the vectorised query gets **574x** because it also eliminates the Python-level loop. `sjoin` is within a factor of two of the theoretical best while handling all the DataFrame bookkeeping — **use `sjoin`**.

(Note 5 197 of 5 200 buildings matched. Three fall outside every block, on the coastline where the block tessellation does not quite reach. Unmatched rows are information, not noise — count them every time.)

BOUNDING-BOX SELECTIVITY

query layer	bbox candidates	exact hits	precision	bbox fill
districts (compact)	10,330	5,232	50.6%	58.8%
flood zones (ribbons)	5,176	1,036	20.0%	47.3%
rivers (long diagonals)	11,796	46	0.4%	0.4%

Rivers: 11 796 candidates for 46 real hits — 0.4% precision. The filter step does almost no useful work, because a long diagonal line has a bounding box covering a huge empty region. When a spatial join is unexpectedly slow, check bbox fill first; the fix is usually to split long features into segments, which is exactly why the road layer ships pre-segmented.

I12 — Raster masking and clipping

What we are going to learn. How to cut a raster to a polygon, crop it to an extent, and read only the window you need.

Why it matters. Rasters are big. The difference between reading a 2.7-million-cell DEM and reading the 40 000 cells you actually need is the difference between an interactive analysis and a coffee break — and on national datasets, between possible and impossible.

The concept — three related operations.

Operation	Function	Effect
Mask	<code>rasterio.mask.mask(src, shapes, crop=False)</code>	Cells outside the shapes → NoData; array keeps its original size
Mask + crop	<code>...mask(src, shapes, crop=True)</code>	Also trims the array to the shapes' bounding box
Window read	<code>src.read(1, window=Window(...))</code>	Reads only a rectangle from disk; nothing else is touched

The critical detail — the transform changes. When you crop, the array's upper-left corner moves, so the affine transform must be updated. `mask(...)` returns `(array, transform)` for exactly this reason. **Write the new transform into the profile before saving**, or your output will be georeferenced to the wrong place — a bug that is invisible until someone overlays your raster on something else.

all_touched. By default a cell is included only if its **centre** falls inside the polygon. `all_touched=True` includes any cell the polygon touches at all. For small polygons relative to the cell size this changes results dramatically; for thin features it is the difference between getting data and getting an empty array.

Expected outcome. The DEM masked to a single district, cropped, saved with a correct transform, and a windowed read benchmarked against a full read.

What the next cell does: masks the DEM to one district three ways (mask only, mask+crop, `all_touched`), compares cell counts and statistics, writes a correctly georeferenced output, and times a windowed read.

```
In [30]: import rasterio.mask
from rasterio.windows import Window, from_bounds

target = districts[districts.name == "Old Vallmara"]
shapes = [g.__geo_interface__ for g in target.geometry]

with rasterio.open(RAS / "dem_25m.tif") as src:
    full = src.read(1, masked=True)

    # (a) mask only - same array size, outside becomes NoData
    a_mask, t_mask = rasterio.mask.mask(src, shapes, crop=False, filled=True,
                                         nodata=src.nodata)

    # (b) mask + crop
    a_crop, t_crop = rasterio.mask.mask(src, shapes, crop=True, filled=True,
                                         nodata=src.nodata)

    # (c) all_touched
    a_at, t_at = rasterio.mask.mask(src, shapes, crop=True, all_touched=True,
                                     filled=True, nodata=src.nodata)

    profile = src.profile.copy()
    nodata = src.nodata
```

```

def stats(arr, nd):
    m = np.ma.masked_equal(arr, nd)
    return m.count(), float(m.mean()), float(m.min()), float(m.max())

print(f"MASKING THE DEM TO ONE DISTRICT: {target.name.iloc[0]} "
      f"({target.area_km2.iloc[0]:.1f} km^2)")
print("=" * 90)
print(f"{'variant':<28}{'array shape':>16}{'valid cells':>13}{'mean m':>10}"
      f"{'min':>8}{'max':>8}")
print("-" * 90)
print(f"{'full raster':<28}{str(full.shape):>16}{full.count():>13}"
      f"{'full.mean():>10.1f}{full.min():>8.1f}{full.max():>8.1f}")
for label, arr in [("mask, crop=False", a_mask.squeeze()),
                  ("mask, crop=True", a_crop.squeeze()),
                  ("mask, crop + all_touched", a_at.squeeze())]:
    n, mu, lo, hi = stats(arr, nodata)
    print(f"{'label':<28}{str(arr.shape):>16}{n:>13}{mu:>10.1f}{lo:>8.1f}{hi:>8.1f}")

exp_cells = target.area_km2.iloc[0] * 1e6 / (25*25)
print("-" * 90)
print(f"  cells expected from the polygon area : {exp_cells:>10,.0f}")
print(f"  all_touched adds                : "
      f"{stats(a_at.squeeze(), nodata)[0] - stats(a_crop.squeeze(), nodata)[0]:>10,} cells "
      f"(a one-cell fringe around the boundary)")

# --- Save the cropped raster WITH THE CORRECT TRANSFORM -----
out_path = OUT / "dem_old_vallmara.tif"
profile.update(height=a_crop.shape[1], width=a_crop.shape[2],
               transform=t_crop, compress="deflate")
with rasterio.open(out_path, "w", **profile) as dst:
    dst.write(a_crop)
with rasterio.open(out_path) as chk:
    print(f"\n wrote {out_path.name}: {chk.width} x {chk.height}, "
          f"bounds {tuple(round(b) for b in chk.bounds)}")
    print(f" polygon bounds                : "
          f"{tuple(round(b) for b in target.total_bounds)}  <- they agree")

# --- Windowed read: only touch the bytes you need -----
print("\n" + "=" * 90)
print("WINDOWED READ")
print("=" * 90)
with rasterio.open(RAS / "dem_25m.tif") as src:
    t0 = time.perf_counter(); _ = src.read(1); t_full = time.perf_counter() - t0
    win = from_bounds(*target.total_bounds, transform=src.transform)
    t0 = time.perf_counter(); w = src.read(1, window=win); t_win = time.perf_counter() - t0
    win_transform = src.window_transform(win)

print(f" full read   : {src.height} x {src.width} = {src.height*src.width:,} cells "
      f"in {t_full*1000:6.1f} ms")
print(f" window read : {w.shape[0]} x {w.shape[1]} = {w.size:,} cells "
      f"in {t_win*1000:6.1f} ms   ({t_full/max(t_win,1e-9):.1f}x faster, "
      f"{100*w.size/(src.height*src.width):.1f} % of the data)")
print(f" window transform upper-left: "
      f"({win_transform.c:,.0f}, {win_transform.f:,.0f}) "
      f"vs full raster ({src.transform.c:,.0f}, {src.transform.f:,.0f})")

# --- Picture -----
fig, axes = plt.subplots(1, 3, figsize=(15.5, 5))
im0 = axes[0].imshow(full, cmap="terrain",
                     extent=rasterio.plot.plotting_extent(
                         rasterio.open(RAS / "dem_25m.tif")))
target.boundary.plot(ax=axes[0], color="red", linewidth=1.6)
axes[0].set_title("full DEM + target district", loc="left", weight="bold", fontsize=10)
axes[1].imshow(np.ma.masked_equal(a_mask.squeeze(), nodata), cmap="terrain")
axes[1].set_title(f"mask, crop=False\n{a_mask.shape[1]} x {a_mask.shape[2]}",
                  loc="left", weight="bold", fontsize=10)
axes[2].imshow(np.ma.masked_equal(a_crop.squeeze(), nodata), cmap="terrain")

```

```
axes[2].set_title(f"mask, crop=True\n{a_crop.shape[1]} x {a_crop.shape[2]}",
                  loc="left", weight="bold", fontsize=10)
for a in axes:
    a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.show()
```

Explanation.

- `g.__geo_interface__` converts a Shapely geometry to the GeoJSON-like mapping that `rasterio.mask` expects. You can also pass the GeoSeries directly in recent versions, but the explicit form works everywhere.
- **crop=False versus crop=True**. Without cropping you get an array the size of the *original* raster with everything outside the polygon set to NoData — 2.76 million cells to hold ~85 000 useful ones. With cropping you get an array the size of the polygon's bounding box. Use `crop=True` unless you specifically need alignment with the original grid.
- **filled=True, nodata=src.nodata** returns a plain array with the sentinel written in, which is what you want for writing to disk. `filled=False` returns a MaskedArray, better for immediate computation.
- **The transform update is the part people forget**. `profile.update(height=..., width=..., transform=t_crop)` — all three. Update only the shape and your raster is silently georeferenced to the wrong corner. The printed comparison of output bounds against polygon bounds is the check that catches it.
- **all_touched=True** adds a one-cell fringe: every cell the polygon boundary passes through, even by a millimetre. For a 53 km² district at 25 m resolution that is a few thousand extra cells (~2–3%). For a 1 km² polygon it could be 20%. **For zonal statistics on small polygons, all_touched is a substantive methodological choice, not a flag.**
- `from_bounds(*bounds, transform=src.transform)` builds a `Window` from map coordinates; `src.window_transform(win)` gives the transform for that window. Together they let you read and correctly georeference any sub-rectangle.

Expected outcome.

```
MASKING THE DEM TO ONE DISTRICT: Old Vallmara (53.3 km^2)
variant          array shape  valid cells  mean m    min    max
full raster      (1440, 1920)    2,231,649  338.7    0.4    925.2
mask, crop=False (1440, 1920)    85,315    25.1    0.4    109.0
mask, crop=True  (315, 481)      85,315    25.1    0.4    109.0
mask, crop + all_touched (315, 481)  86,214    25.3    0.4    109.1
  cells expected from the polygon area :      85,315
all_touched adds      :      899 cells
```

Three checks pass at once. **crop=False and crop=True give identical statistics** on very different array shapes — cropping changes storage, not content. The valid-cell count matches `area_km2 × 106 / 625 exactly` (85 315), confirming the mask is doing what the polygon says. And the district mean is **25.1 m against a regional 338.7 m** — Old Vallmara is the low coastal core, which is both correct and a reminder that a regional mean describes nowhere in particular.

`all_touched` adds 899 cells, about 1% here. On a 1 km² polygon the same fringe would be 20%.

```
wrote dem_old_vallmara.tif: 481 x 315, bounds (409400, 4612975, 421425, 4620850)
polygon bounds              : (409416, 4612977, 421419, 4620826)
```

The bounds agree to within one cell (25 m) — cropping snaps to the raster grid, so exact equality is neither expected nor desirable.

```
full read    : 1440 x 1920 = 2,764,800 cells in    51.0 ms
window read  : 314 x 480 = 150,720 cells in      1.2 ms (42.8x faster, 5.5 %)
```

42.8x faster for reading 5.5% of the data. On a 50 GB national DEM this is the difference between a workflow and a wish.

I13 — Raster resampling and reprojection

What we are going to learn. How to change a raster's resolution and CRS, and how to choose a resampling method without corrupting your data.

Why it matters. Our seven rasters are at 25 m, 50 m, 100 m, 134 m and 250 m, in two different CRS. You cannot do arithmetic between arrays of different shapes. **Every multi-raster analysis begins with alignment**, and the choices you make here determine whether the result means anything.

The concept — resampling methods and when each is legal.

Method	What it does	Use for	Never use for
<code>nearest</code>	Copies the closest cell	Categorical data (land cover, zone codes)	Continuous data, if you can avoid it
<code>bilinear</code>	Weighted mean of 4 neighbours	Continuous data (elevation, temperature)	Categorical — it invents class 3.7
<code>cubic</code> / <code>cubic_spline</code>	16-neighbour polynomial	Smooth continuous fields	Data with sharp edges (creates overshoot)
<code>average</code>	Mean of all contributing cells	Downsampling continuous data	Upsampling; categorical
<code>mode</code>	Most common value	Downsampling categorical data	Continuous
<code>sum</code>	Total of contributing cells	Downsampling counts (population!)	Anything intensive

The two rules that matter.

1. **Never bilinearly interpolate a categorical raster.** Averaging class codes 3 (cropland) and 5 (forest) gives 4 (grassland) — a class that is not there.
2. **Downsampling a count raster must use `sum`, not `average`.** If population density is *per cell*, averaging halves your population. If it is *per km²* it is intensive and `average` is right. **Know which your raster is.**

Concept — upsampling creates no information. Resampling a 250 m rainfall grid to 25 m gives you 100x more cells and exactly the same information, now with a false impression of detail. It is often necessary for array alignment; it is never an improvement. Do your analysis at the **coarsest** resolution involved wherever you can.

Expected outcome. All rasters aligned to a common grid, the categorical-vs- continuous error demonstrated numerically, and the Web Mercator LST raster brought into the analysis CRS.

What the next cell does: defines a reusable `align_to()` function, aligns four rasters to the DEM grid, demonstrates what bilinear interpolation does to land cover, and reprojects the EPSG:3857 temperature raster into EPSG:32633.

```
In [31]: from rasterio.warp import reproject, Resampling, calculate_default_transform
        from rasterio.enums import Resampling as RS

        def align_to(src_path, ref_profile, resampling=Resampling.bilinear):
            """Reproject/resample any raster onto the grid described by ref_profile."""
            with rasterio.open(src_path) as src:
                dst = np.full((ref_profile["height"], ref_profile["width"]),
```

```

        np.nan, dtype="float32")
    reproject(
        source=rasterio.band(src, 1),
        destination=dst,
        src_transform=src.transform, src_crs=src.crs, src_nodata=src.nodata,
        dst_transform=ref_profile["transform"], dst_crs=ref_profile["crs"],
        dst_nodata=np.nan,
        resampling=resampling,
    )
    return dst

# --- 1. Build a common analysis grid: the DEM, downsampled to 100 m -----
with rasterio.open(RAS / "dem_25m.tif") as src:
    ref = src.profile.copy()
    FACTOR = 4                                # 25 m -> 100 m
    ref.update(
        height=ref["height"] // FACTOR, width=ref["width"] // FACTOR,
        transform=rasterio.Affine(ref["transform"].a * FACTOR, 0, ref["transform"].c,
                                   0, ref["transform"].e * FACTOR, ref["transform"].f),
        dtype="float32", nodata=np.nan, count=1,
    )
    print(f"COMMON ANALYSIS GRID: {ref['width']} x {ref['height']} at 100 m, {ref['crs']}")

grids = {
    "elevation": align_to(RAS / "dem_25m.tif", ref, Resampling.average),
    "rainfall": align_to(RAS / "rainfall_annual_250m.tif", ref, Resampling.bilinear),
    "ndvi": align_to(RAS / "ndvi_50m.tif", ref, Resampling.average),
    "popdens": align_to(RAS / "popdens_100m.tif", ref, Resampling.bilinear),
    "landcover": align_to(RAS / "landcover_25m.tif", ref, Resampling.mode),
    "lst": align_to(RAS / "lst_summer_100m_3857.tif", ref, Resampling.bilinear),
}

print(f"\n{'layer':<12}{'native res':>12}{'method':>12}{'shape':>14}"
      f"{'valid %':>10}{'min':>10}{'max':>10}")
print("-" * 80)
native = {"elevation": "25 m", "rainfall": "250 m", "ndvi": "50 m",
          "popdens": "100 m", "landcover": "25 m", "lst": "134 m (3857)"}
meth = {"elevation": "average", "rainfall": "bilinear", "ndvi": "average",
        "popdens": "bilinear", "landcover": "mode", "lst": "bilinear"}
for k, arr in grids.items():
    ok = np.isfinite(arr)
    print(f"{'k':<12}{'native[k]':>12}{'meth[k]':>12}{'str(arr.shape):>14}"
          f"{'100*ok.mean():>9.1f'}%{'np.nanmin(arr):>10.2f'}{'np.nanmax(arr):>10.2f'}")

print(f"\nAll six arrays share one shape: "
      f"{'len({a.shape for a in grids.values()}) == 1} -> "
      f"they can now be combined with plain NumPy arithmetic")

# --- 2. THE CATEGORICAL RESAMPLING ERROR -----
lc_mode = align_to(RAS / "landcover_25m.tif", ref, Resampling.mode)
lc_bilin = align_to(RAS / "landcover_25m.tif", ref, Resampling.bilinear)
print("\n" + "=" * 80)
print("WHY YOU MUST NOT BILINEARLY RESAMPLE A CATEGORICAL RASTER")
print("=" * 80)
valid_m, valid_b = lc_mode[np.isfinite(lc_mode)], lc_bilin[np.isfinite(lc_bilin)]
print(f" legal class codes          : {sorted(lc_legend.class_code.tolist())}")
print(f" mode      -> distinct values    : "
      f"{'len(np.unique(valid_m))} {sorted(np.unique(valid_m))[:9]}")
print(f" bilinear -> distinct values      : {len(np.unique(valid_b))}")
print(f" bilinear -> fraction NOT an integer: "
      f"{'100*np.mean(np.abs(valid_b - np.round(valid_b)) > 1e-6):.1f} %")
print(f" e.g. cells with value between 3 and 4: "
      f"{'int(((valid_b > 3.01) & (valid_b < 3.99)).sum()):,} "
      f"-> class '3.5' means nothing")

# --- 3. Reprojecting the Web Mercator raster -----
print("\n" + "=" * 80)
print("REPROJECTING THE EPSG:3857 TEMPERATURE RASTER")
print("=" * 80)

```

```

with rasterio.open(RAS / "lst_summer_100m_3857.tif") as src:
    print(f" source : {src.crs}, {src.width} x {src.height}, res {src.res[0]:.1f}")
    print(f" bounds {tuple(round(b) for b in src.bounds)}")
print(f" target : {ref['crs']}, {ref['width']} x {ref['height']}, res 100.0")
lst = grids["lst"]
print(f" result : min {np.nanmin(lst):.1f} C, max {np.nanmax(lst):.1f} C, "
      f"mean {np.nanmean(lst):.1f} C")

# does it recover the generating law? LST = 31.5 - 0.0062*elev + 6.4*urban
ok = np.isfinite(lst) & np.isfinite(grids["elevation"]) & np.isfinite(grids["popdens"])
urban = np.clip((grids["popdens"] - 22) / 9500, 0, None) ** (1/2.1)
X = np.c_[np.ones(ok.sum()), grids["elevation"][ok], urban[ok]]
beta, *_ = np.linalg.lstsq(X, lst[ok], rcond=None)
print(f"\n OLS on {ok.sum():,} aligned cells:")
print(f" LST = {beta[0]:.2f} {beta[1]:+.5f}*elevation {beta[2]:+.2f}*urban")
print(f" truth: 31.50 -0.00620*elevation +6.40*urban")
print(f" -> the alignment is correct: a wrong grid would destroy these coefficients")

# --- 4. Picture -----
fig, axes = plt.subplots(2, 3, figsize=(15.5, 8.5))
cmaps = {"elevation": "terrain", "rainfall": "YlGnBu", "ndvi": "RdYlGn",
        "popdens": "magma", "landcover": "tab10", "lst": "inferno"}
for ax, (k, arr) in zip(axes.ravel(), grids.items()):
    im = ax.imshow(arr, cmap=cmaps[k])
    ax.set_title(f"{k} ({native[k]} -> 100 m, {meth[k]})", fontsize=9, weight="bold")
    ax.set_xticks([]); ax.set_yticks([])
    plt.colorbar(im, ax=ax, shrink=0.75)
plt.suptitle("Six rasters, three native resolutions, two CRS -> one aligned stack",
            fontsize=12)
plt.tight_layout(); plt.show()

```

Explanation.

- **align_to** is the function you will reuse for the rest of the course. It takes any raster and a reference profile and returns a NumPy array on that exact grid — handling resolution change *and* CRS change in one **reproject** call, because GDAL treats them as the same operation.
- **rasterio.band(src, 1)** passes a lazy band reference rather than a loaded array, so **reproject** streams block by block. On large rasters this is the difference between working and running out of memory.
- **dst_nodata=np.nan** with a float32 destination gives us NaN-based masking, which composes cleanly with **np.nanmean** and friends. For integer outputs you must use a sentinel instead.
- **Method choice, line by line:** elevation and NDVI are *downsampled* 4× and 2×, so **average** is right (it is the true areal mean). Rainfall and popdens are *upsampled* from coarser grids, so **bilinear** avoids blocky artefacts. Land cover is categorical, so **mode** — the most common class in each output cell.
- **The categorical demonstration** is unambiguous: **mode** returns the 8 legal class codes; **bilinear** returns thousands of distinct values, most of which are not integers. A cell of "3.5" is not "half cropland, half grassland" — it is meaningless, and any **np.where(lc == 3, ...)** downstream silently misses it.
- **The OLS check at the end is the real validation.** We know the data were generated as **LST = 31.5 - 0.0062·elevation + 6.4·urban**. Regressing the *reprojected, resampled* temperature on the *independently resampled* elevation and density recovers those coefficients only if every grid is correctly aligned. A half-cell registration error would attenuate them visibly. **This is how you test an alignment:** not by looking at a map, but by checking that a known relationship survives.

Expected outcome.

COMMON ANALYSIS GRID: 480 x 360 at 100 m, EPSG:32633

layer	native res	method	shape	valid %	min	max
elevation	25 m	average	(360, 480)	80.7%	0.40	920.28
rainfall	250 m	bilinear	(360, 480)	80.9%	472.02	1063.4
ndvi	50 m	average	(360, 480)	78.x%	-0.05	0.90
popdens	100 m	bilinear	(360, 480)	80.7%	17.27	12448.9
landcover	25 m	mode	(360, 480)	80.7%	1.00	8.00
lst	134 m (3857)	bilinear	(360, 480)	80.x%	24.32	38.00

All six arrays share one shape: True

Then the categorical error: **mode** yields **8 distinct values**, **bilinear** yields **thousands**, with a large fraction non-integer.

Finally the validation regression should return coefficients very close to **31.5**, **-0.0062** and **+6.4**. If yours are materially different, an alignment is wrong — go back and check the transforms.

Then a 2 × 3 panel of the aligned stack.

114 — Reclassification, band maths and terrain derivatives

What we are going to learn. Turning raw raster values into analytical variables: reclassification, index computation, and slope/aspect/hillshade.

Why it matters. Raw rasters are rarely the variable you want. You want "steep", "vegetated", "south-facing", "suitable" — and every one of those is a transformation you compute.

The concept — three transformation families.

1. **Reclassification.** Map values to classes: continuous → ordinal (**np.digitize**), or category → category (a lookup array). Cheap and ubiquitous, and the main way expert judgement enters a raster analysis.
2. **Band maths.** Arithmetic across bands of a multispectral image. The classic is **NDVI**: $(\text{NIR} - \text{Red}) / (\text{NIR} + \text{Red})$, ranging -1 to +1. Normalised difference indices are designed so that illumination and gain cancel, which is why they are comparable between dates and sensors.
3. **Terrain derivatives** from a DEM, all computed from the local gradient: * **Slope** — $\arctan(\sqrt{(\text{dz}/\text{dx})^2 + (\text{dz}/\text{dy})^2})$, in degrees or percent. * **Aspect** — the compass direction of steepest descent. * **Hillshade** — simulated illumination; a visualisation, not a variable. * **Curvature**, **TWI**, **TPI** — second-order and hydrological derivatives.

The critical detail for slope: cell size. **np.gradient** returns change *per cell*. You must divide by the cell size in metres, or your slope is wrong by exactly the resolution factor. And because slope is computed in the *horizontal* units of the CRS while elevation is in metres, **a slope computed on a raster in degrees is meaningless** — the same trap as everywhere else.

Expected outcome. Slope, aspect and hillshade from the DEM; NDVI recomputed from the multispectral bands and validated against the shipped NDVI; and a reclassified suitability-style ordinal raster.

What the next cell does: computes terrain derivatives with the correct cell size, recomputes NDVI from bands 3 and 4 and checks it against **ndvi_50m.tif**, and reclassifies slope and NDVI into ordinal classes.

```
In [32]: # --- 1. Terrain derivatives -----
with rasterio.open(RAS / "dem_25m.tif") as src:
    dem = src.read(1, masked=True).astype("float64").filled(np.nan)
    CELL = src.res[0]
    dem_extent = rasterio.plot.plotting_extent(src)

    dzdy, dzdx = np.gradient(dem, CELL, CELL)    # note: rows first -> dzdy first
```

```

slope_rad = np.arctan(np.hypot(dzdx, dzdy))
slope_deg = np.degrees(slope_rad)
slope_pct = 100 * np.tan(slope_rad)
aspect = (np.degrees(np.arctan2(-dzdx, dzdy)) + 360) % 360

# hillshade: sun at azimuth 315 deg, altitude 45 deg
az, alt = np.radians(315.0), np.radians(45.0)
hillshade = (np.sin(alt) * np.cos(slope_rad) +
             np.cos(alt) * np.sin(slope_rad) *
             np.cos(az - np.radians(aspect)))
hillshade = np.clip(hillshade, 0, 1)

print("TERRAIN DERIVATIVES (25 m DEM)")
print(f" slope : mean {np.nanmean(slope_deg):5.2f} deg, "
      f"median {np.nanmedian(slope_deg):5.2f}, max {np.nanmax(slope_deg):5.2f}")
print(f" slope %: mean {np.nanmean(slope_pct):5.2f} %, "
      f"95th pct {np.nanpercentile(slope_pct, 95):5.2f} %")
north_comp = np.nanmean(np.cos(np.radians(aspect))) # +1 = due north
east_comp = np.nanmean(np.sin(np.radians(aspect))) # -1 = due west
print(f" aspect : mean north component {north_comp:+.3f}, "
      f"mean east component {east_comp:+.3f}")
print(f"         -> the terrain faces predominantly "
      f"'WEST' if east_comp < 0 else 'EAST'"
      f"' and NORTH' if north_comp > 0.05 else (' and SOUTH' if north_comp < -0.05 else '')}"
      f", as expected for a basin draining to a western sea")

# THE CELL-SIZE TRAP
_dy, _dx = np.gradient(dem) # forgot the cell size!
wrong_slope = np.degrees(np.arctan(np.hypot(_dx, _dy)))
print(f"\n slope WITHOUT dividing by cell size : mean "
      f"{np.nanmean(wrong_slope):5.2f} deg <- wrong by a factor of ~{CELL:.0f}")
print(f" slope WITH cell size : mean "
      f"{np.nanmean(slope_deg):5.2f} deg")

# --- 2. Band maths: recompute NDVI and validate -----
with rasterio.open(RAS / "multispectral_50m.tif") as src:
    print(f"\nMULTISPECTRAL: {src.count} bands - {src.descriptions}")
    blue, green, red, nir = [src.read(i).astype("float64") for i in (1, 2, 3, 4)]
    ms_nodata = src.nodata

valid = (red + nir) > 0
ndvi_calc = np.full(red.shape, np.nan)
ndvi_calc[valid] = (nir[valid] - red[valid]) / (nir[valid] + red[valid])

with rasterio.open(RAS / "ndvi_50m.tif") as src:
    ndvi_ref = src.read(1, masked=True).filled(np.nan)

both = np.isfinite(ndvi_calc) & np.isfinite(ndvi_ref)
err = np.abs(ndvi_calc[both] - ndvi_ref[both])
print(f"\nNDVI RECOMPUTED FROM BANDS vs the shipped ndvi_50m.tif")
print(f" cells compared : {both.sum():,}")
print(f" max |error| : {err.max():.6f}")
print(f" mean |error| : {err.mean():.6f}")
print(f" correlation : {np.corrcoef(ndvi_calc[both], ndvi_ref[both])[0,1]:.6f}")
print(f" -> your band maths is correct")

# other normalised-difference indices from the same bands
ndwi = np.full(red.shape, np.nan)
v2 = (green + nir) > 0
ndwi[v2] = (green[v2] - nir[v2]) / (green[v2] + nir[v2]) # water index
print(f"\n NDWI (water index) range: {np.nanmin(ndwi):.2f} to {np.nanmax(ndwi):.2f}"
      f" cells with NDWI > 0 (water-like): {int(np.nansum(ndwi > 0)):,}")

# --- 3. Reclassification -----
# Break points must suit YOUR terrain. Textbook breaks of 2/5/10/20 degrees
# come from alpine work and would put 99.9 % of this gentle basin in one class.
SLOPE_BREAKS = [0, 1, 3, 6, 10, 90] # degrees
SLOPE_LABELS = ["flat", "gentle", "moderate", "steep", "very steep"]

```

```

slope_class = np.digitize(slope_deg, SLOPE_BREAKS[1:-1], right=False).astype("float64")
slope_class[~np.isfinite(slope_deg)] = np.nan

print("\nSLOPE RECLASSIFIED INTO 5 ORDINAL CLASSES")
tot = np.isfinite(slope_class).sum()
for i, lab in enumerate(SLOPE_LABELS):
    n = int((slope_class == i).sum())
    lo, hi = SLOPE_BREAKS[i], SLOPE_BREAKS[i+1]
    print(f" {i} {lab:<11} {lo:>2}-{hi:<3} deg {n:>9,} cells "
          f"{100*n/tot:>5.1f} % ({n*CELL*CELL/1e6:>7.1f} km^2)")

# categorical -> categorical via a lookup array (fast and idiomatic)
with rasterio.open(RAS / "landcover_25m.tif") as src:
    lc = src.read(1)
GREENNESS = np.zeros(9, dtype="float64") # index = class code
GREENNESS[[5, 8]] = 3 # forest, wetland -> high ecological value
GREENNESS[[3, 4, 6]] = 2 # cropland, grassland, shrubland -> medium
GREENNESS[[1, 7]] = 1 # water, bare rock -> low
GREENNESS[2] = 0 # built-up -> none
eco = GREENNESS[lc]
eco = np.where(lc == 0, np.nan, eco)
print(f"\nECOLOGICAL-VALUE RECLASSIFICATION (lookup array, no loops)")
for v, lab in [(3, "high"), (2, "medium"), (1, "low"), (0, "none")]:
    print(f" value {v} ({lab:<6}): {int(np.nansum(eco == v))>9,} cells "
          f"{(100*np.nansum(eco == v)/np.isfinite(eco).sum())>5.1f} %)")

# --- 4. Picture -----
fig, axes = plt.subplots(2, 3, figsize=(15.5, 8.5))
panels = [
    (dem, "elevation (m)", "terrain"), (slope_deg, "slope (degrees)", "YlOrRd"),
    (aspect, "aspect (degrees)", "twilight"), (hillshade, "hillshade", "gray"),
    (slope_class, "slope class (0-4)", "viridis"), (eco, "ecological value (0-3)", "YlGn"),
]
for ax, (arr, title, cm) in zip(axes.ravel(), panels):
    im = ax.imshow(arr, cmap=cm, extent=dem_extent)
    ax.set_title(title, fontsize=9, weight="bold")
    ax.set_xticks([]); ax.set_yticks([]); ax.set_aspect("equal")
    plt.colorbar(im, ax=ax, shrink=0.75)
plt.tight_layout(); plt.show()

```

Explanation.

- **np.gradient(dem, CELL, CELL)** — the second and third arguments are the spacing along each axis. NumPy returns gradients **axis by axis**, and for a 2-D array axis 0 is *rows* (which run north→south) and axis 1 is *columns*. So the first returned array is dz/dy and the second is dz/dx . Getting this backwards rotates your aspect by 90° — a bug that looks plausible on a map.
- **The cell-size trap, quantified.** **np.gradient(dem)** without spacing computes change per *cell*, so on a 25 m DEM the resulting slope is roughly 25× too steep. Mean slope goes from a believable ~7° to an impossible ~80°. The check is simple: **is your mean slope physically plausible?**
- **Aspect** uses **arctan2(-dzdx, dzdy)** to produce compass bearings (0 = north, 90 = east). The mean cosine of aspect being negative confirms predominantly west-facing terrain — which is exactly right for a basin draining to a western sea. That is a free validation of the whole computation.
- **Hillshade** is the standard Lambertian formula with the sun at azimuth 315° (north-west) and altitude 45°. Note north-west lighting is a convention, not physics: humans perceive craters as domes under south-east lighting, so cartographers always light from the north-west.
- **NDVI validation.** The dataset was built so **(NIR - Red)/(NIR + Red)** reproduces the shipped NDVI. Getting a max absolute error near **0.003** (the uint16 quantisation) proves your band indexing and dtype handling are right. **Always validate band maths against a known quantity** — band order varies between sensors and providers, and there is no error message for using green as red.

- Note `.astype("float64")` on the bands. They are `uint16`; subtracting them in integer arithmetic **wraps around** for negative results, producing values near 65 535 instead of a small negative number. This is one of the most vicious silent bugs in raster work.
- **The lookup-array reclassification** (`GREENNESS[1c]`) is the idiomatic approach for categorical → categorical: build an array indexed by class code and use fancy indexing. It is a single vectorised operation over millions of cells, where `np.where` chains or dictionary lookups would take seconds.

Expected outcome.

```
TERRAIN DERIVATIVES (25 m DEM)
slope : mean  2.74 deg, median  2.58, max 12.22
slope %: mean  4.80 %,   95th pct 10.07 %
aspect : mean north component +0.016, mean east component -0.4xx
        -> the terrain faces predominantly WEST

slope WITHOUT dividing by cell size : mean 44.58 deg  <- wrong by a factor of ~25
slope WITH      cell size           : mean  2.74 deg
```

The Vallmara Basin is **gentle**: mean slope 2.7°, maximum 12.2°. That is worth knowing before you reach for alpine slope thresholds. The cell-size error inflates the mean to 44.6°, which is a physically absurd landscape-wide average — the kind of number a plausibility check catches instantly.

```
NDVI RECOMPUTED FROM BANDS vs the shipped ndvi_50m.tif
cells compared : 549,298
max |error|    : 0.000533
correlation    : 1.000000
-> your band maths is correct
```

A maximum error of 5×10^{-4} is the `uint16` quantisation of the reflectance bands, nothing more. Your band indexing, dtype casting and NoData handling are all correct.

```
SLOPE RECLASSIFIED INTO 5 ORDINAL CLASSES
0 flat          0-1 deg   ~30 %
1 gentle        1-3 deg   ~35 %
2 moderate      3-6 deg   ~25 %
3 steep         6-10 deg  ~10 %
4 very steep    10-90 deg  ~0.0 %

ECOLOGICAL-VALUE RECLASSIFICATION
value 3 (high ): 1,122,356 cells ( 50.3 %)
value 2 (medium): 744,267 cells ( 33.4 %)
value 1 (low  ): 191,594 cells (  8.6 %)
value 0 (none ): 173,432 cells (  7.8 %)
```

Then a six-panel terrain figure in which the hillshade should look like a photograph of a landscape lit from the upper left. If your hillshade looks *inverted* (valleys reading as ridges), you have the sun in the wrong place or your `dzdx/dzdy` swapped.

I15 — Zonal statistics

What we are going to learn. How to summarise raster values inside vector polygons — from scratch, efficiently, and correctly.

Why it matters. Zonal statistics is the bridge between the raster world and the vector world. "Mean elevation per district", "population per catchment", "percentage forest per block" — these are the columns that go into your model and your report.

The concept — two implementations.

1. **Loop over polygons**, masking the raster each time. Simple, correct, and `O(n)` file reads. Fine for tens of polygons.
2. **Rasterise the zones once** into an integer ID grid, then use grouped reductions (`np.bincount`, `scipy.ndimage`). One pass over the raster, regardless of polygon count. This is what you want for hundreds or thousands of zones, and it is what `rasterstats` does internally.

The three decisions you must make explicitly.

Decision	Options	Consequence
Cell inclusion	centre-in-polygon vs <code>all_touched</code>	Changes small-zone results substantially
NoData	exclude vs treat as zero	"Mean elevation" over sea cells is meaningless; "population count" over them is legitimately zero
Statistic	mean / sum / majority / percentile	Sum is only valid for counts ; mean is only valid for intensive variables

Concept — the small-polygon problem. A polygon smaller than one cell may contain **no cell centres at all**, giving NaN. With 460 census blocks against a 250 m rainfall grid, some blocks will simply have no data. Detect and handle it (fall back to `all_touched`, or to the value at the centroid) — never let it silently propagate.

Expected outcome. A reusable `zonal_stats` function, applied to all 24 districts and all 460 census blocks, validated against the values already stored in the dataset.

What the next cell does: implements zonal statistics both ways, times them, validates the district mean elevation against the layer's own `mean_elev_m` column, and demonstrates the small-polygon problem on the coarse rainfall grid.

```
In [33]: from rasterio.features import rasterize
import time

def zonal_stats(gdf, raster_path, stats=("mean", "min", "max", "std", "count"),
               band=1, all_touched=False, categorical=False):
    """Zonal statistics by rasterising the zones once. Returns a DataFrame
    indexed like `gdf`. This is O(raster), not O(raster x n_polygons)."""
    with rasterio.open(raster_path) as src:
        arr = src.read(band, masked=True)
        zones = rasterize(
            ((geom, i + 1) for i, geom in enumerate(gdf.geometry)),
            out_shape=arr.shape, transform=src.transform,
            fill=0, dtype="int32", all_touched=all_touched,
        )
        valid = (~arr.mask) & (zones > 0)
        z = zones[valid].astype(np.int64)
        v = arr.data[valid].astype("float64")
        n = len(gdf)

        counts = np.bincount(z, minlength=n + 1)[1:]
        sums = np.bincount(z, weights=v, minlength=n + 1)[1:]
        sq = np.bincount(z, weights=v**2, minlength=n + 1)[1:]
        with np.errstate(invalid="ignore", divide="ignore"):
            means = sums / counts
            var = sq / counts - means**2
        out = pd.DataFrame(index=gdf.index)
        if "count" in stats: out["count"] = counts
        if "sum" in stats: out["sum"] = np.where(counts > 0, sums, np.nan)
        if "mean" in stats: out["mean"] = np.where(counts > 0, means, np.nan)
        if "std" in stats: out["std"] = np.where(counts > 0, np.sqrt(np.maximum(var, 0)),
        np.nan)
```

```

if "min" in stats or "max" in stats:
    lo = np.full(n, np.nan); hi = np.full(n, np.nan)
    order = np.argsort(z, kind="stable")
    zs, vs = z[order], v[order]
    edges = np.searchsorted(zs, np.arange(1, n + 2))
    for i in range(n):
        a, b = edges[i], edges[i + 1]
        if b > a:
            lo[i], hi[i] = vs[a:b].min(), vs[a:b].max()
        if "min" in stats: out["min"] = lo
        if "max" in stats: out["max"] = hi
if categorical:
    for code in np.unique(v).astype(int):
        c = np.bincount(z[v == code], minlength=n + 1)[1:]
        out[f"pct_class_{code}"] = np.where(counts > 0, 100 * c / counts, np.nan)
return out

# --- 1. District elevation statistics, and validation -----
t0 = time.perf_counter()
dz = zonal_stats(districts, RAS / "dem_25m.tif")
t_fast = time.perf_counter() - t0

res = districts[["district_id", "name", "district_type", "mean_elev_m"]].join(dz)
res["abs_err"] = (res["mean"] - res["mean_elev_m"]).abs()
print("ZONAL ELEVATION STATISTICS BY DISTRICT (first 10)")
print(res[["district_id", "name", "count", "mean", "std", "min", "max",
            "mean_elev_m", "abs_err"]].head(10).round(2).to_string(index=False))
print(f"\n VALIDATION against the layer's own mean_elev_m column:")
print(f"    max absolute error : {res.abs_err.max():.4f} m")
print(f"    -> our zonal statistics reproduce the values stored in the dataset")

# --- 2. Compare with the loop-and-mask implementation -----
import rasterio.mask

def zonal_mean_loop(gdf, raster_path):
    out = []
    with rasterio.open(raster_path) as src:
        for geom in gdf.geometry:
            a, _ = rasterio.mask.mask(src, [geom.__geo_interface__], crop=True,
                                      filled=False)
            out.append(float(a.mean()) if a.count() else np.nan)
    return np.array(out)

print("\n SCALING: the two implementations, on 24 zones and on 459 zones")
print(f" {'zones':>7}{'rasterize-once':>18}{'loop-and-mask':>16}{'winner':>12}")
bl_z = blocks[~blocks.geometry.is_empty]
for label, gdf in [("24", districts), ("459", bl_z)]:
    t0 = time.perf_counter(); fast = zonal_stats(gdf, RAS / "dem_25m.tif",
                                                stats=("mean",))["mean"].to_numpy()

    tf = time.perf_counter() - t0
    t0 = time.perf_counter(); slow = zonal_mean_loop(gdf, RAS / "dem_25m.tif")
    tl = time.perf_counter() - t0
    win = "rasterize" if tf < tl else "loop"
    print(f"  {label:>7}{tf*1000:>15.0f} ms{tl*1000:>13.0f} ms{win:>12}"
          f"    (agree to {np.nanmax(np.abs(fast-slow)):.2e} m)")
print("\n The rasterize-once cost is fixed (one pass over 2.8 M cells) while")
print("  loop-and-mask cost grows linearly with the number of zones. For a handful")
print("  of zones the loop wins; by a few hundred it has lost badly.")

# --- 3. Categorical zonal statistics: % land cover per district -----
lcz = zonal_stats(districts, RAS / "landcover_25m.tif",
                  stats=("count",), categorical=True)
code2name = dict(zip(lc_legend.class_code, lc_legend.landuse_class))
lcz.columns = ["count"] + [code2name.get(int(c.split("_")[-1]), c)
                           for c in lcz.columns[1:]]
lcz.insert(0, "district", districts["name"].to_numpy())
print("\nLAND COVER COMPOSITION BY DISTRICT (% of cells, first 8)")
print(lcz.drop(columns="count").head(8).round(1).to_string(index=False))

```

```
# --- 4. The small-polygon problem -----
print("\n" + "=" * 84)
print("THE SMALL-POLYGON PROBLEM")
print("=" * 84)
print(f" {'zones':<26}{'raster':<22}{'zero-cell zones':>17}{'median cells':>14}")
print("-" * 84)
samples = [
    ("census blocks (n=459)", blocks[~blocks.geometry.is_empty],
     "rainfall_annual_250m.tif"),
    ("census blocks (n=459)", blocks[~blocks.geometry.is_empty], "dem_25m.tif"),
    ("buildings (n=800)", buildings_clean.iloc[:800], "rainfall_annual_250m.tif"),
    ("buildings (n=800)", buildings_clean.iloc[:800], "dem_25m.tif"),
]
for label, gdf, rname in samples:
    z = zonal_stats(gdf, RAS / rname, stats=("mean", "count"))
    print(f" {label:<26}{rname:<22}{int((z['count']==0).sum()):>17,}"
          f"{z['count'].median():>14,.0f}")

zb = zonal_stats(buildings_clean.iloc[:800], RAS / "rainfall_annual_250m.tif",
                 stats=("mean", "count"))
zb_at = zonal_stats(buildings_clean.iloc[:800], RAS / "rainfall_annual_250m.tif",
                   stats=("mean", "count"), all_touched=True)

print("-" * 84)
print(f" A building footprint is ~100 m^2; a 250 m rainfall cell is 62,500 m^2.")
print(f" Almost NO building contains a cell centre, so centre-in-polygon returns")
print(f" NaN for {int((zb['count']==0).sum())} of 800 buildings.")
print(f" With all_touched=True that falls to {int((zb_at['count']==0).sum())}.")
print(f" For zones smaller than a cell, do not use zonal statistics at all -")
print(f" SAMPLE the raster at the centroid instead (Lesson B13).")
```

Explanation.

- `rasterize((geom, i+1) for ...), ...)` burns each polygon into an integer grid using `i+1` as its value (0 is reserved for "no zone"). This is the key trick: after this single pass, the whole problem becomes grouped array reduction.
- `np.bincount(z, weights=v)` computes per-zone sums in one vectorised call. Combined with `np.bincount(z)` for counts, that gives means. Adding `weights=v**2` gives the variance via $E[X^2] - E[X]^2$. Three `bincount` calls and you have count, mean and standard deviation for any number of zones.
- Min and max need a different approach (they are not sums), so we sort by zone once and slice. `np.searchsorted` finds the group boundaries in $O(n \log n)$.
- **Overlapping zones are silently mishandled** by this implementation: `rasterize` writes the *last* polygon for any overlapping cell. That is correct for a partition (districts, blocks) and wrong for overlapping buffers. Check your zones do not overlap, or fall back to the loop.
- **The validation against `mean_elev_m` is the point of the exercise.** That column was computed by the data generator using exactly this method. Reproducing it to within 10^{-4} m proves the rasterisation, the masking, the transform and the NoData handling are all correct simultaneously.
- **The small-polygon demonstration** shows why resolution matching matters. On the 250 m rainfall grid a typical census block gets a handful of cells and some get none at all; on the 25 m DEM the same block gets ~100× more. If you need rainfall per block, either use `all_touched=True`, or sample at the centroid, or — best — accept that the rainfall raster does not resolve census blocks and aggregate to districts instead. **Do not manufacture precision the source data does not have.**

Expected outcome.

ZONAL ELEVATION STATISTICS BY DISTRICT (first 10)

district_id	name	count	mean	std	min	max	mean_elev_m	abs_err
D01	Old Vallmara	85315	25.07	30.84	0.40	109.03	25.1	0.03
D02	Harbourgate	108316	73.30	32.01	0.40	131.38	73.3	0.00
D05	Sundholm	129219	179.95	49.01	95.52	320.34	180.0	0.05
...								

VALIDATION against the layer's own mean_elev_m column:
max absolute error : 0.0490 m

Max error 0.049 m, which is exactly the rounding of the stored column to one decimal place. Rasterisation, masking, transform and NoData handling are all correct — one number confirming four things.

SCALING: the two implementations, on 24 zones and on 459 zones

zones	rasterize-once	loop-and-mask	winner
24	~300 ms	~140 ms	loop
459	~350 ms	~2,000 ms	rasterize

Read this carefully — it contradicts the usual advice. Rasterise-once has a fixed cost (one pass over 2.8 M cells) while loop-and-mask grows linearly with zone count. For 24 zones the loop wins; by 459 it has lost by 5–6x; at 10 000 zones it is hopeless. **Benchmark on your actual zone count**, do not assume.

The land-cover composition table shows Old Vallmara at **65% Built-up**, Brannock at **72% Cropland**, and the eastern districts dominated by Forest.

Then the small-polygon table, whose last two rows are the point: **a building footprint is ~100 m², a 250 m rainfall cell is 62 500 m²**, so almost no building contains a cell centre and centre-in-polygon returns NaN for nearly all 800.

`all_touched=True` fixes the NaN but each building then gets the value of one whole 6.25-hectare cell. **For zones smaller than a cell, do not use zonal statistics — sample at the centroid instead.**

I16 — Rasterize, polygonize, and designing an analytical map

What we are going to learn. The vector → raster → vector round trip, and how to turn a result into a map that makes an argument.

Why it matters. Some questions are easy in raster space (distance surfaces, focal statistics, overlays of many layers) and some are easy in vector space (topology, attributes, exact areas). Fluency means moving between them deliberately — and knowing what each conversion costs.

The concept — the round trip is lossy.

- **Rasterize** (`rasterio.features.rasterize`): polygons → grid. Loses exact boundaries; a curved coastline becomes a staircase. The error scales with cell size and with the perimeter-to-area ratio.
- **Polygonize** (`rasterio.features.shapes`): grid → polygons. Produces **axis-aligned staircase boundaries** with an enormous number of vertices. Always simplify afterwards, and never present raw polygonized output as if it were surveyed data.

Map design — the six decisions. A thematic map is an argument, and each of these either strengthens or undermines it:

1. **Projection** — equal-area for density, conformal for shape. (Module 2, I1.)
2. **Classification** — quantiles, natural breaks, equal interval? (Module 1, B6.)
3. **Colour** — sequential for magnitude, diverging **only** with a meaningful midpoint, qualitative for nominal. Check colour-blind safety: `viridis`, `cividis`, `RdYlBu` are safe; `jet` and red-green pairs are not.
4. **Missing data** — shown explicitly, never left white.

5. **Context** — coastline, place names, a scale bar, a north arrow.

6. **Honesty** — does the visual emphasis match the statistical strength?

Expected outcome. A distance-to-hospital raster surface built by rasterising and distance-transforming, polygonized back into service-area bands, and a publication-quality map of the result.

What the next cell does: rasterises hospitals, computes a Euclidean distance surface, reclassifies it into access bands, polygonizes them, quantifies the round-trip error, and composes a finished map with a scale bar and north arrow.

```
In [34]: from rasterio.features import rasterize, shapes
from scipy.ndimage import distance_transform_edt
from matplotlib.patches import Rectangle
import matplotlib.pyplot as plt

# --- 1. Vector -> raster: a distance-to-hospital surface -----
CELL_M = 100.0
with rasterio.open(RAS / "dem_25m.tif") as src:
    bounds = src.bounds
    land_mask_hi = src.read(1, masked=True).mask
    h = int((bounds.top - bounds.bottom) / CELL_M)
    w = int((bounds.right - bounds.left) / CELL_M)
    transform = rasterio.Affine(CELL_M, 0, bounds.left, 0, -CELL_M, bounds.top)

hosp = facilities_clean[facilities_clean.facility_type == "hospital"]
hosp_grid = rasterize([(g, 1) for g in hosp.geometry], out_shape=(h, w),
                      transform=transform, fill=0, dtype="uint8")
land_grid = rasterize([(LAND_GEOM, 1)], out_shape=(h, w), transform=transform,
                      fill=0, dtype="uint8").astype(bool)

dist_km = distance_transform_edt(hosp_grid == 0, sampling=CELL_M) / 1000.0
dist_km = np.where(land_grid, dist_km, np.nan)

print("DISTANCE-TO-HOSPITAL SURFACE")
print(f"  grid          : {w} x {h} at {CELL_M:.0f} m")
print(f"  hospitals burnt : {int(hosp_grid.sum())} cells (from {len(hosp)} points)")
print(f"  distance range  : {np.nanmin(dist_km):.2f} - {np.nanmax(dist_km):.2f} km")
print(f"  mean / median   : {np.nanmean(dist_km):.2f} / {np.nanmedian(dist_km):.2f} km")

# --- 2. Reclassify into access bands -----
BANDS = [0, 5, 10, 20, 999]
BAND_LABELS = ["within 5 km", "5-10 km", "10-20 km", "over 20 km"]
band = np.digitize(dist_km, BANDS[1:-1], right=False).astype("float32")
band[~np.isfinite(dist_km)] = np.nan

# population in each band, via the population-density raster
popd = align_to(RAS / "popdens_100m.tif", dict(
    height=h, width=w, transform=transform, crs=CRS_UTM), Resampling.bilinear)
cell_km2 = (CELL_M / 1000) ** 2
print(f"\n{'access band':<14}{'area km2':>12}{'% of land':>11}"
      f"{'population':>14}{'% of people':>13}")
print("-" * 66)
tot_pop = np.nansum(popd * cell_km2)
for i, lab in enumerate(BAND_LABELS):
    m = band == i
    pop = np.nansum(np.where(m, popd, 0) * cell_km2)
    print(f"{'lab':<14}{'m.sum()*cell_km2:>12,.0f}{'100*m.sum()/np.isfinite(band).sum():>10.1f}%"
          f"{'pop:>14,.0f}{'100*pop/tot_pop:>12.1f}%"")

# --- 3. Raster -> vector: polygonize the bands -----
polys = []
band_i = np.where(np.isfinite(band), band, -1).astype("int16")
for geom, val in shapes(band_i, mask=np.isfinite(band), transform=transform):
    polys.append({"band": int(val),
                  "geometry": Polygon(geom["coordinates"][0], geom["coordinates"][1:])})
access = gpd.GeoDataFrame(polys, crs=CRS_UTM)
access["label"] = access.band.map(dict(enumerate(BAND_LABELS)))
```

```

access_d = access.dissolve(by="band").reset_index()
access_d["label"] = access_d.band.map(dict(enumerate(BAND_LABELS)))

print(f"\nPOLYGONIZED: {len(access)} raw polygons -> {len(access_d)} dissolved bands")
raw_v = int(sum(shapely.count_coordinates(g) for g in access_d.geometry))
simp = access_d.copy()
simp["geometry"] = access_d.geometry.simplify(200).buffer(0)
simp_v = int(sum(shapely.count_coordinates(g) for g in simp.geometry))
print(f" vertices, raw staircase : {raw_v:>9,}")
print(f" vertices, simplify(200) : {simp_v:>9,} "
      f"({100*(1-simp_v/raw_v):.1f} % removed)")
print(f" area change from simplifying: "
      f"{100*(simp.geometry.area.sum()-access_d.geometry.area.sum())/
access_d.geometry.area.sum():+.3f} %")

# --- 4. Round-trip error, and what controls it -----
print("\n" + "=" * 86)
print("ROUND-TRIP ERROR: rasterisation loss depends on SHAPE, not just cell size")
print("=" * 86)
targets = {
    "whole basin (compact)": LAND_GEOM,
    "one district": districts.geometry.iloc[5],
    "100-yr flood zone (ribbons)": flood[flood.return_period_yr == 100].geometry.union_all(),
    "riparian strip 50 m": rivers.geometry.union_all().buffer(50),
}
print(f" {'geometry':<28}{'area km2':>10}{'P/sqrt(A)':>11}"
      + "".join(f"{cs:>7} m" for cs in [25, 100, 250, 500]))
print("-" * 86)
for label, geom in targets.items():
    a_v = geom.area / 1e6
    shape_idx = geom.length / np.sqrt(geom.area) # 3.54 for a circle
    row = f" {label:<28}{a_v:>10,.1f}{shape_idx:>11,.1f}"
    for cs in [25, 100, 250, 500]:
        hh = int((bounds.top - bounds.bottom) / cs)
        ww = int((bounds.right - bounds.left) / cs)
        tr = rasterio.Affine(cs, 0, bounds.left, 0, -cs, bounds.top)
        g = rasterize([(geom, 1)], out_shape=(hh, ww), transform=tr,
                      fill=0, dtype="uint8")
        a_r = g.sum() * (cs / 1000) ** 2
        row += f"{100*(a_r-a_v)/a_v:>8.2f}%"
    print(row)
print("-" * 86)
print(" P/sqrt(A) is a shape index: 3.54 for a circle, larger for convoluted shapes.")
print(" Rasterisation error scales with PERIMETER, so it is negligible for compact")
print(" shapes and severe for thin ribbons - exactly the shapes hazard zones have.")

# --- 5. A finished map -----
fig, ax = plt.subplots(figsize=(10.5, 8.5))
COLORS = ["#1a9850", "#a6d96a", "#fdae61", "#d73027"]
sea.plot(ax=ax, facecolor="#cfe3f2", edgecolor="none", zorder=0)
for i, lab in enumerate(BAND_LABELS):
    sub = simp[simp.band == i]
    if len(sub):
        sub.plot(ax=ax, facecolor=COLORS[i], edgecolor="none", alpha=0.85,
                zorder=1, label=lab)
districts.boundary.plot(ax=ax, color="white", linewidth=0.6, zorder=2)
roads[roads.road_class.isin(["motorway", "primary"])]plot(
    ax=ax, color="#d4d4d4", linewidth=0.7, zorder=3)
hosp.plot(ax=ax, color="white", markersize=170, marker="P",
          edgecolor="black", linewidth=1.4, zorder=5)
hosp.plot(ax=ax, color="#b2182b", markersize=90, marker="P", zorder=6)

for _, r in districts[districts.district_type == "urban_core"].iterrows():
    ax.annotate(r["name"], (r.geometry.centroid.x, r.geometry.centroid.y),
               ha="center", fontsize=8.5, weight="bold", color="white", zorder=7,
               path_effects=[pe.withStroke(linewidth=2.4, foreground="#222")])

# scale bar

```

```

x0, y0 = bounds.left + 2_000, bounds.bottom + 2_500
for k in range(2):
    ax.add_patch(Rectangle((x0 + k*5000, y0), 5000, 700,
                           facecolor="black" if k % 2 == 0 else "white",
                           edgecolor="black", linewidth=0.8, zorder=8))
ax.text(x0, y0 + 1200, "0", fontsize=7, ha="center", zorder=8)
ax.text(x0 + 10000, y0 + 1200, "10 km", fontsize=7, ha="center", zorder=8)
# north arrow
ax.annotate("N", xy=(bounds.right - 3_000, bounds.top - 4_500),
            xytext=(bounds.right - 3_000, bounds.top - 9_000),
            arrowprops=dict(arrowstyle="->", linewidth=1.8, color="black"),
            ha="center", fontsize=11, weight="bold", zorder=8)

ax.legend(loc="upper left", fontsize=8, frameon=True, framealpha=0.95,
         title="Distance to nearest hospital", title_fontsize=9)
ax.set_title("Hospital accessibility in the Vallmara Basin",
            fontsize=14, weight="bold", loc="left")
ax.text(0.0, -0.035,
       "Straight-line distance from a 100 m grid. Road-network distance would be "
       "20-50 % longer.\nSource: fictional Vallmara Basin dataset. "
       "CRS: EPSG:32633 (UTM 33N).",
       transform=ax.transAxes, fontsize=7, color="#555", va="top")
ax.set_aspect("equal"); ax.set_xticks([]); ax.set_yticks([])
for sp in ax.spines.values():
    sp.set_visible(False)
plt.tight_layout(); plt.show()

```

Explanation.

- `rasterize([(geom, value), ...])` burns geometries into a grid. Points become single cells — note that 4 hospital points produce exactly 4 cells, so nothing is lost here. Polygons are a different story (see the round-trip table).
- `distance_transform_edt(mask, sampling=CELL_M)` computes, for every `True` cell, the Euclidean distance to the nearest `False` cell. We invert the logic (`hosp_grid == 0`) so the "obstacles" are the hospitals. **sampling= converts the result from cells to metres** — omit it and every distance is out by a factor of 100.
- This is a **Euclidean** distance surface. A true accessibility surface would be a cost surface over the road network (`skimage.graph.MCP` or a routing engine), which is why the map's caption says so explicitly. Stating the limitation on the map itself is not optional.
- `shapes(array, mask=, transform=)` is the inverse operation, yielding `(geojson_geometry, value)` pairs. Note the mask: without it you polygonize the NaN region too and get a giant meaningless polygon.
- **The vertex count is the lesson of polygonization.** Raw output traces every cell corner, so a band boundary crossing 500 cells has ~1 000 vertices. After `simplify(200)` you keep the shape and lose 90%+ of the vertices, changing the area by a fraction of a percent.
- **The round-trip error table** quantifies rasterisation loss directly: at 25 m the land area is right to a few hundredths of a percent; at 500 m it is off by a percent or more, and the sign depends on where the coastline falls relative to the cell centres. **Rule of thumb: your cell must be much smaller than the features you care about**, at least 4–5 cells across the narrowest feature.
- **The map.** Every element earns its place: the sea gives context; white district boundaries separate without competing; roads explain *why* the bands have their shape; the hospital markers are drawn twice (white halo, then red) so they read against any background; labels get a stroke outline for the same reason; the scale bar and north arrow make it a map rather than a picture; and the caption states the method and its limitation. The colour ramp runs green → red, which is the one case where a red-green scheme is defensible (it is *ordered*, and the lightness also varies, so it survives greyscale and most colour-vision deficiencies).

Expected outcome.

DISTANCE-TO-HOSPITAL SURFACE

grid : 480 x 360 at 100 m
hospitals burnt : 4 cells (from 4 points)
distance range : 0.00 – 34.74 km
mean / median : 14.27 / 13.52 km

access band	area km2	% of land	population	% of people
within 5 km	206	14.7%	529,666	78.2%
5–10 km	311	22.3%	75,773	11.2%
10–20 km	488	35.0%	57,492	8.5%
over 20 km	390	28.0%	14,073	2.1%

The headline finding: 14.7% of the land holds 78.2% of the people within 5 km of a hospital, while 28% of the basin — 14 000 people — is more than 20 km away. Read it both ways. As a *population* statistic the service looks excellent: four in five residents are close to a hospital. As a *territorial* statistic it looks poor: over a quarter of the region is more than 20 km out. Neither framing is dishonest; which one you lead with is the political content of your report. Say both.

POLYGONIZED: 5 raw polygons → 4 dissolved bands
vertices, raw staircase : 3,021
vertices, simplify(200) : 166 (94.5 % removed)
area change from simplifying: +0.017 %

94.5% of vertices removed for a 0.017% area change. Never ship raw polygonized output.

Then the round-trip table, whose point is the shape index $P/\sqrt{A}$ (3.54 for a circle):

geometry	area km2	P/sqrt(A)	25 m	100 m	250 m	500 m
whole basin (compact)	1,394.8	4.3	0.00%	0.00%	-0.03%	-0.00%
one district	53.9	4.4	-0.00%	0.05%	-0.50%	-0.73%
100-yr flood zone (ribbons)	186.0	57.9	0.06%	0.50%	0.60%	0.13%
riparian strip 50 m	20.7	82.1	-0.04%	-0.05%	12.46%	1.30%

Rasterisation error scales with perimeter, not area. The compact shapes (index ≈ 4.3 , close to a circle's 3.54) are essentially exact at every resolution. The 50 m riparian strip (index 82) is fine at 25 m and 100 m and then jumps to **+12.5% at 250 m** — the point at which the cell becomes wider than the strip itself.

Note that the error is **not monotone**: it falls back to 1.3% at 500 m. Cells that wrongly include strip-free ground are partly cancelled by cells that wrongly exclude strip ground, and at some resolutions the cancellation is lucky. **Never treat a small error at one resolution as evidence that the discretisation is sound** — check the resolution you will actually use, and prefer the rule of thumb: your cell must be several times smaller than the narrowest feature you care about.

Finally the finished map.

Exercises — Module 2 (Intermediate)

Solutions are in the **Solutions** section at the end. Try each one first.

Exercise 2.1 — A defensible CRS choice

Objective. You are asked to report (a) the total area of protected land, and (b) the total length of the river network, for a client who will publish the figures. Compute each in at least three CRS, including one equal-area and one equidistant projection, and compute the geodesic ground truth with `pyproj.Geod`. Produce a short table and a one-paragraph recommendation stating which figure you would publish for each quantity and why.

Exercise 2.2 — Build a validity-and-cleaning report

Objective. Write a function `qa_report(gdf, name)` that returns a one-row DataFrame with: row count, CRS, geometry types, counts of null / empty / invalid geometries, count of duplicated geometries (compare WKB), number of features with zero area or zero length, and the total area or length. Run it over **all twelve** GeoPackage layers plus the two GeoJSON files and present a single combined table. Then repair every problem you found and show the report again.

Exercise 2.3 — Riparian buffer compliance

Objective. Regional law requires a protected strip along every watercourse: **200 m** for Strahler order 4, **120 m** for order 3, **50 m** for order 2. Building inside the strip is prohibited.

1. How many buildings violate the rule, and what is their total value?
2. Which district has the worst violation rate (violations per 1 000 buildings)?
3. What percentage of the protected strip is currently built-up land cover?
4. Produce a map of violations.

Careful: do not double-count buildings that fall inside two overlapping strips.

Exercise 2.4 — Land-use change accounting

Objective. Suppose a proposed reservoir is defined as the area within 400 m of the Vallmara River **below 30 m elevation**. Build that polygon (you will need both raster and vector operations), then produce a table of how many hectares of each land-use class would be lost, how many buildings would be inundated, their total value, and how many people would be displaced (use dasymetric weighting).

Exercise 2.5 — Correct areal interpolation

Objective. The socio-economic table gives `median_income_vs` per **district**. Estimate median income per **census block** using areal interpolation, then recompute district-level income from your block estimates and check whether you recover the original. Explain in two sentences why the recovery is (or is not) exact, and what that tells you about interpolating an *intensive* variable.

Exercise 2.6 — Zonal statistics at two resolutions

Objective. Compute mean annual rainfall per district (a) from the native 250 m raster and (b) from the same raster bilinearly upsampled to 25 m. Compare the two sets of district means. Are they identical? Should they be? Which would you report, and what does the difference tell you about the value of upsampling?

Exercise 2.7 — Build the analysis-ready feature table

Objective. Produce a single tidy table with **one row per census block** and these columns:

Group	Columns
Identity	<code>block_id</code> , <code>district_id</code> , <code>district_type</code>
Demography	<code>population</code> , <code>households</code> , <code>pop_density_km2</code> , <code>area_km2</code>
Terrain	<code>mean_elev_m</code> , <code>mean_slope_deg</code> , <code>min_elev_m</code>
Climate	<code>mean_rainfall_mm</code> , <code>mean_lst_c</code>
Vegetation	<code>mean_ndvi</code> , <code>pct_forest</code> , <code>pct_builtup</code>
Hazard	<code>pct_in_flood100</code> , <code>pct_in_flood500</code> , <code>dist_river_m</code>
Access	<code>dist_hospital_m</code> , <code>dist_clinic_m</code> , <code>dist_school_m</code> , <code>dist_primary_road_m</code> , <code>n_bus_stops</code>
Assets	<code>n_buildings</code> , <code>total_value_kvs</code> , <code>mean_building_age</code>

Every column must be correct: right CRS, NoData honoured, no double counting, missing values as NaN rather than 0. Save it to `data/outputs/block_features.gpkg`. **You will use this table for the whole of Module 3 and the capstone, so get it right.**

Challenge 2.8 — Reproduce the generating law

Objective. The dataset was generated with `rainfall = 470 + 0.62 × elevation + 150 × (north-south position) + noise`.

Recover all three coefficients from the rasters alone. Then answer: how much does your estimate change if you (a) work at 25 m instead of 250 m, (b) use district means instead of raw cells, (c) forget to exclude NoData? Quantify each effect and explain which is the most dangerous in practice.

Module 3 — Advanced: Spatial Data Science

Modules 1 and 2 were about *doing GIS in Python*. Module 3 is about **spatial data science**: bringing statistics, machine learning and decision analysis to bear on spatial problems, and handling the ways in which space breaks the assumptions those methods were built on.

The central problem. Classical statistics and machine learning assume observations are independent. Spatial observations are not: nearby places resemble each other. That single fact has three consequences that run through this entire module.

Consequence	Symptom	Remedy
Effective sample size < n	Standard errors too small, everything "significant"	Test residual autocorrelation; use spatial error/lag models
Random train/test splits leak	Cross-validated accuracy far above true out-of-area accuracy	Spatially blocked cross-validation
Omitted spatial confounders	Coefficients attributed to the wrong variable	Include the confounder, or a spatial trend / eigenvector filter

The 14 lessons

#	Lesson	Technique
A0	The analysis-ready feature table	Assembling everything from Module 2
A1	Spatial feature engineering	Proximity, density, focal, composition, lag
A2	Multi-criteria decision analysis	Weighted overlay, AHP, sensitivity analysis
A3	Environmental suitability modelling	Raster MCDA with hard constraints
A4	Urban accessibility and equity	Two-step floating catchment area (2SFCA)
A5	Spatial autocorrelation	Moran's I from scratch, permutation inference
A6	Hotspot analysis	Getis-Ord Gi*, LISA, multiple testing
A7	Point pattern analysis	KDE, quadrat test, nearest-neighbour index
A8	Spatial clustering	DBSCAN, K-means, spatially constrained regionalisation
A9	Spatial regression	OLS, residual diagnostics, spatial lag features
A10	Predictive modelling I	Flood susceptibility: design and feature matrix
A11	Predictive modelling II	Spatial cross-validation and the leakage it prevents
A12	Model interpretation	Permutation importance, partial dependence, surfaces
A13	Quantitative risk	Hazard × exposure × vulnerability, expected annual damage
A14	Communicating spatial results	What to show, what to caveat, what not to claim

A0 — The analysis-ready feature table

What we are about to do. Assemble one tidy table with **one row per census block** and every attribute we will need for the rest of the course.

Why it matters. This is the deliverable that separates a spatial *analyst* from a spatial *data scientist*. Everything downstream — clustering, regression, machine learning, risk modelling — consumes this one table. If it is wrong, everything is wrong; if it is right, the rest is ordinary data science.

The concept — the analysis base table. In tabular data science this is routine. In spatial work it is the hard part, because every column comes from a different geometry, a different resolution and possibly a different CRS. The recipe:

1. Choose the **analysis unit** (here: census blocks) and its **support** (polygons, ~3 km² each).
2. For each source, choose the **correct transfer operation**: zonal statistics for rasters, spatial join + aggregate for points, overlay + area weighting for polygons, `sjoin_nearest` for proximity.
3. Recompute every absolute quantity after any geometry-splitting operation.
4. Keep missing values as **NaN**, never as 0.
5. **Validate** each column against something you already know.

A warning about the unit of analysis. Everything you conclude is conditional on this choice — the **modifiable areal unit problem**. Results computed on blocks and on districts can differ in magnitude and even in sign. State the unit, and where it matters, repeat the analysis at a second scale to check robustness.

What the next cell does: builds the full feature table — identity, demography, terrain, climate, vegetation, hazard, accessibility and assets — validates every group, and saves it to `data/outputs/block_features.gpkg`.

```
In [35]: from rasterio.features import rasterize
         from rasterio.warp import Resampling
         from scipy.spatial import cKDTree
```

```

# ----- identity -----
F = blocks[~blocks.geometry.is_empty].copy().reset_index(drop=True)
F = F.merge(districts[["district_id", "district_type", "dist_core_km"]],
            on="district_id", how="left")
F = F[["block_id", "district_id", "district_type", "dist_core_km",
      "area_km2", "population", "households", "pop_density_km2", "geometry"]]
F["centroid"] = F.geometry.representative_point()

# ----- terrain -----
zs = zonal_stats(F, RAS / "dem_25m.tif", stats=("mean", "min", "max", "std"))
F["mean_elev_m"] = zs["mean"].to_numpy()
F["min_elev_m"] = zs["min"].to_numpy()
F["elev_range_m"] = (zs["max"] - zs["min"]).to_numpy()

# slope: write the derived raster once, then run zonal statistics on it
with rasterio.open(RAS / "dem_25m.tif") as src:
    dem_a = src.read(1, masked=True).astype("float64").filled(np.nan)
    prof = src.profile.copy()
    gy, gx = np.gradient(dem_a, prof["transform"].a, prof["transform"].a)
    slope_a = np.degrees(np.arctan(np.hypot(gx, gy)))
    slope_path = OUT / "slope_deg_25m.tif"
    prof.update(dtype="float32", nodata=-9999.0, compress="deflate")
    with rasterio.open(slope_path, "w", **prof) as dst:
        dst.write(np.nan_to_num(slope_a, nan=-9999.0).astype("float32"), 1)
F["mean_slope_deg"] = zonal_stats(F, slope_path, stats=("mean",))["mean"].to_numpy()

# ----- climate -----
F["mean_rainfall_mm"] = zonal_stats(
    F, RAS / "rainfall_annual_250m.tif", stats=("mean",))["mean"].to_numpy()
lst_path = OUT / "lst_utm_100m.tif"
lst_arr = align_to(RAS / "lst_summer_100m_3857.tif", ref, Resampling.bilinear)
lp = ref.copy(); lp.update(dtype="float32", nodata=-9999.0)
with rasterio.open(lst_path, "w", **lp) as dst:
    dst.write(np.nan_to_num(lst_arr, nan=-9999.0).astype("float32"), 1)
F["mean_lst_c"] = zonal_stats(F, lst_path, stats=("mean",))["mean"].to_numpy()

# ----- vegetation -----
F["mean_ndvi"] = zonal_stats(F, RAS / "ndvi_50m.tif", stats=("mean",))["mean"].to_numpy()
lcz = zonal_stats(F, RAS / "landcover_25m.tif", stats=("count",), categorical=True)
for code, name in zip(lc_legend.class_code, lc_legend.landuse_class):
    col = f"pct_class_{code}"
    key = "pct_" + name.lower().split()[0].replace("-", "").replace("/", "")
    F[key] = lcz[col].to_numpy() if col in lcz.columns else 0.0

# ----- hazard -----
for rp in (100, 500):
    z = flood[flood.return_period_yr == rp].geometry.union_all()
    inter = F.geometry.intersection(z).area
    F[f"pct_in_flood{rp}"] = (100 * inter / F.geometry.area).to_numpy()

for name, tgt in [("river", rivers), ("coast", coastline)]:
    geom = tgt.geometry.union_all()
    F[f"dist_{name}_m"] = F["centroid"].distance(geom).to_numpy()

# ----- accessibility -----
cent = gpd.GeoDataFrame(F[["block_id"]], geometry=F["centroid"], crs=CRS_UTM)
for ftype in ["hospital", "clinic", "school", "fire_station"]:
    tgt = facilities_clean[facilities_clean.facility_type == ftype][["geometry"]]
    j = gpd.sjoin_nearest(cent, tgt, how="left",
                          distance_col="d").drop_duplicates("block_id")
    F[f"dist_{ftype}_m"] = j.set_index("block_id")["d"].reindex(F.block_id).to_numpy()

prim = roads[roads.road_class.isin(["motorway", "primary"])[["geometry"]]
j = gpd.sjoin_nearest(cent, prim, how="left", distance_col="d").drop_duplicates("block_id")
F["dist_primary_road_m"] = j.set_index("block_id")["d"].reindex(F.block_id).to_numpy()

sj = gpd.sjoin(stops[["stop_id", "geometry"]], F[["block_id", "geometry"]],

```

```

        predicate="within")
F["n_bus_stops"] = (F.block_id.map(sj.groupby("block_id").size())
                    .fillna(0).astype(int).to_numpy())

# ----- assets -----
bpt = buildings_clean.copy()
bpt["geometry"] = buildings_clean.geometry.representative_point()
bj = gpd.sjoin(bpt, F[["block_id", "geometry"]], predicate="within")
agg = bj.groupby("block_id").agg(
    n_buildings=("building_id", "size"),
    total_value_kvs=("value_kvs", "sum"),
    mean_year_built=("year_built", "mean"),
    total_floorspace_m2=("footprint_m2", "sum"))
for c in agg.columns:
    F[c] = F.block_id.map(agg[c]).to_numpy()
F["n_buildings"] = F["n_buildings"].fillna(0).astype(int)
F["mean_building_age"] = 2025 - F["mean_year_built"]

# ----- validation -----
F = F.drop(columns=["centroid", "mean_year_built"])
FEATS = [c for c in F.columns if c not in ("block_id", "district_id",
                                           "district_type", "geometry")]
print(f"ANALYSIS-READY FEATURE TABLE: {len(F)} blocks x {len(FEATS)} features\n")
chk = pd.DataFrame({
    "dtype": F[FEATS].dtypes.astype(str),
    "n_nan": F[FEATS].isna().sum(),
    "min": F[FEATS].min(numeric_only=True).round(2),
    "median": F[FEATS].median(numeric_only=True).round(2),
    "max": F[FEATS].max(numeric_only=True).round(2),
})
print(chk.to_string())

print("\nVALIDATION")
pct_cols = [c for c in F.columns if c.startswith("pct_class_")
            or c.startswith("pct_") and c.startswith("pct_") and "flood" not in c]
lc_pct = [c for c in F.columns if c.startswith("pct_") and "flood" not in c]
print(f" land-cover percentages sum to 100 : "
      f"{np.allclose(F[lc_pct].sum(axis=1).dropna(), 100, atol=0.5)}")
print(f" population sums to the regional total : "
      f"{F.population.sum():,} (blocks) vs {blocks.population.sum():,} (all blocks incl. empty)")
print(f" buildings assigned : "
      f"{F.n_buildings.sum():,} of {len(buildings_clean):,}")
print(f" blocks with no buildings : {int((F.n_buildings == 0).sum())}")
print(f" every distance is finite : "
      f"{bool(np.isfinite(F[[c for c in F.columns if c.startswith('dist_')]].to_numpy()).all())}")

out_fp = OUT / "block_features.gpkg"
F.to_file(out_fp, layer="block_features", driver="GPKG")
print(f"\nSaved -> {out_fp.name} ({out_fp.stat().st_size/1024:.0f} KB)")

```

Explanation.

- **Identity first.** `block_id` is the primary key; `district_id` and `district_type` come along so we can aggregate or stratify later.
- **representative_point() not centroid** for all proximity work — it is guaranteed to lie inside the block.
- **Terrain.** We write the slope raster to disk before running zonal statistics on it. That may look wasteful, but it means the slope layer is inspectable, reusable and self-documenting — and `zonal_stats` takes a path. In a real pipeline, materialise your derived rasters.
- **Climate.** The LST raster is in EPSG:3857, so it is reprojected onto the common grid with `align_to before` zonal statistics. Running zonal statistics across a CRS mismatch is a silent-wrong-answer generator.

- **Land-cover composition** uses the `categorical=True` branch of `zonal_stats`, producing one percentage column per class. Composition features like `pct_forest` and `pct_builtup` are far more informative to a model than a single majority class.
- **Hazard.** `F.geometry.intersection(z).area / F.geometry.area` gives the exact fraction of each block inside the flood zone — a *continuous* exposure measure rather than a binary flag. Binary flags throw away most of the signal.
- **Accessibility.** `sjoin_nearest` with `drop_duplicates` per block, then `.set_index(...).reindex(...)` to write back positionally-safely.
- **Assets.** Buildings are reduced to representative points before joining, so each building is counted in exactly one block. Joining polygons to polygons with `intersects` would double-count buildings straddling a boundary.
- **`fillna(0)` only where zero is the truth.** `n_bus_stops` and `n_buildings` are genuine counts — a block with no bus stop has zero. But `total_value_kvs` and `mean_building_age` stay NaN, because "no buildings" means *undefined* mean age, not zero.

Expected outcome. A summary table of about **30 features across 459 blocks**, then four validation checks:

- land-cover percentages summing to 100 ($\pm$ rounding) — proves the categorical zonal statistics are complete;
- block population reconciling with the regional total;
- ~5 197 of 5 200 buildings assigned (three fall outside the block tessellation);
- every distance finite — no unmatched `sjoin_nearest`.

If any check fails, fix it **before** going further. Everything from here on consumes this table.

A1 — Spatial feature engineering

What we are going to learn. The five families of spatial feature, and how to build each one.

Why it matters. In tabular ML, feature engineering is where domain knowledge enters. In *spatial* ML, geometry itself is a source of features that no amount of model capacity can substitute for. A gradient-boosted tree cannot invent "distance to the nearest river" from raw coordinates; you must build it.

The five families.

Family	Question it answers	How
Proximity	How far to the nearest X?	<code>sjoin_nearest</code> , <code>cKDTree</code> , distance transforms
Density / intensity	How much X is near here?	Counts in a buffer, KDE, focal sums
Composition	What is this place made of?	Zonal statistics on categorical rasters
Focal / neighbourhood	What is the <i>surroundings</i> like?	Focal statistics on rasters; <code>k</code> -NN means on vectors
Spatial lag	What are my <i>neighbours</i> ' values?	Contiguity or distance weights $\times$ the variable

The spatial lag deserves special attention. For a variable `y` and a row- standardised weights matrix `W`, the spatial lag is `Wy` — the weighted average of the neighbours' values. It is the single most useful engineered spatial feature, and it is also the thing that makes naive cross-validation leak (A11).

Concept — a caution about coordinates as features. Feeding raw `x` and `y` into a tree model lets it memorise the training locations. It will look excellent under random CV and fail completely on new territory. Use coordinates only with spatially blocked validation, and prefer *interpretable* spatial features.

Expected outcome. Twelve new features added to the table, each with a stated rationale, plus a correlation analysis showing which are redundant.

What the next cell does: builds focal, density, lag and interaction features, then examines their correlation structure to find redundancy before modelling.

```
In [36]: from scipy.ndimage import uniform_filter, generic_filter
from scipy.spatial import cKDTree

Fx = Fx.copy()
cxy = np.c_[Fx.geometry.representative_point().x, Fx.geometry.representative_point().y]

# --- (1) DENSITY: how much of X is within r metres? -----
def density_within(points_gdf, radius_m, weight_col=None):
    """Count (or weighted sum) of `points_gdf` within radius of each block."""
    p = np.c_[points_gdf.geometry.x, points_gdf.geometry.y]
    w = (points_gdf[weight_col].fillna(0).to_numpy() if weight_col
         else np.ones(len(points_gdf)))
    tree = cKDTree(p)
    idx = tree.query_ball_point(cxy, r=radius_m)
    return np.array([w[i].sum() for i in idx])

bpts = buildings_clean.copy()
bpts["geometry"] = buildings_clean.geometry.representative_point()
Fx["bldg_density_1km"] = density_within(bpts, 1000) / (np.pi * 1.0**2)
Fx["value_density_1km"] = density_within(bpts, 1000, "value_kvs") / (np.pi * 1.0**2)
Fx["facilities_within_3km"] = density_within(facilities_clean, 3000)
Fx["stops_within_1km"] = density_within(stops, 1000)

# --- (2) FOCAL: what are the SURROUNDINGS like? -----
with rasterio.open(RAS / "dem_25m.tif") as src:
    dem_a = src.read(1, masked=True).astype("float64").filled(np.nan)
    tr = src.transform
# Topographic Position Index: elevation minus the mean of a 1 km neighbourhood
win = int(1000 / tr.a) # 1 km / 25 m = 40 cells
filled = np.where(np.isfinite(dem_a), dem_a, 0.0)
counts = uniform_filter(np.isfinite(dem_a).astype(float), size=win, mode="nearest")
local_mean = uniform_filter(filled, size=win, mode="nearest") / np.maximum(counts, 1e-9)
tpi = np.where(np.isfinite(dem_a), dem_a - local_mean, np.nan)

tpi_path = OUT / "tpi_1km_25m.tif"
tp = prof.copy(); tp.update(dtype="float32", nodata=-9999.0)
with rasterio.open(tpi_path, "w", **tp) as dst:
    dst.write(np.nan_to_num(tpi, nan=-9999.0).astype("float32"), 1)
Fx["mean_tpi"] = zonal_stats(Fx, tpi_path, stats=("mean",))["mean"].to_numpy()

# --- (3) SPATIAL LAG: what are my NEIGHBOURS like? -----
def knn_weights(coords, k=6, row_standardise=True):
    """Row-standardised k-nearest-neighbour spatial weights matrix."""
    tree = cKDTree(coords)
    d, idx = tree.query(coords, k=k + 1) # first neighbour is self
    n = len(coords)
    W = np.zeros((n, n))
    rows = np.repeat(np.arange(n), k)
    W[rows, idx[:, 1:].ravel()] = 1.0
    if row_standardise:
        W = W / W.sum(axis=1, keepdims=True)
    return W

W6 = knn_weights(cxy, k=6)

def lag(values):
    v = np.asarray(values, dtype=float)
    ok = np.isfinite(v)
    Wm = W6 * ok[None, :]
    denom = Wm.sum(axis=1)
    return np.where(denom > 0, (Wm @ np.nan_to_num(v)) / np.maximum(denom, 1e-12), np.nan)

for col in ["pop_density_km2", "mean_elev_m", "pct_in_flood100", "mean_ndvi"]:
    Fx[f"lag_{col}"] = lag(Fx[col])
```

```
# --- (4) INTERACTION / RATIO features -----
Fx["people_per_building"] = Fx.population / Fx.n_buildings.replace(0, np.nan)
Fx["value_per_capita"] = Fx.total_value_kvs / Fx.population.replace(0, np.nan)
Fx["relief_ratio"] = Fx.elev_range_m / np.sqrt(Fx.area_km2 * 1e6)
Fx["access_index"] = -(np.log1p(Fx.dist_hospital_m) + np.log1p(Fx.dist_clinic_m)
                      + np.log1p(Fx.dist_fire_station_m)) / 3

NEW = ["bldg_density_1km", "value_density_1km", "facilities_within_3km",
       "stops_within_1km", "mean_tpi", "lag_pop_density_km2", "lag_mean_elev_m",
       "lag_pct_in_flood100", "lag_mean_ndvi", "people_per_building",
       "value_per_capita", "relief_ratio", "access_index"]
print(f"ENGINEERED {len(NEW)} NEW FEATURES\n")
print(Fx[NEW].describe().T[["mean", "50%", "std", "min", "max"]].round(2).to_string())

# --- (5) Redundancy check -----
NUM = [c for c in Fx.columns
        if Fx[c].dtype.kind in "if" and c not in ("block_id",)]
C = Fx[NUM].corr(numeric_only=True).abs()
Cv = C.to_numpy(copy=True) # pandas 3 returns read-only views
np.fill_diagonal(Cv, 0)
C = pd.DataFrame(Cv, index=C.index, columns=C.columns)
pairs = (C.where(np.triu(np.ones(C.shape), 1).astype(bool))
         .stack().sort_values(ascending=False))
print("\nMOST CORRELATED FEATURE PAIRS (|r| > 0.90)")
strong = pairs[pairs > 0.90]
for (a, b), r in strong.items():
    print(f" {r:.3f} {a:<26} <=> {b}")
print(f"\n {len(strong)} pairs above 0.90 out of {len(pairs):,} - "
      f"these are candidates for removal before any linear model.")

# --- Picture: four engineered surfaces -----
fig, axes = plt.subplots(1, 4, figsize=(19, 4.8))
panels = [("bldg_density_1km", "Building density (per km2, 1 km radius)", "magma"),
          ("mean_tpi", "Topographic Position Index (1 km)", "RdBu_r"),
          ("lag_pct_in_flood100", "Spatial lag of flood exposure", "Blues"),
          ("access_index", "Composite access index (higher = better)", "viridis")]
for ax, (col, title, cm) in zip(axes, panels):
    Fx.plot(ax=ax, column=col, cmap=cm, scheme="quantiles", k=6, legend=True,
            legend_kwds={"loc": "lower left", "fontsize": 5.5}, edgecolor="none")
    ax.set_title(title, fontsize=9, weight="bold", loc="left")
    ax.set_aspect("equal"); ax.set_xticks([]); ax.set_yticks([])
plt.tight_layout(); plt.show()
```

Explanation.

- `ckdtree.query_ball_point(points, r)` returns, for each query point, the list of all tree points within `r`. This is the efficient way to build "count of X within a radius" features — far better than buffering and spatially joining. Passing a `weight_col` turns a count into a weighted sum (here, asset value).
- **TPI (Topographic Position Index)** is elevation minus the mean elevation of a surrounding window. Positive = ridge or local high, negative = valley or depression, near zero = uniform slope. **It is a far better flood-exposure predictor than raw elevation**, because what matters hydrologically is whether you are low *relative to your surroundings*, not low in absolute terms.
- The `uniform_filter` trick handles NaN correctly: filter the *filled* array and the *validity mask* separately, then divide. A plain `uniform_filter` over an array containing NaN propagates NaN across the whole window.
- `knn_weights` builds a row-standardised k-nearest-neighbour weights matrix `W`, where $W[i, j] = 1/k$ if `j` is one of `i`'s `k` nearest neighbours. `W @ y` is then the mean of each block's neighbours. We use `k = 6` — a common default, roughly the number of contiguous neighbours in an irregular tessellation.
- **Why the lag features matter.** `lag_pct_in_flood100` says "how exposed are my neighbours?". A block that is itself dry but surrounded by floodplain is operationally at risk: its access roads flood, its services are disrupted. Raw exposure misses this entirely.

- **Ratio features** (`people_per_building`, `value_per_capita`) normalise out size, which is usually a nuisance variable. `relief_ratio` normalises elevation range by block size so that big and small blocks are comparable.
- **The redundancy check** matters because spatial features are *constructed* from overlapping geometry and are therefore correlated by design. `bldg_density_1km` and `value_density_1km` will be nearly collinear. For tree models this is harmless; for linear models and for *interpretation* it is fatal — collinear features split the coefficient arbitrarily between them.

Expected outcome. A describe table for 13 new features, a list of highly correlated pairs (expect `bldg_density_1km` ↔ `value_density_1km`, `pop_density_km2` ↔ `bldg_density_1km`, and each variable with its own lag), and four maps. Look at the TPI map in particular: the river valleys should appear as connected blue (negative) ribbons, which is a visual confirmation that the focal computation is correct.

A2 — Multi-criteria decision analysis

What we are going to learn. How to combine several incommensurable criteria into one defensible score, and how to test whether the answer depends on your weights.

Why it matters. "Where should we put the next clinic?" has no objective answer — it depends on how you trade off population served against travel distance against deprivation. MCDA makes that trade-off **explicit, auditable and testable**, instead of hiding it inside a modeller's judgement.

The concept — the four steps of weighted overlay.

1. **Choose criteria** and their direction (benefit: more is better; cost: less is better).
2. **Normalise** each to a common scale, usually [0, 1]. Options: min-max, rank-based (robust to outliers), or a value function encoding what the decision-maker actually cares about (often non-linear — travelling 40 km is more than twice as bad as 20 km).
3. **Weight** the criteria so the weights sum to 1.
4. **Aggregate** — either **weighted sum** (compensatory: a great score on one criterion offsets a poor one) or **weighted product / geometric mean** (non-compensatory: a near-zero on any criterion drags the whole score down). Choose deliberately: for siting where a hard requirement exists, the geometric mean is usually more honest.

Where do weights come from? AHP. In the Analytic Hierarchy Process you make *pairwise* comparisons ("access is 3x more important than deprivation") on a 1–9 scale, build a reciprocal matrix, and take its principal eigenvector as the weights. Crucially it also yields a **consistency ratio**: if your comparisons are internally contradictory ($A > B$, $B > C$, $C > A$) the CR exceeds 0.1 and you must revise them. **Report the CR** — it is what makes elicited weights auditable.

Sensitivity analysis is not optional. If the recommended site changes when you perturb a weight by 10%, your recommendation is an artefact of the weights, not of the data. Say so.

Expected outcome. A ranked shortlist of census blocks for a new clinic, built from four criteria with AHP weights, plus a sensitivity analysis showing how stable the top of the ranking is.

What the next cell does: normalises four criteria, derives weights by AHP with a consistency check, aggregates by both weighted sum and geometric mean, runs a 1 000-iteration Monte-Carlo weight perturbation, and maps the result.

```
In [37]: M = Fx.copy()

# --- 1. CRITERIA -----
# benefit = more is better; cost = less is better
CRITERIA = {
    "population":      ("benefit", "people who would be served"),
    "dist_clinic_m":    ("benefit", "distance to the nearest existing clinic"),
    "dist_primary_road_m": ("cost",  "a clinic must be reachable by road"),
    "pct_in_flood100":  ("cost",  "do not build in the floodplain"),
}
```

```

def normalise(v, direction, method="rank"):
    """Scale to [0, 1]. Rank-based normalisation is robust to the extreme
    skew that spatial variables almost always have."""
    v = pd.Series(v).astype(float)
    if method == "rank":
        s = v.rank(pct=True, na_option="keep")
    else:
        # min-max
        s = (v - v.min()) / (v.max() - v.min())
    return s if direction == "benefit" else 1 - s

N = pd.DataFrame({c: normalise(M[c], d) for c, (d, _) in CRITERIA.items()})
print("CRITERIA, normalised to [0, 1] (rank-based)")
print(N.describe().T[["mean", "50%", "min", "max"]].round(3).to_string())

# --- 2. AHP WEIGHTS from pairwise comparisons -----
# how many times more important is the ROW criterion than the COLUMN one?
labels = list(CRITERIA)
A = np.array([
    #      pop    dist_clinic    road    flood
    [1.0,    2.0,    3.0,    4.0], # population served
    [1/2.0, 1.0,    2.0,    3.0], # underserved-ness
    [1/3.0, 1/2.0, 1.0,    2.0], # road access
    [1/4.0, 1/3.0, 1/2.0, 1.0], # flood avoidance
])
eigval, eigvec = np.linalg.eig(A)
k = np.argmax(eigval.real)
w = np.abs(eigvec[:, k].real); w = w / w.sum()
lam_max = eigval.real[k]
n = len(labels)
CI = (lam_max - n) / (n - 1)
RI = {1: 0, 2: 0, 3: 0.58, 4: 0.90, 5: 1.12, 6: 1.24, 7: 1.32}[n]
CR = CI / RI

print("\nAHP WEIGHTS")
for lab, wi in zip(labels, w):
    print(f" {lab:<22} {wi:.3f} ({CRITERIA[lab][1]})")
print(f" lambda_max = {lam_max:.4f}, CI = {CI:.4f}, CR = {CR:.4f} "
      f"'{'CONSISTENT (CR < 0.10)' if CR < 0.10 else 'INCONSISTENT - revise!'}")

# --- 3. AGGREGATE two ways -----
M["score_sum"] = (N.to_numpy() * w).sum(axis=1)
M["score_geom"] = np.exp((np.log(np.clip(N.to_numpy(), 1e-6, None)) * w).sum(axis=1))

print(f"\nrank correlation between the two aggregations: "
      f"{M.score_sum.corr(M.score_geom, method='spearman'):.4f}")
top_sum = set(M.nlargest(10, 'score_sum').block_id)
top_geo = set(M.nlargest(10, 'score_geom').block_id)
print(f"top-10 overlap: {len(top_sum & top_geo)} of 10 blocks")

cols = ["block_id", "district_id", "population", "dist_clinic_m",
        "dist_primary_road_m", "pct_in_flood100", "score_sum"]
print("\nTOP 10 CANDIDATE BLOCKS FOR A NEW CLINIC (weighted sum)")
print(M.nlargest(10, "score_sum")[cols].round(2).to_string(index=False))

# --- 4. SENSITIVITY: does the answer survive perturbed weights? -----
rng = np.random.default_rng(7)
SIMS = 1000
appear = np.zeros(len(M))
rank_sum = np.zeros(len(M))
for _ in range(SIMS):
    wp = np.abs(w * rng.normal(1.0, 0.25, size=n)) # +/-25 % perturbation
    wp = wp / wp.sum()
    sc = (N.to_numpy() * wp).sum(axis=1)
    order = np.argsort(-sc)
    appear[order[:10]] += 1
    rank_sum += pd.Series(-sc).rank().to_numpy()

```

```

M["top10_frequency"] = appear / SIMS
M["mean_rank"] = rank_sum / SIMS
stable = M.nlargest(12, "top10_frequency")[
    ["block_id", "district_id", "score_sum", "top10_frequency", "mean_rank"]]
print(f"\nSENSITIVITY ANALYSIS ({SIMS} simulations, weights perturbed +/-25 %)")
print(stable.round(3).to_string(index=False))
print(f"\n blocks appearing in the top 10 in EVERY simulation : "
      f"{int((M.top10_frequency == 1.0).sum())}")
print(f" blocks appearing at least once : "
      f"{int((M.top10_frequency > 0).sum())}")
print(" -> a robust recommendation names only the blocks with high frequency.")

# --- 5. Map -----
fig, axes = plt.subplots(1, 3, figsize=(17.5, 5.6))
for ax, (col, title, cm) in zip(axes, [
    ("score_sum", "MCDA score (weighted sum)", "YlOrRd"),
    ("score_geom", "MCDA score (geometric mean)", "YlOrRd"),
    ("top10_frequency", "Robustness: P(top 10) over 1000 weightings", "viridis")]):
    M.plot(ax=ax, column=col, cmap=cm, scheme="quantiles", k=7, legend=True,
           legend_kwds={"loc": "lower left", "fontsize": 6}, edgecolor="none")
    facilities_clean[facilities_clean.facility_type == "clinic"].plot(
        ax=ax, color="cyan", markersize=22, marker="o",
        edgecolor="black", linewidth=0.5)
    ax.set_title(title, fontsize=10, weight="bold", loc="left")
    ax.set_aspect("equal"); ax.set_xticks([]); ax.set_yticks([])
plt.tight_layout(); plt.show()

```

Explanation.

- **Rank normalisation over min–max.** Spatial variables are almost always heavily skewed — **population** here runs from 24 to 35 667. Min–max scaling would compress 95% of the blocks into the bottom 10% of the range, so the score would be driven entirely by one or two outliers. **rank(pct=True)** is robust and produces a uniform distribution. The cost is that you lose magnitude information: the difference between rank 0.98 and 0.99 could be 10 people or 10 000.
- **Direction handling** with **1 – s** for cost criteria. Get this backwards and you site the clinic in the floodplain.
- **AHP.** The matrix **A** is *reciprocal* ($A[j, i] = 1/A[i, j]$) with 1s on the diagonal. The principal eigenvector is the weight vector. **$\lambda_{\max}$** equals **n** exactly if your judgements are perfectly consistent; the excess drives the Consistency Index and, divided by a random-matrix baseline **RI**, the **Consistency Ratio**. **CR < 0.10** is the conventional acceptance threshold.
- **Weighted sum vs geometric mean.** The weighted sum is fully compensatory: a block with zero road access can still score well if its population is huge. The geometric mean cannot — a zero on any criterion drives the whole score to zero. For siting problems where every criterion is a genuine requirement, the geometric mean is the more honest aggregation. Comparing the two top-10 lists tells you how much the choice matters.
- **The sensitivity analysis is the most important block here.** We perturb the weights by $\pm 25\%$ a thousand times and record how often each block reaches the top 10. A block that appears every time is a robust recommendation. A block that appears 30% of the time is an artefact of one particular weighting, and presenting it as "the answer" would be indefensible. **Always report the frequency, not just the point ranking.**

Expected outcome.

```
AHP WEIGHTS
population           0.467
dist_clinic_m        0.277
dist_primary_road_m  0.160
pct_in_flood100      0.095
lambda_max = 4.0310, CI = 0.0103, CR = 0.0115  CONSISTENT (CR < 0.10)
```

```
rank correlation between the two aggregations: 0.8338
top-10 overlap: 10 of 10 blocks
```

CR = 0.0115, far below the 0.10 threshold — the pairwise judgements are internally coherent. Note also that the two aggregation rules agree on the whole top 10 here even though their rank correlation across all 459 blocks is only 0.83: they disagree substantially in the *middle* of the ranking and agree at the extremes, which is the usual pattern.

Look carefully at the normalised criteria table: **pct_in_flood100 has median 0.696 and maximum 0.696**. Most blocks have zero flood exposure, so they all tie at the same rank and none can score the full 1.0. Rank normalisation with heavy ties silently caps the achievable score on that criterion, which effectively *reduces* its weight below the 0.095 you specified. If a criterion is mostly zeros, use min-max or a purpose-built value function instead.

```
SENSITIVITY ANALYSIS (1000 simulations, weights perturbed +/-25 %)
block_id district_id score_sum top10_frequency mean_rank
B0422      D04      0.771      0.991      3.040
B0409      D19      0.748      0.942      7.034
B0230      D18      0.773      0.922      3.871
B0224      D22      0.772      0.896      4.813
B0314      D18      0.754      0.802      8.444
...
blocks appearing in the top 10 in EVERY simulation : 0
blocks appearing at least once                    : 41
```

This is the result that matters. Not one block survives every weighting, and **41 different blocks** reach the top 10 at some point. The defensible recommendation is therefore not "build at B0230" (the point-estimate winner) but "**B0422, B0409, B0230 and B0224 are robust candidates, appearing in the top 10 in 90–99% of plausible weightings**". Note that B0230 has the *highest* point score but only the *third* highest robustness — ranking by the point estimate alone would have picked a less stable site.

Three maps; the third (robustness) is the one you would actually publish.

A3 — Environmental suitability modelling on rasters

What we are going to learn. The same MCDA logic applied cell-by-cell to raster surfaces, with **hard constraints** as well as soft preferences.

Why it matters. Vector MCDA is limited to whatever units you happen to have. Raster MCDA works at the resolution of the underlying data and can express continuous preference surfaces, which is how real siting studies (wind, solar, conservation, landfill) are done.

The concept — constraints and factors.

- **Constraints** are Boolean: legal or physical exclusions. Protected area? No. Inside a flood zone? No. Slope over 15°? No. Multiply the suitability by 0.
- **Factors** are continuous preferences, normalised to [0, 1] and weighted.

$$\text{suitability} = (\prod \text{constraints}) \times (\sum w_i \cdot \text{factor}_i)$$

Value functions. Do not assume linear normalisation. For solar irradiance, more is monotonically better. For distance to a grid connection, there is a threshold beyond which the project is uneconomic — a sigmoid or a piecewise function encodes that far better than a straight line. **The shape of the value function is a modelling assumption and should be stated.**

Concept — resolution and the constraint budget. Every hard constraint removes land. Apply five constraints carelessly and you may exclude 99% of the study area, leaving a "suitability map" that is really a map of the one place you forgot to exclude. Always report **how much land each constraint removes**.

Expected outcome. A solar-farm suitability surface for the Vallmara Basin, with a constraint audit, and the top candidate sites extracted as polygons.

What the next cell does: builds four constraint layers and four factor layers on the common 100 m grid, reports how much land each constraint removes, combines them, and extracts contiguous high-suitability parcels above 10 ha.

```
In [38]: from rasterio.features import rasterize, shapes
from scipy.ndimage import label as cc_label, distance_transform_edt

H, Wd = ref["height"], ref["width"]
TR = ref["transform"]
CELL = TR.a

from shapely.geometry.base import BaseGeometry

def burn(gdf_or_geom, invert=False):
    # NOTE: a GeoDataFrame also has a .geom_type attribute, so test the TYPE,
    # not the presence of the attribute – otherwise the whole frame is passed
    # to rasterize as one geometry and silently skipped.
    geoms = ([gdf_or_geom] if isinstance(gdf_or_geom, BaseGeometry)
              else list(gdf_or_geom.geometry))
    a = rasterize([(g, 1) for g in geoms], out_shape=(H, Wd), transform=TR,
                  fill=0, dtype="uint8").astype(bool)
    return ~a if invert else a

land_g = burn(LAND_GEOM)
slope_g = align_to(slope_path, ref, Resampling.average)
elev_g = grids["elevation"]
lc_g = grids["landcover"]
ndvi_g = grids["ndvi"]
pop_g = grids["popdens"]

# --- 1. HARD CONSTRAINTS -----
constraints = {
    "on land": land_g,
    "slope <= 5 deg": np.nan_to_num(slope_g, nan=99) <= 5,
    "outside protected areas": burn(protected, invert=True),
    "outside the 100-yr floodplain": burn(flood[flood.return_period_yr == 100],
                                          invert=True),
    "not built-up or forest": ~np.isin(np.nan_to_num(lc_g, nan=0), [2, 5]),
    "not water or wetland": ~np.isin(np.nan_to_num(lc_g, nan=0), [1, 8]),
}

print("CONSTRAINT AUDIT (cells remaining after each is applied in turn)")
print(f" {'constraint':<32}{'passes':>12}{'km2':>10}{'% of land':>11}{'lost km2':>11}")
print("-" * 78)
mask = np.ones((H, Wd), dtype=bool)
land_km2 = land_g.sum() * (CELL/1000)**2
prev = land_g.sum()
for name, c in constraints.items():
    mask &= c
    n = int(mask.sum())
    print(f" {name:<32}{n:>12},{n*(CELL/1000)**2:>10,.1f}"
          f"{100*n/land_g.sum():>10.1f}%{(prev-n)*(CELL/1000)**2:>11,.1f}")
    prev = n
print("-" * 78)
print(f" {'ELIGIBLE LAND':<32}{int(mask.sum()):>12,}")
```

```

f"{mask.sum()*(CELL/1000)**2:>10,.1f}{100*mask.sum()/land_g.sum():>10.1f}%")

# --- 2. FACTORS with explicit value functions -----
def linear_vf(x, lo, hi):
    """Linear ramp: 0 below lo, 1 above hi (or reversed if lo > hi)."""
    return np.clip((x - lo) / (hi - lo), 0, 1)

def sigmoid_vf(x, mid, steep):
    return 1.0 / (1.0 + np.exp((x - mid) / steep))

# distance to the primary road network (grid connection proxy), in metres
road_g = burn(roads[roads.road_class.isin(["motorway", "primary"])])
d_road = distance_transform_edt(~road_g, sampling=CELL)
# distance to built-up areas (demand centres)
built_g = np.nan_to_num(lc_g, nan=0) == 2
d_built = distance_transform_edt(~built_g, sampling=CELL)

factors = {
    "flat terrain": (linear_vf(np.nan_to_num(slope_g, nan=99), 5, 0), 0.35),
    "close to the grid": (sigmoid_vf(d_road, 3000, 900), 0.30),
    "low ecological value": (linear_vf(np.nan_to_num(ndvi_g, nan=1), 0.75, 0.15), 0.20),
    "close to demand": (sigmoid_vf(d_built, 8000, 2500), 0.15),
}

wsum = sum(w for _, w in factors.values())
print(f"\nFACTORS (weights sum to {wsum:.2f})")
for name, (arr, w) in factors.items():
    print(f"    {name:<24} w={w:.2f}    value-function range "
          f"{np.nanmin(arr):.2f} - {np.nanmax(arr):.2f}")

score = sum(arr * w for arr, w in factors.values()) / wsum
suit = np.where(mask, score, np.nan)

print(f"\nSUITABILITY over eligible land: min {np.nanmin(suit):.3f}, "
      f"median {np.nanmedian(suit):.3f}, max {np.nanmax(suit):.3f}")
for q in [50, 75, 90, 95, 99]:
    t = np.nanpercentile(suit, q)
    n = int(np.nansum(suit >= t))
    print(f"    >= {q}th percentile ({t:.3f}) : {n:>7,} cells "
          f"    f"= {n*(CELL/1000)**2:>7,.1f} km^2")

# --- 3. Extract contiguous candidate parcels -----
THRESH = float(np.nanpercentile(suit, 95))
MIN_HA = 10.0
hi = np.nan_to_num(suit, nan=0) >= THRESH
lbl, nlab = cc_label(hi)
recs = []
for geom, val in shapes(lbl.astype("int32"), mask=hi, transform=TR):
    poly = Polygon(geom["coordinates"][0], geom["coordinates"][1:].buffer(0))
    if poly.area / 1e4 < MIN_HA:
        continue
    m = lbl == int(val)
    recs.append({"parcel_id": f"P{len(recs)+1:03d}", "area_ha": poly.area / 1e4,
                "mean_suit": float(np.nanmean(suit[m])),
                "geometry": poly})
parcels = gpd.GeoDataFrame(recs, crs=CRS_UTM).sort_values(
    "area_ha", ascending=False).reset_index(drop=True)

print(f"\nCANDIDATE PARCELS (suitability >= {THRESH:.3f}, area >= {MIN_HA:.0f} ha)")
print(f"    connected components above threshold : {nlab:,}")
print(f"    parcels large enough to develop      : {len(parcels)}")
if len(parcels):
    parcels["district"] = gpd.sjoin(
        parcels, districts[["name", "geometry"]], how="left",
        predicate="intersects").drop_duplicates("parcel_id")["name"].to_numpy()
    print(parcels.head(8)[["parcel_id", "area_ha", "mean_suit", "district"]]
          .round(2).to_string(index=False))
    print(f"\n    total developable area : {parcels.area_ha.sum():.0f} ha "
          f"    f"({parcels.area_ha.sum()/100:.1f} km^2, ")

```

```

f"{100*parcels.area_ha.sum()*1e4/LAND_GEOM.area:.2f} % of the basin)")

# --- 4. Map -----
fig, axes = plt.subplots(1, 3, figsize=(17.5, 5.6))
ext = (TR.c, TR.c + Wd*CELL, TR.f - H*CELL, TR.f)
axes[0].imshow(np.where(mask, 1, np.nan), cmap="Greens", extent=ext, vmin=0, vmax=1.4)
protected.plot(ax=axes[0], facecolor="none", edgecolor="darkgreen", linewidth=1.0)
flood[flood.return_period_yr == 100].plot(ax=axes[0], facecolor="#9ecae1",
                                           edgecolor="none", alpha=0.6)
axes[0].set_title(f"(a) Eligible land after constraints\n"
                  f"{mask.sum()*(CELL/1000)**2:.0f} km^2 "
                  f"({100*mask.sum()/land_g.sum():.1f} % of the basin)",
                  fontsize=10, weight="bold", loc="left")
im = axes[1].imshow(suit, cmap="RdYlGn", extent=ext, vmin=0, vmax=1)
plt.colorbar(im, ax=axes[1], shrink=0.75, label="suitability")
axes[1].set_title("(b) Suitability score over eligible land",
                  fontsize=10, weight="bold", loc="left")
land.plot(ax=axes[2], facecolor="#f2efe6", edgecolor="#ccc6b8", linewidth=0.5)
if len(parcels):
    parcels.plot(ax=axes[2], column="mean_suit", cmap="RdYlGn", vmin=0.5, vmax=1,
                 edgecolor="black", linewidth=0.6, legend=True,
                 legend_kwds={"shrink": 0.6, "label": "mean suitability"})
roads[roads.road_class.isin(["motorway", "primary"])].plot(
    ax=axes[2], color="#777", linewidth=0.6)
axes[2].set_title(f"(c) {len(parcels)} developable parcels >= {MIN_HA:.0f} ha",
                  fontsize=10, weight="bold", loc="left")
for a in axes:
    a.set_aspect("equal"); a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.show()

```

Explanation.

- **The constraint audit is the ethical core of this lesson.** Applying constraints one at a time and reporting the land removed by each makes the result reproducible and arguable. If a stakeholder disputes the slope threshold, the table tells everyone immediately how much it matters.
- `burn(..., invert=True)` rasterises a layer and negates it, giving "outside X". Note that `rasterize` uses cell centres by default, so a protected-area boundary is respected to within half a cell (50 m here) — acceptable for a screening study, not for a legal determination.
- **Value functions.** `linear_vf(slope, 5, 0)` gives 1 at 0° falling linearly to 0 at 5° — a *descending* ramp, expressed by putting the "good" end second. `sigmoid_vf(d_road, 3000, 900)` is ~1 within 2 km of a road, ~0.5 at 3 km, and ~0 beyond 5 km, encoding the economics of a grid connection far better than a straight line.
State your value functions; they carry as much of your judgement as the weights do.
- `distance_transform_edt(~road_g, sampling=CELL)` gives a continuous distance-to-road surface in metres. Note `sampling=` again.
- **Connected-component labelling** (`scipy.ndimage.label`) groups adjacent high-suitability cells into candidate parcels. This matters because a solar farm needs a *contiguous* site: 500 scattered hectares is not the same as one 500-hectare block. Filtering by minimum parcel area converts a suitability surface into a *deliverable*.
- The 95th-percentile threshold is a choice, not a finding. Report the area available at several thresholds (as we do) so the reader can pick their own.

Expected outcome.

CONSTRAINT AUDIT (cells remaining after each is applied in turn)				
constraint	passes	km2	% of land	lost km2
on land	139,478	1,394.8	100.0%	0.0
slope <= 5 deg	128,541	1,285.4	92.2%	109.4
outside protected areas	124,711	1,247.1	89.4%	38.3
outside the 100-yr floodplain	107,514	1,075.1	77.1%	172.0
not built-up or forest	47,174	471.7	33.8%	603.4
not water or wetland	46,672	466.7	33.5%	5.0
ELIGIBLE LAND	46,672	466.7	33.5%	

The audit immediately tells you where the argument will be: **the "not built-up or forest" rule removes 603 km², nearly six times more than everything else combined.** If a stakeholder wants to challenge one assumption, that is the one — and now they can, precisely, instead of arguing about the map.

SUITABILITY over eligible land: min 0.151, median 0.625, max 0.950
 >= 95th percentile (0.838) : 2,334 cells = 23.3 km²

CANDIDATE PARCELS (suitability >= 0.838, area >= 10 ha)

connected components above threshold : 83
 parcels large enough to develop : 6

parcel_id	area_ha	mean_suit	district
P006	1,179.0	0.87	Kestrel Quay
P004	764.0	0.86	Old Vallmara
P002	148.0	0.87	Ardenfeld
P001	77.0	0.88	Ashcombe
P005	13.0	0.89	Old Vallmara
P003	11.0	0.86	Ardenfeld

total developable area : 2,192 ha (21.9 km², 1.57 % of the basin)

Follow the collapse: 100% → 33.5% eligible → 1.7% above the suitability threshold → and then only 6 of 83 high-suitability clumps are large enough to build on. "One third of the basin is eligible" and "1.57% is actually developable" are both true, and only the second is useful. The contiguity step is what turns a suitability surface into a deliverable, and it is the step most often skipped.

Three panels: eligible land, the suitability surface, and the final parcels.

A4 — Urban accessibility and spatial equity

What we are going to learn. Two-Step Floating Catchment Area (2SFCA) — the standard method for measuring spatial access to services — and how to turn it into an equity statement.

Why it matters. "Distance to the nearest hospital" ignores **capacity** and **competition**. A hospital 2 km away serving 200 000 people may be less accessible than one 8 km away serving 5 000. 2SFCA fixes this, and it is what health-geography and transport-equity work actually uses.

The concept — 2SFCA in two steps.

Step 1. For each supply location j with capacity S_j , find all demand locations within the catchment (travel threshold d_0) and compute the **provider-to-population ratio**:

$$R_j = S_j / \sum_{k \in \text{catchment}(j)} P_k$$

Step 2. For each demand location i , sum the ratios of all supply locations within *its* catchment:

$$A_i = \sum_{j \in \text{catchment}(i)} R_j$$

A_i has units of *providers per person*. Higher is better. It captures supply, demand and competition simultaneously.

Enhanced 2SFCA (E2SFCA) replaces the hard catchment boundary with a distance- decay weight $W(d)$ — because a facility 1 km away is more accessible than one at 29 km, even if both are "within 30 km". We implement E2SFCA with a Gaussian decay.

Concept — measuring inequity. Once you have A_i per block with population P_i , the **population-weighted Gini coefficient** of A summarises how unequally access is distributed: 0 = perfectly equal, 1 = maximally unequal. Pair it with a **concentration curve** (cumulative access against cumulative population, ordered by access) for a defensible equity statement.

Expected outcome. E2SFCA accessibility for clinics and hospitals, a Gini coefficient, a concentration curve, and a map showing which communities are underserved.

What the next cell does: implements E2SFCA with Gaussian decay, computes it for two service types, calculates the population-weighted Gini, and identifies the worst-served populations.

```
In [39]: from scipy.spatial import cKDTree

DEMAND_XY = np.c_[Fx.geometry.representative_point().x,
                  Fx.geometry.representative_point().y]
DEMAND_POP = Fx.population.to_numpy(dtype=float)
P = DEMAND_POP # short alias used in this lesson only

def e2sfca(supply_gdf, capacity_col, d0_m, decay="gaussian",
          demand_xy=None, demand_pop=None):
    """Enhanced two-step floating catchment area accessibility.

    Demand is passed in explicitly (defaulting to the module-level block
    centroids and populations) so the function never silently depends on a
    global that some later cell might rebind.
    """
    demand_xy = DEMAND_XY if demand_xy is None else demand_xy
    P = DEMAND_POP if demand_pop is None else demand_pop
    sx = np.c_[supply_gdf.geometry.x, supply_gdf.geometry.y]
    S = supply_gdf[capacity_col].to_numpy(dtype=float)
    S = np.where(np.isfinite(S), S, np.nanmedian(S)) # impute unknown capacity

    D = np.sqrt(((demand_xy[:, None, :] - sx[None, :, :]) ** 2).sum(axis=2))
    if decay == "gaussian":
        Wt = np.exp(-0.5 * (D / (d0_m / 2.0)) ** 2)
        Wt[D > d0_m] = 0.0
    else: # hard catchment
        Wt = (D <= d0_m).astype(float)

    # STEP 1: provider-to-population ratio at each supply point
    demand_j = (Wt * P[:, None]).sum(axis=0)
    Rj = np.divide(S, demand_j, out=np.zeros_like(S), where=demand_j > 0)
    # STEP 2: sum the weighted ratios reachable from each demand point
    return (Wt * Rj[None, :]).sum(axis=1)

clinics = facilities_clean[facilities_clean.facility_type == "clinic"]
hosps = facilities_clean[facilities_clean.facility_type == "hospital"]

Fx["access_clinic"] = e2sfca(clinics, "capacity", 10_000) * 1000 # per 1,000 people
Fx["access_hospital"] = e2sfca(hosps, "capacity", 30_000) * 1000

print("E2SFCA ACCESSIBILITY (provider capacity per 1,000 residents)")
for col, label, n in [("access_clinic", "clinics (10 km catchment)", len(clinics)),
                      ("access_hospital", "hospitals (30 km catchment)", len(hosps))]:
    a = Fx[col]
    covered = 100 * (a > 0).mean()
```

```

pop_zero = Fx.loc[a == 0, "population"].sum()
print(f"\n {label} ({n} facilities)")
print(f"    blocks with ANY access : {covered:5.1f} %")
print(f"    people with NO access   : {pop_zero:,.0f} "
      f"({100*pop_zero/P.sum():.1f} % of the population)")
print(f"    min / median / max      : {a.min():.3f} / {a.median():.3f} / {a.max():.3f}")

# --- Equity: population-weighted Gini and concentration curve -----
def weighted_gini(x, w):
    x = np.asarray(x, float); w = np.asarray(w, float)
    o = np.argsort(x); x, w = x[o], w[o]
    cw = np.cumsum(w); cxw = np.cumsum(x * w)
    if cxw[-1] == 0:
        return np.nan
    cxw = cxw / cxw[-1]; cw = cw / cw[-1]
    return float(1 - np.sum((cxw[1:] + cxw[:-1]) * np.diff(cw)))

# A Gini is only interpretable for a BENEFIT (more is better). To compare
# E2SFCA against the naive alternative fairly, convert distance to a benefit.
Fx["prox_clinic"]   = 1.0 / (1.0 + Fx.dist_clinic_m / 1000.0)
Fx["prox_hospital"] = 1.0 / (1.0 + Fx.dist_hospital_m / 1000.0)

print("\n" + "=" * 78)
print("SPATIAL EQUITY (population-weighted Gini; all measures are BENEFITS)")
print("=" * 78)
for col, label in [("prox_clinic", "clinic - naive proximity 1/(1+d_km)",
                    "access_clinic", "clinic - E2SFCA (capacity + competition)",
                    "prox_hospital", "hospital - naive proximity 1/(1+d_km)",
                    "access_hospital", "hospital - E2SFCA")]:
    print(f" {label:<46} Gini = {weighted_gini(Fx[col], P):6.3f}")
print("\n 0 = everyone has identical access; 1 = one person has all of it.")
print(" A Gini computed on a COST (distance) is not comparable with one computed")
print(" on a BENEFIT - low inequality in distance and low inequality in access")
print(" mean opposite things. Always convert to a common direction first.")

# who is worst off?
worst = Fx.nsmallest(400, "access_clinic")
cum = worst.population.cumsum()
print(f"\n The 20 % of the population with the LOWEST clinic access:")
thr = Fx.sort_values("access_clinic").assign(c=lambda d: d.population.cumsum())
bottom20 = thr[thr.c <= 0.20 * P.sum()]
print(f"    blocks           : {len(bottom20)}")
print(f"    people           : {bottom20.population.sum():,}")
print(f"    mean access      : {bottom20.access_clinic.mean():.4f} "
      f"vs regional mean {Fx.access_clinic.mean():.4f}")
print(f"    district types   : {bottom20.district_type.value_counts().to_dict()}")
print(f"    mean dist to clinic : {bottom20.dist_clinic_m.mean()/1000:.1f} km "
      f"vs {Fx.dist_clinic_m.mean()/1000:.1f} km regionally")

# --- Figure -----
fig = plt.figure(figsize=(17, 5.4))
ax1 = fig.add_subplot(1, 3, 1)
Fx.plot(ax=ax1, column="access_clinic", cmap="RdYlGn", scheme="quantiles", k=7,
        legend=True, legend_kwds={"loc": "lower left", "fontsize": 6},
        edgecolor="none")
clinics.plot(ax=ax1, color="black", markersize=16, marker="o")
ax1.set_title("Clinic accessibility (E2SFCA)", fontsize=10, weight="bold", loc="left")
ax1.set_aspect("equal"); ax1.set_xticks([]); ax1.set_yticks([])

ax2 = fig.add_subplot(1, 3, 2)
Fx.plot(ax=ax2, column="access_hospital", cmap="RdYlGn", scheme="quantiles", k=7,
        legend=True, legend_kwds={"loc": "lower left", "fontsize": 6},
        edgecolor="none")
hosps.plot(ax=ax2, color="black", markersize=45, marker="P")
ax2.set_title("Hospital accessibility (E2SFCA)", fontsize=10, weight="bold", loc="left")
ax2.set_aspect("equal"); ax2.set_xticks([]); ax2.set_yticks([])

ax3 = fig.add_subplot(1, 3, 3)

```

```

for col, lab, c in [("access_clinic", "clinics", "#2166ac"),
                  ("access_hospital", "hospitals", "#b2182b")]:
    d = Fx.sort_values(col)
    cp = np.cumsum(d.population) / d.population.sum()
    ca = np.cumsum(d[col] * d.population)
    ca = ca / ca.iloc[-1]
    ax3.plot(cp, ca, color=c, linewidth=2,
             label=f"{lab} (Gini = {weighted_gini(Fx[col], P):.3f})")
ax3.plot([0, 1], [0, 1], "k--", linewidth=1, label="perfect equality")
ax3.set_xlabel("cumulative share of population\n(ordered from worst to best access)")
ax3.set_ylabel("cumulative share of accessibility")
ax3.set_title("Concentration curves", fontsize=10, weight="bold", loc="left")
ax3.legend(fontsize=8); ax3.grid(alpha=0.3)
plt.tight_layout(); plt.show()

```

Explanation.

- **The full distance matrix D** is 459×18 here, which is trivial. For a national study with 50 000 demand points and 3 000 facilities you would use a **cKDTree** ball query and a sparse matrix instead — the mathematics is unchanged.
- **The Gaussian decay** $\exp(-\frac{1}{2}(d/(d_0/2))^2)$ truncated at d_0 puts the half-weight point at roughly $0.6 d_0$. Any monotone decreasing kernel works; the choice is a modelling assumption. The literature also uses stepped weights (e.g. 1.0 / 0.68 / 0.22 for successive distance bands), which are easier to explain to non-specialists.
- **Step 1 computes competition.** $demand_j$ is the *weighted* population that can reach facility j . Dividing capacity by it gives capacity **per person** at that facility. This is what makes 2SFCA different from a simple buffer count.
- **Step 2 sums availability.** A block near three modestly-provisioned clinics can score better than one next to a single overwhelmed clinic — which is the behaviour we want and which nearest-distance cannot express.
- **Capacity imputation.** Recall that 11 facilities have $capacity = NaN$ after we cleaned the **-999** sentinel. Here we impute the median. That is a defensible choice **only because we state it**; the alternative (dropping them) would understate supply.
- **weighted_gini** implements the standard trapezoidal Lorenz-curve formula with population weights, so that a block of 20 000 people counts 1 000 times a block of 20. **Unweighted Gini over polygons is a common and serious error** — it treats a vast empty upland block as equal in importance to a dense city block.
- **The concentration curve** is the visual form of the same statistic. The further it sags below the diagonal, the more unequal the distribution.

Expected outcome.

```

clinics (10 km catchment)  (18 facilities)
  blocks with ANY access :   71.7 %
  people with NO access  : 14,919 (2.3 % of the population)

```

```

hospitals (30 km catchment)  (4 facilities)
  blocks with ANY access :   98.3 %
  people with NO access  :   796 (0.1 % of the population)

```

```

SPATIAL EQUITY (population-weighted Gini; all measures are BENEFITS)
clinic - naive proximity 1/(1+d_km)           Gini = 0.336
clinic - E2SFCA (capacity + competition)      Gini = 0.205
hospital - naive proximity 1/(1+d_km)         Gini = 0.388
hospital - E2SFCA                               Gini = 0.103

```

The two measures disagree, and not in the direction people expect. Naive proximity says hospital access is the *more* unequal of the two (0.388 vs 0.336); E2SFCA says it is dramatically *less* unequal (0.103 vs 0.205). Why? Because the four hospitals are large and sited where the population is, so within the 30 km catchment almost everyone

accumulates a similar capacity-per-person ratio. Distance alone sees only that some people live far away; it cannot see that the facility they reach is a big one.

Neither number is "right". They answer different questions — *how far must I travel?* versus *how much provision is available to me?* — and a serious equity assessment reports both. What you must not do is compute a Gini on distance, observe that it is large, and call it a finding.

The 20 % of the population with the LOWEST clinic access:

```
blocks           : 313
people           : 122,020
mean access      : 0.1543 vs regional mean 0.6324
district types   : {'upland_rural': 156, 'suburban': 114, 'rural': 39, 'urban_core': 4}
mean dist to clinic : 9.7 km vs 7.8 km regionally
```

Note the composition: the worst-served fifth is **not** purely rural — 114 of the 313 blocks are **suburban**. Suburban fringe blocks are close enough to be unremarkable on a distance map but sit outside the catchment of any clinic. A distance-based analysis would have sent resources to the uplands and missed them.

Three panels: two accessibility maps and the concentration curves, both sagging below the equality diagonal, the clinic curve noticeably further.

A5 — Spatial autocorrelation: Moran's I from scratch

What we are going to learn. How to measure whether a variable is spatially clustered, and how to test it properly.

Why it matters. This is the diagnostic that tells you whether ordinary statistics apply. If your residuals are spatially autocorrelated, your standard errors are too small, your p-values are too optimistic, and your cross-validation is leaking.

Test it before you trust anything.

The concept — spatial weights come first. Every spatial statistic starts with a **weights matrix** **W** encoding "who is a neighbour of whom":

Scheme	Definition	Good for
Queen contiguity	Share any boundary point	Irregular polygons — the default for areal data
Rook contiguity	Share an edge (not just a corner)	Regular grids
k-nearest neighbours	The k closest	Points; guarantees every unit has neighbours
Distance band	Everything within d	When the process has a known range
Kernel	Continuously decaying weight	Smooth processes

Row-standardise ($W /= W.sum(axis=1)$) so each row sums to 1; then **Wy** is the *mean* of the neighbours and Moran's I is bounded near $[-1, 1]$.

Moran's I.

$$I = \frac{n}{S_0} \times \frac{\sum_i \sum_j w_{ij} (y_i - \bar{y})(y_j - \bar{y})}{\sum_i (y_i - \bar{y})^2}$$

with $S_0 = \sum_{i,j} w_{ij}$. It is essentially a correlation between a variable and its own spatial lag. $I \approx E[I] = -1/(n-1)$ means no spatial structure; $I > 0$ means clustering (like next to like); $I < 0$ means a checkerboard.

Inference: use permutations. The analytical variance of Moran's I relies on assumptions that rarely hold. **Conditional randomisation** — shuffle the values across the fixed geometry many times and see where the observed I falls in the resulting distribution — is assumption-free and takes milliseconds.

Expected outcome. Queen and k-NN weights built from scratch, Moran's I for several variables with permutation p-values, and a Moran scatterplot.

What the next cell does: builds a queen-contiguity weights matrix, implements Moran's I and a 999-permutation test, applies both to six variables, and draws the Moran scatterplot with its four quadrants.

```
In [40]: # --- 1. Weights matrices from scratch -----
def queen_weights(gdf, row_standardise=True):
    """Queen contiguity: neighbours share at least one boundary point."""
    n = len(gdf)
    W = np.zeros((n, n))
    sidx = gdf.sindex
    geoms = gdf.geometry.to_numpy()
    for i, g in enumerate(geoms):
        for j in sidx.query(g, predicate="intersects"):
            if i != j:
                W[i, j] = 1.0
    if row_standardise:
        rs = W.sum(axis=1, keepdims=True)
        W = np.divide(W, rs, out=np.zeros_like(W), where=rs > 0)
    return W

G = Fx.reset_index(drop=True)
Wq = queen_weights(G)
nbrs = (Wq > 0).sum(axis=1)
print("QUEEN CONTIGUITY WEIGHTS")
print(f" units : {len(G)}")
print(f" neighbours: min {nbrs.min()}, median {np.median(nbrs):.0f}, "
      f"mean {nbrs.mean():.2f}, max {nbrs.max()}")
print(f" ISLANDS (0 neighbours) : {int((nbrs == 0).sum())} "
      f"<- islands break every spatial statistic; check for them")
print(f" matrix density : {100*(Wq > 0).mean():.2f} % "
      f"(sparse - use scipy.sparse for large n)")

# --- 2. Moran's I with a permutation test -----
def morans_i(y, W):
    y = np.asarray(y, dtype=float)
    z = y - np.nanmean(y)
    S0 = W.sum()
    num = z @ (W @ z)
    den = (z ** 2).sum()
    return len(y) / S0 * num / den

def moran_test(y, W, permutations=999, seed=0):
    y = np.asarray(y, dtype=float)
    ok = np.isfinite(y)
    Wc = W[np.ix_(ok, ok)]
    rs = Wc.sum(axis=1, keepdims=True)
    Wc = np.divide(Wc, rs, out=np.zeros_like(Wc), where=rs > 0)
    yy = y[ok]
    I = morans_i(yy, Wc)
    rng = np.random.default_rng(seed)
    sim = np.array([morans_i(rng.permutation(yy), Wc) for _ in range(permutations)])
    # two-sided pseudo p-value
    p = (1 + min((sim >= I).sum(), (sim <= I).sum()) * 2) / (permutations + 1)
    EI = -1.0 / (len(yy) - 1)
    z_sim = (I - sim.mean()) / sim.std(ddof=1)
    return dict(I=I, EI=EI, p_sim=min(p, 1.0), z_sim=z_sim,
               sim_mean=sim.mean(), sim_std=sim.std(ddof=1), n=int(ok.sum()))

VARS = ["pop_density_km2", "mean_elev_m", "mean_rainfall_mm", "mean_ndvi",
        "pct_in_flood100", "dist_hospital_m", "mean_lst_c"]
```

```

print("\nMORAN'S I (queen contiguity, 999 permutations)")
print(f"  {'variable':<22}{'I':>8}{'E[I]':>9}{'z':>9}{'p':>9}  interpretation")
print("-" * 82)
res_moran = {}
for v in VARS:
    r = moran_test(G[v], Wq, permutations=999, seed=42)
    res_moran[v] = r
    tag = ("strong clustering" if r["I"] > 0.6 else
          "clustering" if r["I"] > 0.2 else
          "weak/none" if r["I"] > -0.05 else "dispersion")
    star = "***" if r["p_sim"] < 0.001 else "*" if r["p_sim"] < 0.01 else \
          "*" if r["p_sim"] < 0.05 else "ns"
    print(f"  {v:<22}{r['I']:>8.4f}{r['EI']:>9.4f}{r['z_sim']:>9.2f}"
          f"{r['p_sim']:>9.4f}  {tag} {star}")

print("\n EVERY variable is significantly clustered. That is not a finding -")
print("  it is the normal condition of spatial data, and it is exactly why")
print("  ordinary statistical inference cannot be applied to it unmodified.")

# --- 3. Moran scatterplot -----
v = "pct_in_flood100"
y = G[v].to_numpy(dtype=float)
z = (y - y.mean()) / y.std()
Wz = Wq @ z
I = res_moran[v]["I"]

fig, axes = plt.subplots(1, 3, figsize=(17, 5.2))
ax = axes[0]
quad = np.where((z > 0) & (Wz > 0), "HH",
               np.where((z < 0) & (Wz < 0), "LL",
               np.where((z > 0) & (Wz < 0), "HL", "LH")))
COL = {"HH": "#b2182b", "LL": "#2166ac", "HL": "#ef8a62", "LH": "#67a9cf"}
for q in ["LL", "LH", "HL", "HH"]:
    m = quad == q
    ax.scatter(z[m], Wz[m], s=16, c=COL[q], label=f"{q} (n={m.sum()})",
              edgecolor="none", alpha=0.8)
b = np.polyfit(z, Wz, 1)[0]
xx = np.linspace(z.min(), z.max(), 10)
ax.plot(xx, b * xx, "k-", linewidth=1.8, label=f"slope = I = {I:.3f}")
ax.axhline(0, color="grey", linewidth=0.8); ax.axvline(0, color="grey", linewidth=0.8)
ax.set_xlabel(f"z({v})"); ax.set_ylabel(f"spatial lag of z({v})")
ax.set_title("Moran scatterplot", fontsize=10, weight="bold", loc="left")
ax.legend(fontsize=7); ax.grid(alpha=0.3)

# permutation null distribution
r = res_moran[v]
rng = np.random.default_rng(42)
ok = np.isfinite(y)
sim = np.array([moran_i(rng.permutation(y[ok]), Wq[np.ix_(ok, ok)])
               for _ in range(999)])
axes[1].hist(sim, bins=40, color="#cccccc", edgecolor="white")
axes[1].axvline(I, color="crimson", linewidth=2.2, label=f"observed I = {I:.3f}")
axes[1].axvline(sim.mean(), color="black", linestyle="--", linewidth=1.2,
               label=f"null mean = {sim.mean():.4f}")
axes[1].set_xlabel("Moran's I under random reallocation")
axes[1].set_title(f"Permutation null (999 draws)\np = {r['p_sim']:.4f}",
                  fontsize=10, weight="bold", loc="left")
axes[1].legend(fontsize=8)

G.assign(quad=quad).plot(ax=axes[2], column="quad", categorical=True,
                        cmap="coolwarm", legend=True,
                        legend_kwds={"loc": "lower left", "fontsize": 7},
                        edgecolor="none")
axes[2].set_title("Moran quadrants in space", fontsize=10, weight="bold", loc="left")
axes[2].set_aspect("equal"); axes[2].set_xticks([]); axes[2].set_yticks([])
plt.tight_layout(); plt.show()

# --- 4. Does the weights scheme change the answer? -----

```

```

print("\nSENSITIVITY TO THE WEIGHTS SCHEME (variable: pct_in_flood100)")
print(f"  {'scheme':<28}{'I':>10}{'p':>9}")
for label, Wx in [("queen contiguity", Wq),
                  ("k-nearest, k=4", knn_weights(cxy, k=4)),
                  ("k-nearest, k=8", knn_weights(cxy, k=8)),
                  ("k-nearest, k=16", knn_weights(cxy, k=16))]:
    rr = moran_test(G[v], Wx, permutations=499, seed=1)
    print(f"  {label:<28}{rr['I']:>10.4f}{rr['p_sim']:>9.4f}")
print("\n  I falls as the neighbourhood widens – averaging over more, more")
print("    distant units dilutes the local similarity. ALWAYS report your")
print("    weights specification; an unqualified Moran's I is meaningless.")

```

Explanation.

- **queen_weights** uses the spatial index to find intersecting polygons — for a planar partition, "intersects" and "shares a boundary point" are the same thing. For large *n* use `libpysal.weights.Queen.from_dataframe`, which is optimised; building it by hand once is how you understand what it contains.
- **Islands** (units with no neighbours) break everything: their row of *W* is all zeros, so their spatial lag is undefined. Detect them explicitly. Remedies: switch to k-NN weights (guarantees *k* neighbours), or merge the island into its nearest unit, or drop it and say so.
- **morans_i** is a direct transcription of the formula. Note $z @ (W @ z)$ — computing $W @ z$ first is $O(n^2)$; forming the outer product would be $O(n^3)$.
- **The permutation test.** Under the null, the observed values could have landed on any unit. Shuffling *y* while holding *W* fixed generates the exact null distribution for that geometry. $(1 + \text{count}) / (\text{permutations} + 1)$ is the standard pseudo-p-value — the +1 prevents a p-value of exactly 0, which would be an overstatement.
- **Handling NaN properly.** `moran_test` subsets *W* to the finite rows and **re-standardises**. Simply dropping rows without re-standardising leaves rows summing to less than 1 and biases *I* downward.
- **The Moran scatterplot** plots *z* against *Wz*; **the slope of the fitted line is Moran's I**. The four quadrants classify each unit: HH (high value, high neighbours) and LL are clusters; HL and LH are spatial outliers — a high-exposure block surrounded by dry ones, which is often where the interesting story is.
- **The weights-sensitivity block is not optional.** Moran's *I* is a statistic *about a graph*, and the graph is your choice. Reporting "*I* = 0.55" without saying "queen contiguity" is like reporting a correlation without saying which variables.

Expected outcome.

```

QUEEN CONTIGUITY WEIGHTS
units                : 459
neighbours: min 2, median 6, mean 5.64, max 9
ISLANDS (0 neighbours) : 0
matrix density       : 1.23 %

MORAN'S I (queen contiguity, 999 permutations)
variable      I      E[I]      z      p  interpretation
pop_density_km2    0.8614 -0.0022   30.65  0.0010 strong clustering
mean_elev_m        0.9854 -0.0022   35.66  0.0010 strong clustering
mean_rainfall_mm   0.9842 -0.0022   35.14  0.0010 strong clustering
mean_ndvi          0.6940 -0.0022   25.31  0.0010 strong clustering
pct_in_flood100    0.4197 -0.0022   15.60  0.0010 clustering
dist_hospital_m    0.9837 -0.0022   35.71  0.0010 strong clustering
mean_lst_c         0.9720 -0.0022   34.37  0.0010 strong clustering

```

Every variable is significantly clustered, most of them overwhelmingly so. `mean_elev_m` at *I* = 0.985 is almost perfectly smooth — as any physical field must be. Even the most fragmented variable, flood exposure, sits at 0.42 with *z* = 15.6.

That universality is the lesson: **spatial autocorrelation is not an anomaly to be detected, it is the default state of spatial data.** The question is never "is it there?" but "how much, at what scale, and does my method account for it?"

Note the effective sample size implication. With $I \approx 0.98$ on elevation, 459 blocks carry nothing like 459 independent observations — closer to a few dozen. Any t-test or confidence interval computed as if $n = 459$ is badly overconfident.

SENSITIVITY TO THE WEIGHTS SCHEME (pct_in_flood100)		
queen contiguity	0.4197	0.0020
k-nearest, k=4	0.4269	0.0020
k-nearest, k=8	0.3074	0.0020
k-nearest, k=16	0.1896	0.0020

I falls from 0.43 to 0.19 — a factor of more than two — purely by changing the definition of "neighbour".

Averaging over more, more distant units dilutes local similarity. An unqualified Moran's I is meaningless; always report the weights specification.

The Moran scatterplot should show a clear positive slope with dense HH and LL clusters and some HL/LH outliers; the permutation histogram should be centred near $-1/(n-1) \approx -0.002$ with the observed I far outside it.

A6 — Hotspot analysis: Getis-Ord G_i^* and LISA

What we are going to learn. How to move from a *global* statement ("this variable is clustered") to a *local* one ("here is a statistically significant cluster").

Why it matters. Global Moran's I tells you clustering exists but not where. Policy needs the where. Local statistics answer it — but they introduce a multiple-testing problem that is very often ignored in published work.

The concept — two families of local statistic.

Getis-Ord G_i^* measures whether the values *around and including* unit i are unusually high or low:

$$G_i^* = \frac{\sum_j w_{ij} x_j - \bar{x} \sum_j w_{ij}}{S \sqrt{[n \sum_j w_{ij}^2 - (\sum_j w_{ij})^2] / (n-1)}}$$

It is a **z-score**: $G_i > 1.96$ is a *hot spot*, < -1.96 a *cold spot*. It identifies intensity clusters. (G_i , without the star, excludes unit i itself.)

Local Moran's I (LISA) decomposes global I into per-unit contributions and classifies each unit as HH / LL / HL / LH. It identifies both clusters **and spatial outliers**, which G_i^* cannot.

The multiple-testing problem. With 459 units tested at $\alpha = 0.05$ you expect **23 false positives** by chance. Published hotspot maps routinely present these as findings. Corrections:

- **Bonferroni** — α/n . Correct but brutally conservative.
- **False Discovery Rate (Benjamini–Hochberg)** — controls the *expected proportion* of false positives among the rejections. **This is the right default for exploratory spatial work.**
- **Conditional permutation** — the pseudo-p-values themselves, which at least avoid distributional assumptions.

Expected outcome. G_i^* and LISA for flood exposure and for population density, with and without FDR correction, so you can see how many "hotspots" evaporate.

What the next cell does: implements G_i^* and Local Moran with conditional permutation inference, applies Benjamini–Hochberg correction, and maps the corrected and uncorrected results side by side.

```
In [41]: def getis_ord_gstar(y, W):  
        """Getis-Ord  $G_i^*$  z-scores. W should NOT be row-standardised, and the
```

```

        diagonal must be 1 (the 'star' includes the focal unit itself)."""
    y = np.asarray(y, dtype=float)
    n = len(y)
    Wb = (W > 0).astype(float)
    np.fill_diagonal(Wb, 1.0)
    xbar = y.mean()
    S = np.sqrt((y ** 2).sum() / n - xbar ** 2)
    w_sum = Wb.sum(axis=1)
    w_sq = (Wb ** 2).sum(axis=1)
    num = Wb @ y - xbar * w_sum
    den = S * np.sqrt((n * w_sq - w_sum ** 2) / (n - 1))
    return np.divide(num, den, out=np.zeros_like(num), where=den > 0)

def local_moran(y, W, permutations=999, seed=0):
    """Local Moran's I with conditional-permutation pseudo p-values."""
    y = np.asarray(y, dtype=float)
    n = len(y)
    z = (y - y.mean())
    m2 = (z ** 2).sum() / n
    Wz = W @ z
    Ii = z * Wz / m2
    rng = np.random.default_rng(seed)
    nb = (W > 0)
    sims = np.empty((permutations, n))
    for p in range(permutations):
        perm = rng.permutation(n)
        zp = z[perm]
        sims[p] = z * (W @ zp) / m2
    ge = (sims >= Ii).sum(axis=0)
    le = (sims <= Ii).sum(axis=0)
    p_sim = (np.minimum(ge, le) + 1) / (permutations + 1)
    quad = np.where((z > 0) & (Wz > 0), "HH",
                    np.where((z < 0) & (Wz < 0), "LL",
                    np.where((z > 0) & (Wz < 0), "HL", "LH")))
    return Ii, p_sim, quad

def benjamini_hochberg(p, alpha=0.05):
    """Return a boolean array of rejections controlling the FDR at alpha."""
    p = np.asarray(p, dtype=float)
    n = len(p)
    order = np.argsort(p)
    thresh = alpha * (np.arange(1, n + 1)) / n
    passed = p[order] <= thresh
    k = np.max(np.where(passed)[0]) + 1 if passed.any() else 0
    out = np.zeros(n, dtype=bool)
    if k:
        out[order[:k]] = True
    return out

TARGET = "pct_in_flood100"
y = G[TARGET].to_numpy(dtype=float)

# --- Gi* -----
gi = getis_ord_gstar(y, Wq)
from scipy.stats import norm
p_gi = 2 * (1 - norm.cdf(np.abs(gi)))
G["gi_z"] = gi
G["gi_p"] = p_gi

# --- LISA, at two permutation budgets -----
Ii, p_lisa999, quad = local_moran(y, Wq, permutations=999, seed=11)
_, p_lisa, _ = local_moran(y, Wq, permutations=9999, seed=11)
G["lisa_I"] = Ii
G["lisa_p"] = p_lisa
G["lisa_quad"] = quad

print(f"HOTSPOT ANALYSIS OF {TARGET} (n = {len(G)} blocks)")
print("=" * 92)

```

```

print(f" {'method':<28}{'min p':>9}{'alpha=0.05':>12}{'Bonferroni':>12}"
      f"{'FDR (BH)':>10}{'expected FP':>13}")
print("-" * 92)
for label, pv in [("Getis-Ord Gi* (normal)", p_gi),
                  ("LISA, 999 permutations", p_lisa999),
                  ("LISA, 9999 permutations", p_lisa)]:
    n_raw = int((pv < 0.05).sum())
    n_bon = int((pv < 0.05 / len(pv)).sum())
    n_fdr = int(benjamini_hochberg(pv, 0.05).sum())
    print(f" {label:<28}{pv.min():>9.5f}{n_raw:>12}{n_bon:>12}"
          f"{'n_fdr:>10'}{0.05*len(pv):>13.0f}")
print("-" * 92)
print(f" At alpha = 0.05 with {len(G)} tests you expect ~{0.05*len(G):.0f} false")
print(f" positives by chance alone. Report the FDR-corrected count.")
print(f"\n THE PERMUTATION FLOOR. A pseudo p-value cannot be smaller than")
print(f" 1/(permutations+1). With 999 permutations that floor is 0.001, while")
print(f" Benjamini-Hochberg needs the SMALLEST p to clear alpha/n = "
      f"{0.05/len(G):.5f}.")
print(f" No unit can ever pass - which is why LISA at 999 permutations rejects")
print(f" {int(benjamini_hochberg(p_lisa999, 0.05).sum())} and at 9999 rejects "
      f"{int(benjamini_hochberg(p_lisa, 0.05).sum())}.")
print(f" Rule: permutations must exceed ~n/alpha = {int(len(G)/0.05):,} for FDR")
print(f" correction to be able to reject anything at all.")

# --- classify -----
sig_fdr = benjamini_hochberg(p_gi, 0.05)
G["hotspot"] = np.where(sig_fdr & (gi > 0), "hot",
                       np.where(sig_fdr & (gi < 0), "cold", "not significant"))
G["hotspot_raw"] = np.where((p_gi < 0.05) & (gi > 0), "hot",
                            np.where((p_gi < 0.05) & (gi < 0), "cold", "not significant"))

print(f"\n Gi* classification (FDR-corrected):")
print(" ", G.hotspot.value_counts().to_dict())
print(f" Gi* classification (uncorrected):")
print(" ", G.hotspot_raw.value_counts().to_dict())

hot = G[G.hotspot == "hot"]
print(f"\n FLOOD-EXPOSURE HOT SPOTS")
print(f" blocks : {len(hot)}")
print(f" population at risk : {hot.population.sum():,} "
      f"({100*hot.population.sum()/G.population.sum():.1f} % of the region)")
print(f" buildings : {int(hot.n_buildings.sum()):,}")
print(f" asset value : {hot.total_value_kvs.sum():.0f} k VS")
print(f" districts involved : "
      f"{sorted(hot.district_id.unique().tolist())}")

lisa_sig = benjamini_hochberg(p_lisa, 0.05)
G["lisa_class"] = np.where(lisa_sig, G.lisa_quad, "not significant")
print(f"\n LISA classification (FDR-corrected): "
      f"{G.lisa_class.value_counts().to_dict()}")
outliers = G[G.lisa_class.isin(["HL", "LH"])]
print(f" spatial OUTLIERS (HL + LH): {len(outliers)} blocks - these are")
print(f" places that differ sharply from their surroundings, which Gi* misses.")

# --- Maps -----
fig, axes = plt.subplots(1, 4, figsize=(19.5, 5.2))
G.plot(ax=axes[0], column=TARGET, cmap="Blues", scheme="quantiles", k=6,
       legend=True, legend_kwds={"loc": "lower left", "fontsize": 6}, edgecolor="none")
axes[0].set_title(f"(a) Raw variable\n{TARGET}", fontsize=9.5, weight="bold", loc="left")

CMAP_H = {"hot": "#b2182b", "cold": "#2166ac", "not significant": "#eeeeee"}
for ax, col, title in [(axes[1], "hotspot_raw", "(b) Gi* hotspots, UNCORRECTED\nnp < 0.05"),
                       (axes[2], "hotspot", "(c) Gi* hotspots, FDR-CORRECTED\nBenjamini-"),
                       (axes[3], "hotspot", "(d) Gi* hotspots, FDR-CORRECTED\nBenjamini-")]:
    Hochberg)):
        for k_, c_ in CMAP_H.items():
            sub = G[G[col] == k_]
            if len(sub):
                sub.plot(ax=ax, color=c_, edgecolor="none", label=k_)

```

```

ax.legend(fontsize=7, loc="lower left")
ax.set_title(title, fontsize=9.5, weight="bold", loc="left")

CMAP_L = {"HH": "#b2182b", "LL": "#2166ac", "HL": "#f4a582",
          "LH": "#92c5de", "not significant": "#eeeeee"}
for k_, c_ in CMAP_L.items():
    sub = G[G.lisa_class == k_]
    if len(sub):
        sub.plot(ax=axes[3], color=c_, edgecolor="none", label=k_)
axes[3].legend(fontsize=7, loc="lower left")
axes[3].set_title("(d) LISA clusters & outliers\nFDR-corrected", fontsize=9.5,
                  weight="bold", loc="left")

for a in axes:
    a.set_aspect("equal"); a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.show()

```

Explanation.

- **Gi*** uses a binary weights matrix with a 1 on the diagonal. The "star" means the focal unit is included in its own neighbourhood — which is what makes *Gi* an intensity statistic. Row-standardising before *Gi* is a common error that changes the meaning of the numerator.
- $S = \sqrt{(\Sigma y^2/n - \bar{y}^2)}$ is the **population** standard deviation (divide by *n*, not *n-1*), as specified in Getis and Ord's original formulation.
- **Local Moran with conditional permutation.** For each permutation we shuffle the values across *all* units and recompute every local statistic. (A stricter implementation holds unit *i* fixed and shuffles only the others; the difference is negligible for *n* in the hundreds and it is much faster this way.)
- **Benjamini–Hochberg.** Sort the p-values ascending, find the largest *k* such that $p_i \leq \alpha \cdot k/n$, and reject the first *k*. It controls the *expected proportion of false discoveries* among your rejections, which is exactly the right guarantee for exploratory mapping — Bonferroni controls the probability of *any* false positive and is far too strict when you have 459 tests.
- **Gi* versus LISA.** *Gi* finds where values are high or low. *LISA* finds where the local pattern is unusual, including spatial outliers — a dry block surrounded by floodplain (HL), or a flooded block surrounded by dry land (LH). Those outliers are frequently the operationally interesting cases (isolated at-risk communities, or unexpectedly protected pockets), and *Gi* cannot see them.

Expected outcome.

method	min p	alpha=0.05	Bonferroni	FDR (BH)	expected FP
Getis–Ord Gi* (normal)	0.00000	57	8	24	23
LISA, 999 permutations	0.00100	106	0	0	23
LISA, 9999 permutations	0.00010	122	1	3	23

Three findings, each worth more than the map.

1. **Uncorrected Gi* reports 57 significant blocks; FDR keeps 24; the expected false-positive count is 23.** In other words, roughly *half* of the uncorrected "hotspots" are noise. Published hotspot maps very often show the uncorrected version.
2. **The permutation floor.** A pseudo p-value cannot be smaller than $1/(\text{permutations}+1)$. With 999 permutations that floor is 0.001, while Benjamini–Hochberg needs the smallest p-value to clear $\alpha/n = 0.000109$. **No unit can ever pass, so LISA at 999 permutations rejects zero — not because there is no signal, but because the test lacks the resolution to express it.** Raising to 9 999 permutations moves the floor to 0.0001 and three units survive. **Rule of thumb: permutations must exceed n/α — here 9 180 — for FDR correction to be able to reject anything at all.**

3. **Gi* and LISA disagree sharply** (24 vs 3 survivors). They are different statistics answering different questions, and Gi*'s analytical normal p-values have no floor, so they clear FDR more easily. Neither is "more correct"; report which you used and why.

```
FL00D-EXPOSURE HOT SPOTS (Gi*, FDR-corrected)
blocks                : 24
population at risk    : 64,677 (10.1 % of the region)
buildings             : 589
asset value           : 126,369 k VS
districts involved    : ['D01', 'D06', 'D09', 'D11', 'D18']
```

Panels (b) and (c) should differ visibly: a fringe of isolated "significant" blocks in (b) disappears in (c). **That fringe is exactly what uncorrected hotspot maps publish as findings.** Panel (d) shows the LISA classification, which after correction is nearly empty — an honest depiction of what this particular test can and cannot support.

A7 — Point pattern analysis

What we are going to learn. How to characterise a set of *point events* — here, flood incidents — as clustered, random or regular, and how to estimate their intensity surface.

Why it matters. Areal statistics (Moran, Gi*) need a partition into units. Point events do not come that way, and aggregating them to blocks throws away information and imposes the MAUP. Point pattern analysis works on the events themselves.

The concept — the null model. The baseline is **Complete Spatial Randomness (CSR)**: a homogeneous Poisson process where events are independent and equally likely anywhere in the study region. Everything is measured against CSR.

Three tools.

1. **Quadrat counts.** Divide the region into cells, count events per cell. Under CSR the counts are Poisson, so $\text{variance/mean} \approx 1$. The **variance-to-mean ratio (VMR)** is a one-number summary: > 1 clustered, ≈ 1 random, < 1 regular. A χ^2 test gives a p-value. **It is sensitive to quadrat size** — always report it at several scales.
2. **Nearest-neighbour index (Clark–Evans).** $R = \bar{d}_{\text{observed}} / \bar{d}_{\text{expected}}$ where $\bar{d}_{\text{expected}} = 0.5/\sqrt{\lambda}$ for CSR with intensity $\lambda = n/A$. $R < 1$ clustered, $R \approx 1$ random, $R > 1$ regular. Suffers from **edge effects**: points near the boundary have artificially distant neighbours, biasing R upward.
3. **Kernel density estimation.** A smooth intensity surface $\hat{\lambda}(s) = \sum_i K((s - s_i)/h) / h^2$. **The bandwidth h is the entire analysis** — too small and you map individual events, too large and you map the study area's shape.

The crucial caveat — inhomogeneity. Flood incidents cluster because *people and rivers* cluster, not necessarily because floods are contagious. Testing against homogeneous CSR will always reject. The honest comparison is against an **inhomogeneous** null with intensity proportional to population at risk — which is what we do here with a case–control style comparison.

Expected outcome. Quadrat and nearest-neighbour statistics at several scales, a KDE surface at three bandwidths, and a comparison of the raw intensity against a population-adjusted "relative risk" surface.

What the next cell does: runs quadrat and Clark–Evans tests, builds KDE surfaces at three bandwidths, and computes a population-adjusted relative-risk surface to separate "where floods happen" from "where floods happen more than you would expect".

```
In [42]: from scipy.stats import chisquare, gaussian_kde
         from scipy.spatial import cKDTree

         pts = incidents.copy()
         pts = pts[pts.geometry.within(LAND_GEOM.buffer(500))].reset_index(drop=True)
```

```

XY = np.c_[pts.geometry.x, pts.geometry.y]
A_km2 = LAND_GEOM.area / 1e6
lam = len(pts) / LAND_GEOM.area # events per m^2

print(f"POINT PATTERN: {len(pts)} flood incidents over {A_km2:.0f} km^2")
print(f" intensity lambda = {len(pts)/A_km2:.3f} events per km^2")

# --- 1. QUADRAT ANALYSIS at several scales -----
print("\nQUADRAT ANALYSIS")
print(f" {'cell size':>10}{'cells on land':>15}{'mean':>8}{'var':>9}{'VMR':>8}"
      f"{'chi2 p':>10} verdict")
print("-" * 74)
minx, miny, maxx, maxy = LAND_GEOM.bounds
for cell in [2000, 4000, 6000, 8000]:
    nx = int(np.ceil((maxx - minx) / cell)); ny = int(np.ceil((maxy - miny) / cell))
    H2, xe, ye = np.histogram2d(XY[:, 0], XY[:, 1], bins=[nx, ny],
                                range=[[minx, minx + nx*cell], [miny, miny + ny*cell]])
    # keep only quadrats that are mostly on land
    cx = (xe[:-1] + xe[1:]) / 2; cy = (ye[:-1] + ye[1:]) / 2
    XX, YY = np.meshgrid(cx, cy, indexing="ij")
    on_land = np.array([[LAND_GEOM.contains(Point(a, b)) for b in cy] for a in cx])
    counts = H2[on_land]
    if len(counts) < 5:
        continue
    m, v = counts.mean(), counts.var(ddof=1)
    vmr = v / m if m > 0 else np.nan
    exp = np.full(len(counts), m)
    chi2, pval = chisquare(counts, exp)
    verdict = ("CLUSTERED" if vmr > 1.2 else "regular" if vmr < 0.8 else "random")
    print(f" {'cell':>8} m{len(counts):>15}{m:>8.2f}{v:>9.2f}{vmr:>8.2f}"
          f"{'pval:>10.2e} {'verdict}"))
print("\n VMR = variance / mean. Poisson (CSR) implies VMR = 1.")

# --- 2. NEAREST-NEIGHBOUR INDEX (Clark-Evans) -----
tree = cKDTree(XY)
d_nn, _ = tree.query(XY, k=2)
d_obs = d_nn[:, 1].mean()
d_exp = 0.5 / np.sqrt(lam)
R = d_obs / d_exp
se = 0.26136 / np.sqrt(len(pts) ** 2 * lam)
Z = (d_obs - d_exp) / se

# edge-corrected version: drop points within d_exp of the boundary
edge = np.array([LAND_GEOM.boundary.distance(Point(*p)) for p in XY])
keep = edge > d_exp
d_obs_c = d_nn[keep, 1].mean()
R_c = d_obs_c / d_exp

print("\nCLARK-EVANS NEAREST-NEIGHBOUR INDEX")
print(f" observed mean NN distance : {d_obs:>9,.0f} m")
print(f" expected under CSR          : {d_exp:>9,.0f} m")
print(f" R = obs / exp                  : {R:>9.4f} z = {Z:>9.2f}")
print(f" R, edge-corrected             : {R_c:>9.4f} "
      f"({int((~keep).sum())} boundary points removed)")
print(f" verdict: {'CLUSTERED' if R < 0.95 else 'regular' if R > 1.05 else 'random'}"
      f" (R < 1 means clustered)")

# --- 3. KDE at three bandwidths -----
res_k = 200.0
gx = np.arange(minx, maxx, res_k); gy = np.arange(miny, maxy, res_k)
GX, GY = np.meshgrid(gx, gy)
grid_pts = np.vstack([GX.ravel(), GY.ravel()])

def bw_factor(coords, h_m):
    """Convert a bandwidth in metres into scipy's `bw_method` factor.

    scipy scales the kernel by factor * (per-axis data std), so the factor must
    be h divided by the ROOT-MEAN-SQUARE per-axis standard deviation. Using

```

```

coords.std() on a 2-column array of UTM coordinates is a classic error: it
mixes eastings (~4e5) and northings (~4.6e6) and returns ~2e6, giving a
bandwidth hundreds of times too small.
"""
    return h_m / np.sqrt(np.mean(coords.var(axis=0)))

print(f"\n bandwidth scaling check:")
print(f"    WRONG XY.std() over both columns      = {XY.std():>12,.0f} m")
print(f"    RIGHT RMS of per-axis std                = "
      f"{np.sqrt(np.mean(XY.var(axis=0))):>12,.0f} m")

kdes = {}
for name, bw in [("h = 500 m (under-smoothed)", 500),
                 ("h = 1500 m (reasonable)", 1500),
                 ("h = 4000 m (over-smoothed)", 4000)]:
    kd = gaussian_kde(XY.T, bw_method=bw_factor(XY, bw))
    z = kd(grid_pts).reshape(GX.shape) * len(pts) * 1e6      # events per km^2
    kdes[name] = z

# --- 4. POPULATION-ADJUSTED RELATIVE RISK -----
# Control points drawn in proportion to population: "where would incidents be
# if they were purely proportional to people?"
rng = np.random.default_rng(3)
w = Fx.population.to_numpy(float); w = w / w.sum()
pick = rng.choice(len(Fx), size=4000, p=w)
ctrl = []
for i in pick:
    g = Fx.geometry.iloc[i]
    minx_, miny_, maxx_, maxy_ = g.bounds
    for _ in range(50):
        q = Point(rng.uniform(minx_, maxx_), rng.uniform(miny_, maxy_))
        if g.contains(q):
            ctrl.append((q.x, q.y)); break
ctrl = np.array(ctrl)

kd_case = gaussian_kde(XY.T, bw_method=bw_factor(XY, 1500))
kd_ctrl = gaussian_kde(ctrl.T, bw_method=bw_factor(ctrl, 1500))
f_case = kd_case(grid_pts).reshape(GX.shape)
f_ctrl = kd_ctrl(grid_pts).reshape(GX.shape)
rr = np.log((f_case + 1e-12) / (f_ctrl + 1e-12))

on = np.array([[LAND_GEOM.contains(Point(a, b)) for a in gx] for b in gy])
# Relative risk is only interpretable where BOTH densities are supported by data.
# Where the control density is near zero the ratio explodes; mask those cells.
# gaussian_kde underflows to EXACTLY zero far from the data, so a percentile
# floor is useless here - use a floor relative to each surface's peak instead.
valid = (on & (f_ctrl > 1e-4 * f_ctrl[on].max())
        & (f_case > 1e-4 * f_case[on].max()))
rr_m = np.where(valid, np.clip(rr, -3, 3), np.nan)

print("\nPOPULATION-ADJUSTED RELATIVE RISK")
print(f" control points drawn in proportion to population : {len(ctrl):,}")
print(f" cells with adequate support (both densities)      : "
      f"{100*valid.sum()/on.sum():.1f} % of land")
print(f" log relative risk (clipped to +/-3): min {np.nanmin(rr_m):+.2f}, "
      f"median {np.nanmedian(rr_m):+.2f}, max {np.nanmax(rr_m):+.2f}")
print(f" supported land with log-RR > 0 (more incidents than population implies): "
      f"{100*np.nanmean(rr_m > 0):.1f} %")
print(f" ... with log-RR > 1 (2.7x more than expected): "
      f"{100*np.nanmean(rr_m > 1):.1f} %")
print("\n A raw KDE maps WHERE FLOODS HAPPEN, which is largely where people are.")
print(" The relative-risk surface maps where floods happen MORE THAN EXPECTED")
print(" given the population - a genuinely different, and far more useful, map.")

# --- 5. Figure -----
fig, axes = plt.subplots(1, 4, figsize=(19.5, 5.2))
ext = (minx, maxx, miny, maxy)
for ax, (name, z) in zip(axes[:3], kdes.items()):

```

```

zz = np.where(on, z, np.nan)
im = ax.imshow(zz, origin="lower", extent=ext, cmap="inferno")
pts.plot(ax=ax, color="cyan", markersize=1.4, alpha=0.7)
ax.set_title(name, fontsize=9.5, weight="bold", loc="left")
plt.colorbar(im, ax=ax, shrink=0.7, label="events / km2")
vmax = np.nanpercentile(np.abs(rr_m), 98)
im = axes[3].imshow(rr_m, origin="lower", extent=ext, cmap="RdBu_r",
                    vmin=-vmax, vmax=vmax)
rivers.plot(ax=axes[3], color="black", linewidth=0.7)
plt.colorbar(im, ax=axes[3], shrink=0.7, label="log relative risk")
axes[3].set_title("Population-adjusted relative risk\n(red = more than expected)",
                  fontsize=9.5, weight="bold", loc="left")
for a in axes:
    a.set_aspect("equal"); a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.show()

```

Explanation.

- **Quadrat analysis at several scales** is essential because the VMR depends on cell size. A pattern can look clustered at 2 km and random at 8 km — and that *scale dependence is itself the finding*: it tells you the characteristic size of the clusters.
- `chisquare(counts, exp)` tests the Poisson null. Note that the χ^2 approximation requires expected counts of roughly 5+, which is why we skip configurations with too few quadrats.
- **Clark–Evans and edge effects.** `d_expected = 0.5/√λ` assumes an unbounded region. Points near the study-area boundary have no neighbours outside it, so their observed nearest-neighbour distances are inflated and **R** is biased **upward** (towards "regular"). Our correction — dropping points within `d_exp` of the boundary — is the crude version; proper alternatives are Ripley's isotropic correction or a toroidal wrap.
- **KDE bandwidth.** `gaussian_kde`'s `bw_method` is a multiplier on the data's standard deviation, hence `bw / XY.std()` to express it in metres. The three panels show the whole problem: at 500 m you are mapping individual reports; at 4 km you are mapping the shape of the basin. **The bandwidth is a substantive choice and must be reported**; rules of thumb (Silverman, Scott) are starting points, not answers.
- **The relative-risk surface is the most important idea in this lesson.** A raw incident KDE is dominated by population: more people means more reports. By generating **control points in proportion to population** and taking the log ratio of the two densities, we ask a different question — *where do floods occur more often than the population distribution alone would predict?* That is the case–control logic of spatial epidemiology, and it turns a map of "where people are" into a map of "where the hazard is".

Expected outcome.

POINT PATTERN: 386 flood incidents over 1,395 km²
 intensity lambda = 0.277 events per km²

QUADRAT ANALYSIS

cell size	cells on land	mean	var	VMR	chi2 p	verdict
2000 m	349	1.10	13.12	11.93	0.00e+00	CLUSTERED
4000 m	87	4.17	144.40	34.61	0.00e+00	CLUSTERED
6000 m	37	10.35	322.90	31.19	2.23e-212	CLUSTERED
8000 m	21	17.33	1081.53	62.40	4.17e-252	CLUSTERED

CLARK–EVANS NEAREST-NEIGHBOUR INDEX

observed mean NN distance : 430 m
 expected under CSR : 950 m
 R = obs / exp : 0.4525 z = -404.28
 R, edge-corrected : 0.4235 (40 boundary points removed)

VMR **rises** with quadrat size here (11.9 → 62.4), which is the signature of clustering at a scale *larger* than the smallest quadrat: at 2 km many quadrats sit wholly inside a cluster, so the within-quadrat variance is modest; at 8 km each quadrat straddles both cluster and void. Reporting one quadrat size would have told you a fraction of the story.

Clark–Evans R = 0.45 (edge-corrected 0.42) — very strongly clustered, and note the edge correction moves it *down*, confirming that uncorrected R is biased towards "regular".

```
bandwidth scaling check:
  WRONG XY.std() over both columns      = 2,095,966 m
  RIGHT RMS of per-axis std              = 10,909 m
```

Look at that factor of 190. `coords.std()` on a two-column array of UTM coordinates mixes eastings ($\sim 4 \times 10^5$) and northings ($\sim 4.6 \times 10^6$); the resulting "standard deviation" is dominated by the gap between the two means and has nothing to do with the spread of the data. Feed it to `gaussian_kde` and your bandwidth is $\sim 200\times$ too small, producing a "density surface" that is really a map of individual events. This is an easy mistake to make and a hard one to notice, because the output still looks like a plausible heat map.

```
POPULATION-ADJUSTED RELATIVE RISK
  cells with adequate support (both densities)      : 76.5 % of land
  log relative risk (clipped to +/-3): min -3.00, median +0.12, max +3.00
  supported land with log-RR > 0 : 35.4 %
  ... with log-RR > 1 (2.7x more than expected): 17.1 %
```

Three KDE panels showing the bandwidth trade-off, and a fourth panel where the red (elevated relative risk) areas follow **the river corridors**, not the city. That contrast between panel 2 and panel 4 is the lesson: the raw density peaks in Vallmara City because that is where the people are, while the relative-risk surface correctly identifies the riverine floodplain as the hazardous ground.

Note also the masking. `gaussian_kde` underflows to *exactly* zero far from the data, so a ratio of two KDEs is undefined over empty country. We mask cells where either surface falls below 10^{-4} of its peak, and report the supported fraction (76.5%). **A relative-risk map without a support mask will show spectacular, entirely spurious extremes in unpopulated areas.**

A8 — Spatial clustering and regionalisation

What we are going to learn. Three different clustering problems that all get called "spatial clustering", and the right algorithm for each.

Why it matters. "Cluster the data" is ambiguous in a spatial setting. Do you want clusters *in space*, clusters *in attribute space*, or **contiguous regions** that are homogeneous in attributes? These are three different problems.

The three problems.

Problem	Question	Algorithm
Point clustering in space	Where are the dense concentrations of events?	DBSCAN on coordinates
Attribute clustering (typology)	Which places are <i>alike</i> , wherever they are?	K-means / GMM on standardised features
Regionalisation	Partition into contiguous homogeneous regions	Ward with a contiguity constraint

DBSCAN needs `eps` (neighbourhood radius) and `min_samples`. It finds arbitrarily shaped clusters and labels sparse points as **noise (-1)**, which is a genuine advantage over K-means — not every point belongs to a cluster. Choose `eps` from the **k-distance plot**: sort each point's distance to its `k`-th nearest neighbour and look for the knee.

K-means requires standardised features, because it minimises Euclidean distance and a variable measured in metres will otherwise dominate one measured in percent. It also assumes spherical, similarly-sized clusters — often wrong for geographic data.

Regionalisation adds a hard constraint: clusters must be spatially contiguous.

`sklearn.cluster.AgglomerativeClustering` accepts a `connectivity` matrix that does exactly this, turning attribute clustering into region-building. This is how statistical agencies design reporting zones.

Expected outcome. DBSCAN clusters of flood incidents with a k-distance justification for `eps`, a K-means typology of census blocks, and a contiguous regionalisation — with a comparison of what each reveals.

What the next cell does: runs all three, uses the k-distance plot to choose `eps`, evaluates K-means `k` with silhouette scores, and shows that unconstrained clustering produces spatially fragmented "regions" while the constrained version does not.

```
In [43]: from sklearn.cluster import DBSCAN, KMeans, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score

# ===== 1. DBSCAN on incidents =====
XY = np.c_[pts.geometry.x, pts.geometry.y]
MIN_SAMPLES = 8
nn = cKDTree(XY)
kd, _ = nn.query(XY, k=MIN_SAMPLES + 1)
kdist = np.sort(kd[:, MIN_SAMPLES])

# knee = point of maximum curvature of the sorted k-distance curve
x = np.arange(len(kdist)); y_ = kdist
p1, p2 = np.array([x[0], y_[0]]), np.array([x[-1], y_[-1]])
v = p2 - p1
q = np.c_[x, y_] - p1
# 2-D cross product by hand: np.cross no longer accepts 2-D vectors in NumPy 2
d = np.abs(v[0] * q[:, 1] - v[1] * q[:, 0]) / np.linalg.norm(v)
knee = int(np.argmax(d))
EPS = float(kdist[knee])

print("DBSCAN ON FLOOD INCIDENTS")
print(f" min_samples = {MIN_SAMPLES}")
print(f" eps chosen from the k-distance knee : {EPS:.0f} m")
db = DBSCAN(eps=EPS, min_samples=MIN_SAMPLES).fit(XY)
lab = db.labels_
n_cl = len(set(lab)) - (1 if -1 in lab else 0)
print(f" clusters found : {n_cl}")
print(f" noise points : {int((lab == -1).sum())} "
      f"({100*(lab == -1).mean():.1f} % of incidents)")
pts["cluster"] = lab
summary = (pts[pts.cluster >= 0].groupby("cluster")
           .agg(n=("incident_id", "size"),
                mean_depth=("depth_cm", "mean"),
                total_damage=("damage_kvs", "sum")).round(1))
summary["extent_km"] = [
    pts[pts.cluster == c].geometry.union_all().convex_hull.length / 2000
    for c in summary.index]
print("\n cluster summary")
print(summary.head(10).to_string())

print(f"\n sensitivity of the cluster count to eps:")
for f in [0.5, 0.75, 1.0, 1.5, 2.0]:
    l2 = DBSCAN(eps=EPS*f, min_samples=MIN_SAMPLES).fit(XY).labels_
    print(f" eps = {EPS*f:>.0f} m -> {len(set(l2)) - (1 if -1 in l2 else 0):>3} clusters, "
          f"{100*(l2 == -1).mean():>5.1f} % noise")

# ===== 2. K-MEANS typology of blocks =====
TYPO = ["pop_density_km2", "mean_elev_m", "mean_ndvi", "pct_builtup",
```

```

        "dist_hospital_m", "pct_in_flood100", "mean_slope_deg", "mean_lst_c"]
Xt = Fx[TYP0].to_numpy(dtype=float)
Xt = np.where(np.isfinite(Xt), Xt, np.nanmedian(Xt, axis=0))
Xs = StandardScaler().fit_transform(Xt)

print("\n" + "=" * 72)
print("K-MEANS TYPOLOGY OF CENSUS BLOCKS")
print("=" * 72)
print(f"  {'k':>4}{'inertia':>14}{'silhouette':>13}")
best_k, best_s = None, -1
for k in range(2, 9):
    km = KMeans(n_clusters=k, n_init=20, random_state=0).fit(Xs)
    sil = silhouette_score(Xs, km.labels_)
    print(f"  {'k':>4}{'km.inertia':>14,.0f}{'sil':>13.4f}")
    if sil > best_s:
        best_k, best_s = k, sil
print(f"  best silhouette at k = {best_k} ({best_s:.4f})")

K = 5
km = KMeans(n_clusters=K, n_init=50, random_state=0).fit(Xs)
Fx["kmeans"] = km.labels_
prof_km = Fx.groupby("kmeans")[TYP0 + ["population", "area_km2"]].mean()
prof_km["n_blocks"] = Fx.groupby("kmeans").size()
print(f"\n  Cluster profiles at k = {K} (means)")
print(prof_km.round(1).to_string())

# how spatially fragmented are the unconstrained clusters?
frag = []
for c in range(K):
    sub = Fx[Fx.kmeans == c]
    merged = sub.geometry.union_all()
    parts = 1 if merged.geom_type == "Polygon" else len(merged.geoms)
    frag.append(parts)
print(f"\n  spatial fragments per K-means cluster: {frag}")
print(f"  -> unconstrained clustering produces {sum(frag)} disconnected patches")
print(f"  from {K} clusters. They are TYPES, not REGIONS.")

# ===== 3. REGIONALISATION =====
conn = (Wq > 0).astype(int)
conn = conn + conn.T
ward = AgglomerativeClustering(n_clusters=K, linkage="ward",
                              connectivity=conn).fit(Xs)
Fx["region"] = ward.labels_
frag_r = []
for c in range(K):
    sub = Fx[Fx.region == c]
    merged = sub.geometry.union_all()
    frag_r.append(1 if merged.geom_type == "Polygon" else len(merged.geoms))
print(f"\nREGIONALISATION (Ward + queen contiguity constraint)")
print(f"  spatial fragments per region: {frag_r} (total {sum(frag_r)})")
print(f"  within-cluster sum of squares:")
def wcsc(labels):
    return sum(((Xs[labels == c] - Xs[labels == c].mean(axis=0))*2).sum()
               for c in np.unique(labels))
print(f"  K-means (unconstrained) : {wcsc(km.labels_):>10,.0f}")
print(f"  Ward + contiguity        : {wcsc(ward.labels_):>10,.0f}")
print(f"  cost of contiguity       : "
      f"{100*(wcsc(ward.labels_)-wcsc(km.labels_))/wcsc(km.labels_):>9.1f} %")
print("\n  Contiguity always costs homogeneity. The question is whether you need")
print("  regions you can administer, or types you can describe.")

# ----- figure -----
fig, axes = plt.subplots(1, 4, figsize=(19.5, 5.2))
axes[0].plot(kdist, linewidth=1.6)
axes[0].axvline(knee, color="crimson", linestyle="--")
axes[0].axhline(EPS, color="crimson", linestyle="--",
                label=f"eps = {EPS:,.0f} m")
axes[0].set_xlabel("points, sorted"); axes[0].set_ylabel(f"{MIN_SAMPLES}-NN distance (m)")

```

```

axes[0].set_title("k-distance plot -> choose eps", fontsize=9.5, weight="bold", loc="left")
axes[0].legend(fontsize=8); axes[0].grid(alpha=0.3)

land.plot(ax=axes[1], facecolor="#f5f2ea", edgecolor="#ddd6c8", linewidth=0.5)
noise = pts[pts.cluster == -1]
noise.plot(ax=axes[1], color="bbbbbb", markersize=3, label="noise")
pts[pts.cluster >= 0].plot(ax=axes[1], column="cluster", categorical=True,
                           cmap="tab20", markersize=7)
rivers.plot(ax=axes[1], color="#3182bd", linewidth=0.6)
axes[1].set_title(f"DBSCAN: {n_cl} incident clusters", fontsize=9.5,
                  weight="bold", loc="left")

Fx.plot(ax=axes[2], column="kmeans", categorical=True, cmap="Set2",
        legend=True, legend_kwds={"loc": "lower left", "fontsize": 7},
        edgecolor="white", linewidth=0.15)
axes[2].set_title(f"K-means typology (k={K})\n{sum(frag)} disconnected patches",
                  fontsize=9.5, weight="bold", loc="left")

Fx.plot(ax=axes[3], column="region", categorical=True, cmap="Set2",
        legend=True, legend_kwds={"loc": "lower left", "fontsize": 7},
        edgecolor="white", linewidth=0.15)
axes[3].set_title(f"Ward + contiguity (k={K})\n{sum(frag_r)} patches - true REGIONS",
                  fontsize=9.5, weight="bold", loc="left")
for a in axes[1:]:
    a.set_aspect("equal"); a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.show()

```

Explanation.

- **The k-distance knee.** Sort every point's distance to its `min_samples`-th neighbour; the curve is flat for points inside clusters and rises sharply for noise. The knee is where they separate. We locate it as the point of maximum perpendicular distance from the chord joining the curve's endpoints — a reproducible rule, far better than eyeballing.
- **DBSCAN's noise label (-1) is a feature.** Isolated incidents genuinely are isolated; forcing them into a cluster (as K-means must) invents structure.
- **The eps sensitivity table is mandatory.** DBSCAN's output changes a lot with `eps`. If your conclusion holds only at one value, it is not a conclusion.
- **K-means on standardised features.** `StandardScaler` is not optional here: `dist_hospital_m` ranges to 33 000 while `pct_builtup` ranges to 100. Without standardisation the clustering is entirely determined by distance-to-hospital.
- **Silhouette score** measures how well-separated the clusters are; higher is better, and it is the least-bad automatic choice of `k`. In practice, for a *typology*, interpretability usually beats the silhouette optimum — we use `K = 5` because five types are describable.
- **The fragmentation count is the punchline.** Unconstrained K-means produces clusters that are scattered across dozens of disconnected patches. They are perfectly valid **types** ("dense, low-lying, well-served"), but calling them *regions* would be wrong — you cannot administer a region made of 40 disjoint fragments.
- **AgglomerativeClustering(connectivity=...)** restricts merges to spatially adjacent units, so every cluster is contiguous by construction. The `connectivity` matrix is our queen weights. **Note the trade-off we quantify:** contiguity always increases within-cluster variance. Report the cost.

Expected outcome.

DBSCAN ON FLOOD INCIDENTS

eps chosen from the k-distance knee : 2,482 m
clusters found : 3
noise points : 28 (7.3 % of incidents)

cluster summary

	n	mean_depth	total_damage	extent_km
0	81	57.5	63,615.6	13.1
1	268	62.2	239,532.7	30.7
2	9	45.9	6,523.9	5.9

sensitivity of the cluster count to eps:

eps = 1,241 m -> 4 clusters, 17.1 % noise
eps = 1,861 m -> 3 clusters, 10.9 % noise
eps = 2,482 m -> 3 clusters, 7.3 % noise
eps = 3,723 m -> 2 clusters, 5.7 % noise

The sensitivity table is more informative than the headline result. The cluster count is stable at 3 across a wide band of *eps*, which is reassuring — but the *noise fraction* moves from 17% to 6% over the same range, so which incidents count as "isolated" is entirely a function of the parameter. Report the curve, not a single number.

Note the dominant cluster 1: **268 of 386 incidents and 240 000 k VS of damage** in a single 31 km corridor. That is the Vallmara River floodplain through the city, and it is where a mitigation budget should go.

K-MEANS TYPOLOGY

k	inertia	silhouette	
2	2,336	0.3534	
3	1,560	0.3744	<- best
5	1,132	0.3084	

Silhouette prefers $k = 3$, but we use **$k = 5$** because five types are describable and the silhouette difference is small. That is a legitimate choice *provided you say so*. The profiles are interpretable: cluster 2 is the dense urban core (4 596 people/km², 87% built-up, 2.4 km to a hospital); cluster 3 is the flood-exposed riverine fringe (58% in the 100-year zone); cluster 1 is remote upland (697 m, 24 km to a hospital, zero built-up).

spatial fragments per K-means cluster: [6, 1, 2, 6, 1] -> 16 patches

REGIONALISATION (Ward + queen contiguity constraint)

spatial fragments per region: [1, 1, 1, 1, 1] (total 5)

within-cluster sum of squares:

K-means (unconstrained) :	1,130
Ward + contiguity :	1,426
cost of contiguity :	26.2 %

This is the result to take away. Unconstrained K-means gives 5 clusters scattered over 16 disconnected patches — perfectly good *types*, useless as *regions*. Adding the contiguity constraint gives exactly 5 connected regions at a cost of **26.2% more within-cluster variance**. Whether that price is worth paying depends entirely on what the output is for: a descriptive typology, or a map someone has to administer.

A9 — Spatial regression

What we are going to learn. Why OLS on spatial data is usually wrong, how to detect it, and the two standard remedies.

Why it matters. You already know regression. What you may not know is that its central assumption — independent errors — is violated by essentially every spatial dataset, and that the violation inflates your confidence rather than your error. You will not notice unless you look.

The concept — two ways space enters a regression.

Spatial lag model (SAR): the outcome at i depends on the outcome at its neighbours.

$$y = \rho W y + X\beta + \varepsilon$$

Use when there is genuine **interaction** — house prices, disease spread, technology adoption. OLS here is *biased*, not merely inefficient, because $W y$ is correlated with ε .

Spatial error model (SEM): the *unobserved* drivers are spatially structured.

$$y = X\beta + u, \quad u = \lambda W u + \varepsilon$$

Use when a spatially smooth confounder is missing (soil, microclimate, institutions). OLS coefficients stay unbiased but the standard errors are wrong.

Diagnosis. Fit OLS, then compute **Moran's I on the residuals**. If it is significant, OLS is inadequate. Lagrange-Multiplier tests (LM-lag, LM-error, and their robust versions) then indicate *which* specification to prefer.

The pragmatic middle road. A full ML-estimated SAR/SEM needs **spreg** (PySAL). Without it you can go a long way by **adding spatial features** — the spatial lag of the predictors, a smooth spatial trend, or eigenvector spatial filtering — and re-testing the residuals. That is what we do here, and we measure how much of the autocorrelation each step removes.

Expected outcome. An OLS model for land-surface temperature, residual Moran's I showing autocorrelation, and two remedies with the improvement quantified.

What the next cell does: fits OLS for LST, tests residual autocorrelation, then fits three progressively better specifications and shows how residual Moran's I falls as the spatial structure is accounted for.

```
In [44]: import statsmodels.api as sm

D = Fx.copy()
D["urban_intensity"] = np.clip((D.pop_density_km2 - 22) / 9500, 0, None) ** (1/2.1)
y = D["mean_lst_c"].to_numpy(dtype=float)

def ols_report(X, name, ycol=y, W=Wq):
    Xc = sm.add_constant(np.asarray(X, dtype=float), has_constant="add")
    m = sm.OLS(ycol, Xc).fit()
    resid = m.resid
    mt = moran_test(resid, W, permutations=999, seed=5)
    return dict(name=name, model=m, r2=m.rsquared, adj=m.rsquared_adj,
                aic=m.aic, moran_I=mt["I"], moran_p=mt["p_sim"],
                rmse=np.sqrt(np.mean(resid ** 2)))

# --- Model 1: the "true" physical specification -----
X1 = D[["mean_elev_m", "urban_intensity"]]
r1 = ols_report(X1, "1. elevation + urban")

# --- Model 2: a misspecified model, missing elevation -----
X2 = D[["urban_intensity"]]
r2 = ols_report(X2, "2. urban only (misspecified)")

# --- Model 3: model 1 + spatial lags of the predictors (SLX) -----
X3 = D[["mean_elev_m", "urban_intensity"]].copy()
X3["lag_elev"] = lag(D.mean_elev_m)
X3["lag_urban"] = lag(D.urban_intensity)
r3 = ols_report(X3, "3. + spatial lag of X (SLX)")

# --- Model 4: model 1 + a smooth spatial trend surface -----
```

```

cx_ = D.geometry.representative_point().x.to_numpy()
cy_ = D.geometry.representative_point().y.to_numpy()
xs = (cx_ - cx_.mean()) / 1e4; ys = (cy_ - cy_.mean()) / 1e4
X4 = D[["mean_elev_m", "urban_intensity"]].copy()
X4["x"] = xs; X4["y"] = ys
X4["x2"] = xs**2; X4["y2"] = ys**2; X4["xy"] = xs*ys
r4 = ols_report(X4, "4. + quadratic trend surface")

print("SPATIAL REGRESSION OF LAND SURFACE TEMPERATURE")
print("=" * 96)
print(f" {'specification':<34}{'R2':>8}{'adjR2':>8}{'RMSE':>8}{'AIC':>10}"
      f"{'resid I':>10}{'p':>8}")
print("-" * 96)
for r in [r2, r1, r3, r4]:
    print(f" {r['name']:<34}{r['r2']:>8.4f}{r['adj']:>8.4f}{r['rmse']:>8.4f}"
          f"{'r['aic']:>10.1f}{r['moran_I']:>10.4f}{r['moran_p']:>8.4f}")
print("-" * 96)

# --- Coefficients of the correct model -----
m1 = r1["model"]
print("\nMODEL 1 COEFFICIENTS (truth: LST = 31.5 - 0.0062*elev + 6.4*urban)")
names = ["const", "mean_elev_m", "urban_intensity"]
truth = [31.5, -0.0062, 6.4]
print(f" {'term':<18}{'estimate':>12}{'std err':>11}{'t':>9}{'p':>10}{'TRUTH':>12}")
for nm, b, se, t_, p_, tv in zip(names, m1.params, m1.bse, m1.tvalues,
                                m1.pvalues, truth):
    print(f" {nm:<18}{b:>12.5f}{se:>11.5f}{t_>9.1f}{p_>10.2e}{tv:>12.5f}")

# --- What the misspecified model does to the surviving coefficient -----
print(f"\nOMITTED-VARIABLE BIAS")
print(f" urban coefficient, correct model      : {m1.params[2]:>8.3f} (truth 6.400)")
print(f" urban coefficient, elevation omitted : "
      f"{r2['model'].params[1]:>8.3f} "
      f"({100*(r2['model'].params[1]-6.4)/6.4:+.0f} %)")
print(f" residual Moran's I rises from {r1['moran_I']:.3f} to {r2['moran_I']:.3f}")
print(" -> residual autocorrelation is the FINGERPRINT of a missing spatially")
print(" structured covariate. It tells you the model is incomplete.")

# --- Standard errors: how badly does OLS understate them? -----
print(f"\nTHE COST OF IGNORING AUTOCORRELATION")
resid1 = m1.resid
n_eff = len(y) * (1 - r1["moran_I"]) / (1 + r1["moran_I"])
print(f" nominal n          : {len(y)}")
print(f" approximate effective n      : {n_eff:.0f} "
      f"({n_eff/(len(y)-n_eff)} with residual I = {r1['moran_I']:.3f})")
print(f" inflation of standard errors : x{np.sqrt(len(y)/max(n_eff,1)):.2f}")
print(f" -> a t of {m1.tvalues[1]:.0f} becomes "
      f"{m1.tvalues[1]/np.sqrt(len(y)/max(n_eff,1)):.0f}: still significant here,")
print(" but for a marginal predictor this is the difference between p<0.05")
print(" and p>0.05.")

# --- Figure -----
fig, axes = plt.subplots(1, 4, figsize=(19.5, 5))
axes[0].scatter(m1.fittedvalues, y, s=10, alpha=0.6, color="#2166ac")
lims = [min(y.min(), m1.fittedvalues.min()), max(y.max(), m1.fittedvalues.max())]
axes[0].plot(lims, lims, "k--", linewidth=1)
axes[0].set_xlabel("fitted"); axes[0].set_ylabel("observed")
axes[0].set_title(f"Model 1 fit (R2 = {r1['r2']:.3f})", fontsize=9.5,
                  weight="bold", loc="left")
axes[0].grid(alpha=0.3)

for ax, r, title in [(axes[1], r2, "Residuals, model 2 (misspecified)"),
                    (axes[2], r1, "Residuals, model 1 (correct)"),
                    (axes[3], r4, "Residuals, model 4 (+ trend)")]:
    D.assign(res=r["model"].resid).plot(
        ax=ax, column="res", cmap="RdBu_r", vmin=-1.2, vmax=1.2,
        legend=True, legend_kwds={"shrink": 0.65}, edgecolor="none")
    ax.set_title(f"{title}\nMoran's I = {r['moran_I']:.3f} (p={r['moran_p']:.3f})",

```

```

        fontsize=9.5, weight="bold", loc="left")
    ax.set_aspect("equal"); ax.set_xticks([]); ax.set_yticks([])
    plt.tight_layout(); plt.show()

```

Explanation.

- **Model 2 is deliberately wrong** — it omits elevation, which is both a strong predictor of temperature and strongly spatially structured. Watch two things happen: the surviving coefficient on **urban_intensity** becomes biased, and the **residual Moran's I jumps**. That second symptom is the diagnostic: residual spatial autocorrelation is the fingerprint of a missing spatially patterned covariate.
- **Model 3 (SLX)** adds the spatial lags of the *predictors*. This is the simplest spatial specification, estimable by plain OLS, and it captures spillovers — my temperature depends partly on my neighbours' land cover, which is physically true for heat.
- **Model 4** adds a quadratic trend surface in the coordinates. This soaks up any smooth large-scale gradient that the covariates miss. It is a blunt instrument: it will happily absorb *real* effects that happen to be smooth, so it improves the residual diagnostics without necessarily improving understanding. Use it as a diagnostic, not as a final model.
- **The effective sample size formula** $n_{\text{eff}} \approx n(1-I)/(1+I)$ is a rough first-order approximation, but it makes the point concretely: with residual $I = 0.3$, your 459 observations behave like about 250, and every standard error should be inflated by $\sqrt{(459/250)} \approx 1.35$.
- Note that we are recovering **known** coefficients. The truth is $31.5 - 0.0062 \cdot \text{elevation} + 6.4 \cdot \text{urban}$. Model 1 should land essentially on top of it. That is a validation of the entire pipeline — CRS, resampling, zonal statistics, feature construction — in three numbers.

Expected outcome.

specification	R2	adjR2	RMSE	AIC	resid I	p
2. urban only (misspecified)	0.6089	0.6080	1.5686	1719.8	0.8921	0.0010
1. elevation + urban	0.9884	0.9884	0.2697	105.6	0.0524	0.0590
3. + spatial lag of X (SLX)	0.9931	0.9931	0.2077	-130.4	0.1617	0.0010
4. + quadratic trend surface	0.9885	0.9883	0.2689	112.9	0.0584	0.0310

MODEL 1 COEFFICIENTS (truth: LST = 31.5 - 0.0062*elev + 6.4*urban)					
term	estimate	std err	t	p	TRUTH
const	31.60898	0.02678	1180.1	0.00e+00	31.50000
mean_elev_m	-0.00640	0.00005	-122.3	0.00e+00	-0.00620
urban_intensity	6.15119	0.07566	81.3	1.68e-273	6.40000

OMITTED-VARIABLE BIAS	
urban coefficient, correct model	: 6.151 (truth 6.400)
urban coefficient, elevation omitted	: 10.410 (+63 %)
residual Moran's I rises from 0.052 to 0.892	

Model 1 recovers the generating law. Intercept 31.61 against a true 31.50, elevation -0.00640 against -0.00620, urban 6.15 against 6.40 — from zonal statistics over reprojected rasters. That is a validation of the entire pipeline, not just of the regression.

Model 2 is the cautionary tale. Drop elevation — a variable that is both a strong predictor and strongly spatially structured — and the urban coefficient inflates by **63%** while residual Moran's I leaps from **0.052 to 0.892**. The model still has $R^2 = 0.61$, which in many contexts would be reported without comment. The residual autocorrelation is what gives it away.

Model 3 deserves a second look, because it does something unexpected. Adding the spatial lags of the predictors gives the best R^2 and by far the best AIC — and yet residual Moran's I *rises* from 0.052 to 0.162. The lag terms have absorbed variance that the physical model already explained, leaving residuals that are *more* spatially

patterned, not less. **A better AIC does not imply better-behaved residuals**; check both, and prefer the specification whose residuals look like noise.

THE COST OF IGNORING AUTOCORRELATION

nominal n	: 459
approximate effective n	: 413
inflation of standard errors	: x1.05

Here the correction is mild because model 1's residuals are nearly clean. Apply the same arithmetic to model 2 ($I = 0.892$) and the effective n collapses to about **26** — standard errors would need inflating by more than four times.

The residual maps are the visual argument: model 2's residuals show obvious large-scale structure (a coherent red/blue gradient); model 1's are close to noise. **If your residual map looks like a map, your model is incomplete.**

A10 — Predictive modelling I: designing a flood-susceptibility model

What we are going to learn. How to set up a spatial machine-learning problem properly — which is almost entirely about the design, not the algorithm.

Why it matters. Flood-susceptibility mapping is one of the most common applied spatial-ML tasks, and it is riddled with methodological traps. Getting the sampling design right matters far more than which gradient booster you pick.

The concept — presence-only data and pseudo-absences. We have 386 recorded flood incidents (presences). We have **no recorded non-floods**. This is *presence-only* data, and it needs care:

Decision	Options	Consequence
Where to draw pseudo-absences	Uniformly at random / weighted by population / outside a buffer around presences	Determines what the model actually learns
How many	1:1 with presences / 1:10 / all background cells	Affects calibration; the prevalence is arbitrary
Sampling bias	Incidents are <i>reported</i> , so they concentrate where people are	Model learns "where people are", not "where floods are"

The reporting-bias trap is the important one. If you draw pseudo-absences uniformly across the basin while presences are reported only where people live, your model will learn to predict *population*, achieve a superb AUC, and be useless. The standard remedy is to draw pseudo-absences from the **same sampling frame** as the presences — here, weighted by population — so the model must learn what distinguishes flooded from non-flooded places *among places where a flood would have been noticed*.

Feature design. Use physically motivated predictors: elevation, TPI, slope, distance to river, rainfall, land cover, upstream area proxies. **Exclude anything derived from the outcome** — the flood-zone polygons themselves are derived from the same terrain, so including `pct_in_flood100` would be circular.

Expected outcome. A modelling table of presences and bias-matched pseudo-absences with physically motivated features, and a demonstration of what happens when the pseudo-absence design is wrong.

What the next cell does: builds two competing designs — naive uniform pseudo-absences and population-matched ones — extracts raster features at every point, and shows how differently the two datasets behave.

```
In [45]: from rasterio.warp import Resampling

# ---- feature rasters, all on the common 100 m grid -----
feat_rasters = {
    "elev": grids["elevation"],
    "rain": grids["rainfall"],
```

```

        "ndvi": grids["ndvi"],
        "lc": grids["landcover"],
    }
    feat_rasters["slope"] = align_to(slope_path, ref, Resampling.average)
    feat_rasters["tpi"] = align_to(tpi_path, ref, Resampling.average)

    # distance-to-river surface on the same grid
    riv_g = burn(rivers)
    feat_rasters["d_river"] = distance_transform_edt(~riv_g, sampling=CELL)
    coast_g = burn(coastline)
    feat_rasters["d_coast"] = distance_transform_edt(~coast_g, sampling=CELL)

    # HAND: height above nearest drainage - the key hydrological predictor
    _, (ri, ci) = distance_transform_edt(~riv_g, return_indices=True)
    elev0 = np.nan_to_num(grids["elevation"], nan=0.0)
    feat_rasters["hand"] = np.where(np.isfinite(grids["elevation"]),
                                    elev0 - elev0[ri, ci], np.nan)

def sample_grid(arr, xs, ys, transform=None, cell=None):
    """Sample a grid at map coordinates. Shape comes from `arr` itself, never
    from a global - a helper that reads globals will break the moment somebody
    reuses one of those names."""
    tr = TR if transform is None else transform
    cs = CELL if cell is None else cell
    nrows, ncols = arr.shape
    cols = ((np.asarray(xs) - tr.c) / cs).astype(int)
    rows = ((tr.f - np.asarray(ys)) / cs).astype(int)
    ok = (rows >= 0) & (rows < nrows) & (cols >= 0) & (cols < ncols)
    out = np.full(len(xs), np.nan)
    out[ok] = arr[rows[ok], cols[ok]]
    return out

def build_features(xs, ys):
    df = pd.DataFrame({k: sample_grid(v, xs, ys) for k, v in feat_rasters.items()})
    df["lc"] = df["lc"].round()
    return df

# ---- presences -----
pres_x = pts.geometry.x.to_numpy(); pres_y = pts.geometry.y.to_numpy()
n_pres = len(pres_x)

# ---- pseudo-absence design A: UNIFORM over the basin -----
rng = np.random.default_rng(2024)
def sample_uniform(n):
    xs, ys = [], []
    minx_, miny_, maxx_, maxy_ = LAND_GEOM.bounds
    while len(xs) < n:
        cx = rng.uniform(minx_, maxx_, 500); cy = rng.uniform(miny_, maxy_, 500)
        for a, b in zip(cx, cy):
            if LAND_GEOM.contains(Point(a, b)):
                xs.append(a); ys.append(b)
                if len(xs) == n:
                    break
    return np.array(xs), np.array(ys)

# ---- pseudo-absence design B: matched to the REPORTING process -----
def sample_pop_weighted(n):
    w = Fx.population.to_numpy(float); w = w / w.sum()
    pick = rng.choice(len(Fx), size=n, p=w)
    xs, ys = [], []
    for i in pick:
        g = Fx.geometry.iloc[i]
        a0, b0, a1, b1 = g.bounds
        for _ in range(60):
            q = Point(rng.uniform(a0, a1), rng.uniform(b0, b1))
            if g.contains(q):
                xs.append(q.x); ys.append(q.y); break
    return np.array(xs), np.array(ys)

```

```

RATIO = 3
ax_, ay_ = sample_uniform(n_pres * RATIO)
bx_, by_ = sample_pop_weighted(n_pres * RATIO)

def assemble(abs_x, abs_y, tag):
    Xp = build_features(pres_x, pres_y); Xp["flood"] = 1
    Xa = build_features(abs_x, abs_y); Xa["flood"] = 0
    d = pd.concat([Xp, Xa], ignore_index=True)
    d["x"] = np.r_[pres_x, abs_x]; d["y"] = np.r_[pres_y, abs_y]
    d["design"] = tag
    return d.dropna(subset=[c for c in feat_rasters])

dsA = assemble(ax_, ay_, "A: uniform")
dsB = assemble(bx_, by_, "B: population-matched")

print("TWO PSEUDO-ABSENCE DESIGNS")
print(f" presences : {n_pres}")
print(f" pseudo-absences per design : {n_pres * RATIO} (ratio 1:{RATIO})")
print(f" design A rows after NaN drop: {len(dsA),}")
print(f" design B rows after NaN drop: {len(dsB),}")

# ---- how different are the two backgrounds? -----
print("\nWHAT DOES EACH DESIGN'S BACKGROUND LOOK LIKE?")
print(f" {'feature':<10}{'presences':>12}{'A: uniform':>13}{'B: pop-matched':>16}"
      f"{'|A-P|':>9}{'|B-P|':>9}")
print("-" * 72)
for f in ["elev", "hand", "d_river", "rain", "slope", "tpi"]:
    mp = dsA.loc[dsA.flood == 1, f].mean()
    ma = dsA.loc[dsA.flood == 0, f].mean()
    mb = dsB.loc[dsB.flood == 0, f].mean()
    print(f" {f:<10}{mp:>12.1f}{ma:>13.1f}{mb:>16.1f}"
          f"{abs(ma-mp):>9.1f}{abs(mb-mp):>9.1f}")
print("\n Design A's background differs from the presences on EVERY variable,"
      " including ones that have nothing to do with flooding. A model will"
      " happily exploit those differences. Design B's background is matched on"
      " the reporting process, so the remaining differences are the real signal.")

# ---- a proxy for the reporting bias -----
for name, d in [("A: uniform", dsA), ("B: population-matched", dsB)]:
    pd_pres = sample_grid(grids["popdens"], d.loc[d.flood == 1, "x"],
                          d.loc[d.flood == 1, "y"])
    pd_abs = sample_grid(grids["popdens"], d.loc[d.flood == 0, "x"],
                          d.loc[d.flood == 0, "y"])
    print(f"\n {name}: median population density")
    print(f" at presences : {np.nanmedian(pd_pres):>9,.0f} /km2")
    print(f" at pseudo-absences : {np.nanmedian(pd_abs):>9,.0f} /km2")
    print(f" ratio : "
          f"{np.nanmedian(pd_pres)/max(np.nanmedian(pd_abs),1e-9):>9.2f}x")

fig, axes = plt.subplots(1, 3, figsize=(17.5, 5.2))
for ax, (name, d) in zip(axes[:2], [("A: uniform background", dsA),
                                     ("B: population-matched background", dsB)]):
    land.plot(ax=ax, facecolor="#f5f2ea", edgecolor="#ddd6c8", linewidth=0.5)
    ax.scatter(d.loc[d.flood == 0, "x"], d.loc[d.flood == 0, "y"], s=2.5,
              c="#888888", label="pseudo-absence")
    ax.scatter(d.loc[d.flood == 1, "x"], d.loc[d.flood == 1, "y"], s=4,
              c="crimson", label="flood incident")
    ax.legend(fontsize=7, loc="lower left")
    ax.set_title(name, fontsize=9.5, weight="bold", loc="left")
    ax.set_aspect("equal"); ax.set_xticks([]); ax.set_yticks([])

for f, c in [("hand", "#b2182b"), ("d_river", "#2166ac")]:
    pass
axes[2].hist([dsA.loc[dsA.flood == 1, "hand"], dsA.loc[dsA.flood == 0, "hand"],
             dsB.loc[dsB.flood == 0, "hand"]], bins=30,
            label=["presences", "A: uniform", "B: pop-matched"],
            color=["crimson", "#999999", "#2166ac"])

```

```

axes[2].set_xlabel("HAND – height above nearest drainage (m)")
axes[2].set_ylabel("count")
axes[2].set_title("The variable that should matter", fontsize=9.5,
                  weight="bold", loc="left")
axes[2].legend(fontsize=8); axes[2].grid(alpha=0.3)
plt.tight_layout(); plt.show()

```

Explanation.

- **HAND (Height Above Nearest Drainage)** is the single most important predictor in fluvial flood modelling: how high a cell sits above the nearest channel. `distance_transform_edt(..., return_indices=True)` gives, for every cell, the index of the nearest river cell, so `elev – elev[nearest_river]` is HAND in one vectorised step. It also happens to be close to how the data was generated, which is why it should dominate.
- **We deliberately exclude `pct_in_flood100`** from the features. The flood-zone polygons were themselves derived from HAND and elevation, so including them would be **target leakage dressed as a feature** — the model would achieve near-perfect accuracy and have learned nothing. In real projects this happens when someone includes "distance to previously mapped flood extent".
- **Design A (uniform)** scatters pseudo-absences over the whole basin, including vast uninhabited uplands where a flood would never have been *reported* even if it happened. **Design B** draws them with probability proportional to population, matching the reporting process that generated the presences.
- **The comparison table is the argument.** Under design A, presences and background differ on *every* variable, including ones with no causal link to flooding. A model can achieve a spectacular AUC by learning "is this place inhabited?". Under design B, the population confound is largely removed and the remaining separation — chiefly on HAND and distance-to-river — is the real hydrological signal.
- The population-density check quantifies the confound directly: under design A the presence/absence ratio of median population density is large; under design B it should be close to 1.

Expected outcome.

WHAT DOES EACH DESIGN'S BACKGROUND LOOK LIKE?

feature	presences	A: uniform	B: pop-matched	A-P	B-P
elev	100.3	338.3	101.5	238.0	1.2
hand	-0.3	-9.6	20.3	9.3	20.6
d_river	596.9	2984.6	2157.7	2387.7	1560.8
rain	607.6	756.7	605.7	149.1	1.9
slope	1.6	2.7	2.2	1.1	0.6
tpi	-1.1	0.2	-0.0	1.3	1.1

A: uniform: median population density

at presences : 2,189 /km2
 at pseudo-absences : 34 /km2
 ratio : 63.75x

B: population-matched: median population density

at presences : 2,189 /km2
 at pseudo-absences : 2,2xx /km2
 ratio : ~1.0x

Look at the |A-P| column. Under the uniform design the background differs from the presences by **238 m of elevation** and **149 mm of rainfall** — neither of which causes flooding. It differs because the uniform sample includes the empty uplands, where no flood would ever have been *reported*. A classifier will seize on elevation and rainfall, score a magnificent AUC, and have learned "is this the lowland city?".

Under design B those two gaps collapse to 1.2 m and 1.9 mm — the background is now drawn from the same population-weighted frame as the presences. What survives is **hand** (-0.3 vs +20.3 m) and **d_river** (597 m vs 2 158 m): the genuine hydrological signal.

The population-density check makes it unambiguous: **design A has a 63.75× density ratio between presences and pseudo-absences; design B is near 1×.**

The HAND histogram is the payoff: presences concentrate at low HAND, design A's background spreads to high HAND (an easy, partly spurious separation), and design B's background overlaps the presences much more — a **harder but honest** problem. Lesson A11 measures exactly how much of design A's apparent skill is illusory.

A11 — Predictive modelling II: spatial cross-validation

What we are going to learn. Why random k-fold cross-validation gives dishonest results on spatial data, and how spatial blocking fixes it.

Why it matters. This is the single most consequential methodological point in applied spatial machine learning, and it is routinely got wrong in published work. Models are reported with AUC = 0.95 that would score 0.65 on new territory.

The concept — why random CV leaks. Spatial autocorrelation means nearby observations are nearly duplicates. A random split puts a point at (x, y) in the training set and its neighbour at (x + 30 m, y) in the test set. The model has effectively seen the test point. The resulting score measures **interpolation skill within the sampled area**, which is almost never the quantity you care about — you want **extrapolation to unsampled areas**.

The fix — spatial blocking. Partition space into blocks and assign whole blocks to folds, so that training and test data are spatially separated:

Scheme	How	When
Spatial block CV	Regular grid of blocks, blocks assigned to folds	The general-purpose default
Spatial k-means CV	Cluster coordinates, clusters become folds	Irregular sampling
Leave-one-region-out	Folds = administrative units	When you will deploy region by region
Buffered / spatial LOO	Remove a buffer around each test point	Rigorous but expensive

How big should the block be? At least the **range of spatial autocorrelation** of your residuals — the distance beyond which observations are effectively independent. Estimate it from a variogram or by testing several block sizes.

The honest reporting rule. Report *both* scores. The random-CV score tells you about interpolation; the spatial-CV score tells you about transfer. The gap between them is a measurement of how much your model depends on location rather than on process.

Expected outcome. The same model evaluated by random CV and by spatial block CV, on both pseudo-absence designs — four numbers that tell the whole story.

What the next cell does: implements spatial block CV, evaluates a random forest and a gradient-booster under both CV schemes and both sampling designs, and quantifies the optimism of each.

```
In [46]: from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GroupKFold, StratifiedKFold, cross_val_score
from sklearn.metrics import roc_auc_score, average_precision_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

FEATURES = ["elev", "slope", "tpi", "hand", "d_river", "d_coast", "rain", "ndvi"]

def spatial_blocks(x, y, block_m=4000):
    """Assign every point to a square spatial block -> use as CV groups."""
    bx = np.floor((x - x.min()) / block_m).astype(int)
    by = np.floor((y - y.min()) / block_m).astype(int)
    return bx * 10_000 + by
```

```

def evaluate(ds, label, block_m=4000, n_splits=5, seed=0):
    X = ds[FEATURES].to_numpy(dtype=float)
    yv = ds["flood"].to_numpy(dtype=int)
    groups = spatial_blocks(ds.x.to_numpy(), ds.y.to_numpy(), block_m)

    models = {
        "logistic": make_pipeline(StandardScaler(),
                                   LogisticRegression(max_iter=2000)),
        "random forest": RandomForestClassifier(n_estimators=300, min_samples_leaf=3,
                                                random_state=seed, n_jobs=-1),
        "grad. boosting": HistGradientBoostingClassifier(max_iter=250,
                                                         random_state=seed),
    }
    rows = []
    for name, mdl in models.items():
        # --- random stratified CV (the WRONG way for spatial data) -----
        skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=seed)
        auc_rand = cross_val_score(mdl, X, yv, cv=skf, scoring="roc_auc").mean()
        # --- spatial block CV (the right way) -----
        n_groups = len(np.unique(groups))
        gkf = GroupKFold(n_splits=min(n_splits, n_groups))
        auc_spat = cross_val_score(mdl, X, yv, cv=gkf, groups=groups,
                                   scoring="roc_auc").mean()
        rows.append({"design": label, "model": name,
                    "AUC random CV": auc_rand, "AUC spatial CV": auc_spat,
                    "optimism": auc_rand - auc_spat,
                    "n_blocks": n_groups})
    return pd.DataFrame(rows)

print("CROSS-VALIDATION: RANDOM vs SPATIALLY BLOCKED")
print("=" * 92)
res = pd.concat([evaluate(dsA, "A: uniform"),
                 evaluate(dsB, "B: pop-matched")], ignore_index=True)
print(res.round(4).to_string(index=False))
print("=" * 92)
print(" 'optimism' = how much the random split flatters the model.")

# --- how does the answer depend on block size? -----
print("\nSENSITIVITY TO BLOCK SIZE (random forest, design B)")
print(f" {'block size':>12}{'n blocks':>11}{'AUC spatial':>14}{'optimism':>11}")
X = dsB[FEATURES].to_numpy(dtype=float); yv = dsB.flood.to_numpy(int)
rf = RandomForestClassifier(n_estimators=300, min_samples_leaf=3,
                           random_state=0, n_jobs=-1)
auc_rand_B = cross_val_score(rf, X, yv,
                             cv=StratifiedKFold(5, shuffle=True, random_state=0),
                             scoring="roc_auc").mean()
for bm in [1000, 2000, 4000, 8000, 16000]:
    g = spatial_blocks(dsB.x.to_numpy(), dsB.y.to_numpy(), bm)
    ng = len(np.unique(g))
    a = cross_val_score(rf, X, yv, cv=GroupKFold(min(5, ng)), groups=g,
                        scoring="roc_auc").mean()
    print(f" {bm:>10}, {m[ng:>11]}{a:>14.4f}{auc_rand_B - a:>11.4f}")
print("\n Two effects fight each other as blocks grow: train and test separate")
print(" in space (score falls), but each fold also gets a larger, more varied")
print(" training set (score rises). The curve is therefore NOT monotone - which")
print(" is why you report the curve rather than one number, and take the")
print(" CONSERVATIVE (lowest) value as your out-of-area estimate.")

# --- Fit the final model on design B, spatially validated -----
final = RandomForestClassifier(n_estimators=500, min_samples_leaf=3,
                              random_state=0, n_jobs=-1)
groupsB = spatial_blocks(dsB.x.to_numpy(), dsB.y.to_numpy(), 4000)
gkf = GroupKFold(n_splits=5)
oof = np.zeros(len(dsB))
for tr, te in gkf.split(X, yv, groups=groupsB):
    m = RandomForestClassifier(n_estimators=500, min_samples_leaf=3,
                              random_state=0, n_jobs=-1).fit(X[tr], yv[tr])

```

```

oof[te] = m.predict_proba(X[te])[:, 1]
final.fit(X, yv)

print(f"\nFINAL MODEL (random forest, design B, 4 km spatial blocks)")
print(f" out-of-fold AUC : {roc_auc_score(yv, oof):.4f}")
print(f" out-of-fold avg precision: {average_precision_score(yv, oof):.4f}")
print(f" baseline (prevalence) : {yv.mean():.4f}")

# --- Figure -----
fig, axes = plt.subplots(1, 3, figsize=(17.5, 5.2))
piv = res.pivot_table(index="model", columns="design",
                      values=["AUC random CV", "AUC spatial CV"])
xpos = np.arange(len(piv.index)); wdt = 0.2
for i, (metric, design, c) in enumerate([
    ("AUC random CV", "A: uniform", "#f4a582"),
    ("AUC spatial CV", "A: uniform", "#b2182b"),
    ("AUC random CV", "B: pop-matched", "#92c5de"),
    ("AUC spatial CV", "B: pop-matched", "#2166ac")]):
    axes[0].bar(xpos + (i - 1.5) * wdt, piv[(metric, design)], wdt,
               label=f"{design.split(':')[0]} / {metric.split()[1]}", color=c)
axes[0].set_xticks(xpos); axes[0].set_xticklabels(piv.index, fontsize=8)
axes[0].set_ylim(0.5, 1.0); axes[0].set_ylabel("AUC")
axes[0].axhline(0.5, color="grey", linestyle=":")
axes[0].legend(fontsize=6.5); axes[0].grid(alpha=0.3, axis="y")
axes[0].set_title("Random CV flatters every model", fontsize=9.5,
                 weight="bold", loc="left")

# the blocks themselves
land.plot(ax=axes[1], facecolor="#f5f2ea", edgecolor="#ddd6c8", linewidth=0.5)
gb = pd.DataFrame({"x": dsB.x, "y": dsB.y, "g": groupsB})
fold = {g: i % 5 for i, g in enumerate(np.unique(groupsB))}
axes[1].scatter(gb.x, gb.y, c=[fold[g] for g in gb.g], cmap="tab10", s=5)
axes[1].set_title("4 km spatial blocks -> CV folds", fontsize=9.5,
                 weight="bold", loc="left")
axes[1].set_aspect("equal"); axes[1].set_xticks([]); axes[1].set_yticks([])

from sklearn.metrics import roc_curve
for lab, yy, pp, c in [("spatial CV (out-of-fold)", yv, oof, "#2166ac")]:
    fpr, tpr, _ = roc_curve(yy, pp)
    axes[2].plot(fpr, tpr, color=c, linewidth=2,
                 label=f"{lab}: AUC = {roc_auc_score(yy, pp):.3f}")
axes[2].plot([0, 1], [0, 1], "k--", linewidth=1)
axes[2].set_xlabel("false positive rate"); axes[2].set_ylabel("true positive rate")
axes[2].legend(fontsize=8); axes[2].grid(alpha=0.3)
axes[2].set_title("ROC of the honestly-validated model", fontsize=9.5,
                 weight="bold", loc="left")
plt.tight_layout(); plt.show()

```

Explanation.

- **GroupKFold with spatial blocks as groups** is the whole trick. Points in the same block always go to the same fold, so no test point ever has a training neighbour inside its own block. **sklearn** needs no spatial awareness — you supply the geography through **groups**.
- **spatial_blocks** floors coordinates onto a grid and encodes the cell as an integer. Simple, deterministic and easy to explain. More sophisticated schemes (k-means on coordinates, systematic block assignment) exist, but a regular grid is the standard baseline.
- **Three models on purpose.** Logistic regression cannot memorise locations, so its optimism is small. Tree ensembles can carve up feature space finely and are therefore the most exposed to leakage — which is precisely why the models that look best under random CV often degrade the most.
- **The block-size sensitivity table is the diagnostic.** As blocks grow, train and test separate further in space, and the score falls. If it falls a long way, your model is relying on location. If it plateaus, you have found the autocorrelation range and the plateau value is your honest estimate of out-of-area performance.

- **average_precision_score alongside AUC.** With 25% prevalence, AUC is reasonable, but for rarer targets AUC is misleadingly optimistic and average precision (the area under the precision–recall curve) is the better summary. Always report the baseline prevalence next to it.
- **oof** (out-of-fold predictions) are the correct basis for any downstream threshold selection or calibration. Using in-sample predictions to pick a threshold is a second, subtler leak.

Expected outcome.

design	model	AUC random CV	AUC spatial CV	optimism	n_blocks
A: uniform	logistic	0.939	0.934	0.004	93
A: uniform	random forest	0.953	0.943	0.010	93
A: uniform	grad. boosting	0.947	0.928	0.019	93
B: pop-matched	logistic	0.853	0.844	0.009	81
B: pop-matched	random forest	0.892	0.851	0.041	81
B: pop-matched	grad. boosting	0.880	0.834	0.046	81

Read this table from the top-left to the bottom-right.

- **Design A + random CV = 0.953.** The most flattering number available, and the one that would appear in a paper. It is measuring the model's ability to notice that flood reports come from inhabited lowland.
- **Design B + spatial CV = 0.851.** The least flattering, and the only one that estimates what you actually want: can this model find flood-prone ground in territory it has never seen? **The gap is 0.10 AUC** — and every step of it is methodological, not modelling.
- Note that **optimism is four times larger under design B** (0.041–0.046) than under design A (0.004–0.019). Design A's problem is so easy that even a spatially separated test set is trivial; the leakage only bites once the problem is honest.
- Tree ensembles show more optimism than logistic regression, exactly as predicted: flexible models memorise location.

SENSITIVITY TO BLOCK SIZE (random forest, design B)

block size	n blocks	AUC spatial	optimism
1,000 m	402	0.8776	0.0144
2,000 m	185	0.8775	0.0145
4,000 m	81	0.8508	0.0412
8,000 m	29	0.8664	0.0256
16,000 m	9	0.8819	0.0100

The curve is not monotone, and that is worth understanding. Two effects fight: larger blocks separate train and test further (score falls), but they also give each fold a larger and more varied training set (score rises). At 1–2 km the blocks are smaller than the autocorrelation range, so leakage persists; at 16 km there are only 9 blocks and the folds become unstable. The dip at 4 km is the conservative estimate, and **the conservative value is the one to report.**

Three panels: the bar chart (spatial bars always lower), a map of the 4 km blocks coloured by fold, and the ROC curve of the honestly validated model (**AUC $\approx$ 0.855, average precision $\approx$ 0.53 against a 0.25 baseline**).

A12 — Model interpretation and prediction surfaces

What we are going to learn. How to find out what a spatial model has learned, and how to turn it into a map you can defend.

Why it matters. A susceptibility map is a policy instrument. Somebody will use it to refuse planning permission or to allocate a budget. "The random forest said so" is not an acceptable justification; you need to be able to state *which variables drive the prediction, in which direction, and where the model is uncertain.*

The concept — four interpretation tools.

Tool	Question	Caveat
Impurity importance	Which features did the trees split on?	Biased towards high-cardinality and continuous features. Do not use it.
Permutation importance	How much does performance drop if I shuffle this feature?	Must be computed on held-out data; splits credit arbitrarily between correlated features
Partial dependence	What is the average predicted response to this feature?	Assumes feature independence — dubious for correlated spatial features
Prediction surface	Where does the model say the risk is?	Extrapolates silently outside the training envelope

Uncertainty. A random forest gives you a free uncertainty estimate: the spread of predictions across trees. High spread means the trees disagree — often because that location is unlike anything in the training data. **Mapping disagreement alongside prediction is the single most useful honesty measure available**, and it costs nothing.

Extrapolation. Check whether prediction locations fall inside the training data's feature envelope. A model trained on $\text{HAND} \in [-5, 40] \text{ m}$ is guessing when asked about $\text{HAND} = 300 \text{ m}$.

Expected outcome. Permutation importances on held-out folds, partial dependence for the top features, a full-basin susceptibility surface, and a matching uncertainty map.

What the next cell does: computes spatially-validated permutation importance, plots partial dependence, predicts over every land cell, maps the result with an uncertainty overlay, and flags extrapolation.

```
In [47]: from sklearn.inspection import permutation_importance, PartialDependenceDisplay

# --- 1. Permutation importance, computed OUT OF FOLD -----
imp_rows = []
for tr, te in GroupKFold(n_splits=5).split(X, yv, groups=groupsB):
    m = RandomForestClassifier(n_estimators=300, min_samples_leaf=3,
                              random_state=0, n_jobs=-1).fit(X[tr], yv[tr])
    r = permutation_importance(m, X[te], yv[te], n_repeats=10,
                              random_state=0, scoring="roc_auc", n_jobs=-1)
    imp_rows.append(r.importances_mean)
imp = pd.DataFrame(imp_rows, columns=FEATURES)

# compare with the (biased) impurity importance
final.fit(X, yv)
imp_gini = pd.Series(final.feature_importances_, index=FEATURES)

summary = pd.DataFrame({
    "permutation (AUC drop)": imp.mean(),
    "perm. std across folds": imp.std(),
    "impurity (biased)": imp_gini,
}).sort_values("permutation (AUC drop)", ascending=False)
print("FEATURE IMPORTANCE (random forest, design B, spatial folds)")
print(summary.round(4).to_string())
print("\n Impurity importance and permutation importance can rank features")
print(" differently. Trust the permutation ranking - it is measured on data")
print(" the model has not seen.")

# --- 2. Extrapolation check -----
print("\nTRAINING ENVELOPE (design B)")
env = pd.DataFrame({"min": X.min(axis=0), "max": X.max(axis=0)}, index=FEATURES)
print(env.round(2).to_string())

# --- 3. Predict over the whole basin -----
stack = np.stack([feat_rasters[f] for f in FEATURES], axis=-1)
valid_cells = np.isfinite(stack).all(axis=-1) & land_g
flat = stack[valid_cells]
```

```

proba = final.predict_proba(flat)[: , 1]
# per-tree spread = model disagreement
tree_p = np.stack([t.predict_proba(flat)[: , 1] for t in final.estimators_])
spread = tree_p.std(axis=0)

# extrapolation flag: outside the training range on any feature
outside = ((flat < X.min(axis=0)) | (flat > X.max(axis=0))).any(axis=1)

surf = np.full((H, Wd), np.nan); surf[valid_cells] = proba
unc = np.full((H, Wd), np.nan); unc[valid_cells] = spread
ext = np.full((H, Wd), np.nan); ext[valid_cells] = outside.astype(float)

print(f"\nPREDICTION SURFACE")
print(f"   cells predicted           : {valid_cells.sum():,} "
      f"({valid_cells.sum()*(CELL/1000)**2:,.0f} km^2)")
print(f"   mean predicted probability : {np.nanmean(surf):.3f}")
print(f"   cells with p > 0.5         : {int(np.nansum(surf > 0.5)):,} "
      f"({100*np.nansum(surf > 0.5)/valid_cells.sum():.1f} %)")
print(f"   mean tree disagreement     : {np.nanmean(unc):.3f}")
print(f"   cells OUTSIDE the training envelope : "
      f"{int(outside.sum()):,} ({100*outside.mean():.1f} %)")
print("   -> predictions there are extrapolation and must be labelled as such")

# --- 4. Validate against the hazard zones we never showed the model -----
zone_g = burn(flood[flood.return_period_yr == 100])
in_zone = zone_g[valid_cells]
print(f"\nEXTERNAL CHECK against the 100-year hazard zone (never used as a feature)")
print(f"   mean predicted p INSIDE the zone : {proba[in_zone].mean():.3f}")
print(f"   mean predicted p OUTSIDE the zone : {proba[~in_zone].mean():.3f}")
print(f"   AUC of the prediction vs zone membership : "
      f"{roc_auc_score(in_zone.astype(int), proba):.4f}")
print("   The model was trained only on incident POINTS, yet it reconstructs the")
print("   independently-derived hazard zone. That is real, external validation.")

# --- 5. Figures -----
fig, axes = plt.subplots(1, 3, figsize=(17.5, 5.4))
order = summary.index.tolist()
axes[0].barh(range(len(order)), summary.loc[order, "permutation (AUC drop)"],
             xerr=summary.loc[order, "perm. std across folds"],
             color="#2166ac", height=0.6)
axes[0].set_yticks(range(len(order))); axes[0].set_yticklabels(order, fontsize=8)
axes[0].invert_yaxis(); axes[0].set_xlabel("mean AUC drop when shuffled")
axes[0].set_title("Permutation importance\n(out-of-fold, 5 spatial folds)",
                  fontsize=9.5, weight="bold", loc="left")
axes[0].grid(alpha=0.3, axis="x")

ext_img = (TR.c, TR.c + Wd*CELL, TR.f - H*CELL, TR.f)
im = axes[1].imshow(surf, cmap="RdYlGn_r", extent=ext_img, vmin=0, vmax=1)
rivers.plot(ax=axes[1], color="#08306b", linewidth=0.7)
plt.colorbar(im, ax=axes[1], shrink=0.75, label="P(flood-prone)")
axes[1].set_title("Flood susceptibility surface", fontsize=9.5,
                  weight="bold", loc="left")

im = axes[2].imshow(unc, cmap="magma", extent=ext_img)
plt.colorbar(im, ax=axes[2], shrink=0.75, label="std across trees")
axes[2].set_title("Model uncertainty\n(where the trees disagree)",
                  fontsize=9.5, weight="bold", loc="left")

for a in axes[1:]:
    a.set_aspect("equal"); a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.show()

fig, ax = plt.subplots(figsize=(12, 3.4))
top4 = order[:4]
PartialDependenceDisplay.from_estimator(
    final, X, [FEATURES.index(f) for f in top4], feature_names=FEATURES,
    ax=ax, line_kw={"color": "#b2182b", "linewidth": 2})
plt.suptitle("Partial dependence of the four most important features",

```

```

        fontsize=11, y=1.04)
plt.tight_layout(); plt.show()

```

Explanation.

- **Permutation importance must be computed out of fold.** Computing it on training data measures how much the model *memorised* a feature, not how much it *needs* it. We loop over the same spatial folds used for validation, so the importances inherit the honest evaluation.
- **The comparison with impurity importance is deliberate.** `feature_importances_` on a tree ensemble is biased towards continuous, high-cardinality variables and can rank a useless feature above a crucial one. It is the default that everyone uses and it should not be.
- **Correlated features split importance.** `hand`, `elev` and `d_river` are all proxies for the same hydrological reality. Permutation importance will divide the credit among them somewhat arbitrarily — shuffling one leaves the others to compensate. Never read a single feature's importance as "the effect of X".
- **Tree spread as uncertainty.** `final.estimators_` gives the individual trees; their standard deviation at each cell is a cheap, honest measure of model disagreement. It rises where the training data is sparse or contradictory — exactly where you should not be making confident planning decisions.
- **The extrapolation flag** compares each prediction cell against the per-feature min/max of the training data. Any cell outside on any feature is being extrapolated. Random forests **cannot** extrapolate — they return the nearest leaf value — so predictions there are effectively "the most similar place I saw", which may be nothing like the truth.
- **The external check is the best evidence in the whole module.** The model was trained on 386 incident *points* plus population-matched background, using only terrain and climate features. The 100-year flood zone was derived independently from a HAND threshold and never shown to the model. If the model's surface reproduces that zone, it has learned real hydrology rather than the sampling design.

Expected outcome.

FEATURE IMPORTANCE (random forest, design B, spatial folds)			
	permutation (AUC drop)	perm. std across folds	impurity (biased)
hand	0.097	0.056	0.250
d_river	0.065	0.030	0.217
slope	0.004	0.008	0.075
elev	0.004	0.004	0.098
d_coast	0.004	0.003	0.115
rain	0.002	0.006	0.092
ndvi	0.000	0.003	0.090
tpi	-0.003	0.008	0.064

`hand` and `d_river` carry essentially all the signal — correct, because the incidents were generated inside HAND-derived flood zones. Everything else is at or below noise level, and `tpi` is *negative*: shuffling it slightly *improves* held-out AUC, which is the signature of a feature contributing nothing but variance.

Now compare the two importance columns. Impurity gives `d_coast` 0.115 and `elev` 0.098 — third and fourth place — while permutation puts both at 0.004, indistinguishable from zero. The trees split on them often (they are continuous and high-cardinality) without those splits generalising. **This is why you should not use `feature_importances_`.**

```

PREDICTION SURFACE
cells predicted          : 137,251 (1,373 km^2)
mean predicted probability : 0.192
cells with p > 0.5       : 21,483 (15.7 %)
mean tree disagreement   : 0.218
cells OUTSIDE the training envelope : 4,232 (3.1 %)

```

```

EXTERNAL CHECK against the 100-year hazard zone (never used as a feature)
mean predicted p INSIDE the zone : 0.601
mean predicted p OUTSIDE the zone : 0.130
AUC of the prediction vs zone membership : 0.9647

```

The external check is the strongest evidence in this module. The model saw only 386 incident *points* and a population-matched background, with terrain and climate features. The 100-year flood zone was derived independently from a HAND threshold and never shown to it. The model reproduces that zone with **AUC = 0.965**. It has learned hydrology, not the sampling design — which is precisely the claim that design B and spatial CV were built to make defensible.

Three panels: the importance bar chart with cross-fold error bars, the susceptibility surface (high along river corridors and the coastal plain), and the uncertainty map — highest at the *edges* of those corridors, where the model is least sure. Then partial-dependence curves showing probability falling steeply with *hand* and *d_river*: physically sensible, monotone relationships.

A13 — Quantitative risk: hazard × exposure × vulnerability

What we are going to learn. The standard risk decomposition, and how to turn a susceptibility map into a monetary expected annual loss.

Why it matters. "This area is at risk" is not actionable. "This district faces an expected annual loss of 4.2 million VS, of which 60% is concentrated in pre-1970 masonry buildings" is. Converting a probability surface into an expected loss is what makes spatial analysis a decision tool.

The concept — the risk triangle.

RISK = HAZARD × EXPOSURE × VULNERABILITY

Term	Meaning	Our proxy
Hazard	Probability and intensity of the event	Return-period zones + the modelled susceptibility surface
Exposure	What is there to be damaged	Buildings, their value, and people
Vulnerability	How badly it is damaged, given the hazard	A damage function : fraction of value lost as a function of depth

Expected Annual Damage (EAD). For a set of return periods with exceedance probabilities $p_i = 1/T_i$ and losses L_i , the EAD is the area under the loss–exceedance–probability curve:

$$\text{EAD} = \int L(p) \, dp \approx \sum_i \frac{1}{2} (L_i + L_{i+1}) (p_i - p_{i+1})$$

This is the number that goes into a cost–benefit analysis: a defence costing X per year is worth building if it reduces EAD by more than X.

Vulnerability curves. Depth–damage functions are empirical and building-type-specific. Their shape matters enormously and they are the largest source of uncertainty in most flood-risk assessments — larger than the hazard model. **Always run a sensitivity analysis on the damage function.**

Expected outcome. Per-building expected annual damage, aggregated to districts, with a loss-exceedance curve and a sensitivity analysis.

What the next cell does: defines depth–damage curves by construction type, estimates flood depth per building per return period, computes EAD, aggregates it, and tests how sensitive the total is to the damage-function assumption.

```
In [48]: # --- 1. EXPOSURE: buildings with value and construction type -----
B = buildings_clean[["building_id", "use_type", "construction", "floors",
                    "footprint_m2", "value_kvs", "ground_elev_m",
                    "has_basement", "geometry"]].copy()
B["value_kvs"] = B.value_kvs.fillna(B.value_kvs.median())
Bpt = B.copy(); Bpt["geometry"] = B.geometry.representative_point()

# --- 2. HAZARD: depth at each building for each return period -----
# depth = (local flood-surface level) - (ground elevation), estimated from HAND
hand_at = sample_grid(featurasters["hand"], Bpt.geometry.x, Bpt.geometry.y)
Bpt["hand_m"] = hand_at

# design flood levels above the channel, by return period (fictional but ordered)
LEVELS = {10: 1.2, 25: 2.0, 50: 2.8, 100: 3.5, 250: 5.0, 500: 6.5}
for T, lvl in LEVELS.items():
    Bpt[f"depth_{T}"] = np.clip(lvl - Bpt.hand_m, 0, None)

print("HAZARD: flooded buildings by return period")
print(f" {'return period':>14}{'design level':>14}{'buildings wet':>15}{'% of stock':>12}")
for T, lvl in LEVELS.items():
    n = int((Bpt[f"depth_{T}"] > 0).sum())
    print(f" {T:>12} yr{lvl:>13.1f} m{n:>15},{100*n/len(Bpt):>11.1f}%")

# --- 3. VULNERABILITY: depth-damage curves by construction -----
# damage ratio = fraction of building value lost, as a function of depth (m)
CURVES = {
    "masonry": ([0, 0.5, 1, 2, 3, 4, 6], [0, .18, .32, .52, .68, .78, .88]),
    "reinforced_concrete": ([0, 0.5, 1, 2, 3, 4, 6], [0, .10, .20, .36, .50, .60, .72]),
    "timber": ([0, 0.5, 1, 2, 3, 4, 6], [0, .28, .48, .72, .86, .93, .98]),
    "steel": ([0, 0.5, 1, 2, 3, 4, 6], [0, .08, .16, .30, .42, .52, .64]),
}

def damage_ratio(depth, construction):
    out = np.zeros(len(depth))
    for c, (dx, dy) in CURVES.items():
        m = construction == c
        out[m] = np.interp(depth[m], dx, dy)
    return out

constr = Bpt.construction.to_numpy()
for T in LEVELS:
    dr = damage_ratio(Bpt[f"depth_{T}"].to_numpy(), constr)
    dr = np.where(Bpt.has_basement.to_numpy(), np.minimum(dr * 1.15, 1.0), dr)
    Bpt[f"loss_{T}"] = dr * Bpt.value_kvs.to_numpy()

losses = {T: Bpt[f"loss_{T}"].sum() for T in LEVELS}
print("\nLOSS BY RETURN PERIOD (thousand VS)")
print(f" {'T (yr)':>8}{'exceedance p':>15}{'total loss':>16}{'mean loss/wet bldg':>21}")
for T in sorted(LEVELS):
    wet = Bpt[f"depth_{T}"] > 0
    mean_l = Bpt.loc[wet, f"loss_{T}"].mean() if wet.any() else 0
    print(f" {T:>8}{1/T:>15.4f}{losses[T]:>16,.0f}{mean_l:>21,.1f}")

# --- 4. EXPECTED ANNUAL DAMAGE -----
Ts = np.array(sorted(LEVELS))
ps = 1.0 / Ts
Ls = np.array([losses[T] for T in Ts])
order = np.argsort(-ps) # descending probability
ps_s, Ls_s = ps[order], Ls[order]
EAD = np.trapezoid(Ls_s[:-1], ps_s[:-1]) if hasattr(np, "trapezoid") \
    else np.trapz(Ls_s[:-1], ps_s[:-1])
EAD = abs(EAD)
```

```

print(f"\nEXPECTED ANNUAL DAMAGE")
print(f"  EAD (whole basin) : {EAD:,.0f} thousand VS per year")
print(f"  total asset value : {Bpt.value_kvs.sum():,.0f} thousand VS")
print(f"  EAD as % of stock : {100*EAD/Bpt.value_kvs.sum():.3f} % per year")
print(f"  implied payback on a defence costing 10 % of the stock: "
      f"{0.10*Bpt.value_kvs.sum()/EAD:,.0f} years")

# --- 5. Aggregate to districts -----
Bd = gpd.sjoin(Bpt, districts[["district_id", "name", "geometry"]],
              predicate="within").drop_duplicates("building_id")
per_T = {T: Bd.groupby("district_id")[f"loss_{T}"].sum() for T in Ts}
ead_d = {}
for did in districts.district_id:
    L = np.array([per_T[T].get(did, 0.0) for T in Ts])
    ead_d[did] = abs(np.trapezoid(L[:-1], ps[np.argsort(Ts)][:-1])
                    if hasattr(np, "trapezoid") else np.trapz(L[:-1], ps[:-1]))
risk = districts[["district_id", "name", "district_type", "population",
                  "geometry"]].copy()
risk["ead_kvs"] = risk.district_id.map(ead_d)
risk["n_buildings"] = risk.district_id.map(Bd.groupby("district_id").size()).fillna(0)
risk["value_kvs"] = risk.district_id.map(Bd.groupby("district_id").value_kvs.sum()).fillna(0)
risk["ead_per_capita"] = risk.ead_kvs / risk.population
risk["ead_pct_value"] = 100 * risk.ead_kvs / risk.value_kvs.replace(0, np.nan)

print("\nEXPECTED ANNUAL DAMAGE BY DISTRICT (top 8)")
print(risk.nlargest(8, "ead_kvs")[["district_id", "name", "district_type",
                                   "n_buildings", "value_kvs", "ead_kvs",
                                   "ead_pct_value"]].round(1).to_string(index=False))
print(f"\n  top 3 districts hold "
      f"{100*risk.nlargest(3, 'ead_kvs').ead_kvs.sum()/risk.ead_kvs.sum():.0f} % "
      f"of the basin's expected annual damage")

# --- 6. SENSITIVITY to the damage function -----
print("\nSENSITIVITY OF EAD TO THE DAMAGE FUNCTION")
print(f"  {'scenario':<34}{'EAD':>16}{'vs baseline':>14}")
for label, mult in [("baseline curves", 1.00),
                   ("curves 25 % more damaging", 1.25),
                   ("curves 25 % less damaging", 0.75),
                   ("curves 50 % more damaging", 1.50)]:
    tot = []
    for T in Ts:
        dr = damage_ratio(Bpt[f"depth_{T}"].to_numpy(), constr) * mult
        dr = np.where(Bpt.has_basement.to_numpy(), dr * 1.15, dr) # same modifier
        tot.append(np.minimum(dr, 1.0) @ Bpt.value_kvs.to_numpy())
    tot = np.array(tot)
    e = abs(np.trapezoid(tot[:-1], ps[:-1]) if hasattr(np, "trapezoid")
           else np.trapz(tot[:-1], ps[:-1]))
    print(f"  {label:<34}{e:>16,.0f}{100*(e-EAD)/EAD:>13.1f}%")
print("\n  A 25 % change in an EMPIRICAL curve moves the headline number by 20-30 %.")
print("  The response is ASYMMETRIC because damage ratios are capped at 1.0, so")
print("  making curves more damaging saturates while making them less damaging")
print("  does not. In most flood-risk studies the vulnerability function is a")
print("  larger source of uncertainty than the hazard model everyone argues about.")

# --- 7. Figures -----
fig, axes = plt.subplots(1, 3, figsize=(17.5, 5.2))
axes[0].plot(ps_s, Ls_s, "o-", color="#b2182b", linewidth=2)
axes[0].fill_between(np.sort(ps), np.array([losses[T] for T in Ts])[np.argsort(ps)],
                    alpha=0.25, color="#b2182b")
axes[0].set_xlabel("annual exceedance probability")
axes[0].set_ylabel("loss (thousand VS)")
axes[0].set_title(f"Loss-exceedance curve\nshaded area = EAD = {EAD:,.0f} k VS/yr",
                  fontsize=9.5, weight="bold", loc="left")
axes[0].grid(alpha=0.3)

for c, (dx, dy) in CURVES.items():
    axes[1].plot(dx, dy, "o-", label=c, linewidth=1.8)

```

```

axes[1].set_xlabel("flood depth (m)"); axes[1].set_ylabel("damage ratio")
axes[1].set_title("Depth-damage (vulnerability) curves", fontsize=9.5,
                  weight="bold", loc="left")
axes[1].legend(fontsize=7); axes[1].grid(alpha=0.3)

risk.plot(ax=axes[2], column="ead_kvs", cmap="OrRd", scheme="quantiles", k=6,
          legend=True, legend_kwds={"loc": "lower left", "fontsize": 6.5,
                                     "title": "EAD (k VS/yr)"},
          edgecolor="grey", linewidth=0.4,
          missing_kwds={"color": "#eeeeee"})
axes[2].set_title("Expected annual damage by district", fontsize=9.5,
                  weight="bold", loc="left")
axes[2].set_aspect("equal"); axes[2].set_xticks([]); axes[2].set_yticks([])
plt.tight_layout(); plt.show()

```

Explanation.

- **Depth from HAND.** For each return period we assume a design water level above the channel and compute depth as `level - HAND`, floored at zero. This is a **planar/bathtub** approximation — it ignores flow, storage and defences. Real studies use a hydraulic model. State the approximation; do not hide it.
- **The damage curves** are piecewise-linear in depth, interpolated with `np.interp`. Timber is most vulnerable, steel least — the ordering matters more than the exact numbers. The basement adjustment (`×1.15`, capped at 1.0) is a simple example of a modifier; real curves have many.
- **EAD by trapezoidal integration over exceedance probability.** Note the direction: probabilities are sorted ascending for `np.trapezoid`, and we take the absolute value to be direction-agnostic. **The high-probability, low-loss end of the curve dominates the integral** — a 10-year flood contributes far more to EAD than a 500-year flood, which is counter-intuitive and is why designing only for the extreme event is poor economics.
- `np.trapezoid` replaced `np.trapz` in NumPy 2.0; the `hasattr` guard keeps the code working on both.
- **The district aggregation** re-integrates per district rather than apportioning the basin total, which is the correct order of operations: EAD is not linear in the intermediate quantities.
- **The sensitivity analysis is the most important block.** Multiplying every damage ratio by 1.25 moves EAD by roughly 25%. Damage curves are empirical, transferred between countries and building stocks, and rarely validated locally. In practice they carry more uncertainty than the hazard model — and they receive far less scrutiny.

Expected outcome.

LOSS BY RETURN PERIOD (thousand VS)

T (yr)	exceedance p	total loss	mean loss/wet bldg
10	0.1000	144,103	101.4
50	0.0200	211,127	131.7
100	0.0100	236,089	141.2
500	0.0020	321,588	164.2

EXPECTED ANNUAL DAMAGE

EAD (whole basin) : 17,987 thousand VS per year
 total asset value : 1,181,004 thousand VS
 EAD as % of stock : 1.523 % per year
 implied payback on a defence costing 10 % of the stock: 7 years

Look at where the EAD comes from. The 500-year event loses 321 588 k VS — more than twice the 10-year event's 144 103 — but it happens with probability 0.002 against 0.100. In the integral, the 10-year event contributes far more.

Designing only for the extreme event is poor economics; most avoidable loss sits in the frequent, moderate floods.

EXPECTED ANNUAL DAMAGE BY DISTRICT (top 8)

district_id	name	district_type	n_buildings	value_kvs	ead_kvs	ead_pct_value
D01	Old Vallmara	urban_core	1707	509,347.7	10,966.4	2.2
D02	Harbourgate	urban_core	1436	390,643.2	2,469.0	0.6
D09	Tarnwell	suburban	403	61,194.2	1,354.0	2.2
D06	Ardenfeld	suburban	265	25,942.8	1,037.9	4.0

top 3 districts hold 82 % of the basin's expected annual damage

Absolute and relative risk point at different places. Old Vallmara carries 61% of the basin's EAD in absolute terms — it is where the money should go. But **Ardenfeld loses 4.0% of its asset value every year**, nearly twice Old Vallmara's rate, on a stock twenty times smaller. A budget allocated on absolute EAD will never reach Ardenfeld; one allocated on relative loss will never reach the city. Both are legitimate policy targets and you must present both.

SENSITIVITY OF EAD TO THE DAMAGE FUNCTION

scenario	EAD	vs baseline
baseline curves	17,987	0.0%
curves 25 % more damaging	20,757	15.4%
curves 25 % less damaging	12,934	-28.1%
curves 50 % more damaging	23,112	28.5%

A $\pm 25\%$ change in an *empirical, transferred* curve moves the headline number by 15–28%. Note the **asymmetry**: damage ratios are capped at 1.0, so making curves more damaging saturates while making them less damaging does not. In most flood-risk studies the vulnerability function carries more uncertainty than the hazard model everyone argues about — and receives far less scrutiny.

Three panels: the loss-exceedance curve with the EAD shaded, the four vulnerability curves, and the district EAD choropleth.

A14 — Communicating spatial results

What we are going to learn. How to present spatial analysis so that it is useful and not misleading — and the specific claims you must not make.

Why it matters. Maps are unusually persuasive. A reader who would demand a confidence interval from a table will accept a choropleth without question. That asymmetry places a heavier burden of honesty on spatial work than on ordinary analysis.

The seven pitfalls, and what to do instead

#	Pitfall	Why it misleads	Remedy
1	Mapping counts instead of rates	Population maps always look like population	Map rates; show the denominator
2	Unstable rates in small units	A 2-of-50 rate is noisy; extremes are always in small units	Empirical-Bayes smoothing, or show the population
3	Classification shopping	Quantiles vs equal interval can invert the visual story	Fix the scheme <i>before</i> looking; show a histogram
4	Hiding missing data	NaN drawn as white reads as "low"	<code>missing_kwds</code> with hatching and a legend entry
5	The ecological fallacy	District-level correlation $\neq$ individual-level	State the unit of analysis; never infer to individuals
6	MAUP	Different units give different — even opposite — results	Repeat at a second scale; report both
7	Implying causation from a pattern	Co-location is not a mechanism	Report the confounder analysis (A9)

What every serious spatial deliverable must state

1. **The CRS and the units.**
2. **The unit of analysis** and why it was chosen.
3. **The date and provenance** of every layer.
4. **The uncertainty** — model uncertainty, sampling uncertainty, or both.
5. **The validation** — how you know the analysis is right.
6. **The assumptions** — Euclidean rather than network distance; bathtub rather than hydraulic flooding; transferred damage curves.

Expected outcome. A demonstration of pitfalls 1, 2, 3 and 6 on our own data, each with the honest alternative beside it.

What the next cell does: builds four side-by-side comparisons — counts vs rates, raw vs empirical-Bayes-smoothed rates, three classification schemes on the same variable, and the same analysis at two spatial scales.

```
In [49]: fig, axes = plt.subplots(2, 4, figsize=(19.5, 10))

# --- PITFALL 1: counts vs rates -----
Fx["incidents_n"] = 0
sj = gpd.sjoin(pts[["incident_id", "geometry"]], Fx[["block_id", "geometry"]],
               predicate="within")
cnt = sj.groupby("block_id").size()
Fx["incidents_n"] = Fx.block_id.map(cnt).fillna(0).astype(int)
Fx["incidents_per_1k"] = 1000 * Fx.incidents_n / Fx.population.replace(0, np.nan)

Fx.plot(ax=axes[0, 0], column="incidents_n", cmap="Reds", scheme="quantiles", k=6,
        legend=True, legend_kwds={"loc": "lower left", "fontsize": 6}, edgecolor="none")
axes[0, 0].set_title("(1a) MISLEADING: incident COUNT\nlooks exactly like a population map",
                     fontsize=9, weight="bold", loc="left", color="crimson")
Fx.plot(ax=axes[0, 1], column="incidents_per_1k", cmap="Reds", scheme="quantiles",
        k=6, legend=True, legend_kwds={"loc": "lower left", "fontsize": 6},
        edgecolor="none", missing_kwds={"color": "#dddddd", "hatch": "//"})
axes[0, 1].set_title("(1b) BETTER: incidents per 1,000 residents",
                     fontsize=9, weight="bold", loc="left", color="darkgreen")

corr_count = Fx[["incidents_n", "population"]].corr().iloc[0, 1]
corr_rate = Fx[["incidents_per_1k", "population"]].corr().iloc[0, 1]
```

```

# --- PITFALL 2: unstable rates in small populations -----
# Empirical-Bayes smoothing towards the global rate
n_i = Fx.incidents_n.to_numpy(float)
p_i = Fx.population.to_numpy(float)
global_rate = n_i.sum() / p_i.sum()
var_between = max(np.nanvar(n_i / np.maximum(p_i, 1)) - global_rate / np.nanmean(p_i), 1e-12)
shrink = var_between / (var_between + global_rate / np.maximum(p_i, 1))
Fx["rate_eb"] = 1000 * (shrink * (n_i / np.maximum(p_i, 1)) + (1 - shrink) * global_rate)

Fx.plot(ax=axes[0, 2], column="rate_eb", cmap="Reds", scheme="quantiles", k=6,
        legend=True, legend_kwds={"loc": "lower left", "fontsize": 6}, edgecolor="none")
axes[0, 2].set_title("(2) Empirical-Bayes smoothed rate\nsmall-population noise shrunk away",
                    fontsize=9, weight="bold", loc="left", color="darkgreen")

small = Fx.nsmallest(50, "population")
axes[0, 3].scatter(Fx.population, Fx.incidents_per_1k, s=8, alpha=0.5,
                  label="raw rate", color="#b2182b")
axes[0, 3].scatter(Fx.population, Fx.rate_eb, s=8, alpha=0.5,
                  label="EB-smoothed", color="#2166ac")
axes[0, 3].set_xscale("log"); axes[0, 3].set_xlabel("block population (log)")
axes[0, 3].set_ylabel("incidents per 1,000")
axes[0, 3].set_title("(2b) Rate variance explodes in small units",
                    fontsize=9, weight="bold", loc="left")
axes[0, 3].legend(fontsize=7); axes[0, 3].grid(alpha=0.3)

# --- PITFALL 3: classification shopping -----
for ax, scheme, name in [(axes[1, 0], "quantiles", "quantiles"),
                        (axes[1, 1], "equalinterval", "equal interval"),
                        (axes[1, 2], "naturalbreaks", "natural breaks")]:
    Fx.plot(ax=ax, column="total_value_kvs", cmap="viridis", scheme=scheme, k=5,
            legend=True, legend_kwds={"loc": "lower left", "fontsize": 5.5},
            edgecolor="none", missing_kwds={"color": "#dddddd"})
    ax.set_title(f"(3) Same data, scheme = {name}", fontsize=9,
                weight="bold", loc="left")

# --- PITFALL 6: MAUP - the same analysis at two scales -----
corr_block = Fx[["pct_in_flood100", "pop_density_km2"]].corr().iloc[0, 1]
dd = districts.merge(
    Fx.groupby("district_id").agg(flood=("pct_in_flood100", "mean"),
                                dens=("pop_density_km2", "mean")).reset_index(),
    on="district_id")
corr_district = dd[["flood", "dens"]].corr().iloc[0, 1]
dd.plot(ax=axes[1, 3], column="flood", cmap="Blues", scheme="quantiles", k=5,
        legend=True, legend_kwds={"loc": "lower left", "fontsize": 6}, edgecolor="grey",
        linewidth=0.4)
axes[1, 3].set_title(f"(6) Same variable at DISTRICT scale\nr changes "
                    f"{corr_block:+.2f} -> {corr_district:+.2f}",
                    fontsize=9, weight="bold", loc="left")

for ax in axes.ravel():
    if ax not in (axes[0, 3],):
        ax.set_aspect("equal"); ax.set_xticks([]); ax.set_yticks([])
plt.tight_layout(); plt.show()

print("THE NUMBERS BEHIND THE PICTURES")
print("=" * 78)
print(f"PITFALL 1 correlation(incident COUNT, population) = {corr_count:+.3f}")
print(f"correlation(incident RATE, population) = {corr_rate:+.3f}")
print(f"    -> the count map is largely a population map.")
print(f"\nPITFALL 2 raw rate: std = {Fx.incidents_per_1k.std():.2f}, "
      f"max = {Fx.incidents_per_1k.max():.1f}")
print(f"EB rate : std = {Fx.rate_eb.std():.2f}, "
      f"max = {Fx.rate_eb.max():.1f}")
print(f"blocks under 100 people: {int((Fx.population < 100).sum())} "
      f"- their raw rates are essentially noise")
print(f"\nPITFALL 3 class breaks for total_value_kvs:")
import mapclassify

```

```

v = Fx.total_value_kvs.dropna()
for scheme, cls in [("quantiles", mapclassify.Quantiles(v, k=5)),
                    ("equal interval", mapclassify.EqualInterval(v, k=5)),
                    ("natural breaks", mapclassify.NaturalBreaks(v, k=5))]:
    print(f"
          {scheme:<16} top class starts at {cls.bins[-2]:>12,.0f} "
          f"and holds {int(cls.counts[-1]):>3} blocks")
print(f"\nPITFALL 6  corr(flood exposure, pop density) at BLOCK level   = {corr_block:+.3f}")
print(f"                corr(flood exposure, pop density) at DISTRICT level = {corr_district:+.3f}")
print(f"                -> the same question, two units, two different answers.")
print("=" * 78)
print("""
A CHECKLIST FOR YOUR FINAL DELIVERABLE

[ ] CRS stated, with units, on every map
[ ] unit of analysis stated and justified
[ ] denominators shown wherever a rate is mapped
[ ] classification scheme named; histogram available on request
[ ] missing data drawn explicitly, never left white
[ ] uncertainty mapped alongside the estimate
[ ] validation described – how do you know this is right?
[ ] assumptions listed (Euclidean distance, bathtub flooding, transferred curves)
[ ] scale sensitivity checked at a second unit of analysis
[ ] no causal language unless you have a design that supports it
""")

```

Explanation.

- **Pitfall 1** is quantified by the two correlations. Incident *count* correlates strongly with population; incident *rate* does not. A count map answers "where do people live?" dressed as "where do floods happen?".
- **Empirical-Bayes smoothing** shrinks each block's rate towards the global rate, by an amount that depends on the block's population. A block of 40 people with one incident has a raw rate of 25 per 1 000 — an extreme value driven entirely by a small denominator. EB pulls it back towards the regional average; a block of 20 000 people barely moves. The scatter panel makes the mechanism visible: raw rates fan out at small populations, EB rates do not.
- **Pitfall 3** shows the same variable under three schemes. Because asset value is heavily right-skewed, **equal interval** puts almost every block in the bottom class and highlights only the extreme core, while **quantiles** spreads colour evenly and makes the whole region look differentiated. Both are "correct"; they tell different stories. Choose before you look, and say which you chose.
- **Pitfall 6 (MAUP)** is quantified directly: the correlation between flood exposure and population density changes when you move from blocks to districts. Aggregation averages away within-district variation and usually *strengthens* correlations — the classic ecological-correlation inflation. If your headline finding is a correlation, **report it at two scales**.
- The final checklist is the practical output of this lesson. Attach it to your own work.

Expected outcome.

PITFALL 1	correlation(incident COUNT, population)	= +0.526
	correlation(incident RATE, population)	= -0.059
PITFALL 2	raw rate: std = 4.97, max = 44.4	
	EB rate : std = 3.61, max = 28.9	
	blocks under 100 people: 208 – their raw rates are essentially noise	
PITFALL 3	class breaks for total_value_kvs:	
	quantiles top class starts at	1,731 and holds 65 blocks
	equal interval top class starts at	65,730 and holds 4 blocks
	natural breaks top class starts at	57,192 and holds 4 blocks

Pitfall 1: the count map correlates +0.53 with population; the rate map correlates −0.06. The count map is, to a first approximation, a population map with a red colour ramp.

Pitfall 2: 208 of 459 blocks have fewer than 100 residents. A single incident in a block of 40 people gives a rate of 25 per 1 000 — the highest in the region, and pure noise. EB smoothing cuts the maximum from 44.4 to 28.9 and the standard deviation from 4.97 to 3.61, almost all of it from those small blocks.

Pitfall 3 is the starkest. The same variable, five classes: **quantiles puts 65 blocks in the top class; equal interval and natural breaks put 4.** The quantile map will look like widespread high value across the basin; the equal-interval map will look like a single hot spot in the city. Both are faithful renderings of the same numbers. **Choose the scheme before you look at the map, and name it in the caption.**

Pitfall 6: the correlation between flood exposure and population density changes when you move from blocks to districts. Both are true statements about different objects — and only one of them is the answer to the question you were asked.

Exercises — Module 3 (Advanced)

Exercise 3.1 — A defensible siting recommendation

Objective. The regional government will fund **two** new clinics. Using MCDA: 1. Define at least five criteria, including at least one equity criterion, and justify each. 2. Elicit weights by AHP and report the consistency ratio. 3. Aggregate by both weighted sum and geometric mean; explain your choice. 4. Run a Monte-Carlo sensitivity analysis on the weights **and** on the normalisation method. 5. Recommend two blocks, and state the conditions under which your recommendation would change.

Exercise 3.2 — Accessibility with a network-distance correction

Objective. Our E2SFCA used Euclidean distance. Estimate a **detour index** (network distance ÷ straight-line distance) for the Vallmara Basin by sampling 50 origin–destination pairs and measuring the shortest path along the road network (build a graph from the road segments; **networkx** or your own Dijkstra). Re-run the clinic E2SFCA with corrected distances. How much does the Gini change? Which districts change rank most?

Exercise 3.3 — A hotspot analysis you can defend

Objective. Run Getis-Ord Gi on *building asset value at risk** (**`total_value_kvs × pct_in_flood100`**) and produce a map that would survive peer review. It must include: an explicit weights specification, a permutation budget justified against the FDR requirement, an FDR-corrected significance map, and a sensitivity analysis across at least three weights schemes. Report how many blocks are significant under each.

Exercise 3.4 — Regionalisation for service delivery

Objective. The health authority wants to divide the basin into **6 contiguous service regions** that are as equal as possible in population while remaining internally homogeneous in accessibility. Design and implement this. Report the population of each region, the coefficient of variation across regions, and the homogeneity cost relative to unconstrained clustering. Discuss the trade-off.

Exercise 3.5 — A better flood model

Objective. Improve on the Module 3 flood-susceptibility model: 1. Add at least three new physically motivated features (curvature, upslope contributing area, distance to the coast, land-cover composition in a neighbourhood, ...). 2. Use a spatial-block CV with block size justified by a variogram of the residuals. 3. Compare at least three algorithms and calibrate the best one (`CalibratedClassifierCV`). 4. Report AUC, average precision, Brier score and a calibration curve — all spatially validated. 5. Produce a susceptibility map with an uncertainty layer and an extrapolation mask.

Exercise 3.6 — Risk under a climate scenario

Objective. Assume a climate scenario in which all design flood levels rise by 0.5 m and the 100-year event becomes the 50-year event. Recompute EAD. Report: the change in EAD, which districts change most in absolute and in relative terms, how many additional buildings become exposed, and the break-even cost of a defence that would restore the current EAD.

Challenge 3.7 — Recover the full generating process

Objective. Using only the delivered data files, estimate **every** coefficient in the documented generating process:

Relationship	True form
Rainfall	$470 + 0.62 \cdot \text{elevation} + 150 \cdot (\text{north-south position})$
Land surface temperature	$31.5 - 0.0062 \cdot \text{elevation} + 6.4 \cdot \text{urban}$
PM2.5	$7.5 + 16 \cdot \text{urban} + 9 \cdot \exp(-d_{\text{motorway}}/1800)$
Population density	$9500 \cdot \text{urban}^{2.1} + 22$

For each, report your estimate, a confidence interval, and the true value. Then answer: **which estimate is worst, and why?** Diagnose the cause (aliasing, confounding, aggregation bias, measurement error, or insufficient sample) and propose a fix.

Module 4 — Capstone Project

The Vallmara Basin Integrated Climate-Resilience Assessment

The brief

From: Office of the Regional Director, Vallmara Basin Authority **To:** Spatial Data Science Unit **Re:** Integrated climate-resilience assessment — decision support for the 2026–2031 capital programme

The Authority has a capital budget of **120 million VS** over five years. It must be split between flood defence, health-service expansion and building retrofit. Council members disagree about where the money should go, and every district claims to be the worst affected.

We need an evidence base. Specifically:

1. A **Composite Resilience Index** for every district, combining physical hazard, service accessibility and socio-economic vulnerability.

2. A **prioritised intervention list** — which districts, which intervention, what it would cost and what it would avert.

3. An honest statement of **what your analysis cannot tell us**.

The report will be scrutinised by people who want a different answer. Every number must be traceable and every assumption stated.

What you must produce

Stage	Requirement
1. Load and inspect	Every layer loaded, inventoried, and its provenance recorded
2. Clean the attributes	Sentinels, impossibles, duplicates, inconsistent categories — with an audit trail
3. Validate geometries	Invalid, empty and null geometries found and repaired; area reconciliation before/after
4. Resolve CRS	One analysis CRS, justified by measurement, with the error of the alternatives quantified
5. Vector analysis	Exposure of population, buildings and assets to each hazard zone, without double counting
6. Raster analysis	Terrain derivatives, a HAND surface, and zonal statistics validated against a known quantity
7. Raster–vector integration	An aligned multi-layer stack, and every raster variable attached to every district
8. Feature engineering	At least 20 features across proximity, density, composition, focal and lag families
9. Statistical / spatial analysis	Autocorrelation testing, hotspot detection with multiple-testing control, and a spatially validated predictive model
10. Visualisation	A publication-quality map series with explicit classification, missing data and uncertainty
11. Interpretation	What the numbers mean, for whom, with what confidence
12. Conclusion	A ranked intervention list with costed recommendations and stated limitations

Assessment rubric

Mark yourself honestly. You have demonstrated practical proficiency if you can answer **yes** to all of these:

- ☐ Every measurement is in a CRS whose units make it meaningful, and I can say why.
- ☐ No spatial join in my pipeline silently changed a row count.
- ☐ Every absolute quantity was recomputed after any geometry-splitting operation.
- ☐ Missing data is NaN, is visible on my maps, and is counted in my report.
- ☐ I tested residual spatial autocorrelation and acted on the result.
- ☐ My predictive model was validated with spatially blocked folds, and I report the honest score, not the flattering one.
- ☐ I ran a sensitivity analysis on every subjective weight and threshold.
- ☐ I have at least one **external** validation — a check against something the analysis never used.
- ☐ I can state, in one sentence each, the three assumptions most likely to be wrong.
- ☐ Someone else could reproduce my numbers from my code and the raw data.

How to use what follows

Attempt the capstone yourself first. The reference implementation below is one defensible solution, not the only one. Where it makes a judgement call it says so, and where a different call would be equally defensible it says that too.

The reference implementation runs in about a minute and reuses the helper functions built in Modules 2 and 3 (`zonal_stats`, `align_to`, `moran_test`, `getis_ord_gstar`, `benjamini_hochberg`, `knn_weights`, `queen_weights`, `weighted_gini`, `burn`, `sample_grid`).

Capstone Stage 1–4 — Ingest, clean, validate, resolve CRS

What we are about to do. Rebuild the entire data pipeline from the raw files in one auditable block, so the capstone stands alone and every number in it is traceable to a documented step.

Why it matters. The single most common reason an analysis cannot be defended is that nobody can say exactly what was done to the data. A pipeline that logs every decision is worth more than a cleverer model.

Concept — provenance. For each layer, record: source file, original CRS, row count in, row count out, and every transformation applied. This block is the skeleton of the "Data and Methods" section of your report.

What the next cell does: loads every layer from disk, records provenance, applies the cleaning rules from I3, repairs geometry as in I2, reprojects everything to the analysis CRS, and prints a complete audit trail with an area reconciliation.

```
In [50]: # =====
# CAPSTONE STAGES 1-4: INGEST -> CLEAN -> VALIDATE -> CRS
# =====
PROV = []          # provenance log

def logp(layer, source, crs_in, n_in, n_out, notes):
    PROV.append(dict(layer=layer, source=source, crs_in=crs_in,
                     rows_in=n_in, rows_out=n_out, notes=notes))

CAP = {}           # the clean, canonical layer set

# ---- STAGE 1: LOAD ----
raw_specs = [
    ("districts",    GPKG, "districts"), ("blocks",    GPKG, "census_blocks"),
    ("landuse",      GPKG, "landuse"),   ("rivers",    GPKG, "rivers"),
    ("flood",        GPKG, "flood_zones"), ("buildings", GPKG, "buildings"),
    ("facilities",   GPKG, "facilities"), ("stops",     GPKG, "bus_stops"),
    ("routes",       GPKG, "transit_routes"),
    ("land",         GPKG, "land_boundary"), ("sea",     GPKG, "sea"),
    ("coastline",    GPKG, "coastline"),
    ("roads",        VEC / "roads.geojson", None),
    ("protected",    VEC / "protected_areas.geojson", None),
]
for name, src, layer in raw_specs:
    g = gpd.read_file(src, layer=layer) if layer else gpd.read_file(src)
    crs_in = g.crs.to_string()
    n_in = len(g)
    # ---- STAGE 4: single analysis CRS, applied at the door ----
    if g.crs.to_string() != CRS_UTM:
        g = g.to_crs(CRS_UTM)
    CAP[name] = g
    logp(name, str(src.name if hasattr(src, "name") else src), crs_in,
         n_in, len(g), "reprojected" if crs_in != CRS_UTM else "as-is")

# ---- STAGE 3: VALIDATE AND REPAIR GEOMETRY ----
print("STAGE 3 - GEOMETRY VALIDATION")
print(f"  {'layer':<12}{ 'null':>6}{ 'empty':>7}{ 'invalid':>9}{ 'dup geom':>10}"
      f"{'area/len before':>18}{ 'after':>14}")
print("-" * 78)
for name, g in list(CAP.items()):
```

```

n_null = int(g.geometry.isna().sum())
n_empty = int(g.geometry.is_empty.sum())
n_bad = int((~g.geometry.is_valid).sum())
n_dup = int(g.geometry.to_wkb().duplicated().sum())
is_poly = g.geom_type.iloc[0] in ("Polygon", "MultiPolygon")
before = g.geometry.area.sum() / 1e6 if is_poly else g.geometry.length.sum() / 1000

if n_bad:
    g = g.copy()
    g["geometry"] = g.geometry.make_valid()
    g = g.explode(index_parts=False, ignore_index=True)
    keep = ["Polygon", "MultiPolygon"] if is_poly else ["LineString", "MultiLineString"]
    g = g[g.geom_type.isin(keep)]
    g = g[~g.geometry.isna() & ~g.geometry.is_empty].reset_index(drop=True)
    after = g.geometry.area.sum() / 1e6 if is_poly else g.geometry.length.sum() / 1000
    CAP[name] = g
    if n_null or n_empty or n_bad or n_dup:
        print(f"  {name:<12}{n_null:>6}{n_empty:>7}{n_bad:>9}{n_dup:>10}"
              f"{before:>18,.3f}{after:>14,.3f}")
print("  (layers with no problems are omitted)")

# ---- STAGE 2: CLEAN ATTRIBUTES -----
print("\nSTAGE 2 - ATTRIBUTE CLEANING")
SENTINELS = [-999, -9999]

# facilities: sentinel capacity + duplicate features
f = CAP["facilities"]
n0 = len(f)
f = f.assign(_w=f.geometry.to_wkb()).drop_duplicates(["facility_id", "_w"]).drop(columns="_w")
n_sent = int(f.capacity.isin(SENTINELS).sum())
f.loc[f.capacity.isin(SENTINELS), "capacity"] = np.nan
CAP["facilities"] = f.reset_index(drop=True)
print(f"  facilities : {n0 - len(f)} duplicate features dropped, "
      f"{n_sent} sentinel capacities -> NaN")

# buildings: impossible construction years
b = CAP["buildings"]
bad_yr = (b.year_built < 1800) | (b.year_built > 2025)
b.loc[bad_yr, "year_built"] = np.nan
b["building_age"] = 2025 - b.year_built
b["value_kvs_filled"] = b.value_kvs.fillna(b.value_kvs.median())
CAP["buildings"] = b
print(f"  buildings : {int(bad_yr.sum())} impossible years -> NaN, "
      f"{int(b.value_kvs.isna().sum())} missing values median-filled (flagged)")

# landuse: controlled vocabulary
lu = CAP["landuse"]
CONTROLLED = {c.lower(): c for c in lc_legend.landuse_class}
n_lv = lu.landuse_class.nunique()
lu["landuse_class"] = (lu.landuse_class.str.strip().str.lower()
                      .str.replace(r"\s+", " ", regex=True).map(CONTROLLED))
CAP["landuse"] = lu
print(f"  landuse   : {n_lv} raw labels -> {lu.landuse_class.nunique()} controlled, "
      f"{int(lu.landuse_class.isna().sum())} unmapped")

# socio-economic: duplicate + orphan keys
so = pd.read_csv(TAB / "district_socioeconomic.csv")
n0 = len(so)
so = so.drop_duplicates("district_id", keep="first")
valid_ids = set(CAP["districts"].district_id)
orphans = sorted(set(so.district_id) - valid_ids)
so = so[so.district_id.isin(valid_ids)].reset_index(drop=True)
CAP_SOCIO = so
print(f"  socio      : {n0} -> {len(so)} rows "
      f"(orphan keys removed: {orphans})")

# incidents: coordinate quarantine + recovery
ir = pd.read_csv(TAB / "flood_incidents.csv", parse_dates=["date"])

```

```

BOX = dict(lon=(13.6, 14.5), lat=(41.4, 42.0))
inbox = lambda lo, la: lo.between(*BOX["lon"]) & la.between(*BOX["lat"])
ir["reject"] = pd.NA
ir.loc[(ir.lon == 0) & (ir.lat == 0), "reject"] = "null_island"
sw = (~inbox(ir.lon, ir.lat)) & inbox(ir.lat, ir.lon) & ir.reject.isna()
ir.loc[sw, "reject"] = "swapped"
ir.loc[(~inbox(ir.lon, ir.lat)) & ir.reject.isna(), "reject"] = "outside"
rec = ir[ir.reject == "swapped"].copy()
rec[["lon", "lat"]] = rec[["lat", "lon"]].to_numpy()
clean = pd.concat([ir[ir.reject.isna()], rec], ignore_index=True).drop(columns="reject")
clean.loc[clean.damage_kvs < 0, "damage_kvs"] = np.nan
CAP["incidents"] = gpd.GeoDataFrame(
    clean, geometry=gpd.points_from_xy(clean.lon, clean.lat),
    crs=CRS_WGS84).to_crs(CRS_UTM)
print(f" incidents : {len(ir)} raw -> {len(CAP['incidents'])} clean "
      f"({int(sw.sum())} recovered from swapped coordinates, "
      f"{int((ir.reject.notna() & (ir.reject != 'swapped')).sum())} quarantined)")

# ---- STAGE 4: CRS justification -----
from pyproj import Geod
geod = Geod(ellps="WGS84")
poly_ll = CAP["districts"].to_crs(CRS_WGS84).geometry.union_all()
true_km2 = abs(geod.geometry_area_perimeter(poly_ll)[0]) / 1e6
print(f"\nSTAGE 4 - CRS JUSTIFICATION")
print(f" geodesic ground truth for the study area : {true_km2:,.1f} km^2")
print(f" {'candidate CRS':<32}{'area km2':>12}{'error':>10}")
for label, code_ in [("EPSG:32633 UTM 33N (CHOSEN)", CRS_UTM),
                    ("EPSG:3857 Web Mercator", CRS_WEBMERC),
                    ("ESRI:54009 Mollweide", "ESRI:54009")]:
    a = CAP["districts"].to_crs(code_).geometry.area.sum() / 1e6
    print(f" {label:<32}{a:>12,.1f}{100*(a-true_km2)/true_km2:>9.2f}%")

print("\nPROVENANCE LOG")
print(pd.DataFrame(PROV).to_string(index=False))

```

Explanation.

- **The provenance log** is the deliverable, not a debugging aid. Every row records where a layer came from, what CRS it arrived in, and how many rows survived. When a council member asks "why does your building count differ from ours?", this table is the answer.
- **Reprojection at the door.** The `if g.crs != CRS_UTM: g = g.to_crs(...)` line runs before anything else touches the data, so no downstream code has to think about CRS. That is the discipline; the alternative is remembering, every time.
- **Geometry repair is layer-agnostic.** The loop detects the geometry family from the first row and keeps only compatible types after `make_valid`, so it works on polygons and lines alike. It also prints the area/length **before and after**, which is the reconciliation an auditor will ask for.
- **value_kvs_filled rather than overwriting value_kvs.** Keeping the imputed column separate means any downstream result can be recomputed with and without imputation. Overwriting the original destroys that option permanently.
- **Recovering the swapped coordinates rather than dropping them** preserves six genuine observations. The rule is relational — invalid as (lon, lat), valid reversed — not a bounds check.
- **The CRS justification block** turns "we used UTM 33N" into "we used UTM 33N and here is the measured error of the alternatives". That is what makes it a justification rather than an assertion.

Expected outcome.

A geometry-validation table listing only the problem layers:

layer	null	empty	invalid	dup geom	area/len before	after
blocks	0	1	0	0	1,388.848	1,388.848
landuse	0	0	3	2	1,408.500	1,409.715
facilities	0	0	0	27	0.000	0.000

The land-use area **increases** by 1.215 km² after repair — the bow-ties were reporting zero area.

The facilities row is worth pausing on. Twenty-seven facilities share an *exact* location with another facility. Only three of those are true duplicate records (same `facility_id`); the rest are genuinely co-located different services — a school and a clinic on the same civic site. A blanket "drop duplicate geometries" rule would have deleted 27 real facilities. **De-duplicate on the logical key *and* the geometry, never on geometry alone.**

An attribute-cleaning block reporting 3 duplicate facilities, 11 sentinel capacities, 18 impossible building years, 25 raw land-use labels collapsing to 8, the orphan key ['D99'] removed, and 386 clean incidents from 393 raw with **6 recovered**.

A CRS table showing UTM 33N within ~0.1% of the geodesic truth and Web Mercator about **+80%** wrong, and finally a 14-row provenance log.

Capstone Stage 5–8 — Vector analysis, raster analysis, integration, features

What we are about to do. Build the district-level analysis base table: exposure from vector overlay, terrain and hydrology from rasters, and 25+ engineered features.

Why it matters. Stages 5–8 are where most of the analytical value is created and where most of the errors are made. Every double-count, every un-recomputed area, every CRS slip lands here.

Concept — why districts, not blocks. The client allocates budget by district, so the *decision unit* is the district. We compute at block level where the data supports it and aggregate up, because aggregating fine estimates is better than computing coarse ones — but we report at the decision unit. We check the MAUP sensitivity in Stage 9.

What the next cell does: computes hazard exposure by areal and dasymetric weighting, derives slope/TPI/HAND, runs zonal statistics for every raster, attaches accessibility and asset features, and validates the result.

```
In [51]: # =====
# CAPSTONE STAGES 5–8
# =====
from scipy.ndimage import distance_transform_edt, uniform_filter
from rasterio.warp import Resampling

D = CAP["districts"].copy()
BL = CAP["blocks"].copy()
BLD = CAP["buildings"].copy()
BLDpt = BLD.copy(); BLDpt["geometry"] = BLD.geometry.representative_point()

# ---- STAGE 5: VECTOR ANALYSIS - hazard exposure ----
print("STAGE 5 - VECTOR EXPOSURE ANALYSIS")
for rp in (100, 500):
    z = CAP["flood"][CAP["flood"].return_period_yr == rp].geometry.union_all()
    # (a) areal fraction of each district in the zone
    D[f"frac_flood{rp}"] = (D.geometry.intersection(z).area / D.geometry.area).to_numpy()
    # (b) buildings and value exposed - de-duplicated, no double counting
    hit = BLDpt[BLDpt.geometry.within(z)]
    per_d = gpd.sjoin(hit, D[["district_id", "geometry"]],
                      predicate="within").drop_duplicates("building_id")
    D[f"bld_in_flood{rp}"] = D.district_id.map(
        per_d.groupby("district_id").size()).fillna(0).astype(int)
    D[f"value_in_flood{rp}"] = D.district_id.map(
        per_d.groupby("district_id").value_kvs_filled.sum()).fillna(0.0)
    # (c) population exposed - dasymetric, weighted by residential floorspace
```

```

BLDpt["_ls"] = BLDpt.footprint_m2 * BLDpt.floors
bb = gpd.sjoin(BLDpt, BL[["block_id", "geometry"]], predicate="within")
tot_ls = bb.groupby("block_id")["_ls"].sum()
in_ls = bb[bb.geometry.within(z)].groupby("block_id")["_ls"].sum()
w = (in_ls / tot_ls).reindex(BL.block_id).fillna(0).clip(0, 1).to_numpy()
BL[f"pop_flood{rp}"] = BL.population.to_numpy() * w
D[f"pop_in_flood{rp}"] = D.district_id.map(
    BL.groupby("district_id")[f"pop_flood{rp}"].sum()).fillna(0.0)
print(f" {rp}-yr zone: {D[f'bld_in_flood{rp}'].sum():,} buildings, "
      f"{D[f'value_in_flood{rp}'].sum():,.0f} k VS, "
      f"{D[f'pop_in_flood{rp}'].sum():,.0f} people")
print(f" CHECK no double counting: buildings in 100-yr zone counted once = "
      f"{D.bld_in_flood100.sum() == BLDpt.geometry.within(CAP['flood']
[CAP['flood'].return_period_yr==100].geometry.union_all()).sum()}")

# ---- STAGE 6: RASTER ANALYSIS -----
print("\nSTAGE 6 - RASTER ANALYSIS")
with rasterio.open(RAS / "dem_25m.tif") as src:
    dem_a = src.read(1, masked=True).astype("float64").filled(np.nan)
    p25 = src.profile.copy(); C25 = src.res[0]
gy_, gx_ = np.gradient(dem_a, C25, C25)
slope_a = np.degrees(np.arctan(np.hypot(gx_, gy_)))

winc = int(1000 / C25)
fill = np.where(np.isfinite(dem_a), dem_a, 0.0)
cnts = uniform_filter(np.isfinite(dem_a).astype(float), size=winc, mode="nearest")
tpi_a = np.where(np.isfinite(dem_a),
    dem_a - uniform_filter(fill, size=winc, mode="nearest")
    / np.maximum(cnts, 1e-9), np.nan)

riv_mask25 = rasterize([(g, 1) for g in CAP["rivers"].geometry],
    out_shape=dem_a.shape, transform=p25["transform"],
    fill=0, dtype="uint8").astype(bool)
_, (rri, rci) = distance_transform_edt(~riv_mask25, return_indices=True)
e0 = np.nan_to_num(dem_a, nan=0.0)
hand_a = np.where(np.isfinite(dem_a), e0 - e0[rri, rci], np.nan)
driv_a = distance_transform_edt(~riv_mask25, sampling=C25)

CAPR = OUT / "capstone"; CAPR.mkdir(exist_ok=True)
for nm, arr in [("slope", slope_a), ("tpi", tpi_a), ("hand", hand_a), ("driver", driv_a)]:
    pr = p25.copy(); pr.update(dtype="float32", nodata=-9999.0, compress="deflate")
    with rasterio.open(CAPR / f"{nm}_25m.tif", "w", **pr) as dst:
        dst.write(np.nan_to_num(arr, nan=-9999.0).astype("float32"), 1)
print(f" derived rasters written: slope, TPI, HAND, distance-to-river (25 m)")

# validation against a known quantity
zval = zonal_stats(D, RAS / "dem_25m.tif", stats=("mean",))["mean"].to_numpy()
print(f" VALIDATION zonal mean elevation vs the layer's own column: "
      f"max |err| = {np.nanmax(np.abs(zval - D.mean_elev_m.to_numpy())):.4f} m")

# ---- STAGE 7: RASTER-VECTOR INTEGRATION -----
print("\nSTAGE 7 - RASTER-VECTOR INTEGRATION")
RASTERS = {
    "elev": RAS / "dem_25m.tif", "rain": RAS / "rainfall_annual_250m.tif",
    "ndvi": RAS / "ndvi_50m.tif", "popdens": RAS / "popdens_100m.tif",
    "slope": CAPR / "slope_25m.tif", "tpi": CAPR / "tpi_25m.tif",
    "hand": CAPR / "hand_25m.tif", "driver": CAPR / "driver_25m.tif",
}
for nm, path in RASTERS.items():
    zs = zonal_stats(D, path, stats=("mean", "min", "max"))
    D[f"{nm}_mean"] = zs["mean"].to_numpy()
    if nm in ("elev", "hand"):
        D[f"{nm}_min"] = zs["min"].to_numpy()

# LST needs reprojection first
lst_arr = align_to(RAS / "lst_summer_100m_3857.tif", ref, Resampling.bilinear)
lp = ref.copy(); lp.update(dtype="float32", nodata=-9999.0)
with rasterio.open(CAPR / "lst_utm.tif", "w", **lp) as dst:

```

```

dst.write(np.nan_to_num(lst_arr, nan=-9999.0).astype("float32"), 1)
D["lst_mean"] = zonal_stats(D, CAPR / "lst_utm.tif", stats=("mean",))["mean"].to_numpy()

lcz = zonal_stats(D, RAS / "landcover_25m.tif", stats=("count",), categorical=True)
for code_, nm in zip(lc_legend.class_code, lc_legend.landuse_class):
    col = f"pct_class_{code_}"
    if col in lcz.columns:
        D["lc_" + nm.lower().split()[0].replace("-", "")] = lcz[col].to_numpy()
print(f" {len(RASTERS)+1} rasters + land-cover composition attached to {len(D)} districts")

# ---- STAGE 8: FEATURE ENGINEERING -----
print("\nSTAGE 8 - FEATURE ENGINEERING")
Dc = D.geometry.representative_point()
# NOTE: do NOT call these columns "cx"/"cy". `gdf.cx` is GeoPandas' coordinate
# INDEXER, so `D.cx` would return the indexer object, not your column - a
# genuinely baffling bug the first time you meet it.
D["ctr_x"], D["ctr_y"] = Dc.x.to_numpy(), Dc.y.to_numpy()

# proximity
for nm, tgt in [("hospital", CAP["facilities"][CAP["facilities"].facility_type == "hospital"]),
                ("clinic", CAP["facilities"][CAP["facilities"].facility_type == "clinic"]),
                ("fire", CAP["facilities"][CAP["facilities"].facility_type ==
"fire_station"])),
                ("primary_road", CAP["roads"][CAP["roads"].road_class.isin(["motorway",
"primary"])])):
    g = tgt.geometry.union_all()
    D[f"dist_{nm}_m"] = Dc.distance(g).to_numpy()

# density
bxy = np.c_[BLDpt.geometry.x, BLDpt.geometry.y]
tree_b = cKDTree(bxy)
D["bld_within_3km"] = [len(i) for i in tree_b.query_ball_point(np.c_[D.ctr_x, D.ctr_y], 3000)]
sxy = np.c_[CAP["stops"].geometry.x, CAP["stops"].geometry.y]
D["stops_within_3km"] = [len(i) for i in cKDTree(sxy).query_ball_point(np.c_[D.ctr_x,
D.ctr_y], 3000)]

# assets
bd = gpd.sjoin(BLDpt, D[["district_id", "geometry"]],
               predicate="within").drop_duplicates("building_id")
agg = bd.groupby("district_id").agg(
    n_buildings=("building_id", "size"),
    total_value=("value_kvs_filled", "sum"),
    mean_age=("building_age", "mean"),
    pct_timber=("construction", lambda s: 100 * (s == "timber").mean()),
    pct_basement=("has_basement", lambda s: 100 * s.mean()))
for c in agg.columns:
    D[c] = D.district_id.map(agg[c]).to_numpy()

# socio-economic join, checked
before = len(D)
D = D.merge(CAP_SOCIO.drop(columns=[c for c in ["name", "district_type",
"population", "households"]
if c in CAP_SOCIO.columns])),
           on="district_id", how="left", validate="one_to_one")
assert len(D) == before, "socio join changed the row count"

# spatial lag over district contiguity
Wd_ = queen_weights(D)
def dlag(v):
    v = np.asarray(v, float); ok = np.isfinite(v)
    Wm = Wd_ * ok[None, :]; den = Wm.sum(axis=1)
    return np.where(den > 0, (Wm @ np.nan_to_num(v)) / np.maximum(den, 1e-12), np.nan)
for c in ["frac_flood100", "median_income_vs", "dist_hospital_m"]:
    D[f"lag_{c}"] = dlag(D[c])

# ratios
D["value_per_capita"] = D.total_value / D.population
D["ead_exposure_ratio"] = D.value_in_flood100 / D.total_value.replace(0, np.nan)

```

```

D["pop_exposed_pct"] = 100 * D.pop_in_flood100 / D.population

FEATCOLS = [c for c in D.columns if c not in
             ("district_id", "name", "district_type", "geometry", "cx", "cy",
              "survey_year")]
print(f" district table: {len(D)} rows x {len(FEATCOLS)} numeric features")
print(f" feature families: proximity {sum(c.startswith('dist_') for c in FEATCOLS)}, "
      f"density {sum('within' in c for c in FEATCOLS)}, "
      f"composition {sum(c.startswith('lc_') for c in FEATCOLS)}, "
      f"raster {sum(c.endswith('_mean') for c in FEATCOLS)}, "
      f"lag {sum(c.startswith('lag_') for c in FEATCOLS)}")
print(f" missing values remaining: "
      f"{D[FEATCOLS].isna().sum().sum()} across {int((D[FEATCOLS].isna().sum() > 0).sum())} "
      f"columns")
D.to_file(OUT / "capstone_districts.gpkg", layer="districts", driver="GPKG")
print(f" saved -> capstone_districts.gpkg")

```

Explanation.

- **Stage 5 computes exposure three ways on purpose:** areal fraction (crude but transparent), building/value counts (exact, de-duplicated), and dasymetric population (best estimate). Reporting all three lets the reader see how much the method matters.
- The **CHECK no double counting** line compares the summed per-district count against the global count. If a building near a district boundary were counted twice, the two would differ. Assertions like this are cheap and catch the errors that silently inflate headline figures.
- **Stage 6 writes its derived rasters to disk.** Slope, TPI, HAND and distance-to-river become inspectable artefacts, and **zonal_stats** can consume them by path. The alternative — keeping everything in memory — makes the pipeline unauditable.
- **The zonal validation** against the layer's own **mean_elev_m** proves the rasterisation, transform and NoData handling are all correct in one number.
- **Stage 7 reprojects the Web Mercator LST raster before zonal statistics.** Running zonal statistics across a CRS mismatch produces plausible numbers that are wrong.
- **validate="one_to_one"** on the socio-economic merge makes pandas raise if the join is not one-to-one. Combined with the **assert**, a duplicated key can no longer silently multiply rows. **Use validate= on every merge you care about.**
- **The feature-family count** at the end is a completeness check against the brief's requirement of at least 20 features across five families.

Expected outcome.

Stage 5 should report on the order of **1 000 buildings and 250 000 k VS** in the 100-year zone, with a slightly larger figure for the 500-year zone, and the double-counting check returning **True**.

Stage 6 should confirm the zonal validation to within **0.05 m**.

Stage 8 should produce a district table of roughly **50–60 numeric features** with a handful of missing values (the two blanked district populations, and any district with no buildings), and save it to **capstone_districts.gpkg**.

Capstone Stage 9 — Statistical and spatial analysis

What we are about to do. Test for spatial structure, find statistically defensible hotspots, build the Composite Resilience Index, and check that the answer survives a change of spatial unit.

Why it matters. This is where the analysis becomes evidence. An index without a significance test and a scale check is an opinion with decimal places.

Concept — the Composite Resilience Index. We combine three domains:

$$\text{CRI} = w_h \cdot \text{Hazard} + w_a \cdot (1 - \text{Accessibility}) + w_v \cdot \text{Vulnerability}$$

each domain being a normalised aggregate of several indicators, all oriented so that **higher = worse**. The weights are the political content; the sensitivity analysis is what makes them arguable rather than imposed.

What the next cell does: builds the three domain scores, tests each for spatial autocorrelation, runs an FDR-corrected Gi* hotspot analysis on the CRI, performs a Monte-Carlo weight sensitivity, and repeats the whole index at block level to test MAUP sensitivity.

```
In [52]: # =====
# CAPSTONE STAGE 9: STATISTICAL AND SPATIAL ANALYSIS
# =====
Dx = D.copy()

def norm01(v, higher_is_worse=True):
    v = pd.Series(v).astype(float)
    r = v.rank(pct=True, na_option="keep")
    return r if higher_is_worse else 1 - r

# ---- three domain scores ----
HAZ = {"pop_exposed_pct": True, "ead_exposure_ratio": True,
       "hand_mean": False, "frac_flood100": True}
ACC = {"dist_hospital_m": True, "dist_clinic_m": True, "dist_fire_m": True,
       "stops_within_3km": False}
VUL = {"median_income_vs": False, "unemployment_rate": True,
       "pct_over65": True, "mean_age": True, "pct_timber": True,
       "vehicles_per_household": False}

for name, spec in [("hazard", HAZ), ("access", ACC), ("vuln", VUL)]:
    parts = [norm01(Dx[c], hw) for c, hw in spec.items() if c in Dx.columns]
    Dx[f"score_{name}"] = np.nanmean(np.c_[tuple(parts)], axis=1)
    print(f" {name:<8} domain from {len(parts)} indicators: "
          f"range {Dx[f'score_{name}'].min():.3f} - {Dx[f'score_{name}'].max():.3f}")

W_CRI = dict(hazard=0.40, access=0.35, vuln=0.25)
Dx["CRI"] = sum(W_CRI[k] * Dx[f"score_{k}"] for k in W_CRI)
Dx["CRI_rank"] = Dx.CRI.rank(ascending=False).astype(int)

print("\nCOMPOSITE RESILIENCE INDEX (higher = more at risk, less resilient)")
print(Dx.nlargest(8, "CRI")[["district_id", "name", "district_type", "population",
                             "score_hazard", "score_access", "score_vuln", "CRI"]]
      .round(3).to_string(index=False))

# ---- spatial autocorrelation of each domain ----
print("\nSPATIAL AUTOCORRELATION OF THE DOMAIN SCORES (queen weights, 9999 perms)")
print(f" {'score':<16}{'Moran I':>10}{'p':>9}")
for c in ["score_hazard", "score_access", "score_vuln", "CRI"]:
    r = moran_test(Dx[c], Wd_, permutations=9999, seed=3)
    print(f" {c:<16}{r['I']:>10.4f}{r['p_sim']:>9.4f}")

# ---- FDR-corrected hotspots of the CRI ----
gi = getis_ord_gstar(Dx.CRI.to_numpy(dtype=float), Wd_)
from scipy.stats import norm as _norm
p_gi = 2 * (1 - _norm.cdf(np.abs(gi)))
sig = benjamini_hochberg(p_gi, 0.05)
Dx["gi_z"] = gi
Dx["cri_hotspot"] = np.where(sig & (gi > 0), "hot",
                             np.where(sig & (gi < 0), "cold", "not significant"))
print(f"\nCRI HOTSPOTS (Getis-Ord Gi*, FDR-corrected)")
print(f" uncorrected significant : {int((p_gi < 0.05).sum())} of {len(Dx)}")
print(f" FDR-corrected           : {int(sig.sum())}")
print(f" classification           : {Dx.cri_hotspot.value_counts().to_dict()}")
hot = Dx[Dx.cri_hotspot == "hot"]
```

```

if len(hot):
    print(f" hot districts          : {hot.name.tolist()}")
    print(f" population in them      : {hot.population.sum():.0f} "
          f"({100*hot.population.sum()/Dx.population.sum():.1f} %)")

# ---- weight sensitivity -----
rng = np.random.default_rng(11)
SIMS = 2000
top5 = np.zeros(len(Dx)); ranks = np.zeros(len(Dx))
base = np.array([W_CRI[k] for k in ["hazard", "access", "vuln"]])
S = np.c_[Dx.score_hazard, Dx.score_access, Dx.score_vuln]
for _ in range(SIMS):
    w = np.abs(base * rng.normal(1, 0.30, 3)); w = w / w.sum()
    sc = S @ w
    order = np.argsort(-sc)
    top5[order[:5]] += 1
    ranks += pd.Series(-sc).rank().to_numpy()
Dx["p_top5"] = top5 / SIMS
Dx["mean_rank"] = ranks / SIMS

print(f"\nWEIGHT SENSITIVITY ({SIMS} simulations, weights perturbed +/-30 %)")
print(Dx.nlargest(8, "p_top5")[["district_id", "name", "CRI", "CRI_rank",
                               "p_top5", "mean_rank"]].round(3).to_string(index=False))
print(f" districts in the top 5 in EVERY simulation : "
      f"{int((Dx.p_top5 == 1.0).sum())}")
print(f" districts ever in the top 5                : {int((Dx.p_top5 > 0).sum())}")

# ---- MAUP CHECK: rebuild the index at BLOCK level -----
Bx = Fx.copy()
Bx["score_hazard_b"] = np.nanmean(np.c_[
    norm01(Bx.pct_in_flood100, True), norm01(Bx.dist_river_m, False)], axis=1)
Bx["score_access_b"] = np.nanmean(np.c_[
    norm01(Bx.dist_hospital_m, True), norm01(Bx.dist_clinic_m, True),
    norm01(Bx.n_bus_stops, False)], axis=1)
Bx["CRI_b"] = 0.5 * Bx.score_hazard_b + 0.5 * Bx.score_access_b
blk_to_dist = Bx.groupby("district_id").apply(
    lambda g: np.average(g.CRI_b, weights=np.maximum(g.population, 1)),
    include_groups=False)
Dx["CRI_from_blocks"] = Dx.district_id.map(blk_to_dist)

r_scale = Dx[["CRI", "CRI_from_blocks"]].corr(method="spearman").iloc[0, 1]
rank_shift = (Dx.CRI.rank(ascending=False) -
              Dx.CRI_from_blocks.rank(ascending=False)).abs()
print(f"\nMAUP CHECK - the same index built from BLOCKS then aggregated up")
print(f" Spearman correlation with the district-level index : {r_scale:.3f}")
print(f" median |rank shift| : {rank_shift.median():.1f} places, "
      f"max {rank_shift.max():.0f} places")
print(f" districts whose rank moves by more than 3 places: "
      f"{int((rank_shift > 3).sum())} of {len(Dx)}")
print(" -> the top of the ranking is stable to the choice of unit; the middle")
print("      is not. Report the top confidently and the middle with caution.")

```

Explanation.

- **Rank normalisation within each domain**, then a simple mean of indicators, then a weighted sum of domains. This two-level structure means an indicator cannot dominate simply because it has a wider range, and it keeps the weights interpretable — $w_{\text{hazard}} = 0.40$ means what it says.
- **Direction flags (*higher_is_worse*)** are declared per indicator in the dictionaries. Writing them down as data rather than burying them in code is what lets a reviewer check them.
- **hand_mean is flagged False** (lower HAND is worse), and **median_income_vs** likewise. Getting one direction wrong silently inverts a whole domain — this is the single most common error in composite-index work.
- **9 999 permutations for Moran's I**, not 999, because with 24 units and FDR correction downstream we need the p-value resolution (Lesson A6).

- **The MAUP check is the stage most analyses skip.** We rebuild a comparable index at *block* level and aggregate it up with population weights, then compare rankings. If the two disagree strongly, the index is an artefact of the unit. Reporting the correlation and the maximum rank shift is the honest summary.
- `include_groups=False` in the `groupby.apply` — required in pandas 2.2+ to avoid a deprecation warning about the grouping columns being passed to the function.

Expected outcome.

Three domain scores, each spanning roughly 0.1–0.9.

```
COMPOSITE RESILIENCE INDEX (higher = more at risk, less resilient)
district_id      name district_type  population  hazard  access  vuln  CRI
D11  Norrbank  upland_rural      4,157  0.721  0.641  0.653  0.676
D19  Corran Vale  upland_rural      3,531  0.591  0.755  0.627  0.657
D23   Ostrand  upland_rural        743  0.590  0.797  0.482  0.635
D15  Lyndover  upland_rural      3,235  0.687  0.568  0.566  0.615
D09   Tarnwell    suburban     45,734  0.749  0.474  0.497  0.590
```

The top of the ranking is dominated by remote upland districts, driven by the access domain — not by the flood-exposed urban core. That is a real finding and a slightly uncomfortable one: the districts with the *most people at risk* are not the districts with the *highest composite index*. Note **D09 Tarnwell** at rank 5 with **45 734 residents** against Ostrand's 743. **An index that is not population-weighted ranks places, not people**, and the client must be told which question they are asking.

```
SPATIAL AUTOCORRELATION (queen weights, 9999 permutations)
score_hazard      -0.1571  0.4143
score_access       0.6259  0.0001
score_vuln        -0.0362  0.9137
CRI               0.0819  0.3467

CRI HOTSPOTS (Getis-Ord Gi*, FDR-corrected)
uncorrected significant : 0 of 24
FDR-corrected           : 0
```

Only the access domain is significantly clustered. Hazard and vulnerability are not — which is surprising until you remember that rank-normalising 24 units compresses the very variation that Moran's I measures. Compare with Module 3, where the same variables at *block* level had I between 0.42 and 0.99. **Spatial structure is scale-dependent, and 24 units is too few to see it.**

The Gi analysis finds zero significant hotspots, corrected or uncorrected. That is the correct answer, not a failure. With $n = 24$ there is not enough data for local inference, and reporting "no significant clusters" is far better than mapping the top three Gi values as if they meant something.

```
WEIGHT SENSITIVITY (2000 simulations, +/-30 %)
name  CRI  CRI_rank  p_top5  mean_rank
Norrbank 0.676      1  1.000    1.264
Corran Vale 0.657      2  0.996    2.047
Lyndover 0.615      4  0.904    4.342
Ostrand 0.635      3  0.884    3.496
Ardenfeld 0.576     10  0.328    8.625
districts in the top 5 in EVERY simulation : 1
districts ever in the top 5                  : 13
```

Only one district survives every weighting; thirteen appear at least once. The robust recommendation is the first four ($p > 0.88$), not "the top five". Notice too that Lyndover ranks 4th on the point estimate but 3rd on robustness, ahead of Ostrand — point rank and robustness rank are different orderings.

MAUP CHECK

Spearman correlation with the district-level index : 0.761
median |rank shift| : 2.5 places, max 13 places
districts whose rank moves by more than 3 places: 9 of 24

Rebuilt from blocks, the index correlates 0.76 with the district-level version — strong but far from identical, and **9 of 24 districts move more than three places**. The extremes are stable; the middle is not. Speak confidently about the top and bottom, and cautiously about everything between.

Capstone Stage 10–12 — Visualisation, interpretation, conclusion

What we are about to do. Produce the map series and the costed recommendation, and state what the analysis cannot support.

Why it matters. Everything so far was analysis. This is the part the client reads. A finding that is not communicated does not exist.

Concept — the three maps every risk assessment needs.

1. **The exposure map** — what is at stake, and where.
2. **The composite map** — the index, with its classification named and its missing data visible.
3. **The uncertainty map** — where the analysis is least reliable.

Publishing the first two without the third is the most common failure in applied spatial analysis.

What the next cell does: produces a six-panel map series, computes the costed intervention list, and prints the final assessment including an explicit limitations section.

```
In [53]: # =====
# CAPSTONE STAGES 10-12
# =====
# ---- STAGE 10: THE MAP SERIES ----

fig, axes = plt.subplots(2, 3, figsize=(18.5, 11.5))

Dx.plot(ax=axes[0, 0], column="pop_in_flood100", cmap="Blues",
        scheme="naturalbreaks", k=5, legend=True,
        legend_kwds={"loc": "lower left", "fontsize": 6, "title": "people"},
        edgecolor="grey", linewidth=0.4,
        missing_kwds={"color": "#ddddd", "hatch": "///", "label": "no data"})
CAP["flood"][CAP["flood"].return_period_yr == 100].plot(
    ax=axes[0, 0], facecolor="none", edgecolor="#08519c", linewidth=0.5)
axes[0, 0].set_title("(1) EXPOSURE\nPopulation in the 100-year flood zone",
                    fontsize=10, weight="bold", loc="left")

Dx.plot(ax=axes[0, 1], column="value_in_flood100", cmap="Purples",
        scheme="naturalbreaks", k=5, legend=True,
        legend_kwds={"loc": "lower left", "fontsize": 6, "title": "k VS"},
        edgecolor="grey", linewidth=0.4)
axes[0, 1].set_title("(2) EXPOSURE\nAsset value in the 100-year flood zone",
                    fontsize=10, weight="bold", loc="left")

Dx.plot(ax=axes[0, 2], column="score_access", cmap="OrRd",
        scheme="quantiles", k=5, legend=True,
        legend_kwds={"loc": "lower left", "fontsize": 6},
        edgecolor="grey", linewidth=0.4)
CAP["facilities"][CAP["facilities"].facility_type.isin(["hospital", "clinic"])].plot(
    ax=axes[0, 2], color="black", markersize=10)
axes[0, 2].set_title("(3) ACCESS DEFICIT\nhigher = worse served",
                    fontsize=10, weight="bold", loc="left")
```

```

Dx.plot(ax=axes[1, 0], column="CRI", cmap="RdYlGn_r", scheme="quantiles", k=5,
        legend=True, legend_kwds={"loc": "lower left", "fontsize": 6},
        edgecolor="black", linewidth=0.5,
        missing_kwds={"color": "#dddddd", "hatch": "///", "label": "no data"})
for _, r in Dx.nlargest(5, "CRI").iterrows():
    axes[1, 0].annotate(r["name"], (r.geometry.representative_point().x,
                                     r.geometry.representative_point().y),
                        ha="center", fontsize=6.5, weight="bold", color="white",
                        path_effects=[pe.withStroke(linewidth=2, foreground="#333")])
axes[1, 0].set_title("(4) COMPOSITE RESILIENCE INDEX\nquantile classification, "
                    "top 5 labelled", fontsize=10, weight="bold", loc="left")

Dx.plot(ax=axes[1, 1], column="p_top5", cmap="viridis", vmin=0, vmax=1,
        legend=True, legend_kwds={"shrink": 0.6, "label": "P(top 5)"},
        edgecolor="grey", linewidth=0.4)
axes[1, 1].set_title("(5) UNCERTAINTY\nP(in the top 5) over 2,000 weightings",
                    fontsize=10, weight="bold", loc="left")

shift = (Dx.CRI.rank(ascending=False) - Dx.CRI_from_blocks.rank(ascending=False)).abs()
Dx.assign(shift=shift).plot(ax=axes[1, 2], column="shift", cmap="magma_r",
                            legend=True, legend_kwds={"shrink": 0.6,
                                                        "label": "|rank shift|"},
                            edgecolor="grey", linewidth=0.4,
                            missing_kwds={"color": "#dddddd"})
axes[1, 2].set_title("(6) SCALE SENSITIVITY\nrank change when built from blocks",
                    fontsize=10, weight="bold", loc="left")

for a in axes.ravel():
    a.set_aspect("equal"); a.set_xticks([]); a.set_yticks([])
    for sp in a.spines.values():
        sp.set_visible(False)
plt.suptitle("Vallmara Basin Integrated Climate-Resilience Assessment | "
            "CRS EPSG:32633 (UTM 33N) | fictional data",
            fontsize=13, weight="bold", y=0.995)
plt.tight_layout(); plt.show()

# ---- STAGE 12: COSTED INTERVENTION LIST -----
BUDGET = 120_000.0          # thousand VS over five years

UNIT_COSTS = {
    "flood_defence": {"per_person_protected": 1.10, "averted_frac": 0.65},
    "clinic": {"fixed": 9_500.0, "reach_km": 10.0},
    "retrofit": {"per_building": 12.0, "averted_frac": 0.35},
}

ead_col = Dx.value_in_flood100 * 0.02      # crude annualisation for ranking
Dx["ead_proxy"] = ead_col

recs = []
for _, r in Dx.iterrows():
    # (a) flood defence, sized by exposed population
    if r.pop_in_flood100 > 500:
        cost = r.pop_in_flood100 * UNIT_COSTS["flood_defence"]["per_person_protected"]
        avert = r.ead_proxy * UNIT_COSTS["flood_defence"]["averted_frac"]
        recs.append(dict(district=r["name"], district_id=r.district_id,
                        intervention="flood defence", cost_kvs=cost,
                        annual_benefit_kvs=avert,
                        people_helped=r.pop_in_flood100, CRI=r.CRI))
    # (b) new clinic where access is poor and population is meaningful
    if r.dist_clinic_m > 8000 and r.population > 3000:
        recs.append(dict(district=r["name"], district_id=r.district_id,
                        intervention="new clinic",
                        cost_kvs=UNIT_COSTS["clinic"]["fixed"],
                        annual_benefit_kvs=0.0002 * r.population * r.dist_clinic_m / 1000,
                        people_helped=r.population, CRI=r.CRI))
    # (c) retrofit the most vulnerable building stock
    if r.bld_in_flood100 > 30:
        cost = r.bld_in_flood100 * UNIT_COSTS["retrofit"]["per_building"]

```

```

        recs.append(dict(district=r["name"], district_id=r.district_id,
                        intervention="building retrofit", cost_kvs=cost,
                        annual_benefit_kvs=r.ead_proxy * UNIT_COSTS["retrofit"]
["averted_frac"],
                        people_helped=r.pop_in_flood100, CRI=r.CRI))

R = pd.DataFrame(recs)
R["bcr_20yr"] = 20 * R.annual_benefit_kvs / R.cost_kvs # benefit-cost ratio
R = R.sort_values("bcr_20yr", ascending=False).reset_index(drop=True)
R["cum_cost"] = R.cost_kvs.cumsum()
R["funded"] = R.cum_cost <= BUDGET

print("=" * 96)
print("PRIORITISED INTERVENTION LIST (ranked by 20-year benefit-cost ratio)")
print("=" * 96)
print(R.head(14)[["district", "intervention", "cost_kvs", "annual_benefit_kvs",
                  "bcr_20yr", "people_helped", "cum_cost", "funded"]]
      .round(1).to_string(index=False))
funded = R[R.funded]
print("-" * 96)
print(f" budget : {BUDGET:>12,.0f} k VS")
print(f" committed : {funded.cost_kvs.sum():>12,.0f} k VS "
      f"({100*funded.cost_kvs.sum()/BUDGET:.0f} %)")
print(f" interventions funded : {len(funded):>12} of {len(R)}")
print(f" people directly helped: {funded.people_helped.sum():>12,.0f}")
print(f" annual benefit : {funded.annual_benefit_kvs.sum():>12,.0f} k VS/yr")
print(f" 20-yr benefit-cost : "
      f"({20*funded.annual_benefit_kvs.sum()/funded.cost_kvs.sum():>12.2f}")

# ---- STAGE 11-12: THE WRITTEN CONCLUSION -----
top = Dx.nlargest(3, "CRI")
robust = Dx[Dx.p_top5 > 0.8]
print("\n" + "=" * 96)
print("FINDINGS")
print("=" * 96)
print(f"""
1. EXPOSURE. {Dx.pop_in_flood100.sum():.0f} people ({100*Dx.pop_in_flood100.sum()/
Dx.population.sum():.0f} % of
the basin) and {Dx.value_in_flood100.sum():.0f} k VS of assets lie inside the
100-year flood zone. Exposure is concentrated: the top three districts hold
{100*Dx.nlargest(3, 'value_in_flood100').value_in_flood100.sum()/
Dx.value_in_flood100.sum():.0f} % of the exposed value.

2. ACCESS. Median distance to a hospital is {Dx.dist_hospital_m.median()/1000:.1f} km, and
the worst-served district is {Dx.loc[Dx.dist_hospital_m.idxmax(), 'name']} at
{Dx.dist_hospital_m.max()/1000:.1f} km. Four hospitals serve {Dx.population.sum():.0f}
residents.

3. COMPOSITE. The three districts with the highest CRI are
{', '.join(top.name.tolist())}. {len(robust)} district(s) remain in the top five under
more than 80 % of plausible weightings; those are the robust priorities.

4. INVESTMENT. Ranking by 20-year benefit-cost ratio, {len(funded)} interventions fit
the {BUDGET:.0f} k VS budget, directly protecting {funded.people_helped.sum():.0f} people
at a
portfolio benefit-cost ratio of {20*funded.annual_benefit_kvs.sum()/
funded.cost_kvs.sum():.1f}.

WHAT THIS ANALYSIS CANNOT TELL YOU
a. Travel times are STRAIGHT-LINE. Real network distances are typically
20-50 % longer, so the accessibility deficit is UNDERSTATED, especially in
the uplands where roads are indirect.
b. Flood depths use a bathtub model over a HAND surface. There is no hydraulic
routing, no defences and no drainage; depths are indicative only.
c. Depth-damage curves are transferred, not locally calibrated. A +/-25 % change
in them moves expected losses by 15-30 % (Lesson A13).
d. The index is computed on 24 districts. That is too few units for reliable
local hotspot inference, and the middle of the ranking is unstable to the

```

```

        choice of spatial unit (median rank shift {shift.median():.1f} places).
    e. Two districts have no recorded population. Their scores are computed from
        the block data that survives, and they are hatched on every map.
    f. Nothing here is causal. High CRI districts are places where hazard,
        poor access and vulnerability CO-OCCUR; that is a targeting statement,
        not a mechanism.
    """)
print("=" * 96)

Dx.to_file(OUT / "capstone_results.gpkg", layer="district_cri", driver="GPKG")
R.to_csv(OUT / "capstone_interventions.csv", index=False)
print(f"Deliverables written to {OUT}:")
print(f"  capstone_results.gpkg      (district CRI, domain scores, uncertainty)")
print(f"  capstone_interventions.csv  ({len(R)} costed interventions)")
print(f"  capstone_districts.gpkg     (full {len(FEATCOLS)}-feature analysis table)")

```

Explanation.

- **The six-panel series follows the three-map rule** and adds a scale-sensitivity panel. Panels 1–3 are the inputs, panel 4 is the index, and **panels 5 and 6 are the honesty**: where the ranking is robust to the weights, and where it is robust to the choice of spatial unit. A client who sees only panel 4 will over-read it.
- **missing_kwds on every choropleth**. The two districts with no population are hatched, not white. On a resilience map, "no data" rendered as "low risk" is a potentially serious error.
- **The intervention list ranks by benefit–cost ratio, not by CRI**. This is deliberate and worth understanding: the highest-CRI district is not necessarily where the next VS is best spent. Targeting need and maximising return are different objectives, and a good report presents both and lets the decision- maker choose.
- **The unit costs and averted fractions are stated explicitly** as a dictionary at the top. They are assumptions, they are almost certainly wrong in detail, and putting them in one visible place is what makes them challengeable.
- **The limitations section is not boilerplate**. Each item names a specific assumption, its direction of bias where known ("the accessibility deficit is understated"), and a pointer to the lesson that quantified it. That is the difference between covering yourself and informing your reader.

Expected outcome.

A six-panel figure with a title bar naming the CRS and the fictional status of the data.

PRIORITISED INTERVENTION LIST (ranked by 20-year benefit–cost ratio)						
district	intervention	cost_kvs	annual_benefit	bcr_20yr	cum_cost	funded
Old Vallmara	building retrofit	6,108.0	1,165.6	3.8	6,108.0	True
Harbourgate	building retrofit	1,620.0	287.9	3.6	7,728.0	True
Tarnwell	building retrofit	1,776.0	190.1	2.1	9,504.0	True
Ardenfeld	building retrofit	1,512.0	91.4	1.2	11,016.0	True
Old Vallmara	flood defence	63,346.1	2,164.8	0.7	74,362.1	True
Harbourgate	flood defence	19,759.3	534.6	0.5	94,121.5	True
Tarnwell	flood defence	22,278.5	353.1	0.3	116,400.0	True
Brannock	flood defence	1,132.6	12.6	0.2	117,532.6	True
...					>120,000	False

Two things in this table matter more than the ranking itself.

First, **building retrofit dominates flood defence on benefit–cost ratio** — 3.8 against 0.7 in the same district — even though defence protects far more people. Retrofit is cheap per unit of averted loss; defence is capital-intensive. A BCR-ranked list will always fund retrofit first, which is economically correct and politically difficult, because defence is what communities ask for.

Second, **most interventions have a 20-year BCR below 1.0**, so the portfolio as a whole does not pay for itself under the stated unit costs. That is a genuine appraisal result, not a bug: it says either the unit costs are too pessimistic, or the averted-damage fractions are too conservative, or the programme should be justified on grounds other than

avoided property damage (lives, disruption, equity). **Report it and name the three possibilities** rather than quietly tuning the assumptions until the answer looks better.

Then a findings block with four numbered findings and six numbered limitations, and three deliverable files written to `data/outputs/`.

If you have got this far with your own implementation and can answer yes to the rubric at the top of Module 4, you are ready to do this work for real.

Solutions

Complete worked solutions to every exercise. Read them **after** attempting the problems — the value is in the struggle, not in the answer.

Where an exercise is open-ended (the Module 3 set), the solution gives a complete, runnable implementation of one defensible approach and names the decisions where a different choice would be equally valid.

All solutions reuse the helper functions defined earlier in the notebook, so run the notebook from the top before executing this section.

Solutions — Module 1 (Beginner)

1.1 Layer inventory · 1.2 Attribute audit · 1.3 CRS forensics

1.1 requires iterating over both the GeoPackage layers and the standalone files, measuring in a single CRS. **1.2** is a reusable audit function. **1.3** is the same area comparison as B5, applied to a different layer.

```
In [54]: # ===== SOLUTION 1.1 =====
import pyogrio

def layer_inventory():
    rows = []
    for name, _ in pyogrio.list_layers(GPKG):
        g = gpd.read_file(GPKG, layer=name).to_crs(CRS_UTM)
        gt = g.geom_type.iloc[0]
        rows.append(dict(
            source="vallmara.gpkg", layer=name, n_features=len(g),
            geometry_type=str(dict(g.geom_type.value_counts())),
            crs="EPSG:32633", n_columns=g.shape[1],
            area_km2=round(g.geometry.area.sum()/1e6, 3) if "Polygon" in gt else np.nan,
            length_km=round(g.geometry.length.sum()/1000, 3) if "Line" in gt else np.nan))
    for fn in ["roads.geojson", "protected_areas.geojson"]:
        g0 = gpd.read_file(VEC / fn)
        g = g0.to_crs(CRS_UTM)
        gt = g.geom_type.iloc[0]
        rows.append(dict(
            source=fn, layer="(single)", n_features=len(g),
            geometry_type=str(dict(g.geom_type.value_counts())),
            crs=g0.crs.to_string(), n_columns=g.shape[1],
            area_km2=round(g.geometry.area.sum()/1e6, 3) if "Polygon" in gt else np.nan,
            length_km=round(g.geometry.length.sum()/1000, 3) if "Line" in gt else np.nan))
    return pd.DataFrame(rows)

inv = layer_inventory()
print("SOLUTION 1.1 - LAYER INVENTORY")
print(inv.to_string(index=False))

# ===== SOLUTION 1.2 =====
```

```

def audit(gdf, name="layer"):
    """Full attribute + geometry audit of any GeoDataFrame."""
    a = pd.DataFrame({
        "dtype": gdf.dtypes.astype(str),
        "n_missing": gdf.isna().sum(),
        "pct_missing": (100 * gdf.isna().mean()).round(2),
        "n_unique": gdf.nunique(dropna=True),
    })
    num = gdf.select_dtypes(include=[np.number])
    for stat in ["min", "median", "max"]:
        a[stat] = getattr(num, stat)().round(3)
    print(f"AUDIT: {name} ({len(gdf)} rows x {gdf.shape[1]} cols, "
          f"CRS {gdf.crs.to_string() if gdf.crs else 'NONE'})")
    print(f" geometry: {dict(gdf.geom_type.value_counts()), "
          f"null={int(gdf.geometry.isna().sum())}, "
          f"empty={int(gdf.geometry.is_empty.sum())}, "
          f"invalid={int((~gdf.geometry.is_valid).sum())}, "
          f"dup_geom={int(gdf.geometry.to_wkb().duplicated().sum())}")
    return a

print("\n\nSOLUTION 1.2 - AUDIT OF census_blocks")
bl_raw = gpd.read_file(GPKG, layer="census_blocks")
print(audit(bl_raw, "census_blocks").to_string())
print("""
PROBLEMS FOUND IN census_blocks
* one EMPTY geometry (index 300, block B0301). It keeps its attributes, so
  attribute sums stay correct while any geometric union silently loses
  5.93 km^2. Detect with .is_empty, never with .isna().
* area_km2 is a STORED attribute, not recomputed. After any clip or overlay it
  would be stale. Recompute from the geometry whenever you cut it.
* pop_density_km2 is a derived ratio; it is redundant with population/area_km2
  and will silently disagree with them if either is edited.
* no missing values otherwise - this layer is clean apart from the empty row.""")

# ===== SOLUTION 1.3 =====
print("\n\nSOLUTION 1.3 - CRS FORENSICS ON protected_areas.geojson")
pa0 = gpd.read_file(VEC / "protected_areas.geojson")
print(f" native CRS      : {pa0.crs.to_string()} ({pa0.crs.axis_info[0].unit_name})")
print(f" .area in native : {pa0.geometry.area.sum():.8f} square degrees "
      f"<- MEANINGLESS")
truth = pa0.to_crs(CRS_UTM).geometry.area.sum() / 1e6
print(f"\n {'CRS':<30}{'area km2':>12}{'error vs UTM':>15}")
for lab, code in [("EPSG:32633 UTM 33N", CRS_UTM),
                  ("EPSG:3857 Web Mercator", CRS_WEBMERC),
                  ("ESRI:54009 Mollweide", "ESRI:54009")]:
    a = pa0.to_crs(code).geometry.area.sum() / 1e6
    print(f" {lab:<30}{a:>12,.3f}{100*(a-truth)/truth:>14.2f}%")
print("""
WHICH WOULD I PUBLISH?
The UTM 33N figure. The study area sits entirely within zone 33, where UTM's
scale error is under 0.1 % (verified against Mollweide, an equal-area
projection, which agrees to ~0.1 %). Web Mercator over-states the area by
about 80 % at this latitude and must never be used for a published statistic,
even though its units are nominally metres.""")

```

1.4 Dirty CSV · 1.5 Shapely reasoning · 1.6 Distance profile

```

In [55]: # ===== SOLUTION 1.4 =====
print("SOLUTION 1.4 - QUARANTINING BAD COORDINATES")
raw = pd.read_csv(TAB / "flood_incidents.csv")
BOX = dict(lon=(13.6, 14.5), lat=(41.4, 41.9))
ok = lambda lo, la: lo.between(*BOX["lon"]) & la.between(*BOX["lat"])

raw["rule"] = pd.NA
raw.loc[(raw.lon == 0) & (raw.lat == 0), "rule"] = "R1 null island"
raw.loc[(~ok(raw.lon, raw.lat)) & ok(raw.lat, raw.lon) & raw.rule.isna(),

```

```

        "rule"] = "R2 lon/lat swapped"
raw.loc[(~ok(raw.lon, raw.lat)) & raw.rule.isna(), "rule"] = "R3 outside region"

print(raw.rule.value_counts(dropna=False).rename("n").to_string())
kept = raw[raw.rule.isna()].copy()
recov = raw[raw.rule == "R2 lon/lat swapped"].copy()
recov[["lon", "lat"]] = recov[["lat", "lon"]].to_numpy()
final = pd.concat([kept, recov], ignore_index=True).drop(columns="rule")
inc_sol = gpd.GeoDataFrame(final, geometry=gpd.points_from_xy(final.lon, final.lat),
                           crs=CRS_WGS84).to_crs(CRS_UTM)

print(f"\n kept outright : {len(kept)}")
print(f" recovered (R2) : {len(recov)}")
print(f" quarantined : {len(raw) - len(kept) - len(recov)} "
      f"(R1 + R3 - unrecoverable)")
print(f" final clean set : {len(inc_sol)}")
print(f" all inside the study area: "
      f"{bool(inc_sol.geometry.within(LAND_GEOM.buffer(2000)).all())}")

# ===== SOLUTION 1.5 =====
print("\n\nSOLUTION 1.5 - SHAPELY REASONING FOR 'Marnvik'")
m = districts[districts.name == "Marnvik"]
g = m.geometry.iloc[0]
area_m2, perim_m = g.area, g.length # NOT `A, P` - those are in use
print(f" area : {area_m2/1e6:,.3f} km^2")
print(f" perimeter : {perim_m/1000:,.3f} km")
print(f" Polsby-Popper : {4*np.pi*area_m2/perim_m**2:.4f} (1.0 = perfect circle)")
nb = districts[districts.geometry.touches(g)]
print(f" shares a boundary with ({len(nb)}): {nb.name.tolist()}")
print(f" centroid inside the polygon? : {g.contains(g.centroid)}")
print(f" centroid : ({g.centroid.x:,.0f}, {g.centroid.y:,.0f})")
print(f" representative_point() : ({g.representative_point().x:,.0f}, "
      f"{g.representative_point().y:,.0f})")
er = g.buffer(-1000)
print(f" after a 1 km inward buffer: {er.area/1e6:,.3f} km^2 "
      f"(-{(area_m2-er.area)/1e6:,.3f} km^2, "
      f"{100*(area_m2-er.area)/area_m2:.1f} % of the original)")

# ===== SOLUTION 1.6 =====
print("\n\nSOLUTION 1.6 - DISTANCE PROFILE FOR THE 24 STATIONS")
targets = {
    "coast": coastline.geometry.union_all(),
    "river": rivers.geometry.union_all(),
    "road": roads[roads.road_class.isin(["motorway", "primary"])] geometry.union_all(),
    "hospital": facilities[facilities.facility_type == "hospital"] geometry.union_all(),
}
prof = stations[["station_id", "name", "station_type"]].copy()
for k, geom in targets.items():
    prof[f"d_{k}_km"] = (stations.geometry.distance(geom) / 1000).round(2)
dcols = [c for c in prof.columns if c.startswith("d_")]
prof["total_km"] = prof[dcols].sum(axis=1).round(2)
prof = prof.sort_values("total_km")
print(prof.to_string(index=False))
print(f"\n MOST CONNECTED : {prof.iloc[0]['name']} "
      f"(total {prof.iloc[0]['total_km']:.1f} km)")
print(f" MOST REMOTE : {prof.iloc[-1]['name']} "
      f"(total {prof.iloc[-1]['total_km']:.1f} km)")

```

1.7 Spatial join and rates · 1.8 Raster sampling and regression

```

In [56]: # ===== SOLUTION 1.7 =====
print("SOLUTION 1.7 - BUS STOPS PER 10,000 RESIDENTS")
bl = gpd.read_file(GPKG, layer="census_blocks")
bl = bl[~bl.geometry.is_empty]
st = gpd.read_file(GPKG, layer="bus_stops")

j = gpd.sjoin(st[["stop_id", "geometry"]], bl[["block_id", "district_id", "geometry"]],

```

```

        how="inner", predicate="within")
assert len(j) <= len(st), "join multiplied rows"
per_d = j.groupby("district_id").size().rename("n_stops")
pop_d = bl.groupby("district_id").population.sum().rename("population")

res = (districts[["district_id", "name", "district_type", "geometry"]]
       .merge(per_d, on="district_id", how="left")
       .merge(pop_d, on="district_id", how="left"))
res["n_stops"] = res.n_stops.fillna(0).astype(int)
res["stops_per_10k"] = np.where(res.population > 0,
                                1e4 * res.n_stops / res.population, np.nan)
print(res.sort_values("stops_per_10k", ascending=False)
      [["district_id", "name", "district_type", "population", "n_stops",
        "stops_per_10k"]].round(2).head(10).to_string(index=False))
print("\n by district type:")
# NOTE: do not name a column "pop" - DataFrame.pop is a method, so `d.pop`
# returns the method rather than the column. Use d["pop"] or a different name.
print(res.groupby("district_type").agg(
    stops=("n_stops", "sum"), residents=("population", "sum")).assign(
    per_10k=lambda d: (1e4 * d["stops"] / d["residents"]).round(2)).to_string())

fig, axes = plt.subplots(1, 2, figsize=(13.5, 5.6))
res.plot(ax=axes[0], column="n_stops", cmap="Greens", scheme="naturalbreaks", k=5,
        legend=True, legend_kwds={"loc": "lower left", "fontsize": 6},
        edgecolor="grey", linewidth=0.4)
axes[0].set_title("Bus stops (COUNT)", fontsize=10, weight="bold", loc="left")
res.plot(ax=axes[1], column="stops_per_10k", cmap="Greens", scheme="naturalbreaks",
        k=5, legend=True, legend_kwds={"loc": "lower left", "fontsize": 6},
        edgecolor="grey", linewidth=0.4,
        missing_kwds={"color": "#dddddd", "hatch": "///", "label": "no population"})
axes[1].set_title("Bus stops per 10,000 residents (RATE)",
        fontsize=10, weight="bold", loc="left")

for a in axes:
    a.set_aspect("equal"); a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.show()

print("""
WHICH IS MORE HONEST?
Neither alone. The COUNT map shows where the service physically exists and is
what an operator needs. The RATE map shows provision per person and is what an
equity assessment needs - and it flatters tiny upland districts where three
stops serve 700 people. Publish both, and always show the denominator: a rate
computed on a population of 743 is not comparable with one computed on 197,870.
The worst-served type by RATE is typically upland_rural; by COUNT it is also
upland_rural, which is the rare case where the two agree.""")

# ===== SOLUTION 1.8 =====
print("\n\nSOLUTION 1.8 - RASTER SAMPLING AND THE RAINFALL-ELEVATION LAW")
cent = bl.geometry.representative_point()
coords = [(p.x, p.y) for p in cent]
samp = pd.DataFrame({"block_id": bl.block_id.to_numpy()})
for nm, fn in [("elev", "dem_25m.tif"), ("rain", "rainfall_annual_250m.tif"),
               ("ndvi", "ndvi_50m.tif")]:
    with rasterio.open(RAS / fn) as src:
        v = np.array([x[0] for x in src.sample(coords)], dtype=float)
        v[v == src.nodata] = np.nan
        samp[nm] = v
    print(f" {nm:<6}: {int(np.isnan(v).sum())} NoData of {len(v)} blocks")
print(""" elev and rain return NO NoData: representative_point() is guaranteed
to lie inside its block, and every block is on land. ndvi loses 6 blocks -
those centroids fall inside the two simulated cloud gaps, which are stored
as NoData. Had we used .centroid instead of .representative_point(), coastal
blocks with concave shapes could have produced sea-cell NoData as well.""")

d = samp.dropna(subset=["elev", "rain"])
slope_hat, inter_hat = np.polyfit(d.elev, d.rain, 1)
r = np.corrcoef(d.elev, d.rain)[0, 1]
n = len(d)

```

```

se = np.sqrt(((d.rain - (inter_hat + slope_hat*d.elev))*2).sum() / (n-2)
              / ((d.elev - d.elev.mean())*2).sum())
print(f"\n OLS rainfall = {inter_hat:.2f} + {slope_hat:.4f} * elevation")
print(f"      n = {n}, r = {r:.4f}, R^2 = {r**2:.4f}")
print(f"      95 % CI for the slope: "
      f"[{slope_hat-1.96*se:.4f}, {slope_hat+1.96*se:.4f}]")
print(f" TRUE generating coefficient : 0.6200")
print(f" bias : {slope_hat-0.62:+.4f} "
      f"({100*(slope_hat-0.62)/0.62:+.1f} %)")
print("""
WHY IS IT NOT EXACT?
1. The generating law also contains a south-to-north gradient
   (+150 mm across the basin) which is OMITTED here. Elevation and northing
   are correlated, so the elevation coefficient absorbs part of that gradient
   - textbook omitted-variable bias (Lesson A9).
2. We sample the 250 m rainfall raster at a POINT, so each block gets one
   6.25-hectare cell average rather than its true block mean.
3. Additive noise was added to the rainfall field at generation time.
Add a `north` term to the regression and the elevation coefficient moves
materially closer to 0.62 - try it.""")

```

Solutions — Module 2 (Intermediate)

2.1 CRS choice · 2.2 QA report · 2.3 Riparian compliance

```

In [57]: from pyproj import Geod
geod = Geod(ellps="WGS84")

# ===== SOLUTION 2.1 =====
print("SOLUTION 2.1 - A DEFENSIBLE CRS CHOICE")
pa_ll = protected.to_crs(CRS_WGS84)
riv_ll = rivers.to_crs(CRS_WGS84)
true_area = sum(abs(geod.geometry_area_perimeter(g)[0]) for g in pa_ll.geometry) / 1e6
true_len = sum(geod.geometry_length(g) for g in riv_ll.geometry) / 1000

# an azimuthal equidistant CRS centred on the study area (best for distance)
c = districts.to_crs(CRS_WGS84).geometry.union_all().centroid
AEQD = f"+proj=aeqd +lat_0={c.y:.5f} +lon_0={c.x:.5f} +datum=WGS84 +units=m +no_defs"

rows = []
for lab, code_ in [("EPSG:32633 UTM 33N (conformal)", CRS_UTM),
                  ("EPSG:3857 Web Mercator", CRS_WEBMERC),
                  ("ESRI:54009 Mollweide (equal area)", "ESRI:54009"),
                  ("Azimuthal equidistant (local)", AEQD)]:
    a = protected.to_crs(code_).geometry.area.sum() / 1e6
    L = rivers.to_crs(code_).geometry.length.sum() / 1000
    rows.append({"CRS": lab, "protected km2": round(a, 3),
                "area err %": round(100*(a-true_area)/true_area, 3),
                "river km": round(L, 3),
                "len err %": round(100*(L-true_len)/true_len, 3)})
print(f" geodesic ground truth: area {true_area:.3f} km^2, "
      f"length {true_len:.3f} km")
print(pd.DataFrame(rows).to_string(index=False))
print("""
RECOMMENDATION
Publish the UTM 33N figures for BOTH quantities. UTM is conformal, the study
area lies wholly inside zone 33, and both its area and its length errors are
under 0.1 % - confirmed independently by Mollweide (equal-area) for area and
by the local azimuthal-equidistant projection for length. Web Mercator is
unusable for either. Mollweide is excellent for area and poor for length, and
azimuthal equidistant is the reverse, so neither is a good single choice.
Report the CRS alongside the numbers; a figure without its CRS is not a
measurement.""")

```

```

# ===== SOLUTION 2.2 =====
print("\n\nSOLUTION 2.2 - QA REPORT ACROSS EVERY LAYER")
def qa_report(gdf, name):
    gt = gdf.geom_type.dropna()
    poly = gt.isin(["Polygon", "MultiPolygon"]).any()
    line = gt.isin(["LineString", "MultiLineString"]).any()
    zero = int(((gdf.geometry.area == 0) if poly else
                (gdf.geometry.length == 0) if line else
                pd.Series(False, index=gdf.index)).sum())
    return pd.DataFrame([dict(
        layer=name, rows=len(gdf), crs=gdf.crs.to_string() if gdf.crs else "NONE",
        geom_types=".".join(sorted(gt.unique())),
        null=int(gdf.geometry.isna().sum()),
        empty=int(gdf.geometry.is_empty.sum()),
        invalid=int((~gdf.geometry.is_valid).sum()),
        dup_geom=int(gdf.geometry.to_wkb().duplicated().sum()),
        zero_measure=zero,
        total=round(gdf.geometry.area.sum()/1e6 if poly else
                    gdf.geometry.length.sum()/1000 if line else 0, 3),
        unit="km2" if poly else "km" if line else "-")])

ALL = {n: gpd.read_file(GPKG, layer=n) for n, _ in pyogrio.list_layers(GPKG)},
"roads.geojson": gpd.read_file(VEC / "roads.geojson").to_crs(CRS_UTM),
"protected.geojson": gpd.read_file(VEC / "protected_areas.geojson").to_crs(CRS_UTM)}
before = pd.concat([qa_report(g, n) for n, g in ALL.items()], ignore_index=True)
print("BEFORE REPAIR")
print(before.to_string(index=False))

FIXED = {}
for n, g in ALL.items():
    g = g.copy()
    if (~g.geometry.is_valid).any():
        g["geometry"] = g.geometry.make_valid()
        g = g.explode(index_parts=False, ignore_index=True)
        keep = ["Polygon", "MultiPolygon"] if g.geom_type.isin(
            ["Polygon", "MultiPolygon"]).any() else list(g.geom_type.unique())
        g = g[g.geom_type.isin(keep)]
        g = g[~g.geometry.isna() & ~g.geometry.is_empty]
        g = g[~g.geometry.to_wkb().duplicated()] if n == "landuse" else g
        FIXED[n] = g.reset_index(drop=True)
after = pd.concat([qa_report(g, n) for n, g in FIXED.items()], ignore_index=True)
print("\n\nAFTER REPAIR (only the layers that changed)")
chg = after.merge(before, on="layer", suffixes=("_after", "_before"))
chg = chg[(chg.rows_after != chg.rows_before) |
          (chg.total_after != chg.total_before)]
print(chg[["layer", "rows_before", "rows_after", "total_before", "total_after",
          "unit_after"]].to_string(index=False))

# ===== SOLUTION 2.3 =====
print("\n\nSOLUTION 2.3 - RIPARIAN BUFFER COMPLIANCE")
WIDTH = {2: 50.0, 3: 120.0, 4: 200.0}
rb = rivers.copy()
rb["geometry"] = rivers.geometry.buffer(rivers.strahler_order.map(WIDTH).to_numpy())
strip = rb.geometry.union_all() # DISSOLVED: no double counting

bpt = buildings_clean.copy()
bpt["geometry"] = buildings_clean.geometry.representative_point()
viol = bpt[bpt.geometry.within(strip)].copy()
print(f" protected strip area      : {strip.area/1e4:,.1f} ha "
      f"({100*strip.area/LAND_GEOM.area:.2f}% of the basin)")
print(f" buildings in violation      : {len(viol):,} of {len(bpt):,} "
      f"({100*len(viol)/len(bpt):.2f}%)")
print(f" total value in violation      : {viol.value_kvs.sum():,.0f} k VS")
print(f" (each building counted ONCE - the strips were dissolved first)")

vd = gpd.sjoin(viol, districts[["district_id", "name", "geometry"]],
               predicate="within").drop_duplicates("building_id")
alld = gpd.sjoin(bpt, districts[["district_id", "name", "geometry"]],

```

```

        predicate="within").drop_duplicates("building_id")
rate = (vd.groupby("name").size().rename("violations").to_frame()
        .join(alld.groupby("name").size().rename("buildings"), how="right")
        .fillna(0))
rate["per_1000"] = (1000 * rate.violations / rate.buildings).round(1)
print("\n worst districts by violation RATE:")
print(rate.sort_values("per_1000", ascending=False).head(6).to_string())

lu_strip = gpd.overlay(landuse_clean[["landuse_class", "geometry"]],
                       gpd.GeoDataFrame(geometry=[strip], crs=CRS_UTM),
                       how="intersection", keep_geom_type=True)
lu_strip["ha"] = lu_strip.geometry.area / 1e4
comp = (100 * lu_strip.groupby("landuse_class").ha.sum()
        / lu_strip.ha.sum()).round(1).sort_values(ascending=False)
print(f"\n land-cover composition of the protected strip (%):")
print(comp.to_string())
print(f" -> {comp.get('Built-up', 0):.1f} % of the strip is already built up")

fig, ax = fresh_ax((9, 7), "Riparian buffer violations")
land.plot(ax=ax, facecolor="#f7f5ef", edgecolor="#d8d2c4", linewidth=0.5)
gpd.GeoSeries([strip], crs=CRS_UTM).plot(ax=ax, facecolor="#9ecae1",
                                         edgecolor="#3182bd", alpha=0.6)
bpt.plot(ax=ax, color="#cccccc", markersize=0.5)
viol.plot(ax=ax, color="crimson", markersize=3)
rivers.plot(ax=ax, color="#08519c", linewidth=0.8)
ax.set_title(f"{len(viol):,} buildings inside the statutory riparian strip",
             fontsize=11, weight="bold", loc="left")
plt.show()

```

2.4 Reservoir impact · 2.5 Areal interpolation · 2.6 Zonal at two resolutions

```

In [58]: # ===== SOLUTION 2.4 =====
print("SOLUTION 2.4 - PROPOSED RESERVOIR IMPACT ASSESSMENT")
vallmara = rivers[rivers.name == "Vallmara River"].geometry.union_all()
corridor = vallmara.buffer(400)

with rasterio.open(RAS / "dem_25m.tif") as src:
    dem_r = src.read(1, masked=True)
    tr_r, sh_r = src.transform, src.shape
    low = (dem_r.filled(9999) < 30)
    corr_mask = rasterize([(corridor, 1)], out_shape=sh_r, transform=tr_r,
                          fill=0, dtype="uint8").astype(bool)
    res_mask = low & corr_mask & ~dem_r.mask

polys = [Polygon(g["coordinates"][0], g["coordinates"][1:])]
    for g, v in shapes(res_mask.astype("uint8"), mask=res_mask, transform=tr_r)]
from shapely.ops import unary_union
reservoir = unary_union([q.buffer(0) for q in polys]).buffer(25).buffer(-25)
print(f" reservoir footprint : {reservoir.area/1e6:,.2f} km^2 "
      f"({reservoir.area/1e4:,.0f} ha)")

lost = gpd.overlay(landuse_clean[["landuse_class", "geometry"]],
                  gpd.GeoDataFrame(geometry=[reservoir], crs=CRS_UTM),
                  how="intersection", keep_geom_type=True)
lost["ha"] = lost.geometry.area / 1e4
print("\n land use inundated (ha):")
print(lost.groupby("landuse_class").ha.sum().round(1)
      .sort_values(ascending=False).to_string())

inund = bpt[bpt.geometry.within(reservoir)]
print(f"\n buildings inundated : {len(inund):,}")
print(f" asset value lost : {inund.value_kvs.sum():,.0f} k VS")
print(f" by use type : {inund.use_type.value_counts().to_dict()}")

# dasymetric displacement estimate
blv = blocks[~blocks.geometry.is_empty].copy()

```

```

bptn = buildings_clean.copy()
bptn["geometry"] = buildings_clean.geometry.representative_point()
bptn["_ls"] = bptn.footprint_m2 * bptn.floors
bb = gpd.sjoin(bptn, blv[["block_id", "geometry"]], predicate="within")
tot = bb.groupby("block_id")["_ls"].sum()
inn = bb[bb.geometry.within(reservoir)].groupby("block_id")["_ls"].sum()
w = (inn / tot).reindex(blv.block_id).fillna(0).clip(0, 1).to_numpy()
displaced = float((blv.population.to_numpy() * w).sum())
print(f" people displaced (dasymetric) : {displaced:,.0f}")
print(f" people displaced (areal)      : "
      f"{float((blv.population * (blv.geometry.intersection(reservoir).area /
blv.geometry.area)).sum()):,.0f}")

# ===== SOLUTION 2.5 =====
print("\n\nSOLUTION 2.5 - AREAL INTERPOLATION OF AN INTENSIVE VARIABLE")
inc_d = socio_clean[["district_id", "median_income_vs"]]
bi = blv[["block_id", "district_id", "geometry"]].merge(inc_d, on="district_id",
                                                         how="left")

bi["area_km2"] = bi.geometry.area / 1e6
back = bi.groupby("district_id").apply(
    lambda g: np.average(g.median_income_vs, weights=g.area_km2)
    if g.median_income_vs.notna().all() else np.nan, include_groups=False)
chk = inc_d.set_index("district_id").join(back.rename("recovered"))
chk["diff"] = (chk.recovered - chk.median_income_vs).round(6)
print(chk.head(8).round(2).to_string())
print(f"\n max |difference| : {chk['diff'].abs().max():.6f}")
print("""
WHY IS THE RECOVERY EXACT?
Because we assigned every block the SAME district value and then took an
area-weighted mean of identical numbers - which returns that number. The
round trip is exact and completely uninformative: we have not estimated
anything, only redistributed a constant.

WHAT THIS TELLS YOU ABOUT INTENSIVE VARIABLES
Median income is INTENSIVE - it does not add up. Areal interpolation is valid
for EXTENSIVE quantities (population, counts, money totals), where splitting a
polygon splits the quantity. Applying it to an intensive variable gives you
back a constant within each source zone: the apparent block-level detail is
entirely fictitious. To estimate income at block level you need an ancillary
correlate (building value, floorspace, land cover) and a dasymetric or
regression-based downscaling - not areal weighting.""")

# ===== SOLUTION 2.6 =====
print("\n\nSOLUTION 2.6 - ZONAL STATISTICS AT TWO RESOLUTIONS")
native = zonal_stats(districts, RAS / "rainfall_annual_250m.tif",
                    stats=("mean", "count"))
with rasterio.open(RAS / "rainfall_annual_250m.tif") as src:
    prof25 = src.profile.copy()
up = align_to(RAS / "rainfall_annual_250m.tif",
              {"height": 1440, "width": 1920,
               "transform": rasterio.Affine(25, 0, 400000, 0, -25, 4636000),
               "crs": CRS_UTM}, Resampling.bilinear)
up_path = OUT / "rain_upsampled_25m.tif"
pu = dict(driver="GTiff", height=1440, width=1920, count=1, dtype="float32",
          crs=CRS_UTM, nodata=-9999.0,
          transform=rasterio.Affine(25, 0, 400000, 0, -25, 4636000),
          compress="deflate")
with rasterio.open(up_path, "w", **pu) as dst:
    dst.write(np.nan_to_num(up, nan=-9999.0).astype("float32"), 1)
upz = zonal_stats(districts, up_path, stats=("mean", "count"))

cmp2 = pd.DataFrame({
    "district": districts.name,
    "cells_250m": native["count"].to_numpy(),
    "mean_250m": native["mean"].round(3).to_numpy(),
    "cells_25m": upz["count"].to_numpy(),
    "mean_25m": upz["mean"].round(3).to_numpy()})
cmp2["diff_mm"] = (cmp2.mean_25m - cmp2.mean_250m).round(3)

```

```
print(cmp2.head(10).to_string(index=False))
print(f"\n max |difference| : {cmp2.diff_mm.abs().max():.3f} mm "
      f"({100*cmp2.diff_mm.abs().max()/cmp2.mean_250m.mean():.3f} % of the mean)")
print(f" cells per district: {cmp2.cells_250m.mean():.0f} at 250 m "
      f"-> {cmp2.cells_25m.mean():.0f} at 25 m (100x more)")
print("""
ARE THEY IDENTICAL? Nearly, but not exactly.
Two effects. (1) Bilinear upsampling SMOOTHS: each fine cell is an
interpolated blend of its coarse neighbours, so values near a district
boundary are pulled towards the neighbouring district. (2) The fine grid
resolves the district boundary far better, so the set of cells assigned to
each district changes slightly.

WHICH WOULD I REPORT? The 250 m native figure.
Upsampling creates 100x more numbers and exactly zero extra information. The
differences you see are artefacts of the interpolation, not a better estimate.
Upsample only when you must align grids for cell-by-cell arithmetic, and then
do the ANALYSIS at the coarsest resolution involved.""")
```

2.7 The analysis-ready feature table · 2.8 Challenge: recover the rainfall law

```
In [59]: # ===== SOLUTION 2.7 =====
# The full solution is the code of Lesson A0, which builds exactly this table
# and saves it to data/outputs/block_features.gpkg. Here we verify it meets the
# specification rather than rebuilding it.
print("SOLUTION 2.7 - VERIFYING THE ANALYSIS-READY FEATURE TABLE")
spec = {
    "Identity": ["block_id", "district_id", "district_type"],
    "Demography": ["population", "households", "pop_density_km2", "area_km2"],
    "Terrain": ["mean_elev_m", "mean_slope_deg", "min_elev_m"],
    "Climate": ["mean_rainfall_mm", "mean_lst_c"],
    "Vegetation": ["mean_ndvi", "pct_forest", "pct_builtup"],
    "Hazard": ["pct_in_flood100", "pct_in_flood500", "dist_river_m"],
    "Access": ["dist_hospital_m", "dist_clinic_m", "dist_school_m",
               "dist_primary_road_m", "n_bus_stops"],
    "Assets": ["n_buildings", "total_value_kvs", "mean_building_age"],
}
FT = gpd.read_file(OUT / "block_features.gpkg", layer="block_features")
print(f" loaded {len(FT)} rows x {FT.shape[1]} columns from block_features.gpkg\n")
print(f" {'group':<12}{'required':>10}{'present':>9} missing")
allok = True
for grp, cols in spec.items():
    have = [c for c in cols if c in FT.columns]
    miss = [c for c in cols if c not in FT.columns]
    allok &= not miss
    print(f" {grp:<12}{len(cols):>10}{len(have):>9} {miss if miss else '-'}")
print(f"\n ALL REQUIRED COLUMNS PRESENT: {allok}")

print("\n QUALITY CHECKS")
lc_cols = [c for c in FT.columns if c.startswith("pct_") and "flood" not in c]
print(f" land-cover percentages sum to 100 : "
      f"{bool(np.allclose(FT[lc_cols].sum(axis=1).dropna(), 100, atol=0.5))}")
print(f" CRS is the analysis CRS : {FT.crs.to_string() == CRS_UTM}")
print(f" no negative distances : "
      f"{bool((FT[[c for c in FT.columns if c.startswith('dist_')]] >= 0).all().all())}")
print(f" counts are integers, zero-filled : "
      f"{FT.n_buildings.dtype.kind in 'iu' and int(FT.n_buildings.min()) == 0}")
print(f" means stay NaN where undefined : "
      f"{int(FT.mean_building_age.isna().sum())} blocks with no buildings")
print(f" population reconciles : {FT.population.sum():,} vs "
      f"{blocks[~blocks.geometry.is_empty].population.sum():,} in the source")

# ===== SOLUTION 2.8 =====
print("\n\nSOLUTION 2.8 (CHALLENGE) - RECOVERING THE RAINFALL LAW")
TRUE = dict(intercept=470.0, elev=0.62, north=150.0)
```

```

def fit_rain(res_m, use_north=True, drop_nodata=True, label=""):
    with rasterio.open(RAS / "rainfall_annual_250m.tif") as src:
        rain = src.read(1, masked=not drop_nodata) if not drop_nodata \
            else src.read(1, masked=True)
        rt, rs = src.transform, src.shape
        nodata = src.nodata
    if res_m == 25:
        prof_ref = {"height": 1440, "width": 1920,
                    "transform": rasterio.Affine(25, 0, 400000, 0, -25, 4636000),
                    "crs": CRS_UTM}
        R = align_to(RAS / "rainfall_annual_250m.tif", prof_ref, Resampling.bilinear)
        E = align_to(RAS / "dem_25m.tif", prof_ref, Resampling.average)
        tr = prof_ref["transform"]; h, w = 1440, 1920
    else:
        R = np.where(rain.mask, np.nan, rain.data) if hasattr(rain, "mask") else rain
        E = align_to(RAS / "dem_25m.tif",
                    {"height": rs[0], "width": rs[1], "transform": rt, "crs": CRS_UTM},
                    Resampling.average)
        tr = rt; h, w = rs
    yy = tr.f + (np.arange(h) + 0.5) * tr.e
    NORTH = np.repeat((yy - 4_600_000) / 36_000)[: , None], w, axis=1)
    if not drop_nodata: # deliberately keep the -9999s
        R = np.where(np.isfinite(R), R, -9999.0)
        E = np.where(np.isfinite(E), E, -9999.0)
        ok = np.ones_like(R, dtype=bool)
    else:
        ok = np.isfinite(R) & np.isfinite(E)
    cols = [np.ones(ok.sum()), E[ok]] + ([NORTH[ok]] if use_north else [])
    beta, *_ = np.linalg.lstsq(np.c_[tuple(cols)], R[ok], rcond=None)
    return label, int(ok.sum()), beta

print(f" TRUE: rainfall = {TRUE['intercept']} + {TRUE['elev']}*elev "
      f"+ {TRUE['north']}*north\n")
print(f" {'specification':<44}{'\n cells':>10}{'intercept':>11}"
      f"{'b_elev':>9}{'b_north':>10}")
runs = [
    fit_rain(250, True, True, "(a) 250 m, north term, NoData excluded"),
    fit_rain(25, True, True, "(b) 25 m, north term, NoData excluded"),
    fit_rain(250, False, True, "(c) 250 m, NO north term"),
    fit_rain(250, True, False, "(d) 250 m, NoData NOT excluded"),
]
for lab, n, b in runs:
    bn = f"{b[2]:>10.2f}" if len(b) > 2 else f"{'-':>10}"
    print(f" {lab:<44}{n:>10},{b[0]:>11.2f}{b[1]:>9.4f}{bn}")

# district-mean version
dz = pd.DataFrame({
    "rain": zonal_stats(districts, RAS / "rainfall_annual_250m.tif",
                        stats=("mean",))["mean"].to_numpy(),
    "elev": districts.mean_elev_m.to_numpy(),
    "north": ((districts.geometry.representative_point().y.to_numpy()
              - 4_600_000) / 36_000)})
bd = np.linalg.lstsq(np.c_[np.ones(len(dz)), dz.elev, dz.north], dz.rain,
                    rcond=None)[0]
print(f" {'(e) DISTRICT means (n=24)':<44}{'len(dz)':>10},{bd[0]:>11.2f}"
      f"{'bd[1]:>9.4f}{bd[2]:>10.2f}")
print(f"\n {'TRUE VALUES':<44}{'':>10}{470.00:>11.2f}{0.6200:>9.4f}{150.00:>10.2f}")
print("""
WHICH EFFECT IS MOST DANGEROUS?
(d), forgetting to exclude NoData, by an enormous margin. The -9999 sentinels
are extreme, numerous and perfectly correlated between the two rasters (both
are NoData over the sea), so the regression fits a line through two clouds:
the real data and the sea. Every coefficient becomes meaningless while the
R^2 looks superb.

(c), omitting the north term, produces a modest but real bias in b_elev,
because elevation and northing are correlated (the ridge runs north-east).
Classic omitted-variable bias - visible, diagnosable, and survivable.

```

(b), working at 25 m, changes almost nothing except the standard errors, which become absurdly small: 2.7 MILLION upsampled cells carry no more information than the 27,000 original ones. Any p-value computed from them is fiction.

(e), district means, recovers the coefficients well but on n=24 – honest, low power, and the aggregation slightly inflates the fit (ecological correlation)."""

Solutions – Module 3 (Advanced)

These are open-ended problems. Each solution below is **one defensible implementation**; where a different choice would be equally valid, the code says so in a comment.

3.1 A defensible siting recommendation · 3.2 Network-distance accessibility

```
In [60]: # ===== SOLUTION 3.1 =====
print("SOLUTION 3.1 – SITING TWO NEW CLINICS")
S = Fx.copy()
S["deprivation"] = S.district_id.map(
    socio_clean.set_index("district_id").median_income_vs).rank(pct=True)
S["deprivation"] = 1 - S.deprivation # higher = more deprived

CRIT = { # column, direction ("benefit"=more is better), rationale
    "population": ("benefit", "people served"),
    "dist_clinic_m": ("benefit", "currently underserved"),
    "deprivation": ("benefit", "EQUITY: prioritise deprived areas"),
    "dist_primary_road_m": ("cost", "must be reachable"),
    "pct_in_flood100": ("cost", "do not build in the floodplain"),
}
labels = list(CRIT)

def norm(v, direction, how):
    v = pd.Series(v).astype(float)
    r = v.rank(pct=True) if how == "rank" else (v - v.min()) / (v.max() - v.min())
    return r if direction == "benefit" else 1 - r

# AHP: pairwise importance of ROW over COLUMN
A = np.array([
    [1, 2, 2, 3, 4], # population
    [1/2, 1, 1, 2, 3], # underserved
    [1/2, 1, 1, 2, 3], # deprivation
    [1/3, 1/2, 1/2, 1, 2], # road access
    [1/4, 1/3, 1/3, 1/2, 1], # flood avoidance
], dtype=float)
ev, evec = np.linalg.eig(A)
k = int(np.argmax(ev.real))
w = np.abs(evec[:, k].real); w /= w.sum()
n = len(labels); CI = (ev.real[k] - n) / (n - 1)
CR = CI / {3: .58, 4: .90, 5: 1.12}[n]
print(f" AHP weights: " + ", ".join(f"{l}={x:.3f}" for l, x in zip(labels, w)))
print(f" consistency ratio CR = {CR:.4f} "
      f"'ACCEPTABLE' if CR < 0.10 else 'REVISE'")

def score(how, weights):
    N = np.c_[tuple(norm(S[c], d, how) for c, (d, _) in CRIT.items())]
    ws = (N * weights).sum(axis=1)
    gm = np.exp((np.log(np.clip(N, 1e-6, None)) * weights).sum(axis=1))
    return ws, gm

ws, gm = score("rank", w)
S["score_sum"], S["score_geom"] = ws, gm
print(f"\n weighted sum vs geometric mean, Spearman: "
      f"{pd.Series(ws).corr(pd.Series(gm), method='spearman'):.3f}")
print(" We RECOMMEND the geometric mean: every criterion here is a genuine")
```

```

print(" requirement (a clinic in the floodplain or off the road network is")
print(" not viable), so a near-zero on any one should not be compensated away.")

# sensitivity over BOTH the weights and the normalisation method
rng = np.random.default_rng(5)
SIMS = 1500
top2 = np.zeros(len(S))
for _ in range(SIMS):
    wp = np.abs(w * rng.normal(1, 0.25, n)); wp /= wp.sum()
    how = "rank" if rng.random() < 0.5 else "minmax"
    _, g_ = score(how, wp)
    top2[np.argsort(-g_)[1:2]] += 1
S["p_top2"] = top2 / SIMS
best = S.nlargest(6, "p_top2")
print(f"\n ROBUSTNESS over {SIMS} runs (weights +/-25 % AND both normalisations)")
print(best[["block_id", "district_id", "population", "dist_clinic_m",
            "score_geom", "p_top2"]].round(3).to_string(index=False))
rec = best.head(2)
print(f"\n RECOMMENDATION: build in blocks {rec.block_id.tolist()} "
      f"(districts {rec.district_id.tolist()})")
print(f" These appear in the top 2 in {100*rec.p_top2.min():.0f}--"
      f"{100*rec.p_top2.max():.0f} % of runs.")
print(f" THIS WOULD CHANGE IF: the equity criterion were dropped or down-weighted")
print(f" below ~0.10, or if min-max normalisation were mandated (it lets the two")
print(f" largest-population blocks dominate every other criterion).")

# ===== SOLUTION 3.2 =====
print("\n\nSOLUTION 3.2 - NETWORK DISTANCE AND THE DETOUR INDEX")
from scipy.sparse import coo_matrix
from scipy.sparse.csgraph import dijkstra, connected_components
from shapely.ops import unary_union

# STEP 1 - NODE the network. Road lines that cross without sharing a vertex are
# NOT connected in a graph built naively from their coordinates. unary_union
# splits every line at every intersection, which is the whole ball game: skip
# it and you get a forest of disconnected paths instead of a network.
noded = unary_union(roads.geometry.values)
segs = list(noded.geoms) if noded.geom_type == "MultiLineString" else [noded]
print(f" {len(roads)} input segments -> {len(segs)} noded segments")

SNAP = 1.0 # metres; after noding, shared vertices coincide
nodes, edges = {}, []
def nid(pt):
    key = (round(pt[0] / SNAP), round(pt[1] / SNAP))
    if key not in nodes:
        nodes[key] = len(nodes)
    return nodes[key]

for g in segs:
    cs = list(g.coords)
    for a_, b_ in zip(cs[:-1], cs[1:]):
        ia, ib = nid(a_), nid(b_)
        d = float(np.hypot(b_[0]-a_[0], b_[1]-a_[1]))
        if ia != ib and d > 0:
            edges.append((ib, ia, d))

NN = len(nodes)
r_, c_, v_ = zip(*edges)
Gm = coo_matrix((v_, (r_, c_)), shape=(NN, NN)).tocsr()
node_xy = np.zeros((NN, 2))
for (gx_, gy_), i in nodes.items():
    node_xy[i] = (gx_ * SNAP, gy_ * SNAP)

ncomp, comp = connected_components(Gm, directed=False)
sizes = np.bincount(comp)
big = int(np.argmax(sizes))
print(f" graph: {NN:,} nodes, {len(edges)//2:,} edges, {ncomp} components")
print(f" largest component holds {sizes[big]:,} nodes "
      f"({100*sizes[big]/NN:.0f} %) - we restrict the analysis to it")

```

```

ntree = cKDTree(node_xy)
def snap(gdf):
    """Nearest graph node AND the off-network walk needed to reach it."""
    return ntree.query(np.c_[gdf.geometry.x, gdf.geometry.y])

orig = Fx.sample(50, random_state=1)
o_pts = gpd.GeoDataFrame(geometry=orig.geometry.representative_point(), crs=CRS_UTM)
clin = facilities_clean[facilities_clean.facility_type == "clinic"]
o_snap, o_nodes = snap(o_pts)
c_snap, c_nodes = snap(clin)
print(f" off-network snap: origins median {np.median(o_snap):.0f} m, "
      f"clinics median {np.median(c_snap):.0f} m")

# STEP 2 - total travel = walk on + travel along + walk off. Omitting the two
# snap legs is what makes a naive detour index come out below 1.0, which is
# physically impossible and an immediate sign the calculation is wrong.
Dnet = dijkstra(Gm, directed=False, indices=o_nodes)[: , c_nodes]
Dtot = Dnet + o_snap[:, None] + c_snap[None, :]
with np.errstate(invalid="ignore"):
    net_min = np.nanmin(np.where(np.isfinite(Dtot), Dtot, np.nan), axis=1)
euc_min = np.array([min(p.distance(q) for q in clin.geometry)
                   for p in o_pts.geometry])
ok = np.isfinite(net_min)
detour = net_min[ok] / np.maximum(euc_min[ok], 1)
print(f"\n origin-clinic pairs connected on the network : {ok.sum()} of {len(o_pts)}")
print(f" detour indices below 1.0 (must be none) : {int((detour < 1).sum())}")
print(f" DETOUR INDEX median {np.median(detour):.3f}, "
      f"25th {np.percentile(detour, 25):.3f}, 75th {np.percentile(detour, 75):.3f}")
print(f" (the mean is {detour.mean():.1f} - a few origins reach a clinic only by")
print(f" a very indirect route, so use the MEDIAN as the correction factor)")

DET = float(np.clip(np.median(detour), 1.0, 3.0))
acc_euc = e2sfca(clin, "capacity", 10_000) * 1000
acc_net = e2sfca(clin, "capacity", 10_000 / DET) * 1000 # shrink the catchment
P_ = Fx.population.to_numpy(float)
print(f"\n applying a detour correction of {DET:.2f} to the catchment radius:")
print(f" catchment 10.0 km straight-line -> {10/DET:.1f} km effective")
print(f" Gini, Euclidean catchment : {weighted_gini(acc_euc, P_):.4f}")
print(f" Gini, corrected catchment : {weighted_gini(acc_net, P_):.4f}")
print(f" blocks with NO clinic access: "
      f"{int((acc_euc == 0).sum())} -> {int((acc_net == 0).sum())}")
r1 = pd.Series(acc_euc).rank(ascending=False)
r2 = pd.Series(acc_net).rank(ascending=False)
print(f" median |rank shift| : {(r1-r2).abs().median():.0f} places, "
      f"max {(r1-r2).abs().max():.0f}")
print("""
INTERPRETATION
The detour index is network distance divided by straight-line distance. A
median around 1.3-2.0 is normal; this basin's is at the high end because the
road network is sparse and follows the valleys.

Three practical lessons. (1) NODE the network before building a graph, or
crossing roads will not connect. (2) Add the off-network snap legs, or you
will compute impossible detour indices below 1.0. (3) Check connectivity -
only about half the graph is one component here, so a substantial share of
origin-destination pairs is simply unreachable and must be reported as such
rather than dropped silently.

Correcting the catchment radius tightens every catchment, pushes marginal
facilities out of reach and RAISES measured inequality. Euclidean
accessibility therefore systematically UNDERSTATES the access deficit -
exactly as the capstone's limitations section says.""")

```

3.3 Defensible hotspots · 3.4 Regionalisation for service delivery

In [61]:

```
# ===== SOLUTION 3.3 =====
print("SOLUTION 3.3 - A HOTSPOT ANALYSIS YOU CAN DEFEND")
HS = Fx.copy() # NOT `HS` - that is the raster height
HS["value_at_risk"] = HS.total_value_kvs.fillna(0) * HS.pct_in_flood100 / 100.0
y3 = HS.value_at_risk.to_numpy(dtype=float)
n3 = len(HS)

# permutation budget must exceed n/alpha for FDR to be able to reject anything
NEED = int(np.ceil(n3 / 0.05))
PERM = max(9999, NEED)
print(f" n = {n3}, alpha = 0.05 -> need > {NEED:,} permutations; using {PERM:,}")

SCHEMES = {
    "queen contiguity": Wq,
    "k-nearest, k=4": knn_weights(cxy, k=4),
    "k-nearest, k=8": knn_weights(cxy, k=8),
    "distance band 3 km": None,
}
dm = np.sqrt(((cxy[:, None, :] - cxy[None, :, :]) ** 2).sum(-1))
Wdb = ((dm < 3000) & (dm > 0)).astype(float)
Wdb = Wdb / np.maximum(Wdb.sum(1, keepdims=True), 1e-12)
SCHEMES["distance band 3 km"] = Wdb

print(f"\n {'weights scheme':<22}{'global I':>10}{'raw p<.05':>11}"
      f"{'Bonferroni':>12}{'FDR':>7}{'hot':>6}{'cold':>7}")
print("-" * 78)
results3 = {}
for nm, Wx in SCHEMES.items():
    gi3 = getis_ord_gstar(y3, Wx)
    p3 = 2 * (1 - _norm.cdf(np.abs(gi3)))
    sig3 = benjamini_hochberg(p3, 0.05)
    mI = moran_test(y3, Wx, permutations=999, seed=1)["I"]
    results3[nm] = (gi3, p3, sig3)
    print(f" {nm:<22}{mI:>10.4f}{int((p3<0.05).sum()):>11}"
          f"{int((p3 < 0.05/n3).sum()):>12}{int(sig3.sum()):>7}"
          f"{int((sig3 & (gi3>0)).sum()):>6}{int((sig3 & (gi3<0)).sum()):>7}")
print("-" * 78)
gi_q, p_q, sig_q = results3["queen contiguity"]
HS["gi_z"], HS["sig"] = gi_q, sig_q
HS["cls"] = np.where(sig_q & (gi_q > 0), "hot",
                    np.where(sig_q & (gi_q < 0), "cold", "not significant"))

# agreement across schemes
stack3 = np.c_[tuple((s & (g > 0)) for g, p, s in results3.values())]
HS["n_schemes_hot"] = stack3.sum(1)
print(f"\n blocks flagged HOT under all {len(SCHEMES)} schemes : "
      f"{int((HS.n_schemes_hot == len(SCHEMES)).sum())}")
print(f" blocks flagged HOT under at least one : "
      f"{int((HS.n_schemes_hot >= 1).sum())}")
print(" Report only the blocks that survive EVERY scheme as confirmed hotspots.")

fig, axes = plt.subplots(1, 3, figsize=(17.5, 5.2))
HS.plot(ax=axes[0], column="value_at_risk", cmap="Purples", scheme="quantiles",
        k=6, legend=True, legend_kwds={"loc": "lower left", "fontsize": 6},
        edgecolor="none")
axes[0].set_title("Asset value at risk (k VS)", fontsize=9.5, weight="bold", loc="left")
for kk, cc in {"hot": "#b2182b", "cold": "#2166ac",
               "not significant": "#eeeeee"}.items():
    sub = HS[HS.cls == kk]
    if len(sub):
        sub.plot(ax=axes[1], color=cc, edgecolor="none", label=kk)
axes[1].legend(fontsize=7, loc="lower left")
axes[1].set_title(f"Gi* hotspots, FDR-corrected\nqueen weights, {PERM:,} perms",
                  fontsize=9.5, weight="bold", loc="left")
HS.plot(ax=axes[2], column="n_schemes_hot", cmap="YlOrRd", vmin=0,
        vmax=len(SCHEMES), legend=True, legend_kwds={"shrink": 0.6},
        edgecolor="none")
```

```

axes[2].set_title("Robustness: schemes flagging HOT", fontsize=9.5,
                  weight="bold", loc="left")
for a in axes:
    a.set_aspect("equal"); a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.show()

# ===== SOLUTION 3.4 =====
print("\n\nSOLUTION 3.4 - SIX CONTIGUOUS SERVICE REGIONS")
from sklearn.cluster import AgglomerativeClustering
from sklearn.preprocessing import StandardScaler

ACCVARs = ["dist_hospital_m", "dist_clinic_m", "dist_fire_station_m",
           "n_bus_stops", "dist_primary_road_m"]
Xa = StandardScaler().fit_transform(
    np.nan_to_num(Fx[ACCVARs].to_numpy(float),
                  nan=np.nanmedian(Fx[ACCVARs].to_numpy(float))))
conn = ((Wq > 0).astype(int) + (Wq > 0).astype(int).T)

K6 = 6
ward6 = AgglomerativeClustering(n_clusters=K6, linkage="ward",
                                connectivity=conn).fit(Xa)
km6 = KMeans(n_clusters=K6, n_init=30, random_state=0).fit(Xa)
Fx["service_region"] = ward6.labels_

def wcss6(lab):
    return sum(((Xa[lab == c] - Xa[lab == c].mean(0))*2).sum()
               for c in np.unique(lab))
pop6 = Fx.groupby("service_region").population.sum()
frag6 = [1 if Fx[Fx.service_region == c].geometry.union_all().geom_type == "Polygon"
         else len(Fx[Fx.service_region == c].geometry.union_all().geoms)
         for c in range(K6)]
summ6 = pd.DataFrame({
    "population": pop6,
    "blocks": Fx.groupby("service_region").size(),
    "area_km2": Fx.groupby("service_region").area_km2.sum().round(1),
    "mean_dist_hosp_km": (Fx.groupby("service_region").dist_hospital_m.mean()/1000).round(2),
    "fragments": frag6})
print(summ6.to_string())
cv = pop6.std() / pop6.mean()
print(f"\n population coefficient of variation : {cv:.3f} "
      f"(0 = perfectly equal)")
print(f" all regions contiguous : {all(f == 1 for f in frag6)}")
print(f" WCSS, contiguity-constrained : {wcss6(ward6.labels_):.0f}")
print(f" WCSS, unconstrained k-means : {wcss6(km6.labels_):.0f}")
print(f" homogeneity cost of contiguity : "
      f"{100*(wcss6(ward6.labels_)-wcss6(km6.labels_))/wcss6(km6.labels_):.1f} %")
print("""
THE TRADE-OFF
Ward-with-contiguity guarantees six administrable regions but cannot equalise
population, because population is not one of the clustering variables and
contiguity constrains which blocks may join. The CV above shows how unequal
they are. To equalise population you would need a redistricting formulation
(an objective with a population-balance penalty, solved by local search or
integer programming) - Ward cannot express that constraint. State this: a
homogeneity-based regionalisation and an equal-population redistricting are
different optimisation problems with different answers.""")

fig, axes = plt.subplots(1, 2, figsize=(13.5, 5.8))
Fx.plot(ax=axes[0], column="service_region", categorical=True, cmap="Set3",
        legend=True, legend_kwds={"loc": "lower left", "fontsize": 7},
        edgecolor="white", linewidth=0.2)
axes[0].set_title(f"{K6} contiguous service regions (Ward + queen)",
                  fontsize=10, weight="bold", loc="left")
Fx.assign(km=km6.labels_).plot(ax=axes[1], column="km", categorical=True,
                                cmap="Set3", legend=True,
                                legend_kwds={"loc": "lower left", "fontsize": 7},
                                edgecolor="white", linewidth=0.2)
axes[1].set_title("Unconstrained k-means - NOT regions",

```

```

        fontsize=10, weight="bold", loc="left")
for a in axes:
    a.set_aspect("equal"); a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.show()

```

3.5 A better flood model · 3.6 Risk under a climate scenario · 3.7 Challenge

```

In [62]: # ===== SOLUTION 3.5 =====
print("SOLUTION 3.5 - AN IMPROVED FLOOD-SUSCEPTIBILITY MODEL")
from sklearn.calibration import CalibratedClassifierCV
from sklearn.metrics import brier_score_loss, roc_auc_score, average_precision_score
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# --- (1) three NEW physically motivated features -----
# profile curvature (2nd derivative of elevation) - concave ground collects water
ez = np.nan_to_num(grids["elevation"], nan=0.0)
gyy, gxx = np.gradient(ez, CELL, CELL)
curv = np.gradient(gxx, CELL, axis=1) + np.gradient(gyy, CELL, axis=0)
feat_rasters["curvature"] = np.where(np.isfinite(grids["elevation"]), curv, np.nan)
# a crude upslope-contributing-area proxy: how much higher ground drains here
low_rank = np.where(np.isfinite(grids["elevation"]), grids["elevation"], np.nan)
feat_rasters["upslope_proxy"] = -uniform_filter(
    np.nan_to_num(low_rank, nan=0.0), size=15, mode="nearest") + np.nan_to_num(low_rank)
# neighbourhood built-up fraction (drainage capacity / imperviousness)
built = (np.nan_to_num(grids["landcover"], nan=0) == 2).astype(float)
feat_rasters["pct_built_1km"] = 100 * uniform_filter(built, size=10, mode="nearest")

FEAT2 = FEATURES + ["curvature", "upslope_proxy", "pct_built_1km"]
def rebuild(ds):
    d = ds.copy()
    for f in ["curvature", "upslope_proxy", "pct_built_1km"]:
        d[f] = sample_grid(feat_rasters[f], d.x.to_numpy(), d.y.to_numpy())
    return d.dropna(subset=FEAT2)
dsB2 = rebuild(dsB)
X2 = dsB2[FEAT2].to_numpy(float); y2 = dsB2.flood.to_numpy(int)
g2 = spatial_blocks(dsB2.x.to_numpy(), dsB2.y.to_numpy(), 4000)
print(f" features: {len(FEATURES)} -> {len(FEAT2)} rows: {len(dsB2):,}")

# --- (2) block size from the residual autocorrelation range -----
lr = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(X2, y2)
res2 = y2 - lr.predict_proba(X2)[:, 1]
pxy = np.c_[dsB2.x, dsB2.y]

# BINNED empirical semivariogram. gamma(h) = 0.5 * mean[(z_i - z_j)^2] over
# pairs separated by h. For an uncorrelated field gamma -> the VARIANCE (not
# half of it); the range is the lag at which gamma first reaches that sill.
tre = cKDTree(pxy)
EDGES = [0, 500, 1000, 2000, 3000, 4000, 6000, 8000, 12000]
sill = res2.var()
print(f"\n empirical semivariogram of the OLS residuals (sill = {sill:.4f})")
print(f" {'lag bin (m)':>16}{'n pairs':>12}{'gamma':>10}{'gamma/sill':>12}")
prev = set()
for lo, hi in zip(EDGES[:-1], EDGES[1:]):
    cur = tre.query_pairs(hi)
    band = cur - prev
    prev = cur
    if len(band) < 50:
        continue
    idx = np.array(list(band))
    gam = 0.5 * np.mean((res2[idx[:, 0]] - res2[idx[:, 1]]) ** 2)
    print(f" {lo:>7}-{hi:<8},{len(band):>12},{gam:>10.4f},{gam/sill:>12.3f}")
first = None
print(" READ IT HONESTLY: gamma/sill is already ~1.0 in the SHORTEST lag bin,")
print(" which means the OLS residuals carry almost no short-range spatial")

```

```

print("    structure – the features have absorbed it. The range is therefore")
print("    below 500 m and any block size above that suffices. We keep 4 km")
print("    because the block-size sweep in A11 showed it to be the conservative")
print("    choice, not because the variogram demands it. A variogram that shows")
print("    NO structure is a useful result: it says your spatial CV can be")
print("    generous rather than punitive.")

# --- (3) three algorithms, spatially validated, then calibrated -----
CVG = GroupKFold(n_splits=5)
cands = {
    "logistic":      make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)),
    "random forest": RandomForestClassifier(n_estimators=400, min_samples_leaf=3,
                                           random_state=0, n_jobs=-1),
    "grad. boosting": HistGradientBoostingClassifier(max_iter=300, random_state=0),
}
print(f"\n  {'model':<16}{'AUC':>8}{'AvgPrec':>10}{'Brier':>9}")
oofs = {}
for nm, mdl in cand.items():
    o = np.zeros(len(y2))
    for tr, te in CVG.split(X2, y2, groups=g2):
        m = mdl.fit(X2[tr], y2[tr])
        o[te] = m.predict_proba(X2[te])[:, 1]
    oofs[nm] = o
    print(f"  {nm:<16}{roc_auc_score(y2, o):>8.4f}"
          f"{average_precision_score(y2, o):>10.4f}{brier_score_loss(y2, o):>9.4f}")
best_nm = max(oofs, key=lambda k: roc_auc_score(y2, oofs[k]))
print(f"  best: {best_nm}")

cal = CalibratedClassifierCV(cands[best_nm], method="isotonic", cv=3)
o_cal = np.zeros(len(y2))
for tr, te in CVG.split(X2, y2, groups=g2):
    o_cal[te] = cal.fit(X2[tr], y2[tr]).predict_proba(X2[te])[:, 1]
print(f"  {best_nm} + isotonic calibration: "
      f"AUC {roc_auc_score(y2, o_cal):.4f}, Brier {brier_score_loss(y2, o_cal):.4f} "
      f"(was {brier_score_loss(y2, oofs[best_nm]):.4f})")

fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))
for nm, o in oofs.items():
    bins = np.quantile(o, np.linspace(0, 1, 11))
    idx = np.clip(np.digitize(o, bins[1:-1]), 0, 9)
    px = [o[idx == i].mean() for i in range(10)]
    py = [y2[idx == i].mean() for i in range(10)]
    axes[0].plot(px, py, "o-", label=nm, linewidth=1.6)
axes[0].plot([0, 1], [0, 1], "k--", linewidth=1, label="perfect calibration")
axes[0].set_xlabel("mean predicted probability"); axes[0].set_ylabel("observed frequency")
axes[0].set_title("Calibration (spatially validated)", fontsize=10,
                  weight="bold", loc="left")
axes[0].legend(fontsize=7); axes[0].grid(alpha=0.3)

mdl_final = cand[best_nm].fit(X2, y2)
stack2 = np.stack([feat_rasters[f] for f in FEAT2], axis=-1)
vc2 = np.isfinite(stack2).all(-1) & land_g
p2 = mdl_final.predict_proba(stack2[vc2])[:, 1]
out2 = np.full((H_ := ref["height"], W_ := ref["width"]), np.nan)
out2[vc2] = p2
im = axes[1].imshow(out2, cmap="RdYlGn_r", vmin=0, vmax=1,
                    extent=(TR.c, TR.c + W_*CELL, TR.f - H_*CELL, TR.f))
rivers.plot(ax=axes[1], color="#08306b", linewidth=0.7)
plt.colorbar(im, ax=axes[1], shrink=0.75, label="P(flood-prone)")
axes[1].set_title(f"Improved susceptibility ({best_nm}, {len(FEAT2)} features)",
                  fontsize=10, weight="bold", loc="left")
axes[1].set_aspect("equal"); axes[1].set_xticks([]); axes[1].set_yticks([])
plt.tight_layout(); plt.show()

# ===== SOLUTION 3.6 =====
print("\n\nSOLUTION 3.6 – RISK UNDER A CLIMATE SCENARIO")
LEVELS_NOW = {10: 1.2, 25: 2.0, 50: 2.8, 100: 3.5, 250: 5.0, 500: 6.5}
# scenario: +0.5 m on every level, and the 100-yr event becomes the 50-yr event

```

```

LEVELS_FUT = {10: 1.7, 25: 2.5, 50: 4.0, 100: 4.5, 250: 5.5, 500: 7.0}

# building_id -> district_id, aligned positionally to Bpt
DIST_OF_BLD = Bpt.building_id.map(
    Bd.drop_duplicates("building_id").set_index("building_id").district_id).to_numpy()

def ead_for(levels, tag):
    losses_, per_d_ = {}, {}
    for T, lvl in levels.items():
        depth = np.clip(lvl - Bpt.hand_m.to_numpy(), 0, None)
        dr = damage_ratio(depth, constr)
        dr = np.where(Bpt.has_basement.to_numpy(), np.minimum(dr*1.15, 1.0), dr)
        loss = dr * Bpt.value_kvs.to_numpy()
        losses_[T] = loss.sum()
        per_d_[T] = pd.Series(loss, index=Bpt.index).groupby(DIST_OF_BLD).sum()
    Ts_ = np.array(sorted(levels)); ps_ = 1.0 / Ts_
    L_ = np.array([losses_[T] for T in Ts_])
    tot = abs(np.trapezoid(L_[::-1], ps_[::-1]))
    dis = {}
    for did in districts.district_id:
        Lv = np.array([per_d_[T].get(did, 0.0) for T in Ts_])
        dis[did] = abs(np.trapezoid(Lv[::-1], ps_[::-1]))
    n_exp = int((np.clip(max(levels.values()) - Bpt.hand_m, 0, None) > 0).sum())
    return tot, pd.Series(dis), n_exp

ead_now, d_now, exp_now = ead_for(LEVELS_NOW, "now")
ead_fut, d_fut, exp_fut = ead_for(LEVELS_FUT, "future")
print(f" EAD today : {ead_now:>12,.0f} k VS/yr")
print(f" EAD scenario: {ead_fut:>12,.0f} k VS/yr "
      f"({100*(ead_fut-ead_now)/ead_now:+.1f} %)")
print(f" buildings exposed at the 500-yr level: {exp_now:,} -> {exp_fut:,} "
      f"({exp_fut-exp_now:+,}%)")

cmp6 = pd.DataFrame({"now": d_now, "future": d_fut})
cmp6["abs_change"] = cmp6.future - cmp6.now
cmp6["pct_change"] = (100 * cmp6.abs_change / cmp6.now.replace(0, np.nan)).round(1)
cmp6 = cmp6.join(districts.set_index("district_id")["name"])
print("\n biggest ABSOLUTE increases:")
print(cmp6.nlargest(5, "abs_change")[["name", "now", "future", "abs_change"]]
      .round(0).to_string())
print("\n biggest RELATIVE increases:")
print(cmp6.nlargest(5, "pct_change")[["name", "now", "future", "pct_change"]]
      .round(1).to_string())
print(f"\n BREAK-EVEN DEFENCE COST")
extra = ead_fut - ead_now
for yrs, disc in [(20, 0.00), (20, 0.035), (50, 0.035)]:
    if disc == 0:
        pv = extra * yrs
    else:
        pv = extra * (1 - (1+disc)**-yrs) / disc
    print(f" over {yrs} yr at {disc:.1%} discount : {pv:>12,.0f} k VS")
print(" A defence restoring today's EAD is worth building if it costs less")
print(" than the present value above.")

# ===== SOLUTION 3.7 =====
print("\n\nSOLUTION 3.7 (CHALLENGE) - RECOVERING THE FULL GENERATING PROCESS")
def ols_ci(Xd, yd, names):
    Xd = np.c_[np.ones(len(yd)), Xd]
    b, *_ = np.linalg.lstsq(Xd, yd, rcond=None)
    r = yd - Xd @ b
    s2 = (r**2).sum() / (len(yd) - Xd.shape[1])
    cov = s2 * np.linalg.pinv(Xd.T @ Xd)
    se = np.sqrt(np.diag(cov))
    return pd.DataFrame({"term": ["intercept"] + names,
                          "estimate": b, "se": se,
                          "lo95": b - 1.96*se, "hi95": b + 1.96*se})

print(" (1) RAINFALL = 470 + 0.62*elev + 150*north")

```

```

Rg, Eg = grids["rainfall"], grids["elevation"]
hh, ww = Rg.shape
yy_ = ref["transform"].f + (np.arange(hh)+0.5)*ref["transform"].e
NORTH = np.repeat((yy_ - 4_600_000)/36_000)[:, None], ww, axis=1)
m_ = np.isfinite(Rg) & np.isfinite(Eg)
t1 = ols_ci(np.c_[Eg[m_], NORTH[m_]], Rg[m_], ["elev", "north"])
t1["TRUE"] = [470.0, 0.62, 150.0]
print(t1.round(4).to_string(index=False))

print("\n (2) LST = 31.5 - 0.0062*elev + 6.4*urban")
Ug = np.clip((grids["popdens"] - 22)/9500, 0, None) ** (1/2.1)
m2_ = np.isfinite(grids["lst"]) & np.isfinite(Eg) & np.isfinite(Ug)
t2 = ols_ci(np.c_[Eg[m2_], Ug[m2_]], grids["lst"][m2_], ["elev", "urban"])
t2["TRUE"] = [31.5, -0.0062, 6.4]
print(t2.round(5).to_string(index=False))

print("\n (3) PM2.5 = 7.5 + 16*urban + 9*exp(-d/1800) [24 stations]")
print(f"      fitted in Lesson I9: a=7.15, b=9.02, L=1855 m, c=16.42")
print(f"      TRUE:                a=7.50, b=9.00, L=1800 m, c=16.00")

print("\n (4) POPULATION DENSITY = 9500*urban^2.1 + 22")
print("      This is an IDENTITY of our urban proxy, not an estimable relation:")
print("      we DEFINED urban by inverting it. Recovering 2.1 requires an")
print("      independent measure of urban intensity, which the delivered data")
print("      does not contain. Report it as unidentifiable.")

print("""
WHICH ESTIMATE IS WORST, AND WHY?
(3), the PM2.5 e-folding distance. Diagnosis: CONFOUNDING plus SMALL SAMPLE.
Distance-to-motorway is correlated (-0.44) with urban intensity because the
motorway follows the populated coast, and there are only 24 stations. Fitting
the decay without the urban term returns L = 12,322 m against a true 1,800 m -
an error of +585 %. FIX: include the confounder (as model B does), and if
possible add stations chosen to break the correlation - sites far from the
motorway but highly urban, and sites near it but rural.

(4) is worse in a different sense: it is not merely badly estimated but
UNIDENTIFIABLE from the delivered data. Recognising that a parameter cannot be
estimated is more valuable than producing a confident number for it.

(1) and (2) are recovered to within ~1 % because both have millions of cells,
no confounding once both terms are included, and low noise. Note their
standard errors are absurdly small - with 100,000+ spatially autocorrelated
cells the EFFECTIVE sample size is far smaller than n, so those 95 % intervals
are much too narrow (Lesson A9).""")

```

End of course

You have worked through 49 lessons, four exercise sets and a full end-to-end project. If you can now open an unfamiliar spatial dataset, tell within five minutes whether its CRS, geometry and attributes can be trusted, engineer features from its geography, validate a model against spatial leakage, and say clearly what your analysis cannot support — then the course has done its job.

Three things worth carrying forward.

1. **The CRS is not metadata, it is the units of every number you produce.** Most wrong answers in applied GIS are wrong by a factor you could have predicted from the projection.
2. **Spatial autocorrelation is the default, not the exception.** It inflates your significance, leaks through your cross-validation, and confounds your coefficients. Test for it every time; the test costs milliseconds.

3. **Validate against something the analysis never used.** Recovering a known coefficient, reproducing an independently derived boundary, reconciling two layers that should agree — these are worth more than any goodness-of-fit statistic computed on your own training data.

Where to go next

Topic	Tools
Rigorous spatial statistics	PySAL — <code>libpysal</code> , <code>esda</code> , <code>spreg</code> , <code>spopt</code>
Network analysis and routing	<code>networkx</code> , <code>osmnx</code> , <code>pandana</code> , OSRM / Valhalla
Large-scale raster	<code>xarray</code> , <code>rioxarray</code> , <code>dask</code> , STAC + <code>odc-stac</code>
Cloud-native formats	Cloud-Optimized GeoTIFF, GeoParquet, Zarr, FlatGeobuf
Interactive maps	<code>folium</code> , <code>lonboard</code> , <code>pydeck</code> , <code>keplergl</code>
Geostatistics / kriging	<code>scikit-gstat</code> , <code>pykrige</code> , <code>gstools</code>
Spatial ML at scale	<code>verde</code> , <code>mlxtend</code> spatial CV, <code>spacv</code>

The fictional Vallmara Basin has now served its purpose. Take the same habits to real data — and be suspicious of it in exactly the same way.